

Ingeniería de Software

Apunte de estudio — Unidades 1 a 3 · UTN-FRC

Índice

- UNIDAD 1 - Introducción a la Ingeniería de Software
 - 1.1. Un poco de historia y evolución de la disciplina
 - 1.1.1. Conferencia de la OTAN (NATO conference) (Garmisch, 1968)
 - 1.1.2. Hitos de la evolución histórica (1975 - 2026)
 - 1.2. El concepto de Software y su naturaleza compleja
 - 1.2.1. Definición formal de Software y su representación abstracta
 - 1.2.2. Esencia y accidente en la creación de software (“No Silver Bullet”)
 - 1.2.3. Las cuatro propiedades esenciales del software complejo
 - Puntos clave para el examen — 1.2.3
 - Preguntas de autoevaluación — 1.2.3
 - 1.3. La Ingeniería de Software como disciplina formal
 - 1.3.1. Definición formal de la Ingeniería de Software
 - 1.3.2. SWEBOOK (Software Engineering Body of Knowledge)
 - 1.3.3. SEBoK (Systems Engineering Body of Knowledge)
 - 1.4. Disciplinas de la Ingeniería de Software (Materia en Contexto)
 - 1.4.1. Disciplinas Técnicas (Relacionadas con el Producto)
 - 1.4.2. Disciplinas de Gestión (Relacionadas con el Proyecto)
 - 1.4.3. Disciplinas de Soporte o transversales/protectoras (La Actividad Paraguas)
 - 1.4.4. Relación e interdependencia de las disciplinas en el ciclo de desarrollo
 - Puntos clave para el examen — 1.4.4
 - Preguntas de autoevaluación — 1.4.4
 - 1.5. El Ecosistema del Software en contexto: Las 5 Dimensiones Críticas
 - 1.5.1. Análisis Detallado de las Cinco Dimensiones
 - 1.5.2. Análisis Comparativo de las Cinco Dimensiones
 - 1.5.3. Interacción Sistémica del Ecosistema de Software
 - Puntos clave para el examen — 1.5.3
 - Preguntas de autoevaluación — 1.5.3
- UNIDAD 2 - SCM (Gestión de Configuración del Software)
 - 2.1. El software en contexto y la necesidad de la disciplina SCM
 - 2.1.1. Definición formal de SCM (ANSI/IEEE 828, 1990)
 - 2.1.2. El software como “blanco móvil” (movable target) y la volatilidad del cambio
 - 2.1.3. Las 7 problemáticas típicas en la gestión del software (según Bersoff)
 - Puntos clave para el examen — 2.1.3
 - Preguntas de autoevaluación — 2.1.3
 - 2.2. Conceptos clave para la Gestión de Configuración de Software
 - Tabla General de Contraste Conceptual de SCM
 - 2.2.1. Ítem de Configuración de Software (SCI / IC - Software Configuration Item)
 - 2.2.2. Repositorio (Repository)
 - 2.2.3. Línea Base (Baseline)
 - 2.2.4. Ramas (Branch) y Fusión (Merge)

- 2.2.5. Configuración de Software
- Puntos clave para el examen — 2.2.5
- Preguntas de autoevaluación — 2.2.5
- 2.3. Actividades fundamentales de la Gestión de Configuración (SCM) (Cont.)
 - 2.3.1. Identificación de ítems de configuración
 - 2.3.2. Clasificación de ICs por su ciclo de vida y ámbito
- Puntos clave para el examen — 2.3.2
- Preguntas de autoevaluación — 2.3.2
- 2.3. Actividades fundamentales de la SCM (Cont.)
 - 2.3.3. Control de Cambios (Configuration Control)
 - 2.3.4. Auditorías de Configuración (Configuration Auditing)
 - 2.3.5. Informes de Estado / Status Accounting (Configuration Status Accounting)
- Puntos clave para el examen — 2.3.5
- Preguntas de autoevaluación — 2.3.5
- 2.4. Plan de Gestión de Configuración (PGC)
 - 2.4.1. Definición formal y propósito del PGC
 - 2.4.2. Plantilla de estructura formal y secciones del PGC (Estándar TP4)
- 2.5. Gestión de Configuración de Software en ambientes Ágiles
 - 2.5.1. El cambio de paradigma: SCM sin fricción
 - 2.5.2. Patrones de Compilación en Agile SCM (Kinds of Builds)
- Puntos clave para el examen — 2.5.2
- Preguntas de autoevaluación — 2.5.2
- 2.6. Casos prácticos de SCM (TP4)
 - 2.6.1. Enunciado general y objetivos del Trabajo Práctico N° 4
 - 2.6.2. Casos de discusión en aula con resolución y fundamento teórico
 - 2.6.3. Tabla Comparativa de Casos Prácticos de SCM
- Puntos clave para el examen — 2.6.3
- Preguntas de autoevaluación — 2.6.3
- UNIDAD 3 - Requerimientos ágiles: user stories y estimaciones ágiles
 - 3.1. Requerimientos ágiles: Introducción y bases del Manifiesto Ágil
 - 3.1.1. Filosofía ágil y el Manifiesto Ágil (2001)
 - 3.1.2. Procesos Empíricos vs. Procesos Definidos (La base del empirismo)
 - 3.2. Requerimientos en el Paradigma Ágil
 - 3.2.1. El Concepto de “Valor” y la diferencia entre Output y Outcome
 - 3.2.2. El 50% de los requerimientos emergentes
 - Puntos clave para el examen — 3.2.2.1
 - Preguntas de autoevaluación — 3.2.2.1
 - 3.3. Estructura de una User Story: Tarjeta, Conversación, Confirmación y Criterios de Aceptación
 - 3.3.1. Las 3 C’s de las User Stories (Kent Beck / Ron Jeffries)
 - 3.3.2. El Modelo INVEST (Bill Wake)
 - 3.3.3. Contradicción de Longevidad: ¿Se descartan o se conservan las User Stories?
 - Puntos clave para el examen — 3.3.3
 - Preguntas de autoevaluación — 3.3.3

- 3.4. Estimación ágil y Story Points (Puntos de Historia)
 - 3.4.1. Definición y Fundamento de la Estimación Relativa
 - 3.4.2. Las tres dimensiones de un Story Point
 - 3.4.3. La User Story Canónica (Base Story)
 - 3.4.4. Escalas de Estimación: Fibonacci Modificada y Taller de Remera
- 3.5. La Dinámica de Planning Poker (Scrum Poker)
 - 3.5.1. Definición y Características de Base
 - 3.5.2. El Procedimiento de Planning Poker Paso a Paso
 - 3.5.3. Decodificación de las cartas especiales del mazo
 - 3.5.4. Políticas de Reestimación de Historias en SCM (Mike Cohn - Cap. 7)
- 3.6. Relación entre Estimación, Compromiso, Objetivo de Negocio y Velocidad
 - 3.6.1. La Ecuación Ágil Central (La derivación del Calendario)
 - 3.6.2. Matriz de Desambiguación Conceptual: Estimación, Compromiso y Objetivo
 - 3.6.3. El Cono de la Incertidumbre (Barry Boehm) y el Rol de la Velocidad
 - 3.6.4. Patologías de la estimación: Sobreestimación y Subestimación
- Puntos clave para el examen — 3.6.4
- Preguntas de autoevaluación — 3.6.4
- 3.7. El espectro de los Productos Mínimos (MVP, MMP y MLP) y la justificación de la escala
 - 3.7.1. Producto Mínimo Viable (MVP - Minimum Viable Product)
 - 3.7.2. Producto Mínimo Comercializable (MMP - Minimum Marketable Product)
 - 3.7.3. Producto Mínimo Adorable (MLP - Minimum Lovable Product)
 - 3.7.4. Justificación valor por valor de la Escala de Fibonacci Modificada (Mazu de Cartas del TP4)
 - 3.7.5. La Contradicción de Estimación: Story Points frente a Días Ideales (Ideal Days)
- Puntos clave para el examen — 3.7.5
- Preguntas de autoevaluación — 3.7.5
- 3.8. Poker de Estimación (Poker Estimation)
 - 3.8.1. Definición y Fundamento Metodológico
 - 3.8.2. Contradicción Terminológica: “Poker Estimation” vs. “Planning Poker”
 - 3.8.3. La Dinámica Paso a Paso y el Rol de cada Participante
- 3.9. Políticas de Reestimación (Re-Estimating)
 - 3.9.1. Cuándo SÍ se debe reestimar una US:
 - 3.9.2. Cuándo NO se debe reestimar una US:
- 3.10. La Ecuación Central Ágil y la Métrica de Velocidad
 - 3.10.1. La Ecuación Central de SCM y Derivación del Calendario
 - 3.10.2. El Criterio Binario de la Velocidad (La Regla de Oro)
- Puntos clave para el examen — 3.10.2
- Preguntas de autoevaluación — 3.10.2
- 3.11. El espectro de los Productos Mínimos (MVP, MMP, MLP), validación de hipótesis y el corte vertical
 - 3.11.1. Producto Mínimo Viable (MVP - Minimum Viable Product)

- 3.11.2. Producto Mínimo Comercializable (MMP - Minimum Marketable Product)
 - 3.11.3. Producto Mínimo Adorable (MLP - Minimum Lovable Product)
 - 3.11.4. Matriz Comparativa: El Espectro de los Productos Mínimos
 - 3.11.5. El Arte del Corte de Requerimientos: Corte Vertical vs. Construcción Horizontal
- Puntos clave para el examen — 3.11.5
- Preguntas de autoevaluación — 3.11.5
- 3.12. Caso de Estudio Resuelto: “El Parque del Sol” (Gestión de Requerimientos y Estimación)
 - 3.12.1. Introducción al Caso de Estudio
 - 3.12.2. Listado de Historias de Usuario (US) Identificadas
 - 3.12.3. Definición del Producto Mínimo Viable (MVP)
 - 3.12.4. Ejercicio Práctico de Poker de Estimación (Poker Estimation)
 - 3.12.5. La Contradicción Terminológica de Examen: “Poker Estimation” vs “Planning Poker”
- Puntos clave para el examen — 3.12.5
- Preguntas de autoevaluación — 3.12.5
- 3.13. Spikes: investigación, gestión de incertidumbre y reducción de riesgos
 - 3.13.1. Definición conceptual y formal de “Spike”
 - 3.13.2. Clasificación completa de Spikes
 - 3.13.3. Lineamientos y políticas de gestión de Spikes
 - 3.13.4. Ejemplos concretos de la cátedra y la bibliografía
- 3.14. Estructura y jerarquía del backlog (líneas de abstracción)
 - 3.14.1. Tema (Theme)
 - 3.14.2. Épica (Epic)
 - 3.14.3. Historia de usuario (User Story)
 - 3.14.4. Tarea (Task)
- 3.15. Técnicas de particionado de épicas (splitting / slicing vertical)
 - 3.15.1. Qué problema resuelve el particionado
 - 3.15.2. Los cuatro métodos técnicos de particionado (Cohn, Cap. 12)
 - 3.15.3. El concepto de “tracer bullet” (bala de trazado / slicing vertical)
- 3.16. Errores comunes y advertencias en Spikes y estructura del backlog
- 3.17. Relación de Spikes y backlog con otros conceptos del apunte
- APÉNDICE A — Glosario bilingüe (ES/EN)
 - A
 - B
 - C
 - D
 - E
 - G
 - I
 - L
 - M
 - O
 - P

- R
- S
- T
- U
- V
- APÉNDICE B — Mapa de relaciones entre unidades
 - 1. CONEXIONES CONCEPTUALES DESDE LA UNIDAD 1 (Introducción e Integridad)
 - 1.1. El Pentágono del Ecosistema del Software (Proceso, Proyecto, Producto, Personas, Herramientas)
 - 1.2. Procesos Empíricos (Scrum / Kanban)
 - 1.3. Inherencia de la Complejidad del Software (Fred Brooks)
 - 2. CONEXIONES CONCEPTUALES DESDE LA UNIDAD 2 (SCM - Control de Configuración)
 - 2.1. Identificación de Ítems de Configuración (ICs)
 - 2.2. Control de Cambios (Procedimiento formal: CCB, CRF, ECP, ECO)
 - 2.3. Auditorías de Configuración (Física - PCA y Funcional - FCA)
 - 2.4. Informes de Estado / Status Accounting
 - 3. CONEXIONES CONCEPTUALES DESDE LA UNIDAD 3 (Requerimientos y Estimación)
 - 3.1. User Stories (Dimensión de las 3 C's e INVEST)
 - 3.2. Story Points y Escala de Fibonacci Modificada
 - 3.3. El Espectro de Mínimos (MVP, MMP y MLP)
 - 4. LAS TRES CONTRADICCIONES CLAVE DE EXAMEN (Bibliografía vs. Cátedra)
 - CONTRADICCIÓN 1: Longevidad de las User Stories (¿Se descartan o se conservan?)
 - CONTRADICCIÓN 2: Story Points frente a Días Ideales (Ideal Days)
 - CONTRADICCIÓN 3: Planning Poker frente a Poker de Estimación (Poker Estimation)

UNIDAD 1 - Introducción a la Ingeniería de Software

1.1. Un poco de historia y evolución de la disciplina

1.1.1. Conferencia de la OTAN (NATO conference) (Garmisch, 1968)

- **Definición precisa e histórica:** Reunión de carácter internacional celebrada del 7 al 11 de octubre de 1968 en Garmisch, Alemania, patrocinada por el Comité Científico de la OTAN (Organización del Tratado del Atlántico Norte). Es el hito fundacional donde se acuñó y formalizó el término “**Ingeniería de Software**” para abordar la “**crisis del software**” (**software crisis**) [Sommerville, Prefacio].
- **Qué problema resuelve:** Antes de 1968, el software se construía de manera puramente artesanal, sin métodos científicos, predictibilidad de costos, ni planificación de tiempos. La conferencia se convocó para resolver la alarmante tasa de fracaso en grandes sistemas lógicos, los cuales sufrían de rezagos masivos, costos que triplicaban las estimaciones, inestabilidad crónica y falta de adecuación a las necesidades de los usuarios.
- **Cómo se hace (Paso a paso histórico):** El procedimiento para fundar la disciplina se basó en establecer un consenso entre científicos de la computación, líderes industriales y el sector público para:
 - Rechazar el enfoque artesanal e individual del desarrollo de software.
 - Adoptar formalmente una base conceptual de ingeniería, fundamentada en teorías, herramientas y métodos sistemáticos, disciplinados y cuantificables.
 - Estructurar el ciclo de vida del producto mediante actividades explícitas de especificación, diseño, validación y mantenimiento.
- **Clasificación de los síntomas de la “Crisis del Software”:** El material de clase y Sommerville agrupan las problemáticas identificadas en:
 - *Desvíos temporales (Schedule Overruns):* Proyectos de software crónicamente demorados que nunca cumplían los plazos originales.
 - *Desvíos económicos (Budget Overruns):* Costos financieros de desarrollo muy superiores a los presupuestados.
 - *Falta de fiabilidad (Unreliability):* Software inestable que fallaba en producción y provocaba pérdidas críticas.
 - *Brecha funcional (Functional Gap):* Sistemas terminados que no brindaban la funcionalidad que realmente requerían los usuarios finales.
- **Ejemplo concreto de la cátedra:** El docente en los audios teóricos cita el colosal proyecto del sistema operativo **OS/360** de IBM, liderado por Frederick Brooks. Fue uno de los primeros y más costosos sistemas operativos de la industria, el cual sufrió desvíos temporales masivos, requirió millones de dólares adicionales y estuvo plagado de fallas de estabilidad conceptuales, obligando a repensar las bases de la administración de proyectos lógicos.
- **Errores comunes y advertencias:**

- **△ Error común en el examen:** Pensar que la crisis del software es un problema del siglo pasado que ya está solucionado gracias a las computadoras modernas.
- **Advertencia del docente:** El docente enfatiza repetidamente en las clases grabadas que **60 años después seguimos cometiendo exactamente los mismos errores** que Brooks documentó en sus libros. La tecnología avanzó, pero el factor humano (capa 8), las malas estimaciones y la falta de control de configuración siguen saboteando proyectos de manera sistemática.
- **Relación con otros conceptos:** Dispara de manera directa la necesidad de crear la disciplina de **SCM (Gestión de Configuración de Software)** para garantizar la integridad y el origen del producto a lo largo de su ciclo de vida volátil.

1.1.2. Hitos de la evolución histórica (1975 - 2026)

- **Definición:** Secuencia cronológica de publicaciones teóricas, estándares y cambios de paradigmas que estructuran la evolución del desarrollo de software.
- **Qué problema resuelve:** Proporciona el mapa contextual necesario para entender por qué las técnicas tradicionales (dirigidas por planes rígidos) colisionaron con la realidad dinámica del mercado, forzando la transición hacia la agilidad y los procesos empíricos.
- **Evolución histórica detallada:**

Año	Hito de la Evolución Histórica	Autor / Entidad	Aporte Técnico e Impacto en la Disciplina
1968	Conferencia de la OTAN	Naur y Randell	Nacimiento de la disciplina Ingeniería de Software y delimitación de la crisis del software [Sommerville Prefacio].
1975	The Mythical Man-Month	Frederick Brooks	Publicación del libro basado en los errores de gestión del OS/360 de IBM. Introduce la Ley de Brooks (“Agregar personal a un proyecto de software retrasado lo retrasa más”) y desmitifica al “hombre-mes” como métrica lineal de intercambio.
1978	Structured Analysis	Tom DeMarco	Introducción de formalidades estructuradas para el relevamiento de requerimientos y diseño de procesos de datos mediante diagramas de flujo (<i>Data Flow Diagrams</i>).
1980s	Grandes errores conocidos	Varios	Fallos de software catastróficos que generaron pérdidas multimillonarias o pusieron en riesgo vidas humanas, exponiendo la vulnerabilidad del software crítico de protección (<i>safety-critical software</i>).
1987	No Silver Bullet	Frederick Brooks	Publicación del ensayo fundacional en la revista <i>IEEE Computer</i> . Distingue entre esfuerzo

Año	Hito de la Evolución Histórica	Autor / Entidad	Aporte Técnico e Impacto en la Disciplina
			esencial (esencia/conceptualización) y esfuerzo accidental (accidente/representación).
1989	Managing the Software Process	Watts Humphrey	Sienta las bases para caracterizar el software como un proceso repetible, predecible y optimizable, dando lugar al nacimiento de los modelos de evaluación y madurez.
1990	Object Oriented e Internet	Varios	Masificación del paradigma de Orientación a Objetos (diseño conceptual mediante clases y encapsulamiento) y la explosión de la Web, cambiando de forma drástica el diseño arquitectónico y de interfaces.
1991-2000	CMM a CMMI	SEI (Software Engineering Institute)	Lanzamiento del Capability Maturity Model (CMM) y su evolución a CMMI (CMM Integration) en el año 2000, estructurando la mejora de procesos en base a niveles de madurez definidos.
2001	Agile Manifesto	Beck, Cohn, et al.	Publicación del Manifiesto Ágil, formalizando la adopción de procesos empíricos frente a los predictivos tradicionales.
2003	Lean Software Development	Poppendieck	Incorporación de los principios de manufactura esbelta (<i>Lean</i>) al software, focalizados en la eliminación sistemática de desperdicios.
2026	Dominio del software corporativo	Ecosistema Global	El software se consolida como el activo de mayor valuación de mercado del mundo. Siete de las diez empresas más grandes del planeta en capitalización bursátil son compañías de base tecnológica pura (NVIDIA, Alphabet, Apple, Microsoft, Amazon, TSMC, Meta).

1.2. El concepto de Software y su naturaleza compleja

1.2.1. Definición formal de Software y su representación abstracta

- **Definición de Sommerville (BIBLIO_ - Cap. 1.1):** El software no es sólo el código fuente de los programas de cómputo. El software desarrollado profesionalmente consta de **los programas en sí, toda la documentación asociada y los datos de configuración requeridos para hacer que estos programas operen de manera correcta**. Esto incluye documentación del sistema (arquitectura, diseño), documentación del usuario (manuales, guías) y sitios web de actualización.

- **Definición integradora de la cátedra (SLIDE_ / NC_):** “El software es conocimiento representado en distintos niveles de abstracción”. Es un activo intangible que empaqueta ideas, reglas de negocio y flujos lógicos abstractos, refinándolos capa por capa hasta convertirlos en código binario (ceros y unos) interpretable por la electrónica del hardware.
- **Qué problema resuelve:** Elimina la visión simplista y artesanal del programador que cree que su trabajo termina cuando el código compila. Forzar la concepción del software como “conocimiento empaquetado y documentado” garantiza que el sistema sea mantenible, almacenable en un repositorio y transferible a nuevos desarrolladores cuando los autores originales se vayan.
- **Cómo se representa (Capas de Abstracción de Conocimiento):** El conocimiento del dominio se desglosa progresivamente en las siguientes capas lógicas:
 1. **La Idea Genial:** La concepción lúdica y de negocio abstracta del sistema (visión, reglas de juego, historia).
 2. **Cómo se hace:** Planos de diseño técnico representados mediante diagramas arquitectónicos, UML, casos de uso y diagramas de base de datos.
 3. **El Código de Verdad:** La representación formal de los planos en un lenguaje de programación de alto nivel (Python, Java, C++).
 4. **Las Herramientas Mágicas:** Las bibliotecas de software (*libraries*), frameworks de soporte y el sistema operativo que unifican el código del programador.
 5. **El Idioma del Chip:** El código binario (ceros y unos) que representa instrucciones puras entendidas directamente por la CPU.
 6. **El Hardware Físico:** Los transistores de silicio y la memoria electrónica por donde fluyen impulsos eléctricos reales.
- **Clasificación de los tipos de software:** El material de Sommerville y las clases lo clasifican en tres tipos básicos:
 1. **System Software (Software de Sistema):** Administra y controla los recursos de hardware del sistema (ej. sistemas operativos como Linux o Windows).
 2. **Utilitarios (Utilities):** Programas de soporte operativo o técnico (ej. herramientas de control de versiones como Git, depuradores de código).
 3. **Software de Aplicación (Application Software):** Programas orientados a resolver necesidades directas de usuario de negocio (ej. navegadores, software contable, videojuegos).
- **Ejemplo concreto de la cátedra:** La ingeniera de software **Margaret Hamilton** (NASA), quien lideró el desarrollo del código para el Proyecto Apolo. La famosa fotografía de Hamilton de pie al lado de la pila de listados impresos de código fuente ilustra cómo el conocimiento matemático y físico abstracto de navegación espacial quedó representado y empaquetado formalmente como software.
- **Cinco razones académicas para no comparar software con manufactura física (¡PREGUNTA DE PARCIAL!):**
 1. *El software es menos predecible:* No responde a parámetros físicos repetitivos; su estimación está gobernada por variables humanas de alta incertidumbre.

2. *No existe la producción en masa*: En manufactura, reproducir cada auto o tornillo tiene un costo físico unitario alto. En software, una vez diseñado el “plano lógico” (código), duplicarlo cuesta literalmente cero pesos.
 3. *El software no se gasta*: Las piezas de un auto se desgastan por fricción mecánica; el software funciona de manera idéntica infinitamente si el hardware no falla y las variables no se modifican.
 4. *Deterioro por introducción de cambios*: El software no se rompe por el uso, sino por la continua inserción de parches lógicos apresurados (deuda técnica) que degradan su estructura conceptual original.
 5. *No gobernado por leyes físicas*: La manufactura está limitada por la gravedad, inercia y masa. El software, al carecer de masa física, genera en los gerentes la ilusión de que realizar cambios masivos de último minuto es sumamente fácil, rápido y gratuito.
- **Relación con otros conceptos**: El software es la entidad lúdica del desarrollo. Cada uno de sus planos, códigos y documentos son los **Ítems de Configuración de Software (ICs)** que la disciplina **SCM** debe registrar de manera unívoca para mantener la trazabilidad pre y post especificación (ERS).

1.2.2. Esencia y accidente en la creación de software (“No Silver Bullet”)

- **Definición precisa de Brooks (BIBLIO_ - Cap. 16)**:
 - **Esencia (Esfuerzo Esencial)**: La tarea de moldear mentalmente (*mental crafting*) las complejas estructuras lógicas conceptuales que componen la entidad de software abstracta (conjuntos de datos, algoritmos, llamadas a funciones y sus interrelaciones). Consiste en descifrar de manera exacta e inequívoca “el qué” construir.
 - **Accidente (Esfuerzo Accidental)**: La tarea operativa de representar físicamente estas estructuras conceptuales abstractas en un lenguaje de programación específico y mapearlas en instrucciones físicas del procesador bajo restricciones de velocidad y espacio. Consiste en resolver de forma pragmática “el cómo” implementarlo.
- **Qué problema resuelve**: Desmitifica las promesas de la industria de software que periódicamente venden metodologías de gestión (ej. Scrum), lenguajes de programación o herramientas de inteligencia artificial como la “bala de plata” (*silver bullet*) mágica que solucionará el problema del desarrollo lúdico de la noche a la mañana. Divide el problema para demostrar que los grandes saltos del pasado solucionaron únicamente accidentes representacionales y no la complejidad conceptual inherente a la esencia.
- **Cómo se analiza matemáticamente**: Brooks postula que, por definición, el esfuerzo de desarrollo total es la suma de ambos componentes:

$\text{Esfuerzo Total} = \text{Esfuerzo Esencial} + \text{Esfuerzo Accidental}$

Si lográramos diseñar una herramienta con magia absoluta que redujera el esfuerzo de codificación y sintaxis (accidental) a **cero por ciento (0%)**, la mejora global de la productividad quedaría limitada por la porción de tiempo dedicada a la esencia conceptual. Si tu equipo destina el 10% del esfuerzo a escribir código (accidente) y el 90% a descifrar la lógica del negocio (esencia), tu productividad máxima esperada al optimizar al extremo el accidente será de apenas 1.11x:

Esfuerzo Optimizado = 90% (Esencial) + 0% (Accidental) = 90% del esfuerzo original

- **Clasificaciones de las dificultades del Software (Esencia vs. Accidente):**

Característica de Comparación	Esencia (Conceptualización)	Accidente (Representación)
Pregunta fundamental	“¿Qué construir?”* (Reglas de negocio, algoritmos lógicos, flujos lógicos).	¿Cómo codificarlo?* (Sintaxis del lenguaje, compilación de archivos, memoria física).
Naturaleza del esfuerzo	Inevitable e intrínseca al problema real del cliente y su complejidad de negocio.	Artificial y circunstancial de las herramientas de software disponibles en cada época.
Hitos que lo solucionaron	No se resuelve con herramientas. Exige desarrollo incremental, prototipado y grandes diseñadores.	Lenguajes de alto nivel (C, Python), entornos integrados, sistemas de tiempo compartido (<i>timesharing</i>).
Fallas comunes	Errores lógicos graves, requerimientos ambiguos, inconsistencias funcionales del negocio.	Errores de sintaxis (<i>fuzz</i>), punteros nulos, incompatibilidades de variables, fallas de compilación.
Costo de reparación	Altísimo e incremental a medida que avanza el proyecto. Extremadamente difícil de revertir en fases tardías.	Bajo. Los compiladores modernos e IDEs detectan y resuelven el error de forma automatizada y veloz.

- **Ejemplo concreto de la cátedra:** Un programador se sienta a desarrollar un sistema de cobro impositivo. Si pasa dos horas renegando con el editor de código porque la variable del IVA tiene un error de sintaxis de tipo o el indentado de Python falla, el desarrollador está consumiendo su tiempo en una **dificultad accidental** (representacional). En cambio, si el programa corre sin un solo fallo de compilación, pero calcula mal el impuesto correspondiente a los productos exentos de la AFIP por un error en las fórmulas financieras conceptuales, cometió un **error esencial** de conceptualización.
- **Errores comunes y advertencias:** El docente advierte que el error de gestión más recurrente es el de **Capa 8 (nosotros)**. Caemos en el facilismo de pensar que un versionador de código como Git, o un framework ágil como Scrum, solucionarán mágicamente la falta de entendimiento con el cliente. Las herramientas agilizan los accidentes operativos (ej. hacer *push* rápido), pero el diseño de la arquitectura conceptual de calidad depende estrictamente del análisis humano y la comunicación.

1.2.3. Las cuatro propiedades esenciales del software complejo

Brooks postula que la esencia lógica abstracta del software arrastra consigo cuatro propiedades intrínsecas e inevitables que no se pueden simplificar de manera analítica mediante ninguna herramienta tecnológica:

Propiedad 1: Complejidad (Complexity)

- **Definición de Brooks (BIBLIO_)**: Las entidades de software son inherentemente más complejas por unidad de escala que cualquier otro constructor humano debido a que **ninguna de sus partes de diseño lúdico es igual a otra**. En un edificio físico o puente, los elementos lógicos y estructurales (vigas, columnas, ventanas) se repiten simétricamente y de forma idéntica; en el software de mediana a gran escala, no existe repetición física de elementos: cada línea de código, regla de negocio y flujo lúdico de ejecución representa una abstracción conceptual única y distinta. El número de estados lógicos posibles es astronómico y no lineal respecto a su tamaño físico, lo que provoca que un incremento lineal en el tamaño multiplique exponencialmente las interacciones colaterales.
- **Qué problema o consecuencias genera:**
 1. Impide que una única mente humana comprenda la totalidad del sistema lúdico de software, destruyendo la **integridad conceptual** del diseño.
 2. Hace sumamente difícil mapear y resguardar los “cabos sueltos” (loose ends) de interrelaciones lógicas.
 3. Provoca que la inducción de nuevos programadores al equipo tenga una carga de aprendizaje inmensa; por esto, la rotación de personal (*personnel turnover*) es una catástrofe total en software.
- **Ejemplo concreto:** Los colosales sistemas críticos de control de tráfico aéreo de aeropuertos o el sistema operativo OS/360 de IBM. Al modificar una variable global insignificante en una pantalla cosmética secundaria de la terminal, se genera un impacto colateral imprevisto sobre un módulo de bajo nivel de acceso a disco, provocando la corrupción física y la caída general de los datos de vuelo lúdicos sin que ningún desarrollador pudiera anticiparlo por la complejidad de acoplamiento lateral de los módulos.

Propiedad 2: Conformidad (Conformity)

- **Definición de Brooks (BIBLIO_)**: El software carece de leyes y principios lógicos y simétricos derivados de la naturaleza. A diferencia de los físicos, que trabajan bajo la fe de que el universo responde a leyes simétricas simplificadas y unificadoras (ej. teoría de cuerdas, relatividad) porque “Dios no es caprichoso ni arbitrario”, los ingenieros de software deben someter su diseño abstracto a la **conformidad forzada de regulaciones gubernamentales cambiantes, interfaces de sistemas heredados (*legacy systems*) obsoletos y presiones de negocios arbitrarias** diseñadas de manera asimétrica por distintas mentes humanas en momentos de tiempo históricos aleatorios.
- **Qué problema o consecuencias genera:**
 1. La complejidad conceptual del software no se puede simplificar mediante elegantes reglas lógicas de alto nivel, porque las fuerzas a las que se conforma son inherentemente caprichosas y carecen de simetría matemática.
 2. Gran parte del esfuerzo lúdico de programación no se destina a resolver el problema real del usuario, sino a encastrar el sistema de manera forzada y artificial en un ecosistema preexistente hostil y desordenado.
- **Ejemplo concreto:** El docente explica en los audios que el software se inserta en realidades organizacionales con “muchos alguienes” con intereses opuestos (clientes, directores, auditores, normas gubernamentales). Si se desarrolla un sistema de liquidación de sueldos

para una universidad, el software está obligado a conformarse de manera asimétrica con: las leyes impositivas del estado (AFIP) que sufren parches excepcionales de cobro impositivo anual asimétricos, los convenios laborales arbitrarios del gremio docente, y los formatos de intercambio de archivos planos binarios obsoletos exigidos por el sistema del Banco Central de la República Argentina, llenando el código de condicionales y parches asimétricos inevitables para poder operar.

Propiedad 3: Mutabilidad / Variabilidad (Changeability)

- **Definición de Brooks (BIBLIO_):** El software está sometido de forma continua a una presión de cambio evolutivo drástica y única. Esta mutabilidad radical ocurre por dos factores centrales:
 1. *Maleabilidad física:* El software es puro pensamiento intangible representado en un medio digital, careciendo de la inercia física de la materia.
 2. *Presión social del entorno:* El software exitoso se sitúa en el centro operativo de procesos institucionales y de negocios volátiles; si el negocio cambia, el software debe mutar para seguir siendo de utilidad.
- **Qué problema o consecuencias genera:**
 1. Un software exitoso sufrirá mutación continua de requerimientos. El software que no recibe pedidos de cambio sistemáticos a lo largo del tiempo es, por definición, un software que nadie está usando en producción.
 2. Los usuarios asumen erróneamente que cambiar el software es “gratis” e inmediato porque no requiere demoler muros físicos ni comprar hormigón, forzando al equipo a meter parches lúdicos rápidos (deuda técnica) que corrompen progresivamente el diseño arquitectónico de calidad del sistema, derivando en caídas de servicio lúdicas.
- **Ejemplo concreto de la cátedra:** El escenario típico de aula descrito por el docente: el equipo de alumnos desarrolla una pantalla de carga para una aplicación, corre las pruebas unitarias y el cliente les da el “okay” formal. Cinco minutos después, el cliente los llama por teléfono y les dice: “*Che, ahora quiero que los datos se guarden en una planilla Excel interactiva y que mande un correo por cada registro*”. Para cumplir con el plazo de entrega urgente, el equipo mete mano apresurada sobre el código sin rediseñar la arquitectura, introduciendo deuda técnica que destruye la cohesión del módulo y rompe colateralmente las pantallas de carga secundarias lúdicas.

Propiedad 4: Invisibilidad (Invisibility)

- **Definición de Brooks (BIBLIO_):** El software es intrínsecamente unvisualizable (no representable en un espacio físico tridimensional). A diferencia de los ingenieros civiles, químicos o mecánicos que pueden diagramar planos detallados en escala geométrica real que representan la totalidad tridimensional del artefacto físico de forma intuitiva, el software es un conjunto hipercomplejo de abstracciones conceptuales acopladas que no poseen correspondencia en el espacio geométrico real.
- **Qué problema o consecuencias genera:**
 1. Priva a la mente de los diseñadores de su herramienta cognitiva más potente: la visualización espacial.

2. Cualquier intento de representación gráfica de software (ej. diagramas de flujo de datos DFD, modelos de entidad-relación, diagramas de clases UML) es incapaz de mostrar el todo de manera unificada. Cada diagrama es una simplificación sesgada y abstracta: si dibujamos el flujo de datos lúdicos, ocultamos el control de concurrencia; si dibujamos la topología física, ocultamos la herencia conceptual.
 3. Obstaculiza drásticamente la comunicación fluida entre desarrolladores y destruye el entendimiento mutuo con los usuarios no técnicos.
- **Ejemplo concreto de la cátedra:** El debate surgido en el aula a partir de la pregunta del alumno sobre si la invisibilidad causa la proliferación de tantas representaciones de diseño incompatibles. El docente coincide plenamente: dado que el software no se ve físicamente, no existe una plantilla unificada para dibujarlo. Un programador visualiza el sistema como una red de clases POO acopladas, el administrador de base de datos lo ve como un plano de tablas relacionadas con claves foráneas, y el cliente final visualiza pantallas aisladas lúdicas, complicando enormemente la alineación conceptual para coordinar cambios sin romper nada.

Puntos clave para el examen — 1.2.3

Durante las clases teóricas y prácticas, los profesores de la materia recalcaron de forma sistemática los siguientes puntos, que constituyen preguntas obligatorias de examen:

1. **¡SABER PROGRAMAR NO ALCANZA!:** El docente es tajante al remarcar que codificar es meramente la resolución de una dificultad accidental (representación). Para aprobar el examen final se exige manejar el vocabulario técnico de Ingeniería de Sistemas de manera precisa, descartando de cuajo expresiones vagas como “el coso”, “el cosito” o “el pituto” para referirse a artefactos o procesos.
2. **DIFERENCIA ENTRE PROCESOS DEFINIDOS Y EMPÍRICOS:** El docente resalta esta distinción de manera recurrente. Para ejemplificar el agilismo empírico utiliza el caso de la **Universidad de California en Irvine**: sembraron pasto en todo el predio, esperaron un año a observar dónde la gente pisaba de manera natural para hacer “caminito” (inspección empírica de uso real), y recién ahí construyeron los caminos de hormigón sobre esos senderos empíricos. Los procesos definidos tradicionales, en cambio, trazan senderos artificiales rígidos de antemano basados en la teoría del planificador y fracasan porque no se adaptan a la realidad del uso humano.
3. **GESTIÓN BINARIA EN AGILIDAD:** El docente subraya que en agilidad no existe el avance porcentual formal: un entregable o una historia de usuario está **100% terminada y aceptada por el PO, o bien tiene un 0% de avance lúdico** para el cálculo de la velocidad del sprint. Los puntos porcentuales intermedios de avance lúdico parcial no suman velocidad.
4. **ADVERTENCIA EXPLICITA DE COPIA DE IA EN TP4:** Durante la clase práctica se resalta formalmente la política de la cátedra respecto al uso de Inteligencia Artificial para el práctico de SCM: no está prohibida, pero es obligatorio declarar qué herramienta se utilizó. **Si el grupo copia y pega un prompt que contenga placeholders como “reemplazar las comillitas dobles con el nombre de tu proyecto” o entrega código genérico sin contextualizar a su grupo, se les coloca un DOS (2) directo, inapelable y sin derecho a recuperatorio inmediato.**

Preguntas de autoevaluación — 1.2.3

Pregunta 1: ¿Cuál es la importancia histórica de la Conferencia de la OTAN de 1968 en Garmisch?

- **Respuesta breve:** Es el hito fundacional donde se acuñó formalmente el término “Ingeniería de Software” y se caracterizó la “crisis del software” para forzar la transición del desarrollo lógico artesanal a una disciplina de ingeniería predictiva [Sommerville Prefacio].

Pregunta 2: ¿Qué elementos integran al software profesional según Sommerville?

- **Respuesta breve:** Sommerville define que el software profesional no consta únicamente de los programas de cómputo en sí, sino de la integración de estos con toda la documentación asociada del sistema y usuario, y los datos de configuración necesarios para su correcto funcionamiento.

Pregunta 3: ¿Cómo define la cátedra al software de manera integradora y qué niveles de abstracción recorre?

- **Respuesta breve:** Se define como **conocimiento representado en distintos niveles de abstracción**. El conocimiento de negocio se refina progresivamente desde la idea lúdica abstracta inicial, planos conceptuales UML y código de alto nivel, hasta convertirse en código binario (ceros y unos) ejecutable por impulsos eléctricos.

Pregunta 4: ¿En qué consiste la distinción de Brooks entre dificultades de “Esencia” y “Accidente”?

- **Respuesta breve:** La **esencia** es la dificultad intrínseca de moldear mentalmente la compleja estructura lógica abstracta del software (el qué construir); el **accidente** es la dificultad circunstancial de representar físicamente ese concepto abstracto en un lenguaje de programación y hardware específico (el cómo codificarlo).

Pregunta 5: ¿Por qué Brooks postula que no existen “balas de plata” para optimizar drásticamente el desarrollo de software?

- **Respuesta breve:** Porque los grandes avances históricos redujeron a casi cero el esfuerzo de las dificultades **accidentales** (ej. sintaxis, compilación). Al representar el accidente una porción minoritaria del tiempo de desarrollo, optimizarlo más no genera impactos colosales de productividad; para lograr un salto de 10x, se debe atacar necesariamente la **esencia** conceptual del desarrollo.

Pregunta 6: ¿Cuáles son las cuatro propiedades esenciales del software complejo enumeradas por Brooks?

- **Respuesta breve:** Las propiedades son: **Complejidad, Conformidad, Mutabilidad / Variabilidad e Invisibilidad**.

Pregunta 7: ¿Por qué la “Complejidad” del software crece de manera no lineal respecto a su tamaño conceptual?

- **Respuesta breve:** Porque, a diferencia de otras ingenierías, en el software de gran escala **ninguna de sus partes de diseño lúdico es igual a otra**. Cada sentencia de código representa un comportamiento conceptual único, lo que provoca que agregar código multiplique exponencialmente los caminos de ejecución e interacciones laterales posibles.

Pregunta 8: ¿Por qué la “Conformidad” del software es inevitable y asimétrica?

- **Respuesta breve:** Porque el software no se rige por leyes universales simétricas de la naturaleza; está obligado a conformarse asimétricamente a las regulaciones impositivas, interfaces humanas históricas y sistemas heredados obsoletos creados de manera asimétrica por la sociedad.

Pregunta 9: ¿Qué impacto genera la “Invisibilidad” en el trabajo en equipo de desarrollo?

- **Respuesta breve:** Al ser el software intrínsecamente unvisualizable geométricamente en su totalidad, impide que los ingenieros tengan un estándar de representación unificado, forzándolos a utilizar diagramas simplificados que ocultan múltiples aristas lógicas y dificultan la alineación conceptual del equipo.

Pregunta 10: ¿Por qué Sommerville postula que el software no se rompe mecánicamente, sino que se deteriora por cambios?

- **Respuesta breve:** El software no sufre desgaste mecánico; si funciona bien y el hardware se mantiene estable, no fallará. El software se deteriora y “envejece” debido a la continua y apresurada introducción de parches lógicos que degradan la consistencia y la elegancia del diseño original de calidad.

1.3. La Ingeniería de Software como disciplina formal

1.3.1. Definición formal de la Ingeniería de Software

- **Definiciones formales del concepto (BIBLIO_ / SLIDE_):**
 - **David Parnas (SLIDE_01):** Definida sintéticamente como:
“La Ingeniería de Software es la construcción por múltiples personas de software multi-versión (multi-person construction of multi-version software)”.
 - **Definición oficial del IEEE, formalizada en el SWEBOK (SLIDE_01):**
“La ingeniería de software es la aplicación de un enfoque sistemático, disciplinado y cuantificable al desarrollo, operación y mantenimiento del software; es decir, la aplicación de la ingeniería al software”.
 - **Ian Sommerville (BIBLIO_ - Cap. 1.1.1):**
“La ingeniería de software es una disciplina de la ingeniería que se interesa por todos los aspectos de la producción de software, desde las primeras etapas de la especificación del sistema hasta el mantenimiento del sistema después de que se pone en operación”.
- **Análisis detallado de las frases clave de Sommerville (¡PREGUNTA DE EXAMEN!):**
 - *Disciplina de ingeniería:* Significa que los ingenieros hacen que las cosas funcionen aplicando teorías, métodos y herramientas selectivas y pragmáticas. Buscan soluciones a problemas reales incluso cuando no existen teorías científicas de soporte, trabajando bajo restricciones financieras, operativas y organizacionales.
 - *Todos los aspectos de la producción:* Excede las actividades técnicas del desarrollo de código (construcción); abarca la administración del proyecto (planificación, monitoreo), la gestión de calidad, la creación de herramientas y el diseño de teorías de soporte de la ingeniería.

- **Qué problema resuelve:** Desierra de cuajo la informalidad lúdica del desarrollo artesanal e individual (*programación personal*). Permite que organizaciones complejas construyan y evolucionen sistemas lógicos masivos garantizando predictibilidad de costos, plazos de entrega estimados técnicamente, fiabilidad conceptual y la trascendencia del producto más allá de la permanencia de sus autores originales en el equipo.
- **Cómo se hace (La técnica):** Se implementa mediante el modelado y ejecución de un **proceso de software**. Este proceso consiste en un conjunto estructurado de actividades de trabajo, métodos, prácticas y transformaciones que las personas ejecutan (apoyadas por herramientas de ingeniería) para traducir requerimientos de negocio en un producto de software de calidad.

1.3.2. SWEBOK (Software Engineering Body of Knowledge)

- **Definición precisa:** Estándar internacional consensuado, diseñado y publicado por la **IEEE Computer Society (Versión 3.0 de 2014)**, que unifica, delimita y describe formalmente el cuerpo de conocimientos que componen la disciplina de la Ingeniería de Software.
- **Qué problema resuelve:** Evita la fragmentación conceptual de la disciplina. Al definir de manera consensual “qué es y qué no es” Ingeniería de Software, sirve como marco de referencia para la industria, las certificaciones profesionales y el diseño de currículas académicas universitarias a nivel global.
- **Las 15 Áreas de Conocimiento (KA - Knowledge Areas) de SWEBOK v3.0 (Explicadas una por una)** [138, Image 30]:

[SWEBOK v3.0]	
[TÉCNICAS Y CICLO DE VIDA]	[SOPORTE, GESTIÓN Y TEORÍA]
1. Software Requirements	6. Software Configuration Mgmt.
2. Software Design	7. Software Engineering Mgmt.
3. Software Construction	8. Software Engineering Process
4. Software Testing	9. Models and Methods
5. Software Maintenance	10. Software Quality
11. Professional Practice	

[KA DE FUNDAMENTOS TEÓRICOS] 12. Software Engineering Economics

13. Computing Foundations
14. Mathematical Foundations
15. Engineering Foundations
16. **Software Requirements (Requerimientos de Software):** Actividades de elicitación (adquisición), análisis, especificación, validación y gestión de los requisitos que describen el comportamiento y las restricciones de operación del sistema.
17. **Software Design (Diseño de Software):** Definición de la arquitectura del sistema lúdico, componentes, interfaces, flujos lógicos y estructuras de datos internas a partir de los requerimientos.
18. **Software Construction (Construcción de Software):** Creación física detallada de software mediante la combinación de codificación (programación), verificación, pruebas unitarias, integración de componentes y depuración de errores (*debugging*).

19. **Software Testing (Pruebas de Software):** Verificación y validación empírica y dinámica del software mediante la ejecución de casos de prueba controlados para evaluar el comportamiento del programa y detectar defectos.
20. **Software Maintenance (Mantenimiento de Software):** Actividades de ingeniería ejecutadas tras la entrega del producto (soporte post-despliegue) para corregir fallos latentes, adaptar el sistema a cambios ambientales o agregar mejoras lúdicas.
21. **Software Configuration Management (Gestión de Configuración de Software):** SCM. Disciplina transversal encargada de identificar los ítems de configuración, controlar sistemáticamente los cambios de estado, registrar la trazabilidad e informes, y auditar las líneas base del producto.
22. **Software Engineering Management (Gestión de la Ingeniería de Software):** Planificación de proyectos, estimación cuantitativa de esfuerzo/costo, monitoreo y control lúdico de la ejecución del equipo de desarrollo [138, Image 18].
23. **Software Engineering Process (Proceso de la Ingeniería de Software):** Definición de modelos de ciclo de vida del software, evaluación, medición de madurez, diagnóstico y mejora incremental del proceso de software de la organización.
24. **Software Engineering Models and Methods (Modelos y Métodos de la Ingeniería de Software):** Teorías y marcos lógicos estructurados que guían el desarrollo (métodos orientados a objetos, métodos estructurados, métodos ágiles, y métodos formales/matemáticos).
25. **Software Quality (Calidad del Software):** Definición, medición y aseguramiento de que el software cumple con los atributos de calidad esperados (mantenibilidad, portabilidad, seguridad, fiabilidad, etc.).
26. **Software Engineering Professional Practice (Práctica Profesional de la Ingeniería de Software):** Estándares de conducta ética, normas de comportamiento profesional y responsabilidad legal frente a la sociedad y los clientes.
27. **Software Engineering Economics (Economía de la Ingeniería de Software):** Análisis financiero de costo-beneficio, decisiones de inversión lúdica en ingeniería, valor de negocio generado y estimación de retorno de inversión de sistemas [138, Image 16].
28. **Computing Foundations (Fundamentos de Computación):** Conceptos informáticos teóricos basales (sistemas operativos, algoritmos, arquitectura de computadoras, redes y telecomunicaciones).
29. **Mathematical Foundations (Fundamentos Matemáticos):** Elementos de matemática discreta, lógica matemática, teoría de conjuntos, probabilidad y estadística necesarios para la consistencia algorítmica.
30. **Engineering Foundations (Fundamentos de Ingeniería):** Conceptos metodológicos y empíricos clásicos de la ingeniería, como el modelado por simulación, análisis cuantitativo de fallas y controles empíricos.
- **Ejemplo concreto de la cátedra:** El docente resalta en los audios que los planes de estudio vigentes de la carrera Ingeniería en Sistemas de Información en la UTN-FRC toman como base de diseño técnico los estándares curriculares emanados de la ACM y la IEEE, estructurando las materias lógicas y de programación para cubrir integralmente estas 15 áreas de SWEBOOK.

1.3.3. SEBoK (Systems Engineering Body of Knowledge)

- **Definición precisa:** Cuerpo de conocimientos unificado enfocado en la **Ingeniería de Sistemas** en general (Versión 1.9.1 de 2018), orientada a la concepción, diseño, procuración, despliegue y mantenimiento de **sistemas sociotécnicos complejos** que integran hardware, software y personas dentro de una organización.
- **Qué problema resuelve:** SWEBOK delimita únicamente el software como entidad lógica. SEBoK resuelve la necesidad de integrar múltiples disciplinas de ingeniería disímiles para coordinar grandes artefactos tecnológicos complejos en los cuales el software es solo una pieza del rompecabezas sociotécnico.
- **Estructura y Relación con SWEBOK (Las Partes de SEBoK v1.9.1)** [35, Image 33 / Image 35]:
 - *Part 1: Introduction (Introducción):* Contextualización de la disciplina e historia [Image 33].
 - *Part 2: Systems Foundations (Fundamentos de Sistemas):* Ciencia, pensamiento y modelado de sistemas lúdicos [Image 33].
 - *Part 3: Systems Engineering & Management (Ingeniería de Sistemas y Gestión):* Ciclo de vida y procesos de control [Image 33].
 - *Part 4: Applications of Systems Engineering (Aplicaciones):* Aplicación de la ingeniería a sistemas de producto, servicios, empresas y salud.
 - *Part 5: Enabling Systems Engineering (Facilitadores):* Mecanismos para facilitar la coordinación a nivel de individuos, equipos y empresas.
 - *Part 6: Related Disciplines (Disciplinas Relacionadas):* Describe los encastrés y límites de la Ingeniería de Sistemas con la Ingeniería de Software, Gestión de Proyectos e Ingeniería Industrial.
 - *Part 7: Implementation Examples (Ejemplos de Implementación):* Estudios de caso prácticos reales [Image 33].
- **Hueco del material cargado:**

[HUECO: Las partes de SEBoK se nombran e ilustran de manera puramente diagramática en la SLIDE_01 (01 Introducción a la Ingeniería de Software.pdf), pero no existe ningún desarrollo narrativo ni explicación de los subtemas técnicos de cada una de estas partes en ninguna de las notas de compañero ni en los audios de teoría o práctica de la materia. Se marcan como un tema del programa de examen sin desarrollo detallado cargado en el entorno].
- **Relación con otros conceptos:** La Ingeniería de Sistemas es el contenedor macro. El software (definido por SWEBOK) es un componente de software que corre sobre el hardware (Ingeniería Electrónica) y que interactúa con la infraestructura civil y la ergonomía de los usuarios operativos.

1.4. Disciplinas de la Ingeniería de Software (Materia en Contexto)

La cátedra organiza académicamente las disciplinas del proceso de desarrollo en **tres grandes grupos categorizados**, según su foco estratégico en el ecosistema (Producto, Proyecto o Actividad transversal):

[DISCIPLINAS EN LA MATERIA]
[D. TÉCNICAS] [D. GESTIÓN] [D. SOPORTE]

(Foco: EL PRODUCTO) (Foco: EL PROYECTO) (Foco: TRANSVERSAL)

1. Requerimientos 1. Planificación 1. SCM (Configuración)
2. Análisis y Diseño 2. Monitoreo y Control 2. PPQA (Calidad)
3. Construcción 3. Métricas
4. Prueba [Foco UTN]
5. Despliegue [Foco UTN]

1.4.1. Disciplinas Técnicas (Relacionadas con el Producto)

Tienen como objetivo directo la especificación, diseño, codificación y puesta en producción del software como producto tangible lúdico.

Disciplina 1: Requerimientos (Requirements)

- **Definición:** Disciplina técnica que se ocupa de descubrir (elicitar), analizar, negociar, especificar y validar las necesidades y expectativas del cliente para plasmarlas en requerimientos funcionales y no funcionales.
- **Qué problema resuelve:** Evita la “brecha de expectativas” o el desvío funcional. Ataca la problemática típica donde el usuario final “recibe algo totalmente distinto a lo que él pensaba que había pedido”.
- **Cómo se hace (Entregable principal):** Se genera la **ERS (Especificación de Requerimientos de Software)**, documento formal que detalla el acuerdo funcional con el cliente. En entornos ágiles, se reemplaza por el refinamiento incremental de *user stories* anotadas en tarjetas.
- **Relación con otros conceptos:** Es el punto de partida de la trazabilidad. Todas las demás disciplinas técnicas deben rastrearse hacia atrás hasta validar qué requerimiento originó ese diseño, código o caso de prueba.

Disciplina 2: Análisis y Diseño (Analysis and Design)

- **Definición:** Transformación sistemática de los requerimientos funcionales abstractos de la ERS en planos arquitectónicos y lógicos detallados del sistema.
- **Qué problema resuelve:** Evita que los desarrolladores comiencen a escribir código sin una arquitectura de software pensada, lo cual provocaría colapsos de estabilidad lúdica colaterales, incoherencias estructurales e imposibilidad de mantener el software.

- **Cómo se hace (Entregables):** Se diseñan modelos estáticos (diagramas de clases, paquetes de subsistemas) y modelos dinámicos (diagramas de secuencia, máquinas de estado) que configuran la arquitectura lúdica del producto.

Disciplina 3: Construcción / Implementación (Construction)

- **Definición:** Conversión física de los diagramas y especificaciones de diseño lúdico en código ejecutable en un lenguaje de programación específico, acoplado con estructuras de bases de datos operacionales.
- **Qué problema resuelve:** Resuelve la dificultad accidental de representación. Traduce el plano lógico formal de la mente del diseñador al silicio y la CPU lúdica.
- **Cómo se hace (Entregables):** Programación modular, compilación de fuentes (.py,.java,.cpp), scripts de ensamblado e inicialización de la base de datos.

Disciplina 4: Prueba (Testing) [¡FOCO BAJO RESPONSABILIDAD DE ESTA MATERIA!]

- **Definición:** Disciplina de validación y verificación dinámica encaminada a ejecutar de forma planificada el código construido frente a entradas de control para descubrir la presencia de fallos latentes y bugs.
- **Qué problema resuelve:** Evita que el software se despliegue a producción con fallas críticas que pongan en riesgo el negocio del cliente.
- **Cómo se hace:** Se diseñan **casos de prueba (test cases)** que describen precondiciones, pasos lúdicos secuenciales de entrada, y el resultado esperado que se confrontará con el resultado real.
- **Nota del docente:** Esta materia se hace responsable directa del dictado y evaluación de la disciplina Prueba, siendo evaluada teórica y prácticamente en el segundo parcial.

Disciplina 5: Despliegue (Deployment) [¡FOCO BAJO RESPONSABILIDAD DE ESTA MATERIA!]

- **Definición:** Actividad técnica de transferir de forma segura, controlada e integrada el software compilado y verificado desde el entorno local de desarrollo hacia el entorno de producción final para su explotación real por parte del usuario final.
- **Qué problema resuelve:** Evita el clásico error de programador de: *“En mi computadora compila y funciona”*. Resuelve la complejidad de instalación, migración de datos y configuración del ambiente de servidor de producción.
- **Cómo se hace:** Generación de paquetes de instalación, instructivos de ensamble, dockerización y scripts de despliegue automatizado continuous delivery.

1.4.2. Disciplinas de Gestión (Relacionadas con el Proyecto)

Tienen como objetivo la administración de la variable organizacional, temporal y financiera del proyecto, planificando y controlando los desvíos.

PLANIFICACIÓN (PP)	-> Genera el Plan de Proyecto (Project Plan)
MONITOREO Y CONTROL	<- Compara real vs. planificado mediante métricas

Disciplina 1: Planificación de Proyecto (Project Planning - PP)

- **Definición:** Disciplina de gestión enfocada en predecir y calendarizar el alcance, los recursos requeridos, las tareas lógicas secuenciales, el personal y los costos de desarrollo.
- **Qué problema resuelve:** Evita la incertidumbre económica y temporal de la gerencia. Habilita saber de antemano el camino crítico y qué perfiles de programadores se necesitarán a lo largo del tiempo.
- **Cómo se hace (Entregables):** Generación formal del **Plan de Proyecto** (también adaptado en la industria como Plan de Desarrollo de Software), que incluye diagramas de Gantt (calendarización), planes de asignación de personal y cronogramas estimados de entrega.

Disciplina 2: Monitoreo y Control de Proyecto (Project Monitoring and Control - PMC)

- **Definición:** Actividad directiva continua que recolecta el estado real del avance técnico del proyecto para compararlo sistemáticamente con la línea base del Plan de Proyecto y tomar decisiones correctivas oportunas frente a los desvíos.
- **Qué problema resuelve:** Evita desvíos catastróficos que se detectan tarde. Habilita a la gerencia a recortar alcance, renegociar compromisos con el cliente o reasignar recursos antes de que se agote el presupuesto.
- **Cómo se hace:** Reuniones de estado periódicas, auditorías de cronograma y cálculo de desvíos mediante el análisis del esfuerzo remanente frente a la velocidad real.

1.4.3. Disciplinas de Soporte o transversales/protectoras (La Actividad Paraguas)

Son actividades transversales que se ejecutan desde el día cero hasta el final de la vida útil del software. Funcionan como una “actividad paraguas” o escudo protector que estabiliza el entorno para que las disciplinas técnicas y de gestión puedan operar.

Disciplina 1: Gestión de Configuración de Software (SCM)

- **Definición:** Disciplina de soporte para documentar e identificar unívocamente los ítems de configuración, controlar sus cambios evolutivos de estado, auditar la correspondencia con la ERS y mantener la integridad del producto a lo largo del ciclo de vida.
- **Qué problema resuelve:** Evita el caos absoluto en el repositorio (pérdida de cambios de archivos, sobreescrituras destructivas entre desarrolladores, regresiones de fallas lúdicas descontroladas).
- **Cómo se hace:** Creación del Plan de SCM (PGC),agueo de Líneas Base (checkpoints), creación de ramas de desarrollo (*branching*) y fusiones estables (*merges*).

Disciplina 2: Aseguramiento de Calidad (PPQA - Process and Product Quality Assurance)

- **Definición:** Disciplina transversal enfocada en definir estándares de proceso (cómo trabajar) y estándares de producto (cómo deben ser los entregables), y en verificar sistemáticamente su cumplimiento de manera preventiva a lo largo del proyecto.
- **Qué problema o confusión resuelve:**

- $\triangle$ *Explicación del docente contra la mezcla terminológica de la industria:* El docente subraya en los audios teóricos que las empresas confunden recurrentemente los roles de Aseguramiento de Calidad (*CUA Analysis / QA*) con los de Testing o Control de Calidad.
- *Resolución académica de Sommerville (BIBLIO_ - Cap. 24):* **QA (Aseguramiento de Calidad) es preventivo y de proceso** (establece estándares, inspecciones formales de Fagan y audita que se cumpla el proceso de desarrollo de calidad); mientras que el **Control de Calidad (QC/Testing) es correctivo y de producto** (aplica pruebas de ejecución técnica sobre el código entregado para encontrar fallas del sistema lúdico antes de la liberación).
- **Cómo se hace (Entregables):** Creación de plantillas de planes de calidad, rúbricas de revisión de código, auditorías de cumplimiento de proceso y reportes formales de PPQA.
- **Transcripción** **Dudosa** **corregida:**

[TRANSCRIPCIÓN DUDOSA: En el audio “2 SCM teórico.m4a” figura “la simple de inglés es esta PPQA”. Se corrige formalmente por: “la sigla en inglés es esta: PPQA (Process and Product Quality Assurance)”].

Disciplina 3: Métricas (Metrics / Medición y Análisis)

- **Definición:** Disciplina transversal que diseña y ejecuta la recolección cuantitativa de métricas objetivas de Proceso, Proyecto y Producto.
- **Qué problema resuelve:** Evita basar el control de gestión del software en sensaciones subjetivas (“creo que vamos bien”). Brinda visibilidad cuantitativa real al monitoreo del proyecto.
- **Cómo se divide el dominio de las Métricas:**
 1. *Métricas de Proceso:* Miden la eficiencia del proceso de trabajo lúdico (ej. densidad de defectos descubiertos en las inspecciones preventivas, tiempo de ciclo).
 2. *Métricas de Proyecto:* Miden variables de administración del proyecto (ej. horas consumidas reales vs estimadas, Story Points quemados de velocidad).
 3. *Métricas de Producto:* Miden atributos estáticos y de calidad del software como artefacto (ej. complejidad ciclomática del código, líneas de código totales).
- **Ubicación transversal de las Métricas en la materia:** El docente recalca que la disciplina de métricas es tan transversal que **no posee una unidad asignada en el programa**, sino que está embebida conceptualmente: se habla de métricas de proyecto en la unidad 2 (estimaciones), métricas de SCM en la unidad 3, y métricas de proceso/producto en la unidad 4 (calidad).

1.4.4. Relación e interdependencia de las disciplinas en el ciclo de desarrollo

Existe una interdependencia absoluta en cascada y transversal en el ecosistema.

```
PPQA (Calidad)
(Audita el cumplimiento)
PLANIFICACIÓN (PP) <--> SCM (Control Repositorio) <--> MONITOREO Y CONTROL
(Contiene todos los artefactos)
```


- **Interdependencia de Soporte y Técnicas:** Las disciplinas de soporte estabilizan el terreno. El software construido en la fase técnica fluye al repositorio ordenado por SCM para ser rastreado.
- **El ejemplo real del dormitorio desordenado dictado por el docente (AUDIO_01):**

“Para que yo pueda aplicar actividades de Aseguramiento de Calidad (PPQA), o hacer testing formal, necesito obligatoriamente que antes funcione de base una buena gestión de configuración de software (SCM). Es como si quisiera limpiar el dormitorio. Sí, pero no puedo pisar porque están las medias, la gemela, los pantalones, la ropa sucia mezclada con la ropa limpia en la cama. ¿Cómo hago para limpiar? Es imposible de esa manera. Primero tengo que ordenar las prendas, guardarlas en el ropero en su estante asignado, y recién ahí voy a poder limpiar la habitación de forma efectiva. SCM hace exactamente eso en el software: ordena el repositorio contenedor para que luego podamos auditar, probar, verificar y garantizar calidad”.

- **Tabla de Comparación entre Categorías de Disciplinas:**

Eje de Comparación	Disciplinas Técnicas	Disciplinas de Gestión	Disciplinas de Soporte (Transversales)
Foco estratégico principal	El Producto físico de software como artefacto entregable.	El Proyecto como unidad temporal, presupuestaria y organizacional.	El Ecosistema de resguardo y la consistencia sistémica transversal.
Mecanismo de acción	Secuencial o iterativo lúdico en fases del ciclo de desarrollo (PUD / RUP).	Temporal. Planificación JIT e hitos de control del sprint.	Continuo e ininterrumpido desde el inicio de la idea hasta la muerte del producto.
Entregables representativos	ERS (User Stories), arquitectura de software, código compilado, casos de prueba.	Plan de Proyecto (Project Plan), cronogramas Gantt, planes de capacidad de personal.	Repositorio estructurado, Líneas Base (Tags), Informes de SCM, métricas, auditorías de proceso.
Confusiones de examen comunes	Pensar que la construcción (escribir código) es la única disciplina importante del software.	Pensar que planificar el proyecto es un proceso estático que no cambia durante el desarrollo.	Confundir QA (prevención/proceso) con Testing (detección de fallas/producto).

Puntos clave para el examen — 1.4.4

1. **DIFERENCIA ENTRE QA Y TESTING:** Es una de las “trampas mortales” del examen oral final. Bajo ningún punto de vista se debe convalidar en el examen que “hacer QA es correr pruebas de software”. QA es preventivo, audita procesos y define estándares. El testeo o prueba de software es destructivo, de producto, y busca defectos sobre el código construido.
2. **LA DEFINICIÓN SINTÉTICA DE SCM DE MELES:** Meles resalta en sus audios teóricos que, si no se recuerda la definición de SCM de la IEEE de 5 líneas, al menos se debe ser capaz de enunciar de forma precisa la definición corta:

“SCM es una disciplina de soporte que intenta administrar los ítems de configuración para mantener la integridad del software a lo largo de toda su vida útil”.

3. **TRASCIENDE A SUS AUTORES:** El docente hace hincapié en que los ingenieros profesionales no programan para sí mismos, sino para que el software trascienda en el tiempo. Un repositorio versionado con trazabilidad permite que si un programador se va “a criar caracoles a un cerro” o es atropellado por un camión, un nuevo integrante pueda comprender de dónde vino cada regla de negocio lúdica.
4. **MÉTRICAS UNIVERSALES:** Las métricas no pertenecen a una unidad única de estudio porque, por su naturaleza sistémica, se mide el Proyecto (horas, desvíos), el Proceso (eficiencia en la detección de bugs) y el Producto (complejidad conceptual).

Preguntas de autoevaluación — 1.4.4

Pregunta 1: ¿Cómo define David Parnas sintéticamente a la Ingeniería de Software?

- **Respuesta breve:** Parnas la define formalmente como la construcción por múltiples personas de software multi-versión (*multi-person construction of multi-version software*).

Pregunta 2: ¿Qué diferencia existe en el enfoque de Sommerville entre “Disciplina de ingeniería” y “Todos los aspectos de la producción”?

- **Respuesta breve:** Disciplina de ingeniería implica aplicar selectivamente teorías, métodos y herramientas lógicas dentro de restricciones financieras y organizacionales; todos los aspectos de la producción engloba tanto las tareas puramente técnicas como la administración de proyectos, calidad y mejora del proceso.

Pregunta 3: ¿Qué es el SWEBOK y por qué es importante para la carrera de Sistemas en la UTN-FRC?

- **Respuesta breve:** Es el Cuerpo de Conocimientos de la Ingeniería de Software oficial de la IEEE. Es importante porque sus 15 áreas de conocimiento técnico funcionan como marco para el diseño curricular del plan de estudios de la carrera en la universidad.

Pregunta 4: ¿Cuál es la relación jerárquica existente entre SWEBOK y SEBoK?

- **Respuesta breve:** SEBoK abarca la Ingeniería de Sistemas sociotécnicos complejos globales (que integran hardware, software y personas). SEBoK contiene a la Ingeniería de Software (delimitada en el SWEBOK) como una de sus disciplinas técnicas y profesionales asociadas (Part 6).

Pregunta 5: ¿Cuáles son las disciplinas técnicas del desarrollo y cuáles de ellas caen bajo la órbita de examen de esta materia?

- **Respuesta breve:** Las disciplinas técnicas son: Requerimientos, Análisis y Diseño, Construcción, Prueba y Despliegue. En esta materia los docentes se responsabilizan directamente de las disciplinas de **Prueba (Testing)** y **Despliegue (Deployment)**.

Pregunta 6: ¿Cuáles son las disciplinas de gestión del software y a qué ecosistema se aplican?

- **Respuesta breve:** Son la Planificación del Proyecto y el Monitoreo y Control del Proyecto. Se aplican de manera restrictiva al ecosistema y ciclo de vida del **Proyecto**.

Pregunta 7: ¿Por qué se denomina “transversales o protectoras” a las disciplinas de soporte?

- **Respuesta breve:** Se las denomina transversales o protectoras porque no se ejecutan en un momento acotado del tiempo lúdico, sino que brindan cobertura continua y ordenada desde el inicio hasta el fin del proyecto para resguardar la consistencia conceptual y la integridad de los entregables.

Pregunta 8: ¿Cuál es la diferencia académica de fondo entre QA (Aseguramiento de Calidad) y Testing (Pruebas de software)?

- **Respuesta breve:** QA es preventivo y está enfocado en evaluar e inspeccionar el cumplimiento de los **procesos** y estándares de trabajo; mientras que el Testing es correctivo, dinámico y busca fallas sobre el **producto** o código construido ejecutable.

Pregunta 9: Explique la analogía del “dormitorio desordenado” explicada por el docente respecto a SCM.

- **Respuesta breve:** Explica que no se puede aplicar calidad (QA/Testing) en un ambiente caótico. Al igual que no se puede limpiar un dormitorio lleno de prendas sucias tiradas por todos lados antes de guardarlas y ordenarlas en el placard; no se puede auditar ni verificar software si antes SCM no resguarda un repositorio de ICs estructurado y versionado estable.

Pregunta 10: ¿Por qué la disciplina de Métricas no está circunscripta a una unidad del programa lúdico de la materia?

- **Respuesta breve:** Porque las métricas son transversales. Se aplican métricas de proyecto en la planificación, métricas de proceso en el aseguramiento de calidad de las metodologías de trabajo, y métricas de producto para medir la calidad estructural del software compilado.

1.5. El Ecosistema del Software en contexto: Las 5 Dimensiones Críticas

La disciplina de la Ingeniería de Software no opera en el vacío ni se reduce únicamente a la escritura lineal de código de programación. Académicamente, la cátedra define que la disciplina se desenvuelve dentro de un **ecosistema dinámico e interactivo estructurado en cinco dimensiones críticas**:

1. **Proceso (Process)**
2. **Proyecto (Project)**
3. **Producto (Product)**
4. **Personas / Personal (People / Personnel)**
5. **Herramientas (Tools)**

Este ecosistema sistémico representa un patrón de interdependencias que determina el éxito o fracaso de cualquier desarrollo tecnológico profesional. Si una de estas aristas se descuida o se rompe, la integridad del producto y la predictibilidad del proyecto colapsan de manera inmediata.

1.5.1. Análisis Detallado de las Cinco Dimensiones

```
[ EL PENTÁGONO DEL ECOSISTEMA ]
PROCESO
/ \
(Se adapta) (Automatizado con)
/ \
PROYECTO HERRAMIENTAS
/ \
(Se incorpora) (Obtiene como resultado)
/ \
PERSONAS PRODUCTO
```

Dimensión 1: Proceso (Process)

- **Definiciones Conceptuales y Formales:**
- **Definición General de la Cátedra (SLIDE_ / NC_):** El proceso es la **definición teórica, conceptual y más abstracta de lo que se debería hacer** para construir software de calidad. Funciona como una guía metodológica descriptiva que desglosa el trabajo en etapas, flujos de trabajo, disciplinas, actividades y tareas lógicas.
- **Definición Formal de la IEEE (SLIDE_01 / Passage 2):**

“Proceso: La secuencia de pasos ejecutados para un propósito dado”.

- **Definición Formal del Sw-CMM / CMMI (SLIDE_01 / Passage 3):**

“Proceso de Software: Un conjunto de actividades, métodos, prácticas, y transformaciones que la gente usa para desarrollar o mantener software y sus productos asociados”.

- **Definición Formal de Ian Sommerville (BIBLIO_ - Cap. 2.1):**

“Un proceso de software es una secuencia de actividades que conducen a la elaboración de un producto de software”.

- **Qué Problema Resuelve:** Evita el enfoque caótico, indisciplinado e individual del desarrollo artesanal. Al proveer un marco metodológico estandarizado y repetible, resuelve la falta de predictibilidad, la inestabilidad en la calidad de los entregables y la imposibilidad de diagnosticar desvíos o aplicar planes de mejora organizacional continua.
- **Cómo se Hace (Procedimiento y Estructura):** Un proceso de software formal se modela y describe explícitamente detallando los siguientes componentes lógicos indispensables:
 1. **Actividades (Activities):** Grandes bloques de trabajo técnico, administrativo o de soporte (ej. Análisis de Requerimientos, Diseño Arquitectónico, Pruebas Unitarias).
 2. **Roles:** Definición abstracta de las responsabilidades asociadas a las personas lúdicas que intervienen en el proceso (ej. Administrador de Proyecto, Ingeniero de Requerimientos, Diseñador de Interfaces, Programador, Administrador de Configuración). Se definen en base a roles para evitar que las actividades queden rígidamente acopladas a nombres propios de personas físicas que podrían rotar o abandonar la organización.

3. **Artefactos (Artifacts / Productos de Trabajo):** Las entradas y salidas tangibles lúdicas de cada actividad (ej. documento de visión, modelo UML, código ejecutable, reporte de defectos).
4. **Precondiciones y Postcondiciones:** Declaraciones que deben ser obligatoriamente válidas antes y después de ejecutar una actividad. Por ejemplo:
 - **Precondición para el Diseño Arquitectónico:** Requerimientos funcionales de la línea base aprobados por el Product Owner (PO).
 - **Postcondición:** Diagrama de subsistemas y componentes revisado y guardado en el repositorio.
 - **Clasificación Completa de Procesos:**
 - **Procesos Definidos (Predictivos):** Basados en la suposición lúdica de que el desarrollo es una línea de producción estable que se puede planificar y repetir de manera idéntica. Se adaptan de manera formal y estricta antes del inicio del proyecto y exigen un cumplimiento rígido de las fases secuenciales (ej. Modelo en Cascada, PUD / RUP tradicional).
 - **Procesos Empíricos (Adaptativos):** Diseñados para lidiar con variables altamente cambiantes e incertidumbre tecnológica intrínseca. No pretenden prescribir de antemano cada paso, sino que operan en ciclos de vida muy cortos basados en tres pilares: **Inspección, Adaptación y Transparencia** (ej. Framework de trabajo Scrum).
 - **Ejemplo Concreto de la Cátedra:** El **Proceso Unificado de Desarrollo (PUD)** en su versión teórica abstracta, el cual prescribe que para construir software se deben ejecutar de manera iterativa e incremental las disciplinas técnicas de Requerimientos, Análisis, Diseño, Implementación, Prueba y Despliegue.
 - **Errores Comunes y Advertencias:**
 - **⚠ El Pecado de la Burocracia:** Creer que “más proceso es siempre mejor proceso”. Intentar definir procesos excesivamente documentados y rígidos para equipos pequeños destruye su productividad lúdica y genera rechazo en los desarrolladores.
 - **⚠ Confusión Académica:** Confundir un proceso teórico (ej. PUD) con un framework de gestión ágil (ej. Scrum) o con un ciclo de vida representativo (ej. Cascada).
 - **Relación con otros Conceptos del Apunte:** El proceso teórico e impersonal es adaptado por el líder y el equipo para dar origen al **Plan de Proyecto** que calendarizará el trabajo del **Proyecto** real.

Dimensión 2: Proyecto (Project)

- **Definición de la Cátedra (Passage 14 / 82):** El proyecto es la **unidad organizativa, temporal y de gestión de recursos, tiempos y personas** que se diseña y ejecuta de manera planificada para obtener como resultado final un producto de software o servicio único.
- **Qué Problema Resuelve:** Resuelve la restricción fáctica de recursos de la realidad económica corporativa. Sin un proyecto estructurado, el desarrollo de software sería infinito, sin control de costos, sin plazos lógicos de entrega, y sin mecanismos formales para gestionar los riesgos de desvíos, la capacidad humana disponible y los compromisos de entrega contractuales.

- **Cómo se Hace (Procedimiento y Técnica):** Se gestiona mediante la elaboración, monitoreo y adaptación de un **Plan de Proyecto (Project Plan)** que debe documentar rigurosamente los siguientes factores:
 1. **Alcance (Scope):** Delimitación precisa de todo el trabajo lúdico y las tareas técnicas y de gestión indispensables para cumplir el objetivo acordado.
 2. **Cronograma (Schedule):** Planificación temporal que vincula tareas secuenciales o paralelas en el tiempo, identificando el “camino crítico” y fijando hitos de control de avance lúdico.
 3. **Presupuesto (Budget):** Recursos financieros asignados y estimados de costo de personal, infraestructura y herramientas.
 4. **Matriz de Asignación de Roles:** Mapeo unívoco con nombre y apellido que asocia a cada integrante físico del equipo con uno o más roles contemplados conceptualmente en el proceso.
- **Clasificación de los Ciclos de Vida del Proyecto:**
- **Ciclo de Vida Secuencial (Cascada):** El proyecto avanza en fases rígidas de un único paso; no se obtienen versiones operacionales parciales utilizables por el cliente hasta el cierre final del proyecto.
- **Ciclo de Vida Iterativo e Incremental (PUD / Agile):** El proyecto se subdivide en bloques de tiempo más pequeños de duración fija (sprints o iteraciones) donde en cada incremento se planifica, construye, prueba y se obtiene como resultado una versión operable estable del producto.
- **Ciclo de Vida en Espiral:** Enfocado en el análisis recursivo y mitigación sistemática de riesgos técnicos y de negocio a lo largo de giros incrementales.
- **Ejemplo Concreto de la Cátedra:** El proyecto de desarrollo simulado denominado “**Matrix**”. Este proyecto tiene una fecha de inicio (marzo de 2026), una fecha de finalización rígida (noviembre de 2026) y un equipo de cinco integrantes de la materia asignados formalmente mediante el PGC [3 Practico SCM.m4a].
- **Errores Comunes y Advertencias:**
- **⚠ La Confusión Proyecto-Producto:** Es el “error original” más recurrente en el diseño de IDEs y entornos de desarrollo (ej. cuando Visual Studio te pide crear un “New Project” en lugar de un “New Product”). El proyecto es temporal (nace, se ejecuta y muere de manera planificada). El producto de software tiene una vida útil inmensamente más larga y trasciende por completo a los proyectos secuenciales que lo crean, modifican o evolucionan a lo largo de los años.
- **⚠ Sobreestimación Defensiva (Padding):** El error común de los estimadores que inflan de manera artificial el tamaño técnico de las tareas para tener “colchones de tiempo” en el proyecto, destruyendo la transparencia y la fidelidad de las métricas lúdicas de gestión.
- **Relación con otros Conceptos del Apunte:** El proyecto es el “medio o vehículo de gestión” que toma el proceso abstracto y lo ejecuta materialmente utilizando la capacidad de las **Personas** para construir de forma incremental el **Producto** de software estable.

Dimensión 3: Producto (Product)

- **Definición de la Cátedra (Passage 5, 84, 86):** El producto es el **software profesional final entregado al cliente, concebido como conocimiento representado en distintos niveles de abstracción junto con toda su documentación asociada y datos de configuración.**
- **Qué Problema Resuelve:** Constituye el resultado fáctico de valor (outcome) que el cliente necesita para operar su negocio. Resuelve la necesidad funcional real que disparó la inversión financiera inicial del proyecto.
- **Cómo se Hace (Técnica):** El producto se construye pieza por pieza (incrementos lúdicos) a través de las actividades técnicas del proceso (Requerimientos -> Diseño -> Código -> Testing). Cada una de las salidas físicas o digitales generadas en estas etapas (ej. la ERS, el modelo arquitectónico, el código ejecutable, el script de base de datos) constituyen un **Ítem de Configuración de Software (IC)**. Estos ICs deben ser registrados de manera unívoca con nombre único y versión en un repositorio para resguardar su integridad evolutiva.
- **Clasificación de los Tipos de Producto (Sommerville Cap. 1.1):**
- **Productos Genéricos (Embalados / Off-the-shelf):** Desarrollados de forma abierta para el mercado de consumo general (ej. sistemas operativos, suites ofimáticas como Microsoft Word). La especificación de requerimientos de estos productos la maneja y prioriza internamente la empresa desarrolladora [Sommerville Cap. 1.1].
- **Productos a Medida (Customizados / Bespoke):** Desarrollados específicamente bajo un contrato y acuerdo para un cliente particular que tiene el control del alcance y los requerimientos funcionales (ej. el sistema de autogestión académica de la UTN-FRC).
- **Ejemplo Concreto de la Cátedra:** El producto de software meteorológico bautizado “Clima”. Sus ítems de configuración de producto se identifican bajo reglas de nombrado claras en el PGC, por ejemplo: `ERS_Clima.docx` (la Especificación de Requerimientos de Software), `Arquitectura_Clima.pdf` (el diseño arquitectónico) y los módulos de código fuente correspondientes.
- **Errores Comunes y Advertencias:**
- **⚠ Miopía de Código:** Pensar que el producto es “solamente el código ejecutable”. Omitir la documentación de diseño o los casos de prueba dentro del control de configuración del producto destruye la mantenibilidad y la trazabilidad del sistema lúdico a largo plazo.
- **⚠ Falta de Versionado de Documentos:** Guardar archivos de documentación del producto con nombres asimétricos y caóticos como `ERS_Clima_v2_final_estaSI.docx` en carpetas locales en vez de versionarlos de manera unívoca con herramientas de SCM, perdiendo el control de cambios de la configuración.
- **Relación con otros Conceptos del Apunte:** El producto es el contenedor macro de todos los **Ítems de Configuración de Software (ICs)** y **Líneas Base (Baselines)** que la disciplina de soporte **SCM** debe proteger de forma continua.

Dimensión 4: Personas / Personal (People / Personnel)

- **Definición de la Cátedra (Passage 1, 21):** Es el **factor crítico de éxito número uno, el motor dinámico y el corazón del ecosistema de software**. Sommerville y los docentes enfatizan teóricamente que el desarrollo de software es un **esfuerzo intelectual y**

creativo puramente humano, lo que implica que la calidad de los procesos y productos resultantes depende de forma directa del intelecto, la competencia técnica, la motivación, la ética profesional y la cohesión de los miembros de los equipos de trabajo.

- **Qué Problema Resuelve:** Los procesos teóricos, los planes de proyecto y las herramientas de desarrollo son entes inertes e ineficaces por sí mismos. Las personas resuelven la complejidad lúdica de conceptualización de la esencia (el qué construir) mediante el análisis, la comunicación interpersonal asertiva con el cliente, el diseño innovador y la resolución pragmática de las limitaciones accidentales.
- **Cómo se Hace (Procedimiento de Asignación):** Se seleccionan las personas idóneas para el proyecto según su experiencia técnica y de negocio. Al instanciarse el proyecto, se les asignan **roles formales** definidos de manera abstracta en el proceso para evitar acoplamiento rígido de nombres propios. En metodologías ágiles, se potencia la autoorganización y los equipos de desarrollo multifuncionales (cross-functional).
- **Clasificación de Perfiles y Conductas:**
- **Perfiles Técnicos de Software (Especialistas):** Analistas, Arquitectos de Software, Desarrolladores (full-stack, backend, frontend), Analistas de Calidad / QA, Testers de Software, Administradores de Configuración (SCL - Software Configuration Librarian).
- **Perfiles de Negocio (Stakeholders):** Clientes finales, Usuarios operativos directos, Product Owners (PO).
- **Tipos de Motivación Conductual de Sommerville (BIBLIO_ - Cap. 22):**
 1. **Personas orientadas a tareas:** Motivadas por el desafío técnico intelectual y el trabajo analítico de resolución de problemas complejos.
 2. **Personas orientadas a la interacción:** Motivadas por la comunicación social, el compañerismo y el trabajo colaborativo en equipo.
 3. **Personas autoorientadas:** Motivadas por el crecimiento profesional individual, el reconocimiento personal y el éxito económico propio.
- **Ejemplo Concreto de la Cátedra:** El caso citado en el aula teórica donde el programador “**Salva**” es una persona física real del equipo. En el plan del proyecto **Matrix**, se le asigna el rol abstracto de **Arquitecto de Software**, mientras que en otro proyecto paralelo de la organización podría asignársele el rol de desarrollador de código, garantizando la flexibilidad operativa.
- **Errores Comunes y Advertencias:**
- **⚠ El Error de la Linealidad de Recursos:** Creer que los programadores son piezas mecánicas intercambiables u “horas-hombre” de rendimiento idéntico. Sustituir un desarrollador senior experto por dos desarrolladores juniors sin capacitación en medio de un proyecto no duplica la productividad, sino que la reduce por la inmensa carga de inducción de personal.
- **⚠ Aplicar la Ley de Brooks en Demoras:** Intentar resolver desvíos de cronograma en el proyecto agregando más desarrolladores de forma tardía, lo que incrementa el desvío temporal por las necesidades de comunicación y acoplamiento técnico lateral de los nuevos integrantes [The Mythical Man-Month].

- **Relación con otros Conceptos del Apunte:** Las personas son quienes interpretan los requerimientos del cliente, configuran y utilizan las **Herramientas** de desarrollo, ejecutan los **Procesos** y construyen materialmente el **Producto** bajo las restricciones del **Proyecto**.

Dimensión 5: Herramientas (Tools)

- **Definición de la Cátedra (Passage 20):** Es el conjunto de **utilitarios, entornos integrados de desarrollo (IDEs), versionadores, sistemas compiladores, herramientas de gestión administrativa y motores de automatización** que se utilizan para mecanizar, dar soporte físico y optimizar la productividad de las actividades operativas descritas en el proceso de software.
- **Qué Problema Resuelve:** Automatiza y reduce las dificultades de carácter puramente accidental (ej. verificación sintáctica automática, compilación repetitiva de binarios, rastreo del historial de código, almacenamiento físico ordenado de archivos). Esto libera el tiempo y la capacidad cognitiva del equipo humano, permitiéndoles enfocar su esfuerzo intelectual en conceptualizar de forma correcta la esencia del software [No Silver Bullet].
- **Cómo se Hace (Técnica y Selección):** Las herramientas de soporte y desarrollo se definen formalmente en las etapas iniciales de planificación (ej. en el Plan de Gestión de Configuración y Plan de Calidad). El equipo de ingeniería las instala, unifica versiones y automatiza flujos operativos mediante canalizaciones de integración continua (CI) y entrega continua (CD).
- **Clasificación Completa de Herramientas:**
- **Herramientas de Desarrollo Técnico (CASE - Computer-Aided Software Engineering):** Editores de código, IDEs (ej. Visual Studio Code, IntelliJ, PyCharm), compiladores de lenguajes, frameworks de desarrollo, depuradores de código y automatizadores de pruebas unitarias.
- **Herramientas de Soporte y SCM:** Sistemas de gestión de versiones (ej. Git, Subversion, CVS), plataformas de resguardo y control cooperativo (ej. GitHub, GitLab, Bitbucket).
- **Herramientas de Gestión de Proyectos:** Software para seguimiento de tareas, cronogramas y tableros ágiles (ej. Jira, Trello, tableros Kanban interactivos).
- **Herramientas de Comunicación y Soporte de Equipos:** Wikis, blogs internos, canales de comunicación colaborativa (ej. Slack, Discord, Microsoft Teams).
- **Ejemplo Concreto de la Cátedra:** La utilización obligatoria del versionador **Git** y la plataforma pública **GitHub** en el práctico de SCM (TP4) como las herramientas del repositorio destinadas a resguardar la estructura unificada de carpetas (**trunk, branches y documentos**) [3 Practico SCM.m4a].
- **Errores Comunes y Advertencias:**
- **⚠ La Fantasía de la Automatización Mágica:** Creer que instalar una herramienta sofisticada o un bot de Inteligencia Artificial (IA) solucionará mágicamente los desvíos conceptuales en los requerimientos o los problemas conductuales y éticos del equipo de trabajo. Una gran herramienta sobre un mal proceso genera “un desastre automatizado más rápido”.
- **⚠ Violación de Confidencialidad:** El docente advierte en los audios que el equipo técnico comete el error grave de “meter datos públicos y sensibles de la empresa

alegremente” en inteligencias artificiales externas de terceros, violando las políticas de confidencialidad y propiedad intelectual de la organización.

- **Relación con otros Conceptos del Apunte:** Las herramientas automatizan las actividades descritas formalmente en los **Procesos**, facilitando a las **Personas** la construcción integrada del **Producto** digital dentro del marco del **Proyecto**.

1.5.2. Análisis Comparativo de las Cinco Dimensiones

Dimensión Crítica	Foco de Gestión	Naturaleza Conceptual	Entregables / Artefactos Típicos	Métrica Asociada
PROCESO	¿Cómo trabajar? (El método y las reglas).	Abstracta: Definición de directrices, flujos y metodologías teóricas organizacionales.	Modelos de proceso, rúbricas de QA, flujos de actividades, manuales de proceso.	Eficiencia del proceso, cobertura de auditorías, niveles de madurez CMMI.
PROYECTO	¿Cuándo, con qué y con quién entregar? (La gestión de restricciones).	Temporal: Nace, se planifica, se ejecuta y muere con fecha definida de cierre lúdico.	Plan de Proyecto (PP), cronograma de Gantt, planes de recursos, planes de iteración.	Desvíos presupuestarios, avance del cronograma, desvío de esfuerzo en horas.
PRODUCTO	¿Qué valor entregar? (El resultado operacional para el cliente).	Evolutiva: Trasciende la vida de los proyectos secuenciales; acompaña la vida útil del cliente.	Código fuente, base de datos, ERS, manual de usuario, casos de prueba.	Densidad de defectos, complejidad ciclomática del código, mantenibilidad.
PERSONAS	¿Quiénes conceptualizan y ejecutan? (El motor del éxito).	Intelectual: El factor humano y conductual crítico; gobernado por variables cognitivas y éticas.	Matriz de asignación de roles, convenios de equipo, actas de compromiso de negocio.	Clima laboral, índice de rotación de personal (turnover), nivel de seniority lúdico.
HERRAMIENTAS	¿Cómo optimizar y automatizar? (La eficiencia instrumental).	Tecnológica: Soporte instrumental para agilizar accidentes operativos y de	Scripts de compilación automática (CI), topologías de	Cobertura de pruebas automáticas, velocidad de compilación,

Dimensión Crítica	Foco de Gestión	Naturaleza Conceptual	Entregables / Artefactos Típicos	Métrica Asociada
		versionado.	repositorio, entornos IDE instalados.	estabilidad del entorno.

1.5.3. Interacción Sistémica del Ecosistema de Software

Las cinco dimensiones no actúan de manera aislada; el desarrollo de software profesional es el resultado de un acoplamiento sistémico dinámico. Las slides de la cátedra e Ian Sommerville (BIBLIO_ - Cap. 26.1) grafican esta interacción de la siguiente manera:

1. **El Proceso se Adapta al Proyecto:** Las directrices teóricas abstractas de la organización (el proceso de software global) son recortadas, adaptadas y personalizadas por el Project Manager y el equipo de ingeniería para ajustarse a las restricciones reales de tiempo, presupuesto y tipología del proyecto fáctico.
2. **El Personal (Personas) se Incorpora al Proyecto:** El proyecto asigna recursos humanos a su cronograma. Las personas físicas reales (ej. Salva, Franco) son incorporadas al proyecto asumiendo temporalmente los roles teóricos (ej. Arquitecto, Administrador de Configuración) especificados formalmente en el proceso adaptado.
3. **El Proceso se Automatiza con las Herramientas:** El equipo selecciona e implementa herramientas automatizadas (ej. compiladores, Git, scripts de construcción automática) que “mecanizan” y dan soporte físico a las actividades, disminuyendo las dificultades accidentales y acelerando los tiempos de ejecución del proceso lúdico.
4. **El Proyecto Obtiene como Resultado el Producto:** Las personas, trabajando de manera coordinada dentro de las restricciones temporales del proyecto, ejecutando las actividades del proceso y asistidos por las herramientas tecnológicas, traducen paulatinamente los requerimientos de negocio en los entregables digitales estructurados (código, base de datos, especificaciones) que dan forma definitiva al Producto de software de calidad.

SISTEMA DE EQUILIBRIO DE SOMMERVILLE (Capítulo 26) El éxito global de un software de calidad es un balance integrado. Si tenés un PROCESO de excelencia pero el PROYECTO no cuenta con presupuesto lúdico realista, la calidad del PRODUCTO se degradará. Si el equipo de PERSONAS es de bajo seniority, un buen PROCESO puede “limitar el daño”, pero no creará software de alta calidad por sí solo; requiriendo HERRAMIENTAS potentes de soporte para mitigar riesgos.

Puntos clave para el examen — 1.5.3

1. **DIFERENCIA ASOCIADA EN EL REPOSITORIO (¡PREGUNTA DE PARCIAL!):** Los profesores de práctico hacen mucho hincapié en que en el Trabajo Práctico de SCM (TP4) los alumnos deben segmentar rigurosamente los ítems de configuración de acuerdo a su ciclo de vida. Es un error grave de examen mezclar los ítems de producto (vida larga que

trasciende el proyecto, ej. código, casos de uso) con ítems de proyecto (vida acotada al proyecto, ej. planes, minutas, roadmap) en la misma estructura física de repositorio.

2. **POR QUÉ EL PROCESO USA ROLES:** Se pregunta sistemáticamente por qué el proceso teórico no menciona personas con nombre propio. La respuesta académica es que el proceso se define en términos de roles para dar flexibilidad organizacional; las personas asumen roles de manera temporal y dinámica proyecto a proyecto, evitando que el proceso dependa de nombres de ingenieros que hoy están y mañana se van de la empresa.
3. **HERRAMIENTA VS. PROCESO:** El docente de teoría resalta de manera incisiva en los audios que las herramientas automatizan los procesos, pero **no los reemplazan**. Un programador “que hace commit y push de forma descontrolada sin respetar las directrices de cambios del PGC” está usando la herramienta Git pero violando el proceso de SCM, destruyendo la integridad del producto.
4. **MÉTRICAS QUE MIDEN CADA ÁMBITO:** Se exige saber clasificar el dominio de las métricas lúdicas de software. Si se mide la complejidad ciclomática del código, es una métrica de **Producto**; si se miden las horas consumidas reales vs. estimadas, es una métrica de **Proyecto**; si se mide la eficiencia del proceso de detección de fallos en auditorías de calidad, es una métrica de **Proceso**.

Preguntas de autoevaluación — 1.5.3

Pregunta 1: ¿Cuáles son las cinco dimensiones del ecosistema en el que se desenvuelve la Ingeniería de Software?

- **Respuesta breve:** Las cinco dimensiones son: Proceso, Proyecto, Producto, Personas y Herramientas.

Pregunta 2: ¿Qué es un Proceso de Software según la CMMI y en qué términos se definen sus participantes?

- **Respuesta breve:** Es un conjunto de actividades, métodos, prácticas y transformaciones que las personas usan para desarrollar o mantener software. Sus participantes se definen estrictamente en términos de **roles abstractos** para garantizar la flexibilidad de la organización frente a la rotación de personal.

Pregunta 3: ¿Cuál es la diferencia de ciclo de vida e implicancia temporal entre un Proyecto y un Producto de software?

- **Respuesta breve:** El proyecto es estrictamente temporal y tiene una fecha definida de inicio y finalización planificada; el producto de software tiene un ciclo de vida mucho más largo, está activo mientras se use y trasciende de manera evolutiva a los diversos proyectos que lo modifican o actualizan.

Pregunta 4: ¿Cómo se define el Producto de software de manera profesional superando la visión reduccionista del código?

- **Respuesta breve:** Se define como el conocimiento representado en múltiples niveles de abstracción. Comprende de forma unificada el código fuente ejecutable, la base de datos operacional, los datos de configuración, la ERS, la documentación arquitectónica del sistema y los manuales de usuario.

Pregunta 5: ¿Por qué se afirma teóricamente que las Personas son el factor crítico de éxito número uno en el software?

- **Respuesta breve:** Porque el desarrollo de software es un esfuerzo intelectual y creativo puramente humano. Sin personas capacitadas, motivadas y cohesionadas, los procesos y herramientas resultan inertes e incapaces de conceptualizar la esencia lógica compleja de los requerimientos de negocio del cliente.

Pregunta 6: ¿Cuál es la función estratégica primordial de las Herramientas en el ecosistema?

- **Respuesta breve:** Su función es automatizar las tareas operativas repetitivas del proceso (agilizar las dificultades accidentales) para liberar la capacidad cognitiva y el tiempo de los ingenieros humanos, permitiéndoles abocarse al diseño conceptual de la esencia de la solución.

Pregunta 7: ¿Cómo interactúan el Proceso y el Proyecto en el inicio de un desarrollo real de software?

- **Respuesta breve:** El proceso teórico general de la organización es analizado y **adaptado** por el líder del proyecto para ajustarse a las restricciones temporales, financieras y de alcance específicas del proyecto fáctico lúdico.

Pregunta 8: ¿Por qué no se deben mezclar ítems de Producto e ítems de Proyecto en la estructura física del repositorio de SCM?

- **Respuesta breve:** Porque responden a ciclos de vida de longitudes drásticamente asimétricas. Los ítems de proyecto (como las minutas o planes) mueren al cerrar el proyecto lúdico, mientras que los ítems de producto (código, arquitectura, casos de prueba) deben preservarse y versionarse de por vida para dar soporte evolutivo al cliente en producción.

Pregunta 9: Según el análisis de Sommerville (Cap. 26.1), ¿qué ocurre en un proyecto con un proceso excelente pero presupuesto y cronograma poco realistas?

- **Respuesta breve:** La calidad del producto de software resultante se verá afectada negativamente de manera inevitable (provocando funcionalidad reducida o fallos lúdicos en producción), a menos que un personal de excelencia técnica excepcional logre salvar el desarrollo en base a un esfuerzo humano desmedido.

Pregunta 10: En el contexto de las Métricas, proporcione un ejemplo de métrica de Proceso, uno de Proyecto y uno de Producto de software.

- **Respuesta breve:** Una métrica de Proceso es la densidad de defectos descubiertos preventivamente en revisiones de código; una de Proyecto son las horas de esfuerzo reales consumidas frente a las estimadas en el plan; y una de Producto es la complejidad ciclomática de las funciones programadas.

UNIDAD 2 - SCM (Gestión de Configuración del Software)

2.1. El software en contexto y la necesidad de la disciplina SCM

2.1.1. Definición formal de SCM (ANSI/IEEE 828, 1990)

Definiciones de SCM en la materia

- **Definición formal ANSI/IEEE 828 (1990)** (SLIDE_02):
“La Gestión de Configuración de Software (SCM) es una disciplina que aplica dirección y monitoreo administrativo y técnico a: identificar y documentar las características funcionales y físicas de los ítems de configuración, controlar los cambios de esas características, registrar y reportar los cambios y su estado de implementación y verificar la correspondencia con los requerimientos”.
- **Definición sintética y académica de Judith Meles** (AUDIO_01, NC_Mansilla):
“Es una disciplina de soporte (transversal) que intenta administrar los ítems de configuración (ICs) para mantener la integridad del software a lo largo de todo su ciclo de vida”.
- **Definición de Sommerville (BIBLIO_ - Cap. 25.1):**
“La administración de la configuración se ocupa de la gestión de un sistema de software en evolución. Implica la planeación de la configuración, la gestión de versiones, la construcción del sistema y la administración del cambio”.

El concepto central de Edward Bersoff: “Integridad del Producto” (Product Integrity)

Bersoff postula que el fin último de la disciplina SCM no es “restringir el trabajo de los programadores” ni “escribir documentación burocrática”, sino **establecer y mantener la integridad del producto de software** a lo largo de su ciclo de vida.

Sistémicamente, un producto posee **integridad** cuando cumple de forma simultánea con las siguientes cinco condiciones de satisfacción (congruencia técnica y de gestión):

INTEGRIDAD DEL PRODUCTO

Satisface las Es trazable/ Cumple con Cumple con las Cumple con las
necesidades del rastreable en criterios expectativas de expectativas
usuario su ciclo de perf. costo de entrega

- **Qué problema resuelve SCM:** SCM erradica el caos en el repositorio de desarrollo lúdico. Evita desastres típicos de gestión como la pérdida de código fuente, la inconsistencia entre lo que se programó y lo que se entregó al cliente, el retrabajo por sobreescrituras accidentales

entre programadores, y el desgaste operativo de “perder tiempo buscando la versión correcta de un archivo”.

- **Cómo se hace (Las 4 Actividades Fundamentales de SCM) (ANSI/IEEE 828):**
 1. **Identificación de Ítems de Configuración (SCI/IC):** Determinar qué elementos del producto y del proyecto se colocarán bajo control, asignándoles reglas de nombrado unívocas y versiones.
 2. **Control de Cambios:** Flujo de procesos mediante el cual se proponen, evalúan, aprueban e implementan modificaciones sobre las líneas base ya conformadas.
 3. **Informes de Estado (Status Accounting):** Registro administrativo histórico e inalterable que detalla qué se cambió, cuándo, quién lo hizo, qué línea base afectó y el estado del cambio.
 4. **Auditorías de Configuración (Física y Funcional):** Evaluaciones formales para asegurar que el producto construido coincide de manera exacta con la documentación y los requerimientos aprobados.

Comparativa de SCM según el Proceso

Variable de Comparación	SCM en Procesos Tradicionales / Definidos (PUD)	SCM en Procesos Ágiles (Scrum, Lean)
Foco Estratégico	Control preventivo, rigidez de cambios, documentación formal estricta (Petición de Cambio, CRF).	Facilitar y agilizar el cambio (“sin fricción”), sirviendo al desarrollador y no al revés.
Rol del CCB	Órgano formal de toma de decisiones estratégicas de alta burocracia.	Embebido en el Product Owner (PO) y el equipo; las decisiones de cambio se priorizan dinámicamente en el backlog de sprints.
Auditorías	Hitos manuales rigurosos ejecutados por un equipo de QA externo.	Automatizadas continuamente mediante Testing e Integración Continua (CI).
Mecanismo de Control	Check-in / Check-out formal de archivos con bloqueos estrictos.	Repositorios distribuidos (Git) con ramificaciones (branching) y fusiones (merges) frecuentes.

- **Ejemplo de clase:** El docente de teoría explica que usar una herramienta de control de versiones como Git para hacer commit y push **no es hacer gestión de configuración**. La herramienta es un acelerador tecnológico del accidente; SCM es la disciplina académica de gestión lúdica que define qué taguear, cuándo congelar una línea base para que nadie la modifique sin permiso del CCB, y cómo estructurar el repositorio para resguardar la trazabilidad.
- **Advertencia de examen de Judith Meles:**

⚠️ *“El usar una herramienta de versionado no significa que ustedes estén respetando la

disciplina de SCM. No cometan el error típico de decir en el examen final: ‘¿Qué voy a estudiar gestión de configuración si yo uso Git todos los días en mi trabajo?’ La experiencia laboral no alcanza para rendir en esta materia si no manejan los conceptos académicos de los libros”.

- **Relación con otros conceptos del apunte:** SCM es la disciplina protectora de base indispensable. Sin SCM, es imposible aplicar actividades de aseguramiento de calidad (PPQA) o testing efectivo, porque no tendríamos certeza de qué versión de código estamos auditando o probando (analogía del “dormitorio desordenado”).

2.1.2. El software como “blanco móvil” (movable target) y la volatilidad del cambio

Definición precisa

El software se comporta en producción como un **“blanco móvil” (movable target)**. Debido a la propiedad esencial de **mutabilidad** de Brooks, el software carece de inercia física, lo que facilita y fomenta que sea modificado continuamente a lo largo de su vida útil por presiones del entorno operacional en el que está inserto.

Qué problema resuelve

Asumir la realidad del software como un blanco móvil destruye la utopía tradicional de ingeniería que pretendía “definir el 100% de los requerimientos al inicio del proyecto y congelarlos de forma inmutable hasta la entrega”. Comprender este concepto impulsa la adopción de arquitecturas modulares de alta cohesión y bajo acoplamiento (para mitigar el impacto de los cambios) y disciplinas de agilidad preparadas para “abrazar el cambio”.

Causas y Orígenes del Cambio (Clasificación de la Cátedra)

El material de clase de Meles y Covaro clasifica las fuentes de modificación del software en dos grandes grupos (Externos e Internos):

ORÍGENES DEL CAMBIO

[ORÍGENES EXTERNOS] [ORÍGENES INTERNOS]

- Cambios de negocio/procesos - Defectos encontrados a corregir
- Nuevas leyes y regulaciones - Deuda Técnica (technical debt)
- Reorganización de prioridades del cliente - Oportunidades de refactorización
- Cambios en el presupuesto disponible - Desactualización de software de base
- **Cómo se hace (Procedimiento técnico):** La volatilidad del cambio se gestiona estructurando el código mediante **Líneas Base (Baselines)** sucesivas. Todo cambio posterior sobre una línea base congelada requiere la apertura de una **Rama (Branch)** aislada para desarrollo y pruebas, evitando alterar el código estable en producción hasta que se complete la fusión de manera consistente.
- **Ejemplo de clase:** El profesor describe la llamada telefónica del cliente que pide un “cambio chiquito” o una “tontera” (ej. *“sacarle la obligatoriedad a un campo del formulario”*)

o “que el botoncito sea azul verdoso en vez de celeste”). El cliente asume que el cambio es gratis y rápido porque el software no se ve (invisibilidad). El programador cede de buena persona, pero al no realizar un análisis de impacto de arquitectura, el cambio desprotegido corrompe la consistencia de las consultas de base de datos colaterales y tira el sistema de facturación en producción.

- **Errores comunes / Advertencias:** El error más grave es el de **Capa 8 (nosotros)**: resistirse burocráticamente al cambio intentando bloquear el repositorio. En Ingeniería de Software, un producto que no recibe pedidos de cambio sistemáticos a lo largo del tiempo es, por definición, un software que nadie está usando en producción. El cambio debe ser gestionado y facilitado por SCM, no prohibido.
- **Relación con otros conceptos:** La volatilidad del cambio se conecta de forma directa con la **deuda técnica (technical debt)**: cuando se introducen parches apresurados para responder al blanco móvil sin rediseñar la arquitectura, se degrada la mantenibilidad del sistema, encareciendo exponencialmente los cambios futuros.

2.1.3. Las 7 problemáticas típicas en la gestión del software (según Bersoff)

Cuando un proyecto de desarrollo carece de un sistema formal de Gestión de Configuración (SCM), los componentes lógicos del repositorio sufren de manera inevitable siete problemáticas patológicas clásicas identificadas por la teoría de SCM de Bersoff.

A continuación se detalla cada una por separado:

Problemática 1: Pérdida de un componente

- **Explicación teórica:** Desaparición física o lógica de un entregable del proyecto (código fuente, base de datos, diagrama UML, plan de iteración) del ecosistema accesible del equipo de desarrollo, provocado por la ausencia de un repositorio centralizado, respaldado y con políticas claras de almacenamiento unificado.
- **Ejemplo concreto de la cátedra:** El caso planteado en el aula donde un programador guarda el código de integración de Mercado Pago de manera local en su computadora o en un pendrive. El desarrollador falta dos días por enfermedad, y el resto del equipo no puede avanzar con las pruebas porque no tiene acceso físico al archivo lúdico del código, viéndose forzados a reprogramarlo de cero.
- **Mecanismo SCM de Resolución:** El **Repositorio Único y Centralizado (digitalizado)** con estructura de carpetas estandarizada y unificada definida en el PGC (Trunk/Main, Branches, Docs).

Problemática 2: Pérdida de cambios (Sobreescritura accidental)

- **Explicación teórica:** Ocurre cuando dos o más ingenieros de software trabajan de forma concurrente y paralela sobre el mismo archivo de código fuente sin aislamiento de espacios de trabajo ni control de versiones, provocando que el último programador en guardar sus cambios destruya de manera silenciosa las modificaciones ingresadas previamente por sus compañeros.

Ejemplo de la industria:

Día 1: Archivo de login en versión 1.0 (Sin soporte de tarjetas de crédito).

Día 2, 09:00: Alicia descarga el login v1.0 localmente para programar el soporte de Visa.

Día 2, 10:00: Roberto descarga el mismo login v1.0 localmente para programar Mastercard.

Día 2, 14:00: Alicia termina, sube su código. Repositorio avanza a v1.1 (Login con Visa).

Día 2, 15:00: Roberto termina, sube su código encima de la v1.1.

Resultado catastrófico: Repositorio avanza a v1.2 (Login con Mastercard, pero soporte de Visa de Alicia desapareció por sobreescritura).

- **Mecanismo SCM de Resolución:** El **Control de Versiones** mediante el acoplamiento de un repositorio público y **espacios de trabajo privados (workspaces / playgrounds)** con operaciones estrictas de check-out/check-in (pull/push) y bloqueo de escritura o herramientas automáticas de resolución de conflictos (*diff/merge*).

Problemática 3: Sincronía fuente - objeto - ejecutable

- **Explicación teórica:** Inconsistencia y desvinculación física de versiones entre las líneas de código fuente lógicas que editan los desarrolladores (.java,.py,.cpp) y los correspondientes componentes objeto compilados (.class,.pyc,.o) o ejecutables empaquetados finales que se despliegan en los servidores de producción.
- **Ejemplo concreto de la cátedra (¡CASO REAL DE TESTING!):** El caso real en el que participó la cátedra haciendo testing para una empresa de software:

“Nos tocó hacer testing de un producto. Reportamos un defecto, fue al área de desarrollo, volvió arreglado, y cuando lo testeamos a través de un acceso andaba perfecto. Pero cuando el usuario entraba a la aplicación a través de la ruta del acceso directo de la pantalla principal, el error de validación seguía estando idéntico. ¿Qué había pasado? Que el servidor web tenía desplegadas dos rutas físicas distintas: si entrabas por un lado accedías al ejecutable nuevo compilado con el parche de código, pero si entrabas por el acceso directo la ruta apuntaba a un ejecutable viejo desactualizado de una compilación anterior”.

- **Mecanismo SCM de Resolución:** El **Build Automatizado de Integración de Sistemas** que vincula de manera rígida las dependencias utilizando firmas digitales unívocas (marcas de tiempo - *timestamps* o sumas de verificación - *checksums* de código) para obligar a la recompilación limpia de ejecutables ante cualquier cambio en el fuente.

Problemática 4: Regresión de fallas

- **Explicación teórica:** Reintroducción de errores lógicos o bugs que ya habían sido descubiertos, analizados y subsanados con éxito en iteraciones previas del proyecto. Sucede cuando un desarrollador trabaja sobre una base de código desactualizada que carece de los últimos parches integrados y la sube al repositorio central, eliminando las correcciones maduras.
- **Ejemplo práctico:** Se corrige el fallo de seguridad de la base de datos de usuarios en el Sprint 1 (v1.0). En el Sprint 3, un programador rescata un código viejo de su disco local del Sprint 1 para implementar una mejora en el perfil de usuario, y al subir su código “nuevo” sin el parche integrado del Sprint 1, el fallo de seguridad que ya estaba solucionado vuelve a aparecer en producción.

- **Mecanismo SCM de Resolución:** El etiquetado riguroso de **Líneas Base (Baselines / Tags)** estables en el historial de evolución del versionador, que sirven como puntos de referencia y recuperación inmutables ante fallas o regresiones.

Problemática 5: Doble mantenimiento (Trabajo duplicado)

- **Explicación teórica:** Escenario ineficiente de desperdicio operativo donde el equipo de desarrollo mantiene y edita de forma asincrónica el mismo módulo de software o fragmento de funcionalidad que se encuentra duplicado y guardado de manera redundante en dos o más proyectos o rutas físicas separadas en el servidor.
- **Ejemplo concreto de la cátedra:** Un equipo copia y pega la carpeta completa del módulo de “Generación de PDF” en el proyecto de “Facturación” y en el de “Reportes de Stock”. Cuando el fisco cambia las regulaciones impositivas y el diseño del PDF debe mutar, el equipo se ve obligado a reprogramar las mismas modificaciones de forma separada en ambos proyectos, duplicando el costo de desarrollo y multiplicando el riesgo de que una de las implementaciones quede desalineada.
- **Mecanismo SCM de Resolución:** La identificación de **Ítems de Configuración de Producto modularizados**, gestionados en bibliotecas de código compartidas bajo un esquema de dependencias automatizado en el repositorio público.

Problemática 6: Superposición de cambios (Shared updates)

- **Explicación teórica:** Interferencia destructiva sobre la estabilidad de la compilación de la rama principal (master o main) del repositorio, provocada por el intento de múltiples desarrolladores de subir de manera simultánea modificaciones lógicas pesadas de distintos requerimientos directamente sobre los mismos archivos, corrompiendo la consistencia de la construcción del sistema.
- **Ejemplo práctico:** El Programador A está modificando la lógica de conexión a la base de datos para migrar a PostgreSQL; al mismo tiempo, el Programador B está metiendo mano sobre las mismas clases de conexión para implementar un login de Google. Si ambos suben sus cambios a la par sin aislamiento, el sistema de base de datos se rompe por inconsistencia semántica y el sistema general deja de compilar para todo el equipo.
- **Mecanismo SCM de Resolución:** El uso ingenieril de **Ramas (Branches)** semánticas de aislamiento para el desarrollo independiente de funcionalidades o experimentos, las cuales se integran a la rama principal únicamente tras superar con éxito las pruebas en el espacio privado mediante un proceso de **Fusión (Merge)** controlado.

Problemática 7: Cambios no validados

- **Explicación teórica:** Modificación arbitraria de las características funcionales o técnicas de un componente que ya se encontraba bajo el cofre congelado de una Línea Base, realizada sin pasar por un análisis de impacto de arquitectura, sin actualizar los casos de prueba de testing ni la documentación técnica asociada.
- **Ejemplo práctico:** El cliente llama por teléfono al desarrollador para pedirle una “tontera” de último minuto en los horarios de una base de datos. El programador mete mano de manera informal directo en la base de datos de la línea base operativa. Como no se validó, el cambio rompe la lógica de todas las consultas históricas de horarios, tirando la aplicación móvil del EcoParque.

- **Mecanismo SCM de Resolución:** El proceso formal de **Control de Cambios** coordinado por el Comité de Control de Cambios (CCB), sustentado en el documento de propuesta (ECP) y el análisis de impacto preventivo de costos y arquitectura.

Tabla Comparativa de las 7 Problemáticas de Bersoff

# *	<i>Problemática en SCM**</i>	<i>Causa de Base (Qué la genera)**</i>	<i>Impacto Crítico en el Proyecto**</i>	<i>*Mecanismo SCM que la Soluciona**</i>
1	Pérdida de un componente	Falta de un contenedor físico unificado; archivos en pendrives o discos locales.	Imposibilidad de construir el producto; desorganización total.	Repositorio Único Estructurado (Trunk, Branches, Docs) en el PGC.
2	Pérdida de cambios	Modificación concurrente sin aislamiento de espacios de trabajo privados.	Destrucción de código de compañeros; retrabajo crónico.	Control de Versiones con espacios de trabajo (<i>workspaces</i>) y diff/merge.
3	Sincronía fuente-objeto	Compilación desvinculada de firmas; archivos objeto con el mismo nombre.	Despliegue en producción de código viejo o inconsistente con el fuente.	Build Automatizado de Integración con firmas de <i>checksum</i> o <i>timestamp</i> .
4	Regresión de fallas	Trabajar sobre ramificaciones desactualizadas del repositorio.	Reaparición de bugs viejos que ya se habían corregido y testeado.	Líneas Base (Baselines / Tags) del repositorio con control estricto.
5	Doble mantenimiento	Duplicación física redundante de módulos lógicos idénticos.	Desperdicio de horas de desarrollo; riesgo de inconsistencia lógica.	Ítems de Configuración de Producto modularizados y reutilizables.
6	Superposición de cambios	Subir parches pesados directo a la línea principal a la vez sin aislamiento.	Rotura sistemática de la compilación lúdica de la rama master para todo el equipo.	Branching (Ramas de desarrollo) aisladas por funcionalidad con merges controlados.
7	Cambios no validados	Modificación informal de código base por “tonteras” de pedidas al programador.	Ruptura colateral de la arquitectura lúdica de calidad todo el CC sistema.	Control de Cambios formal gestionado por el B (ECP y análisis de impacto).

Puntos clave para el examen — 2.1.3

Durante las clases, las docentes hicieron especial hincapié en estos ejes temáticos, los cuales constituyen preguntas de examen clásicas de la UTN-FRC:

1. **DIFERENCIA ENTRE GESTIÓN DE CONFIGURACIÓN Y VERSIONADO:** Es la pregunta preferida de Judith Meles para bochar alumnos en el examen oral. No se puede responder que SCM es “usar Git”. El versionado es una función técnica automática de la herramienta (el accidente); la Gestión de Configuración (SCM) es la disciplina administrativa y de ingeniería (la esencia) que define las reglas, roles, políticas de congelamiento de líneas base y auditorías que el equipo debe cumplir obligatoriamente para garantizar la integridad.
2. **LA CONFORMACIÓN DEL COMITÉ DE CONTROL DE CAMBIOS (CCB):** Las docentes aclaran que los miembros del CCB no son “marcianos externos a la empresa”. El CCB está compuesto por representantes del **mismo equipo del proyecto** (analista de requerimientos para cuidar la ERS, líder técnico para cuidar la arquitectura, gestor de configuración para operar el repositorio, testing para validar las pruebas) y, si el cambio tiene envergadura estratégica, se debe incorporar obligatoriamente al cliente o PO para que asuma el impacto del costo y los tiempos.
3. **EL CONCEPTO DE INTEGRIDAD DE BERSOFF:** Saber enumerar de forma precisa los 5 atributos que Bersoff define para caracterizar un producto de software con integridad: satisfacción del usuario, trazabilidad completa, cumplimiento de performance, cumplimiento de presupuesto/costos, y plazos de entrega.
4. **CUIDADO CON LAS REGLAS DE NOMBRADO EN EL TP4:** En el Trabajo Práctico de SCM, la cátedra exige que los Ítems de Configuración del repositorio **no tengan el número de versión escrito en su nombre físico** (ej. está mal llamar a un archivo `codigo_v1.py` o `ers_v3.docx`), dado que el versionador Git ya maneja el control de versiones de forma automática internamente. Romper esta regla en la entrega del TP4 es causa directa de reprobación del práctico.

Preguntas de autoevaluación — 2.1.3

Pregunta 1: ¿Cómo define formalmente la ANSI/IEEE 828 (1990) a la disciplina SCM?

- **Respuesta breve:** Es una disciplina de ingeniería que aplica dirección y monitoreo administrativo y técnico a identificar e identificar unívocamente las características de los ítems de configuración, controlar sus cambios evolutivos, registrar e informar de su estado de implementación, y verificar la correspondencia exacta con los requerimientos lógicos aprobados.

Pregunta 2: ¿Cuál es la definición académica abreviada de SCM que exige Judith Meles en los exámenes?

- **Respuesta breve:** SCM es una disciplina de soporte (transversal) que intenta administrar los ítems de configuración (ICs) para mantener la integridad del software a lo largo de todo su ciclo de vida útil.

Pregunta 3: Según Bersoff, ¿qué es la “Integridad del Producto” y qué condiciones de satisfacción lúdicas de base debe cumplir el software?

- **Respuesta breve:** Es el conjunto intrínseco de atributos lúdicos que caracterizan a un producto de software de calidad. Debe satisfacer las necesidades funcionales del usuario, ser fácil y completamente rastreable en su ciclo de vida, cumplir con criterios de performance y requerimientos no funcionales (disponibilidad/seguridad), y alinearse con las expectativas de costo y entrega.

Pregunta 4: ¿Por qué se afirma académicamente que el software es un “blanco móvil” (movable target)?

- **Respuesta breve:** Debido a su mutabilidad inherente y la maleabilidad de su naturaleza digital intangible, el software está sujeto a una presión continua de cambio para adaptarse a la evolución de las leyes impositivas, los procesos de negocio y las necesidades volátiles del cliente.

Pregunta 5: ¿Cuáles son las fuentes típicas de cambio que justifican la necesidad de SCM en un proyecto?

- **Respuesta breve:** Los cambios del negocio y nuevos requerimientos del cliente, actualizaciones de plataformas de hardware o software de base asociadas, corrección de bugs lúdicos, oportunidades de refactorización de código, deuda técnica (technical debt) acumulada y modificaciones de presupuesto del proyecto.

Pregunta 6: ¿En qué consiste la problemática de “Doble Mantenimiento” descrita por Bersoff y cómo se soluciona?

- **Respuesta breve:** Consiste en tener el mismo código o funcionalidad guardado y duplicado de manera redundante en múltiples proyectos o carpetas físicas distintas. Se soluciona abstrayendo la funcionalidad en un **Ítem de Configuración de Producto modularizado y compartido** en el repositorio bajo un esquema único de dependencias.

Pregunta 7: Explique la problemática de “Sincronía fuente-objeto” y describa el caso real de testing de la cátedra que la ilustra.

- **Respuesta breve:** Es la inconsistencia física entre los archivos lógicos de código fuente y los ejecutables finales desplegados. Se ilustra con el caso donde un error de validación corregido persistía si el usuario ingresaba a la app por el acceso directo del escritorio porque ese acceso apuntaba a una ruta con un binario compilado viejo, mientras que por otra ruta accedía al ejecutable nuevo con el parche.

Pregunta 8: ¿Cuál es la diferencia de fondo entre la “versión” de un IC y una “variante” de un IC según la cátedra?

- **Respuesta breve:** La **versión** representa la evolución cronológica y secuencial de un mismo ítem (ej. para corregir bugs o agregar requerimientos); mientras que la **variante** representa una línea paralela del componente diseñada para coexistir en el mismo momento de tiempo para adaptarse a distintos entornos o sistemas operativos (ej. variante para iOS y variante para Android).

Pregunta 9: ¿Por qué se enseña en la UTN-FRC que tener una herramienta como Git no equivale a hacer Gestión de Configuración de Software?

- **Respuesta breve:** Porque Git es solo un versionador de archivos (el instrumento tecnológico). Hacer SCM implica establecer un marco administrativo de ingeniería de calidad: definir el plan formal de gestión de configuración (PGC), establecer comités de cambios (CCB), definir reglas de nombrado unívocas y consensuar los criterios y eventos estables para congelar y marcar Líneas Base.

Pregunta 10: ¿Por qué es obligatorio que figuren los nombres y apellidos de las personas reales en el Plan de Gestión de Configuración (PGC)?

- **Respuesta breve:** Porque si en el plan solo se colocan los roles abstractos (ej. “el analista”, “el gestor”), operativamente nadie se hace responsable técnico de la integridad del repositorio. Colocar nombre y apellido garantiza la rendición de cuentas (*accountability*) y que los procesos se ejecuten realmente en la práctica lúdica.

2.2. Conceptos clave para la Gestión de Configuración de Software

Antes de desglosar cada concepto por separado, es fundamental que comprendas cómo encastran de forma sistémica estos cuatro pilares de SCM. Para el examen, las docentes exigen diferenciar de forma tajante las definiciones de **Ítem de Configuración**, **Repositorio**, **Línea Base** y **Configuración de Software**.

Tabla General de Contraste Conceptual de SCM

Concepto de Control	¿Qué representa en la realidad del proyecto?	Foco o Unidad de Medida	¿Quién tiene autoridad para modificarlo?	¿Cómo se representa físicamente?
Ítem de Configuración (IC / SCI) [Sommerville p Cap. 25.2] d	Cualquier artefacto del producto o del proyecto digitalizado y C colocado bajo control [Sommerville Cap. 25.2].	Individual. Un único elemento con su propia historia de cambios [Sommerville no ap. 25.2].	El desarrollador o autor asignado al archivo (mientras sea Línea Base). es di	Un archivo con nombre unívoco y extensión específica en el scc [Sommerville Cap. 25.2].
Repositorio	El contenedor físico y lógico ordenado de todos los ICs con su historial evolutivo.	Macro-Estructura. El placard contenedor del proyecto.	Administrado por el Gestor de Configuración; acceso de lectura/escritura controlado.	Estructura de carpetas en un servidor (local, centralizado o distribuido).
Línea Base (Baseline) [Sommerville a Cap. 25.2] s	Un cofre cerrado y probado que sirve como v checkpoint de referencia estable [Sommerville Cap. 25.2].	Consensuado. Un subconjunto estable de Cs en sus versiones d alidadas [Sommerville tr Cap. 25.2]. f	Únicamente el Comité de Control e Cambios (CCB) (as un proceso ap ormal. c	Una etiqueta inalterable Tag) licada a un ommit específico.
Configuración de Software	Una instantánea o	Estado General. El tado sistémico	Cambia automáticamente	El estado actual l sistema de

Concepto de Control	¿Qué representa en la realidad del proyecto?	Foco o Unidad de Medida	¿Quién tiene autoridad para modificarlo?	¿Cómo se representa físicamente?
	“foto” de es todos los ICs del repositorio en un instante preciso de tiempo.	au instantáneo de todas las versiones.	con de cada commit o modificación de un desarrollador.	versionado en un timestamp dado.

2.2.1. Ítem de Configuración de Software (SCI / IC - Software Configuration Item)

2.2.1.1. Definición y diferencia conceptual con Software

- **Definición de las Slides de la Cátedra (SLIDE_o2):**

“Se llama ítem de configuración (IC) a todos y cada uno de los artefactos que forman parte del producto o del proyecto, que pueden sufrir cambios o necesitan ser compartidos entre los miembros del equipo y sobre los cuales necesitamos conocer su estado y evolución”.

- **Definición de Ian Sommerville (BIBLIO_ - Cap. 25.2):**

“Cualquier aspecto asociado con un proyecto de software (diseño, código, datos de prueba, documento, etcétera) se coloca bajo control de configuración. Por lo general, existen diferentes versiones de un ítem de configuración. Los ítems de configuración tienen un nombre único” [Sommerville Cap. 25.2].

- **Qué problema resuelve:** Resuelve la informalidad y la pérdida de trazabilidad. Al declarar un artefacto como IC, el equipo se obliga a identificarlo unívocamente, registrar cada cambio en su historial de evolución y evitar que se pierda o sobrescriba.
- **La técnica de digitalización (Diferencia conceptual entre Software e IC):**
 - **El Software es intangible y abstracto:** Puede ser una idea en el pizarrón, un bosquejo de interfaz dibujado en un papel durante una reunión o un diagrama de arquitectura pegado en una pared. Todo eso es software, pero la disciplina SCM **no puede gestionarlo en ese estado físico analógico.**
 - **La digitalización como puente hacia el IC:** Para que una pieza de software califique formalmente como Ítem de Configuración, **debe ser digitalizada** (convertida en un archivo computable) y guardada físicamente en el repositorio.
 - **El procedimiento:**
 1. Se genera o captura el artefacto (ej. se le saca una foto de alta resolución al diagrama del pizarrón).
 2. Se le asigna un formato digital estandarizado (ej..png o .pdf).
 3. Se le aplica la regla de nombrado unívoca definida en el PGC.
 4. Se lo almacena en su carpeta correspondiente dentro del repositorio.

2.2.1.2. Clasificaciones y tipos de ICs

La cátedra en sus clases prácticas (3 Practico SCM.m4a) exige clasificar de forma exhaustiva cada IC en tres tipos mutuamente excluyentes según su ámbito y ciclo de vida:

[CLASIFICACIÓN DE ICs]
[IC DE PRODUCTO] [IC DE PROYECTO] [IC DE ITERACIÓN]

- Viven más que el proyecto - Viven lo que el proyecto - Mueren en la iteración
- ERS, Código, Manuales - Plan, PGC, Minutas, RMI - Reporte Bugs, Plan de Iteración
- 1. **Ítems de Configuración de Producto:** Son aquellos elementos de software lúdicos que forman parte del entregable final de la aplicación y **viven más allá de la finalización del proyecto** (en etapa de mantenimiento y operación de la empresa).
 - *Ejemplos:* Documento de requerimientos (ERS), Casos de Uso, arquitectura del sistema, código fuente (.py,.java), scripts de bases de datos (.sql), manuales de usuario, instructivos de despliegue.
- 2. **Ítems de Configuración de Proyecto:** Son aquellos artefactos administrativos que sirven de soporte para la gestión, ejecución y control de las variables del proyecto. **Nacen con el proyecto y mueren con el cierre del mismo.**
 - *Ejemplos:* Plan de Desarrollo del Proyecto (PP), Plan de Gestión de Configuración (PGC), planes de aseguramiento de calidad (Plan de PPQA), reportes mensuales de estado, minutas de reunión con el cliente.
- 3. **Ítems de Configuración de Iteración:** Elementos tácticos de cortísimo plazo que asisten al desarrollo de un incremento particular. **Mueren o pierden relevancia formal al cerrarse la iteración correspondiente.**
 - *Ejemplos:* Plan de la iteración actual (ej. pi_sprint1.docx), reportes semanales de testing de la iteración, listado transitorio de defectos encontrados en el sprint.
- **Ejemplo de clase:** El docente de práctica describe el caso de un examen final corregido o de un material de cátedra. Si descargamos la diapositiva de la UV de SCM, le agregamos nuestras notas personales y la guardamos en la carpeta de nuestro grupo, ese archivo es un **IC de producción propia** (un entregable de proyecto para estudiar), el cual debe llamarse, por ejemplo: VTS_apunte_SCM_estudiante_go2.pdf.
- **Errores comunes y advertencias:**
 - △ *El error del versionado manual:* No incluyas el número de versión de forma física y estática dentro del nombre del archivo (ej. llamar a un archivo ERS_v3_final.docx).
 - *La regla de examen:* En el TP4, si se utiliza un versionador como Git, el versionado lo gestiona la herramienta por detrás de manera transparente. El nombre del archivo debe permanecer limpio e idéntico a lo largo del tiempo (ej. ERS_clima.docx). El número de versión física en el nombre solo es aceptable si se exporta el archivo fuera del repositorio para entregárselo a alguien sin acceso.

2.2.2. Repositorio (Repository)

- **Definición de las Slides de la Cátedra (SLIDE_o2):**

“Un repositorio de información que contiene los ítems de configuración (ICs) con su

versión. Mantiene la historia de cada IC con sus atributos y relaciones, y es usado para hacer evaluaciones de impacto de los cambios propuestos”.

- **Definición de Sommerville (BIBLIO_ - Cap. 6.3.2):**

“Todos los datos en un sistema se gestionan en un repositorio central, accesible a todos los componentes del sistema. Los componentes no interactúan directamente, sino tan sólo a través del repositorio” [Sommerville Cap. 6.3].

- **Qué problema resuelve:** Elimina el desorden físico de los entregables y la asincronía. Evita el dolor de cabeza típico del estudiante: “La matriz de trazabilidad la tiene mi compañero en su pendrive y hoy faltó”. Concentra todos los activos del proyecto en un espacio seguro que permite evaluar el impacto cruzado de cambiar un archivo (por ejemplo, ver qué clases de código se romperán si modifico la base de datos).

2.2.2.1. Clasificaciones y Tipos de Repositorios

A. Repositorio Centralizado (Client-Server Version Control)

- **Definición:** Estructura de versionado basada en un único servidor central que almacena la base de datos de versiones completa [Sommerville Cap. 25.2]. Los desarrolladores extraen (*check-out*) una copia de trabajo local del archivo actual y, tras editarla, la devuelven (*check-in / commit*) directamente al servidor central [Sommerville Cap. 25.2].
- **Ejemplos en la industria:** Subversion (SVN), CVS [Sommerville Cap. 25.2].
- **Ventajas:** Los administradores poseen un control absoluto de accesos y permisos jerárquicos sobre el repositorio de forma centralizada.
- **Desventajas:** El servidor único representa un “punto único de falla” (*single point of failure*) [Sommerville Cap. 6.3]. Si el servidor se cae, el equipo no puede versionar ni integrar cambios; y si el disco del servidor se corrompe físicamente sin respaldos externos, se pierde todo el historial del proyecto.

B. Repositorio Descentralizado / Distribuido (Distributed Version Control System)

- **Definición:** Estructura donde cada desarrollador clona de manera exacta la base de datos del repositorio completo localmente en su propia máquina [Sommerville Cap. 25.2]. El historial evolutivo de cambios se almacena de forma local, y la sincronización con el servidor de la nube se realiza mediante operaciones de envío (*push*) y recepción (*pull*) de metadatos completos.
- **Ejemplos en la industria:** Git (GitHub, GitLab, Bitbucket).
- **Ventajas:** Si el servidor en la nube falla, la recuperación es tan sencilla como copiar y pegar el repositorio completo de cualquiera de las computadoras de los desarrolladores. Permite el desarrollo 100% offline y habilita flujos de trabajo avanzados sin fricción como las solicitudes de fusión (*Pull Requests / Merge Requests*).
- **Desventajas:** Requiere mayor capacitación del equipo de desarrollo para asimilar la complejidad del flujo de metadatos locales y remotos y el manejo de conflictos.

2.2.2.2. Estructura de carpetas unificada en SCM (Trunk/Main, Branches, Docs)

El docente de práctico resalta que una de las decisiones más estratégicas del Plan de Gestión de Configuración (PGC) es definir la estructura física del repositorio. Un repositorio desordenado es una “bolsa de gatos” que anula los beneficios del versionador.

La cátedra exige estructurar el repositorio en tres ramas o directorios principales de primer nivel:

/mi-proyecto-bts/

trunk/ # Contiene la línea principal de desarrollo (el código activo y compilable) branches/ # Aloja las bifurcaciones y ramas de experimentación o hotfixes de soporte documentos/ # Carpeta para ICs administrativos de proyecto (Minutas, Planes, PGC, ERS)

- **Cómo se hace (La técnica del orden):** Esta estructura debe ser plasmada físicamente en el repositorio público Git desde el primer día del proyecto (TP4) utilizando archivos marcadores de posición (.gitkeep) si las carpetas están temporalmente vacías. Esto evita que los integrantes creen carpetas arbitrarias a mitad de cuatrimestre, manteniendo la consistencia de accesos de testing y QA.
- **Ejemplo de clase (La analogía del ropero de Meles):** Meles asocia el repositorio al placard del dormitorio. La estructura de carpetas unificada es como definir que en el cajón superior van las medias y en el estante del medio los pantalones. Buscar un archivo en una estructura desordenada destruye la eficiencia de ingeniería y la predictibilidad del proyecto.

2.2.3. Línea Base (Baseline)

- **Definición de Ian Sommerville (BIBLIO_ - Cap. 25.2):**

“Una línea base es una colección de versiones de componente que construyen un sistema. Las líneas base están controladas, lo que significa que las versiones de los componentes que conforman el sistema no pueden ser cambiadas. Por lo tanto, siempre debería ser posible recrear una línea base a partir de los componentes que lo constituyen” [Sommerville Cap. 25.2].

- **Definición de las Slides de la Cátedra (SLIDE_02):**

“Una configuración que ha sido revisada formalmente y sobre la que se ha llegado a un acuerdo. Sirve como base para desarrollos posteriores y puede cambiarse sólo a través de un procedimiento formal de control de cambios. Permiten ir atrás en el tiempo y reproducir el entorno de desarrollo en un momento dado del proyecto”.

- **Qué problema resuelve:** Detiene el “daño colateral” de los cambios informales. Al congelar un conjunto de ICs bajo una línea base, se garantiza que todo el equipo trabaja sobre el mismo suelo firme y estable. Si el cliente o el equipo deciden de común acuerdo descartar un experimento técnico fallido, la línea base es el punto de restauración seguro al cual regresar el sistema.

2.2.3.1. Tipos de Líneas Base y sus Criterios de Creación

Las líneas base se dividen según la naturaleza del contenido que “congelan”:

1. **Líneas Base de Especificación:** Congelan requerimientos de negocio y planos de diseño arquitectónico aprobados por el cliente.

- *Ejemplo:* Línea base de requisitos (tras la firma formal de la ERS), línea base de diseño (tras la validación técnica del documento de arquitectura por revisores de QA).
2. **Líneas Base Operacionales / de Producto:** Congelan código ejecutable, bases de datos y entornos configurados que superaron las pruebas de testing de la iteración.
- *Ejemplo:* Línea base de codificación (antes de mandar el build a testing), línea base de entrega de producción (*release*).

[PLAN DE PROYECTO] -> [REQUERIMIENTOS (ERS)] -> [DISEÑO DE ARQUITECTURA] -> [CÓDIGO FUENTE]

LÍNEA BASE DE LÍNEA BASE DE LÍNEA BASE DE LÍNEA BASE DE
PLANIFICACIÓN REQUISITOS DISEÑO CODIFICACIÓN

Criterios y eventos para conformar una Línea Base

El PGC debe detallar los hitos o eventos exactos en los cuales se creará una línea base. Estos eventos **no se planifican con fecha y hora del calendario**, sino que responden al cumplimiento de un nivel de madurez técnica del proceso de software:

- *Hito de Requisitos:* “La ERS es formalmente aceptada y aprobada por el cliente”.
- *Hito de Arquitectura:* “El documento de arquitectura supera con éxito las revisiones técnicas de los líderes de desarrollo”.
- *Hito de Testing:* “Se congela el código para pruebas de humo; el compilador certifica que el código no posee errores sintácticos de compilación”.

2.2.3.2. Cómo se representa en Git: El etiquetado de Líneas Base (Tags)

- **La técnica del etiquetado:** En los versionadores distribuidos modernos, una línea base se conforma aplicando una **etiqueta (Tag)** inalterable sobre un commit de la rama principal (main/master). El tag asocia un nombre semántico estandarizado y una firma inmutable a ese momento preciso de la historia del repositorio.
- **Ejemplo concreto de plantilla de comandos:**

Paso 1: Nos situamos en la rama de producción estable (main)

git checkout main

Paso 2: Nos traemos la última actualización del servidor central

git pull origin main

Paso 3: Aplicamos un Tag anotado para congelar la Línea Base de la ERS de Requerimientos

git tag -a LB_ERS_V1.0 -m “Línea Base formal de Requerimientos ERS firmada por el cliente”

Paso 4: Subimos la etiqueta del repositorio local al servidor central de GitHub

git push origin LB_ERS_V1.0

2.2.4. Ramas (Branch) y Fusión (Merge)

- **Definición de Ramificación (Branching) (Sommerville Cap. 25.2):**

“La creación de una nueva línea de código a partir de una versión en una línea de código existente. La nueva línea de código y la existente pueden desarrollarse de manera independiente” [Sommerville Cap. 25.2].

- **Definición de Combinación (Merging) (Sommerville Cap. 25.2):**

“La creación de una nueva versión de un componente de software al combinar versiones separadas en diferentes líneas de código. Dichas líneas de código pueden crearse mediante una rama anterior de una de las líneas de código implicadas” [Sommerville Cap. 25.2].

- **Qué problema resuelve:** Resuelve la superposición de cambios y la ruptura del repositorio principal. Permite que múltiples programadores experimenten de forma aislada y segura en sus espacios de trabajo locales sin alterar la compilación limpia del código estable en la rama master.

2.2.4.1. Concepto de Branching: Ramificaciones semánticas

La cátedra destaca que la creación de ramas no debe ser arbitraria. Cada rama debe tener una **razón semántica** de creación explícita:

- *Ramas de funcionalidad (Feature Branches):* Para el desarrollo aislado de una historia de usuario del backlog.
- *Ramas de soporte (Hotfix Branches):* Para corregir errores críticos detectados de urgencia en producción.
- *Ramas de experimentación (Spikes):* Para crear prototipos tecnológicos de prueba que luego se integrarán o descartarán en su totalidad.

2.2.4.2. El Merge y la Resolución de Conflictos

Cuando el desarrollo del programador en la rama aislada finaliza y es aprobado por revisores pares mediante un Pull Request (PR), se procede a integrar la rama secundaria con la principal (main/master).

- **El surgimiento de Conflictos:** Si otro desarrollador modificó los mismos archivos en las mismas líneas lógicas y subió sus cambios a master antes que nosotros, el motor automático de Git se detiene al no poder decidir de forma inteligente qué cambio priorizar. Esto genera un **conflicto de merge**.
- **Cómo se resuelve técnicamente (Procedimiento de control):**
 1. Se descarga la última versión de la rama principal a nuestro repositorio local.
 2. Se ejecuta un merge manual en nuestro entorno de desarrollo privado.
 3. Se inspeccionan los bloques en conflicto indicados por las marcas del versionador:

<<<<<<< HEAD

Lógica modificada en master por Roberto para Mercado Pago

def calcular_impuesto(monto):

```
return monto * 0.21
```

```
=====
```

Lógica modificada en nuestra rama feature por Alicia para IVA exento

```
def calcular_impuesto(monto, exento=False):
```

```
    if exento:
```

```
        return 0
```

```
    return monto * 0.21
```

```
| feature_iva_exento
```

4. Se reúne el equipo técnico implicado para evaluar el impacto, decidir de forma unificada la lógica definitiva, limpiar las marcas del versionador (<<<<<<, =====, >>>>>>), probar localmente la compilación y realizar el `commit` definitivo para cerrar el conflicto de merge.

2.2.5. Configuración de Software

- **Definición de las Slides de la Cátedra (SLIDE_02):**

“Un conjunto de ítems de configuración con su correspondiente versión en un momento determinado” [Sommerville Cap. 25.2].

- **El concepto del “instante de tiempo” (La foto del repositorio):**
 - La configuración del software es una **entidad sumamente dinámica e instantánea**. Es una instantánea o instantánea matemática de los activos de software.
 - Si un programador realiza un cambio minúsculo sobre una sola clase o un documento del proyecto y le da commit, la configuración del software cambia inmediatamente y pasa a ser otra diferente.
 - Las líneas base (baselines), en contraste, permanecen estables e inmutables. Son conjuntos de configuraciones consensuadas que se “congelan” y no varían ante los cambios de desarrollo diarios.

Evolución temporal de la Configuración de Software

```
[Enero: Configuración A] -> [Cambio de código de login]
(Meles le saca una foto y la congela)
|| LÍNEA BASE DE REQUISITOS (Tag) || <- Inalterable y de referencia estable
```

Puntos clave para el examen — 2.2.5

1. **DIFERENCIA ENTRE VERSIONADO DE ARCHIVOS Y SCM:** En el examen oral de Meles, bajo ningún punto de vista respondas que SCM “es subir cosas a GitHub”. SCM es la **disciplina académica de ingeniería** (la esencia) que define las reglas, el PGC, quién conforma el CCB, cómo se resguarda la integridad de Bersoff, y cuándo se decide crear una línea base formal. Las herramientas son el acelerador técnico instrumental (el accidente) de estas actividades.

2. **REGLAS DE REPOSITORIO EN EL EXAMEN PRÁCTICO:** Cuando entregues el TP4 del repositorio Git, los profesores prácticos validarán de forma estricta que:
 - **Ningún archivo físico de Git contenga el número de versión escrito en su nombre** (ej. desaprobado directo si subís un archivo llamado ERS_v1.docx). El nombre del archivo se mantiene inalterable, ya que Git gestiona la evolución de la versión por detrás automáticamente.
 - La estructura de carpetas unificada (trunk/, branches/, documentos/) esté **completamente creada desde el día uno** en Git. Está mal ir creando las carpetas incrementalmente sobre la marcha.
3. **EL COMITÉ DE CONTROL DE CAMBIOS NO ES BUROCRACIA INÚTIL:** El CCB no paraliza el proyecto. El CCB es un engranaje ágil integrado por el Product Owner, el líder técnico, testing y QA para realizar análisis de impacto preventivo de costos y arquitectura antes de corromper la estabilidad de una línea base con cambios informales.

Preguntas de autoevaluación — 2.2.5

Pregunta 1: ¿Cómo se define un Ítem de Configuración de Software (IC / SCI) y qué rol juega la digitalización?

- **Respuesta breve:** Es cualquier artefacto de producto o de proyecto que se coloca bajo control de configuración por su valor lúdico de trazabilidad [Sommerville Cap. 25.2]. La digitalización es el puente obligatorio que convierte al software abstracto en un archivo físico computable almacenable en el repositorio.

Pregunta 2: Nombre dos ejemplos de IC de Producto y dos de IC de Proyecto.

- **Respuesta breve:** De **Producto:** La Especificación de Requerimientos de Software (ERS) y las clases de código fuente (.py). De **Proyecto:** El Plan de Gestión de Configuración (PGC) y las minutas de reunión con el cliente.

Pregunta 3: ¿Qué diferencia conceptual existe entre un “repositorio centralizado” y uno “descentralizado / distribuido” en SCM?

- **Respuesta breve:** El centralizado aloja la base de datos de versiones en un único servidor donde los clientes extraen copias transitorias; el descentralizado replica la copia idéntica de la base de datos de versiones completa localmente en la máquina de cada desarrollador [Sommerville Cap. 25.2].

Pregunta 4: ¿Cuáles son las tres carpetas de primer nivel exigidas por la cátedra para la estructura del repositorio?

- **Respuesta breve:** Las carpetas son: trunk/ (línea principal de desarrollo), branches/ (ramas secundarias) y documentos/ (documentación y planes administrativos).

Pregunta 5: ¿Qué es una Línea Base (Baseline) según Sommerville?

- **Respuesta breve:** Es una colección de versiones de componentes que construyen un sistema, la cual está formalmente controlada y congelada, de modo que sus componentes no pueden ser alterados de forma directa y permite recrear el entorno de desarrollo estable en cualquier momento [Sommerville Cap. 25.2].

Pregunta 6: ¿Quién es la única entidad autorizada para modificar una Línea Base conformada en el proyecto?

- **Respuesta breve:** El Comité de Control de Cambios (CCB) es el único órgano directivo autorizado para evaluar, aprobar e implementar una modificación de línea base a través de un procedimiento formal.

Pregunta 7: ¿Cómo se representa físicamente una Línea Base en un versionador como Git?

- **Respuesta breve:** Se representa aplicando un **Tag (etiqueta)** inalterable sobre un commit específico de la rama principal (main/master), identificando de forma unívoca la versión y fecha de conformación.

Pregunta 8: ¿En qué consiste la técnica de “Branching” y para qué sirve?

- **Respuesta breve:** Consiste en bifurcar el desarrollo de código para crear una línea de trabajo secundaria paralela e independiente de la rama principal [Sommerville Cap. 25.2]. Sirve para aislar la programación de nuevas funcionalidades o experimentos sin alterar el código estable en producción [Sommerville Cap. 25.2].

Pregunta 9: ¿Qué es un conflicto de merge y cuándo ocurre técnicamente?

- **Respuesta breve:** Es la detención automática del proceso de integración de ramas cuando dos desarrolladores modificaron las mismas líneas del mismo archivo a la vez, obligando a realizar una resolución técnica y humana manual de compatibilidad de código [Sommerville Cap. 25.2].

Pregunta 10: ¿Cuál es la diferencia entre “Configuración de Software” y “Línea Base”?

- **Respuesta breve:** La configuración es una “foto” instantánea y sumamente variable que cambia con cada commit de desarrollo; la línea base es un checkpoint congelado de configuraciones validadas que permanece inalterable ante el trabajo diario y sirve como base para desarrollos posteriores.

2.3. Actividades fundamentales de la Gestión de Configuración (SCM) (Cont.)

2.3.1. Identificación de ítems de configuración

Definición precisa del concepto

- **Definición formal (Edward Bersoff, 1984 - BIBLIO_):** La identificación de la configuración es la actividad fundamental de SCM encaminada a establecer un esquema estructurado para rotular (*labeling*) y catalogar los componentes de un sistema de software, de manera tal que se pueda registrar su evolución de forma inalterable y trazar un camino documentado que vincule todas las etapas del ciclo de vida del producto.
- **Definición de Ian Sommerville (BIBLIO_ - Cap. 25.2):** La identificación de la configuración se ocupa de la selección de los componentes del sistema (códigos, planos, bases de datos) que se colocarán bajo control, asignándoles a cada uno de ellos un nombre único inalterable a lo largo del tiempo.
- **Qué problema resuelve:** Erradica el caos del almacenamiento informal y el desorden sistemático en el repositorio (la clásica problemática del “dormitorio desordenado” explicada por el docente). Evita que cada programador llame a los archivos de manera arbitraria (lo que impediría automatizar las tareas de integración y empaquetado del software), previene la

desactualización o pérdida de componentes y permite saber exactamente qué tipo de archivo contiene cada entregable.

Cómo se hace: El procedimiento de Identificación en el PGC

Para que un entregable califique como un **Ítem de Configuración de Software (IC / SCI - Software Configuration Item)** y pueda ser correctamente identificado, el Plan de Gestión de Configuración (PGC) debe declarar de forma obligatoria las siguientes tres variables estructurales:

```
[ REQUISITOS DE IDENTIFICACIÓN DE UN IC EN EL PGC ]  
[ REGLA DE NOMBRADO ] [ UBICACIÓN FÍSICA ] [ EXTENSIÓN / FORMATO ]
```

Plantillas con placeholders Ruta exacta de carpetas Formatos unificados y rígidos
unívocos y variables en el repositorio (evita redundancias)

1. Reglas de nombrado unívocas (Naming Rules)

- Se definen plantillas sintácticas rígidas que utilicen prefijos del proyecto para evitar colisiones de nombres.
- Para representar los datos variables de la plantilla, la cátedra exige de forma obligatoria el uso de **delimitadores especiales con menor y mayor dobles (<< >> o < >)**. Cualquier integrante que vaya a crear un archivo del repositorio debe reemplazar de forma obligatoria lo que está dentro de las llaves por el nombre y número real correspondiente.

2. Ubicación física en la estructura del repositorio

- Se debe declarar explícitamente en qué carpeta o directorio del repositorio vivirá el archivo (ej: en trunk/, documentos/ o branches/). No se admite que los archivos floten sueltos en el directorio raíz.

3. Extensión y formato del archivo

- Se debe fijar de antemano qué extensión de archivo se utilizará para cada entregable (ej:.docx,.py,.sql). El PGC debe prohibir la coexistencia de formatos duplicados para el mismo entregable (como subir la matriz de trazabilidad en.xls y.pdf simultáneamente, lo cual destruye el control de versiones al obligar a actualizar ambos archivos de forma paralela).

Ejemplos concretos de la cátedra para el PGC

- **Plantilla para un Caso de Uso (IC Repetitivo):**
 - *Regla de nombrado:* <>u_caso_uso<>_<>.<>
 - *Ubicación física:* /trunk/proyecto/diseño/casos_de_uso/
 - *Extensión admitida:* .docx o.pdf (especificado de manera unívoca).
 - *Ejemplo real instanciado:* BTS_u_caso_uso_consultarHorario_01.docx
- **Plantilla para un Plan de Iteración:**
 - *Regla de nombrado:* pi_<>_<>.<>
 - *Ubicación física:* /documentos/planes/iteraciones/
 - *Ejemplo real instanciado:* pi_matrix_01.docx

- **Plantilla para Código Fuente:**
 - *Regla de nombrado:* <>.py
 - *Ubicación física:* /trunk/proyecto/codigo/
 - *Ejemplo real instanciado:* calendario.py (los nombres físicos del código fuente en entornos controlados por Git **no deben llevar número de versión**).

Errores comunes y advertencias de examen

- ⚠ *El pecado original en Git:* Colocar el número de versión de forma estática en el nombre físico del archivo dentro de un versionador (ej: llamar a tu código calendario_v2.py o ERS_v3.docx).
- *Advertencia del docente:* Git gestiona internamente el historial de cambios y la evolución de las versiones de forma automática y transparente para el usuario. Si vos renombrás físicamente el archivo agregándole un número de versión, Git interpretará que borraste el archivo anterior y creaste uno nuevo de la nada, **destruyendo por completo el historial de cambios acumulado y la trazabilidad evolutiva** del componente.
- *Excepción admitida:* Solo se permite escribir la versión en el nombre físico del archivo si el entregable va a ser **exportado fuera del ámbito del repositorio** (ej: un PDF de ejercicios o exámenes subido a la UV) para que un tercero sin acceso al versionador pueda identificar cuál es la versión vigente que está leyendo.

Relación con otros conceptos

La identificación unívoca es el soporte primario indispensable para poder establecer la **trazabilidad delantera y trasera (pre-ERS y post-ERS)**, y es el requisito básico para poder congelar y taggear conjuntos estables de componentes bajo una **Línea Base (Baseline)**.

2.3.2. Clasificación de ICs por su ciclo de vida y ámbito

Definición precisa del concepto

- **Definición teórica:** Es la taxonomía conceptual que divide a todos los artefactos de software generados a lo largo del proceso de desarrollo en tres categorías mutuamente excluyentes, basándose en la correspondencia de su ciclo de vida con las entidades del negocio: el **Producto**, el **Proyecto** y la **Iteración**.
- **Qué problema resuelve:** Evita el desperdicio de almacenamiento y la sobrecarga de control. Al clasificar los ICs según su duración y trascendencia, el equipo de SCM sabe qué archivos se deben resguardar y mantener de forma permanente a largo plazo (los de Producto, mientras la app siga en uso) y qué archivos se pueden archivar de forma transitoria o descartar administrativamente una vez cerrada una etapa (los de Proyecto e Iteración).

Clasificaciones y tipos completos (La Taxonomía Teórica)

[TAXONOMÍA TEÓRICA DE ICs]
 [IC DE PRODUCTO] [IC DE PROYECTO] [IC DE ITERACIÓN]

- Ciclo de vida largo. - Ciclo de vida medio. - Ciclo de vida corto.
- Sobrevive al proyecto. - Dura lo que el proyecto. - Muere con el Sprint.

- Ej: Código, ERS, Manual. - Ej: Planes, Minutas, PGC. - Ej: Reporte de Defectos.

1. **Ítems de Configuración de Producto:**

- *Definición:* Son todos aquellos artefactos directamente vinculados con la especificación, diseño, construcción, prueba y operación del software entregable. **Poseen el ciclo de vida más largo del desarrollo** porque deben mantenerse, versionarse y resguardarse de forma activa mientras el software esté vivo y operando en producción en los servidores del cliente.
- *Ejemplos:* Especificación de Requerimientos de Software (ERS), diagramas de arquitectura de software, código fuente, scripts de creación de base de datos, manuales de usuario, instructivos de despliegue y casos de prueba de aceptación.

2. **Ítems de Configuración de Proyecto:**

- *Definición:* Son artefactos puramente administrativos y de gestión diseñados para coordinar los recursos, costos y plazos del desarrollo. **Su ciclo de vida dura exactamente lo que dura el proyecto;** una vez que el proyecto se cierra formalmente y el software se entrega, estos archivos pierden vigencia operativa y se archivan.
- *Ejemplos:* Plan de Desarrollo del Proyecto (PP), cronograma de hitos, Roadmap de liberación del producto, Plan de Gestión de Configuración (PGC) y las minutas de reunión con el cliente.

3. **Ítems de Configuración de Iteración:**

- *Definición:* Son elementos tácticos de cortísimo plazo diseñados para gestionar el trabajo de un sprint o incremento específico. **Su ciclo de vida útil muere de forma definitiva al cerrarse la iteración correspondiente,** no teniendo sentido realizarles un seguimiento evolutivo de largo plazo.
- *Ejemplos:* Plan de la iteración actual (ej: pi_sprint1.docx), reportes semanales de testing técnico de la iteración y reportes de defectos activos del sprint.

CONTRADICCIÓN EN PROFUNDIDAD: TAXONOMÍA TEÓRICA VS. ESQUEMA OPERATIVO DEL TP4

Uno de los errores conceptuales más frecuentes y destructivos que cometen los alumnos de la materia es mezclar la taxonomía teórica del software comercial con la simplificación práctica utilizada para los trabajos prácticos de la cátedra de la UTN-FRC.

1. ¿En qué consiste la contradicción?

- **La Taxonomía Teórica (Teórico de Examen):** Exige clasificar los ICs en **Producto, Proyecto e Iteración**. Es el estándar formal de la industria de software real.
- **El Esquema Operativo de la Cátedra (Práctico TP4):** Dado que en la materia los alumnos simulan que cursar y aprobar la asignatura es su “proyecto de software”, no existe en la realidad académica una fase de mantenimiento post-despliegue del producto en producción comercial. Intentar aplicar la taxonomía teórica de forma rígida generaría una sobrecarga burocrática inútil. Por ende, para el TP4, **los profesores de práctico exigen de forma obligatoria simplificar la clasificación en:**

1. **Material provisto por la cátedra (Clase/Cátedra):** Todo archivo de origen externo que se descarga de la UV y se incorpora al repositorio Git como material de lectura de base (slides, apuntes teóricos, bibliografías, enunciados de TPs).
2. **Producción propia:** Todo entregable y código que el grupo de alumnos diseña, codifica y documenta de forma proactiva para ser evaluado (los TPS resueltos, el propio PGC de su grupo, el código de los parciales prácticos, resúmenes grupales de estudio).

2. Cuándo aplica cada clasificación y por qué

- **La clasificación teórica (Producto/Proyecto/Iteración)** aplica en el ejercicio profesional de la Ingeniería de Software y es de **evaluación obligatoria en el examen final y en las preguntas teóricas de los parciales**.
- **La clasificación práctica (Cátedra/Producción Propia)** aplica **únicamente dentro del Plan de Gestión de Configuración (PGC) del TP4 de tu grupo** para ordenar los archivos reales de tu repositorio Git de estudio.

3. Por qué confundirlas es motivo de desaprobado directo según el docente

El docente en los audios (3 Practico SCM.m4a) es contundente:

- **Bochazo en el parcial/final:** Si en una pregunta de examen teórico te piden la clasificación de los ICs según su ciclo de vida y vos respondés: “*Se clasifican en material provisto por la cátedra y producción propia*”, estás demostrando que no entendés la base teórica de SCM de los libros y que solo te acordás mecánicamente del atajo operativo que usaste para entregar el práctico de tu grupo.
- **Reprobación en la entrega del TP4:** Si en la entrega del TP4 copiás y pegás una tabla de SCM genérica de Internet que lista Ítems de Producto, Proyecto e Iteración comerciales teóricos desvinculados de tu realidad académica real, los docentes de práctico te desaprobarán el trabajo por falta de pragmatismo, consistencia y realismo operacional.

Tabla de Trazabilidad y Mapeo Cruzado de ICs Académicos

Para evitar confusiones en las entregas y parciales, acá tenés el mapeo exacto de cómo se categorizan los mismos archivos lúdicos del repositorio según el prisma taxonómico que se aplique:

Nombre del Archivo Físico en el Repositorio	Clasificación Teórica (Parcial Teórico / Final)	Clasificación Práctica (Documento del TP4)	Justificación de la Categoría en SCM
Sommerville_Ing_Software_9ed.pdf	Ítem de Producto (Bibliografía técnica que da soporte conceptual al software)	Material de Cátedra / Clase (Material provisto de origen externo en la UV)	Da sustento al dominio lúdico de la materia. El grupo no lo editó y sobrevivirá a la cursada.

Nombre del Archivo Físico en el Repositorio	Clasificación Teórica (Parcial Teórico / Final)	Clasificación Práctica (Documento del TP4)	Justificación de la Categoría en SCM
o2_SCM_diapositivas.pdf	Ítem de Producto (Especificación de proceso y directivas técnicas)	Material de Cátedra / Clase (Entregado por el docente)	Contiene las reglas arquitectónicas y de gestión que el producto final debe cumplir obligatoriamente.
BTS_Plan_Gestion_Configuracion.pdf	Ítem de Proyecto (Plan de control administrativo y de repositorio)	Producción Propia (Desarrollado de cero por el grupo de alumnos)	Es el plan que define las reglas de SCM del grupo para transcurrir la materia (proyecto). Muere con la cursada.
TP4_enunciado_SCM.pdf	Ítem de Proyecto (Requerimiento administrativo de la iteración)	Material de Cátedra / Clase (Provisto por los profesores de práctico)	Define el alcance del entregable del TP4 para que el grupo pueda ser evaluado.
BTS_codigo_TP6_test_driven.py	Ítem de Producto (Código fuente ejecutable)	Producción Propia (Programado por los integrantes del grupo)	Es el código fuente real programado para resolver el práctico de TDD. Sobrevive en producción.
BTS_RMI_sprint_01.docx	Ítem de Iteración (Registro de retrospectiva y mejora del primer sprint)	Producción Propia (Minuta generada de forma interna por el equipo)	Es una minuta de trabajo táctica para corregir el funcionamiento del equipo durante un sprint. Muere al cerrar la iteración.

Puntos clave para el examen — 2.3.2

1. **LA VERDADERA DIFERENCIA ENTRE VERSIÓN Y ARTEFACTO (AUDIO_o2):**
Durante la discusión de teoría en el aula, el docente de teoría aclara una confusión recurrente de examen respecto al término “versión” de un artefacto no digitalizado. Recuerda que, en el contexto de SCM, **la versión solo tiene sentido de ser controlada si el artefacto se**

ha convertido físicamente en un IC a través de la digitalización (un archivo guardado en el disco). Una idea en el pizarrón puede tener “variantes lógicas” en tu mente, pero para SCM no existe como ítem hasta que no le saques una foto, le pongas una extensión de archivo y la subas al repositorio.

2. **RESTRICCIÓN DE EXTENSIONES EN EL PGC (AUDIO_03):** El profesor práctico advierte de forma explícita que en el TP4 es un error grave de gestión generalizar las extensiones de los ICs de forma libre (ej: poner en la columna de extensión de la tabla: .doc / .pdf / .xls). El PGC debe ser rígido y específico: si la matriz de trazabilidad se definió en Excel, la única extensión permitida para ese IC en la tabla de nombrado es .xls o .xlsx. Permitir múltiples formatos abre la puerta a que los integrantes del grupo suban archivos duplicados de distintas extensiones desincronizados, corrompiendo la consistencia del repositorio.
3. **EL PLAN DE CONFIGURACIÓN SE ENTREGA EN PDF (AUDIO_03):** En el práctico se resalta que el PGC es un documento formal de entrega de proyecto. Por lo tanto, aunque trabajen colaborativamente con herramientas en la nube (como Google Docs), la entrega en la UV exige subir el documento formal de PGC exportado de forma limpia en formato **PDF**. No se admite entregar links sueltos a documentos editables de Google Docs sin el PDF de respaldo.
4. **LA ESTRUCTURA SE SUBE DE ENTRADA (AUDIO_03 / AUDIO_06):** Durante la corrección de errores típicos, el docente resalta que está mal la excusa de: *“Las carpetas de los prácticos 7, 8 y 9 las íbamos a ir creando en Git a medida que transcurriera la materia porque todavía no las usamos”*. La estructura completa de carpetas del repositorio debe estar **definida físicamente en Git desde el día uno del TP4** (usando archivos vacíos.gitkeep para que Git permita subir las carpetas vacías). Esto evita que el día de mañana un integrante nuevo guarde los archivos donde quiera por no tener la estructura de carpetas configurada.

Preguntas de autoevaluación — 2.3.2

Pregunta 1: ¿Cómo define Edward Bersoff (1984) a la actividad de “Identificación de la Configuración”?

- **Respuesta breve:** La define como el proceso de establecer un esquema de codificación y etiquetado para identificar unívocamente las Líneas Base y sus actualizaciones sucesivas, permitiendo trazar con precisión la historia de evolución del software.

Pregunta 2: ¿Por qué la cátedra exige que los Ítems de Configuración (ICs) que se suben a Git no lleven la versión escrita en su nombre físico?

- **Respuesta breve:** Porque el versionador Git ya rastrea y maneja de manera automática y transparente el historial de versiones por detrás. Modificar físicamente el nombre de un archivo cada vez que cambia destruye el historial evolutivo del componente en Git.

Pregunta 3: ¿En qué casos particulares sí se admite escribir el número de versión en el nombre físico de un archivo de SCM?

- **Respuesta breve:** Únicamente cuando el archivo se va a exportar fuera del entorno del repositorio Git para ser entregado a un usuario o cliente externo que no tiene acceso al versionador (ej: los ejercicios de la materia publicados en la UV en formato PDF).

Pregunta 4: ¿Cómo se representan las variables dinámicas en las plantillas de reglas de nombrado según las pautas de examen del práctico de la materia?

- **Respuesta breve:** Se representan obligatoriamente encerrando los placeholders dinámicos que el programador debe reemplazar entre signos de menor y mayor dobles (ej: <> o <>).

Pregunta 5: ¿Cuáles son las tres categorías en las que se dividen los ICs según la clasificación teórica del ciclo de vida?

- **Respuesta breve:** Se dividen en: Ítems de **Producto**, Ítems de **Proyecto** e Ítems de **Iteración**.

Pregunta 6: ¿Qué define si un IC pertenece a la categoría de “Producto” o de “Proyecto” en la taxonomía conceptual teórica?

- **Respuesta breve:** Su ciclo de vida. Si el artefacto debe mantenerse y versionarse de forma activa mientras el software esté vivo en producción, es de **Producto**; si el artefacto pierde vigencia formal y se archiva al finalizar la etapa de desarrollo y entregarse el software, es de **Proyecto**.

Pregunta 7: ¿Cuál es la clasificación de ICs exigida de forma operativa para la confección del Plan de Configuración del TP4 del grupo?

- **Respuesta breve:** Exige clasificar los ítems únicamente en: **Material provisto por la cátedra / Clase** y **Producción propia del grupo**.

Pregunta 8: ¿Por qué es causa de desaprobado directo responder en el examen teórico usando la taxonomía del TP4 (Cátedra/Producción Propia)?

- **Respuesta breve:** Porque esa clasificación es solo una simplificación operativa ficticia para el ámbito de la cursada. Responder eso en el examen teórico demuestra que el alumno no asimiló los conceptos de ciclo de vida del software comercial real dictados en los libros.

Pregunta 9: Dé un ejemplo de un documento de “Producción propia” que sea clasificado teóricamente como “Ítem de Proyecto”.

- **Respuesta breve:** El Plan de Gestión de Configuración (PGC) diseñado de cero por el grupo de alumnos para definir las reglas de nombrado y el repositorio.

Pregunta 10: ¿Para qué sirve el archivo marcador de posición .gitkeep en la estructura de carpetas inicial de Git exigida en el TP4?

- **Respuesta breve:** Git no permite registrar ni subir carpetas vacías al repositorio central. El archivo .gitkeep se coloca como un marcador de posición temporal para obligar a Git a subir la carpeta vacía, manteniendo la estructura de directorios unificada desde el inicio del proyecto.

2.3. Actividades fundamentales de la SCM (Cont.)

2.3.3. Control de Cambios (Configuration Control)

Definición precisa del concepto

- **Definición formal (Edward Bersoff, 1984 - BIBLIO_):** El control de configuración es el proceso y el mecanismo administrativo y técnico mediante el cual se proponen, evalúan, aprueban o rechazan, implementan y verifican de manera consistente todas las modificaciones realizadas sobre los ítems de configuración (ICs) que ya forman parte de una **Línea Base (Baseline)**.

- **Qué problema resuelve:** Detiene la degradación descontrolada de la arquitectura de software (evita meter parches “a lo pampa”). Resuelve la problemática de **cambios no validados** y la **superposición de cambios**, garantizando que ninguna persona del equipo de desarrollo modifique unilateralmente un archivo congelado sin analizar previamente el costo, los tiempos de entrega y los impactos colaterales en otros módulos del sistema.

El flujo completo de Control de Cambios paso a paso

El procedimiento formal para aplicar un cambio sobre una línea base congelada sigue una secuencia estricta de 9 etapas. Cada paso genera un entregable y requiere la intervención de roles específicos de la cátedra:

```
[ EL FLUJO FORMAL DE CONTROL DE CAMBIOS ]
[Paso 1: Solicitud] -> [Paso 2: Registro] -> [Paso 3: Análisis de Impacto (ECP)]
[Paso 6: Pruebas (CI)] <- [Paso 5: Ejecución] <- [Paso 4: Decisión del CCB]
[Paso 7: Auditoría] -> [Paso 8: Nueva LB (Tag)] -> [Paso 9: Notificación]
```

Paso 1: Generación de la Solicitud de Cambio (Change Request)

- **Qué sucede:** Un cliente, un usuario operativo, el equipo de marketing o un desarrollador identifica un bug o una nueva necesidad de negocio y envía formalmente la petición.
- **Quién interviene:** El **Solicitante del Cambio** (Client / Solicitor).
- **Artefacto generado:** **Formato de Petición de Cambio (CRF - Change Request Form)** o Solicitud de Cambio (CR) parcialmente completado con la descripción del problema.

Paso 2: Recepción, Comprobación y Registro

- **Qué sucede:** Se verifica que la solicitud sea técnicamente válida. Si el bug reportado ya fue corregido, si es un error de interpretación del manual, o si la funcionalidad ya existe, se rechaza y se cierra la solicitud. Si es válida, se registra de forma oficial en la cola de cambios pendientes.
- **Quién interviene:** El **Soporte Técnico o el Gestor de Configuración**.
- **Artefacto generado:** **CRF actualizado** con estado “Registrado / Bajo Análisis”.

Paso 3: Análisis de Impacto y Estimación de Costos

- **Qué sucede:** Se realiza un estudio profundo del cambio propuesto para identificar qué componentes del sistema se verán afectados directamente y cuáles de manera colateral. Se estima el esfuerzo de desarrollo (en horas o Story Points) y el impacto en la arquitectura de software.
- **Quién interviene:** El **Analizador del Cambio / Líder Técnico / Arquitecto**.
- **Artefacto generado:** **Propuesta de Cambio de Ingeniería (ECP - Engineering Change Proposal)**, o bien el CRF actualizado con la lista de archivos afectados y la estimación de esfuerzo en horas.

Paso 4: Evaluación y Decisión del CCB (Comité de Control de Cambios)

- **Qué sucede:** El Comité se reúne formalmente para debatir si el cambio es rentable y estratégico desde la perspectiva del negocio y del proyecto. El CCB toma una decisión binaria: **Aprobar, Rechazar o Diferir** (postergar para futuras versiones) el cambio.
- **Quién interviene:** El **CCB (Comité de Control de Cambios)** en su conjunto (Analista, Líder Técnico, Gestor de Configuración, Testing y Cliente/Product Owner si el cambio impacta en costos/plazos).
- **Artefacto generado:** **Orden de Cambio de Ingeniería (ECO - Engineering Change Order)** u orden formal de cambio aprobada, indicando en qué versión del software se incorporará la modificación.

Paso 5: Implementación del Cambio (Codificación)

- **Qué sucede:** El programador asignado descarga la línea base estable del repositorio a su espacio de trabajo privado (*workspace*), abre una rama semántica aislada (*feature branch* o *hotfix branch*) y modifica el código fuente de los componentes autorizados.
- **Quién interviene:** El **Desarrollador (Developer)**.
- **Artefacto generado:** **Ítems de Configuración modificados** en el entorno privado de desarrollo.

Paso 6: Pruebas de Regresión y Validación

- **Qué sucede:** Se compila el sistema en el servidor de construcción y se ejecutan las pruebas de testing automatizadas para garantizar que el cambio soluciona el problema y **no introdujo regresión de fallas** en otras partes estables del software.
- **Quién interviene:** El **Tester / Integrador Técnico**.
- **Artefacto generado:** **Reporte de Ejecución de Pruebas (Test Report)** con estado "OK".

Paso 7: Auditoría de Configuración del Cambio (Revisión de Partes)

- **Qué sucede:** Se verifica de forma formal que el desarrollador haya modificado **únicamente** los componentes que el CCB le autorizó en el ECO, controlando que no se hayan colado cambios informales no aprobados.
- **Quién interviene:** El **Gestor de Configuración de Software o Auditor de QA**.
- **Artefacto generado:** **Acta de Auditoría de Configuración / Minuta de Aprobación del Cambio**.

Paso 8: Conformación de la Nueva Línea Base

- **Qué sucede:** El Comité da el visto bueno formal para fusionar (*merge*) la rama de desarrollo con la rama estable (main o master). El Gestor de Configuración aplica una etiqueta inalterable (**Tag**) para congelar la nueva línea base en el repositorio público.
- **Quién interviene:** El **Gestor de Configuración** operando la herramienta Git.
- **Artefacto generado:** **Nueva Línea Base (Tag de Git)** conformada (ej: LB_PROD_V1.1).

Paso 9: Notificación a las Partes e Informe al Cliente

- **Qué sucede:** Se comunica oficialmente a todos los involucrados en el proyecto (desarrolladores, analistas, testing, cliente) que el cambio fue integrado con éxito, indicando la versión que lo contiene.
- **Quién interviene:** El **Gestor de Configuración o el Líder de Proyecto**.
- **Artefacto generado:** **Nota de Lanzamiento (Release Notes)** o Documento de Notificación de Cambio.

El CCB (Comité de Control de Cambios)

- **Definición de las Slides (SLIDE_02):** El Comité de Control de Cambios (CCB - Change Control Board) es el órgano colegiado responsable de evaluar de forma sistemática el impacto técnico y financiero de las solicitudes de cambio para decidir de manera soberana su aprobación, rechazo o postergación.
- **Quiénes lo integran:** Como el docente repite en clase, **el CCB no está formado por marcianos**. Está integrado por representantes del mismo equipo del proyecto:
 - *El Analista Funcional:* Vela por la consistencia funcional y el impacto sobre la Especificación de Requerimientos de Software (ERS).
 - *El Arquitecto o Líder Técnico:* Analiza el impacto del cambio en la estructura del código y cuida que no se degrade la cohesión ni se aumente el acoplamiento del software.
 - *El Líder de Proyecto / Gestor de SCM:* Evalúa el desvío de plazos y costos organizacionales, y operativamente taguea las líneas base.
 - *Un Representante de Testing / QA:* Evalúa cuántos casos de prueba hay que rehacer y el esfuerzo que requerirán las pruebas de regresión.
 - *El Cliente / Product Owner (PO):* Interviene de forma obligatoria cuando el cambio solicitado tiene tal envergadura estratégica que **afecta de forma directa los plazos finales de entrega o el presupuesto económico acordado**.
- **Ejemplo de clase:** El docente de teoría describe la llamada telefónica del cliente pidiendo que una validación deje de ser obligatoria en el sistema. El programador “de buena onda” la modifica en dos minutos, pero como no pasó por el CCB (no hubo análisis de impacto), el cambio de base de datos provoca un error en cascada que cuelga las transacciones en producción. Pasar por el CCB evita esta catástrofe analizando el impacto de manera colectiva.
- **Errores comunes / Advertencias:**
 - ⚠ *El error de pensar que el CCB es burocracia inútil para dilatar el proyecto.* El CCB es un engranaje de calidad preventivo. No paraliza el desarrollo; evita que el equipo trabaje sobre bases de código inestables que luego deban tirarse a la basura.

2.3.4. Auditorías de Configuración (Configuration Auditing)

Definición del concepto

- **Definición formal (Edward Bersoff, 1984 - BIBLIO_)**: La auditoría de configuración es la actividad de aseguramiento de calidad (Soporte) orientada a realizar una evaluación formal e independiente de los ítems de configuración para garantizar que el producto construido coincida de manera exacta y sin discrepancias con sus especificaciones de requerimientos aprobadas y con la documentación registrada en el Plan de Configuración.
- **Qué problema resuelve**: Erradica la falta de correspondencia entre lo que se programó, lo que se testeó y lo que se documentó (problemática de **cambios no validados** y falta de **sincronía fuente-objeto**). Evita que le entreguemos al cliente un código fuente que no compile, al que le falten manuales de usuario obligatorios, o que contenga funcionalidades “fantasma” que el CCB nunca aprobó.

Contraste Técnico: Auditoría Física vs. Auditoría Funcional

La cátedra exige diferenciar de forma rigurosa las dos vertientes de la auditoría de configuración:

Criterio de Comparación	Auditoría Física de Configuración (PCA - Physical Configuration Audit)	Auditoría Funcional de Configuración (FCA - Functional Configuration Audit)
Foco de Inspección	La presencia física, completitud e integridad de los artefactos lógicos en el repositorio.	El comportamiento dinámico, funcionalidad y rendimiento del software construido.
La Pregunta Clave	“¿Están todos los archivos físicos que el plan decía que debían estar?”*	“¿Funciona el software de acuerdo a los requerimientos aprobados en la ERS?”*
Mecanismo de Acción	Estático . Se controla visual y administrativamente el repositorio y sus documentos de soporte.	Dinámico . Se ejecutan pruebas de software en entornos controlados y se analizan reportes de pruebas.
Documento contra el que se contrasta	El Plan de Gestión de Configuración (PGC) , el Plan de Proyecto y el listado de ICs del inventario.	La Especificación de Requerimientos de Software (ERS) y los Criterios de Aceptación.
Ejemplo de Chequeo	Verificar que la ERS final, los manuales de instalación en PDF y el código fuente estén físicamente subidos en Git.	Verificar que el sistema de facturación compute de forma exacta el 21% del IVA de la tasa impositiva.
Artefacto de Trazabilidad Clave	El Glosario, Plan de SCM y la estructura de carpetas unificada en Git.	La Matriz de Trazabilidad de Requerimientos acoplada con los Casos de Prueba con estado “OK”.

Relación de la Auditoría con Verificación y Validación (V&V)

La auditoría de configuración actúa como soporte directo y se alimenta de las actividades de Verificación y Validación del software:

AUDITORÍA DE CONFIGURACIÓN

[VERIFICACIÓN] (Static/Rules) [VALIDACIÓN] (Dynamic/User)

- ¿Construimos el producto correctamente? - ¿Construimos el producto correcto?
- Contraste contra las Líneas Base. - Satisface las necesidades del usuario.
- Todos los casos de prueba con estado “OK” - El cliente firma la aceptación del

o reportados en la nota de release. producto terminado en producción.

- **Verificación:** Asegura que el producto de software cumple de manera estricta con los objetivos y especificaciones preestablecidas en la documentación de las Líneas Base. Desde el punto de vista de la auditoría, la verificación se da por cumplida cuando **todas las funciones fueron ejecutadas con éxito y el reporte de testing arroja que los casos de prueba están en estado “OK”** (o en su defecto, las discrepancias conocidas constan de forma explícita como “problemas reportados” en la nota de release).
- **Validación:** Garantiza que el software resuelve de manera apropiada el problema real de negocio del cliente (que el usuario obtenga el producto correcto que satisfaga sus expectativas reales).

2.3.5. Informes de Estado / Status Accounting (Configuration Status Accounting)

Definición precisa del concepto

- **Definición formal (Edward Bersoff, 1984 - BIBLIO_):** El registro e informe del estado de la configuración es la actividad de soporte (transversal) que se ocupa de recopilar, almacenar e informar de manera histórica e inalterable la información administrativa y de ingeniería necesaria para gestionar de manera efectiva la evolución del sistema de software a lo largo de su ciclo de vida.
- **Definición de las Slides de la Cátedra (SLIDE_02):** Actividad transversal que se ocupa de mantener los registros de la evolución del sistema. Dado que maneja un gran volumen de información y metadatos, se suele implementar dentro de los procesos automáticos de la herramienta de versionado.
- **Qué problema resuelve:** Erradica la pérdida de memoria histórica del proyecto (problemática de la rotación de personal o “el programador que se fue a criar caracoles al cerro” explicada en los audios). Resuelve la incertidumbre gerencial y técnica respondiendo preguntas sobre el origen de las modificaciones, quién las autorizó, qué versión contiene una mejora específica y qué diferencias de código existen entre dos entregas.

Qué registra e informa formalmente la disciplina (Los 9 Puntos de Bersoff)

Sistémicamente, Edward Bersoff (1984) establece en su paper clásico que para llevar un control inalterable del estado del repositorio, la actividad de Informes de Estado (Status Accounting) debe capturar, registrar y reportar de forma obligatoria los siguientes nueve eventos de información:

1. **El momento de tiempo exacto** (timestamp) en el que cada representación de una Línea Base y su correspondiente actualización entraron físicamente en existencia.
2. **El momento de tiempo exacto** en el que cada Ítem de Configuración de Software (IC) individual fue creado e ingresado al repositorio por primera vez.
3. **La información descriptiva detallada** de los metadatos asociados a cada IC (nombre unívoco, tipo, autor original, tamaño inicial del archivo).
4. **El estado administrativo actual** en el que se encuentra cada Propuesta de Cambio de Ingeniería (ECP) ingresada (estado: aprobado, rechazado, diferido, o bajo análisis del CCB).
5. **La información descriptiva detallada** de cada ECP (quién la solicitó, fecha de ingreso, descripción del bug o requerimiento asociado).
6. **El estado del cambio de software en sí** (estado: en cola de espera, en desarrollo activo en rama aislada, en proceso de testing de regresión, o integrado en main).
7. **La información descriptiva detallada** de cada cambio de código integrado (qué líneas de código se modificaron, por qué razón técnica, quién fue el programador responsable).
8. **El estado de la documentación técnica y administrativa** vinculada de forma directa con una Línea Base (por ejemplo, el estado de aprobación del plan de pruebas o de la matriz de trazabilidad del release).
9. **Las discrepancias y deficiencias técnicas** encontradas en una Línea Base específica durante los procesos de auditoría de configuración (física o funcional), registrando cómo y cuándo se subsanaron.

Preguntas clave del negocio que responde Status Accounting

Durante la cursada, la cátedra enfatiza que un buen sistema de registro e informes debe estar capacitado para responderle de forma inmediata y automática al equipo y al cliente las siguientes cuatro preguntas críticas de control:

- **¿Cuál es el estado del ítem de configuración?** (Saber si un documento de diseño o un módulo de código está en borrador, bajo revisión de QA, congelado en línea base o en etapa de mantenimiento).
- **¿La petición de cambio fue aprobada o rechazada por el CCB?** (Tener trazabilidad administrativa del flujo formal del cambio para no iniciar código sin autorización).
- **¿Qué versión específica del ítem de configuración implementa el cambio aprobado?** (Saber con precisión matemática qué componente de software o archivo físico del repositorio contiene la mejora o el parche del bug solicitado por el cliente).
- **¿Cuál es la diferencia exacta entre una versión y otra dada de un archivo?** (Habilitar el análisis de diferencias mediante comandos diff para conocer qué líneas de código se introdujeron entre la línea base anterior y la actual).

Puntos clave para el examen — 2.3.5

1. **DIFERENCIA ENTRE VERSIÓN Y LÍNEA BASE (AUDIO_02):** Judith Meles hace un hincapié absoluto en que **no todas las versiones de un ítem de configuración van a conformar una Línea Base**. Un desarrollador puede guardar 40 commits locales con 40

versiones de una clase en Git para hacer backups de su trabajo (control de versiones automático), pero ese componente **solo formará parte de una Línea Base cuando el CCB lo revise, lo valide y lo apruebe** colectivamente como punto de referencia inmutable para que todo el equipo empiece a programar el siguiente incremento.

2. **EL ROL DEL GESTOR DE CONFIGURACIÓN OPERATIVO (AUDIO_02):** El docente aclara que el Gestor de Configuración es un **rol de soporte** (administra la herramienta, maneja los permisos del repositorio Git, crea físicamente las ramas y aplica los tags de líneas base). Sin embargo, **él solo no tiene la autoridad técnica ni gerencial para aprobar un cambio**. El único que autoriza o rechaza es el CCB; el gestor es el brazo operativo y administrativo de sus decisiones.
3. **EL CLIENTE EN EL CCB (AUDIO_02):** Una pregunta recurrente de parcial es si el cliente debe estar en el CCB. El docente explica que **depende de la envergadura y característica del cambio**. Si el cliente pide una “tontera” o un cambio menor, no tiene sentido citarlo para no encarecer el proceso. Pero si la modificación altera la arquitectura o el costo-beneficio del proyecto, es **obligatorio que participe** para que escuche el impacto en plazos y presupuesto, asuma la responsabilidad y tome la decisión de si está dispuesto a pagarlo.
4. **CÓMO VOLVER ATRÁS EN GIT (AUDIO_02):** En los exámenes prácticos se evalúa el concepto de resguardo. El docente recuerda que cuando decidimos descartar un desarrollo fallido y volver atrás en la historia del repositorio, **siempre se vuelve a una Línea Base formal (a un Tag aprobado)**, nunca a un commit aleatorio de desarrollo intermedio de un programador.

Preguntas de autoevaluación — 2.3.5

Pregunta 1: ¿Qué es el Comité de Control de Cambios (CCB) y quiénes lo integran según la cátedra?

- **Respuesta breve:** Es el órgano colegiado responsable de evaluar de forma sistemática el impacto técnico, financiero y temporal de los cambios propuestos sobre una línea base congelada. Lo integran representantes del mismo equipo de proyecto (analista, líder técnico, gestor de SCM, tester) y el cliente/PO en caso de cambios estratégicos.

Pregunta 2: ¿Qué es la Propuesta de Cambio de Ingeniería (ECP) y en qué paso del control de cambios se genera?

- **Respuesta breve:** Es el documento técnico y administrativo que detalla la descripción del cambio solicitado, los archivos y líneas base que se verán afectados por el impacto lateral, y la estimación de costos y plazos requeridos. Se genera en el **Paso 3: Análisis de Impacto**.

Pregunta 3: ¿Por qué la Orden de Cambio de Ingeniería (ECO) es indispensable antes de escribir código sobre una línea base?

- **Respuesta breve:** Porque el ECO representa la autorización formal emanada del CCB que habilita al desarrollador a modificar un componente congelado. Programar sin un ECO aprobado introduce cambios informales que corrompen la integridad y consistencia de la línea base.

Pregunta 4: ¿Cuál es la diferencia fundamental entre una “Auditoría Física” (PCA) y una “Auditoría Funcional” (FCA)?

- **Respuesta breve:** La **Física (PCA)** es un control estático y administrativo que verifica la presencia, completitud y consistencia física de los entregables en el repositorio; la **Funcional (FCA)** es un control dinámico que evalúa que la funcionalidad y performance reales del software construido coincidan con la ERS.

Pregunta 5: ¿A qué procesos de calidad de software asiste la Auditoría de Gestión de Configuración?

- **Respuesta breve:** Asiste y se alimenta de los dos procesos básicos de aseguramiento de calidad: la **Verificación** (confrontar el producto con sus líneas base) y la **Validación** (confrontar el producto final con las necesidades reales del usuario).

Pregunta 6: ¿Qué es el “Registro e Informe de Estado” (Status Accounting) en SCM?

- **Respuesta breve:** Es la actividad transversal encargada de registrar administrativamente el historial evolutivo del sistema de software, recopilando metadatos temporales de creación de ICs, el estado de las propuestas de cambio (ECPs) y las diferencias técnicas entre versiones.

Pregunta 7: Nombre al menos tres de los nueve eventos obligatorios de información que Status Accounting debe registrar según Edward Bersoff.

- **Respuesta breve:** El momento exacto de conformación de cada Línea Base; el estado actual de las propuestas de cambio de ingeniería (ECPs) en el CCB; y el detalle técnico de qué líneas de código se modificaron y quién fue el programador responsable.

Pregunta 8: ¿Qué diferencia de fondo existe entre el “Control de Versiones automático” de una herramienta y el “Control de Cambios” de la SCM?

- **Respuesta breve:** La herramienta versiona automáticamente todo archivo y cambio que el programador guarda localmente (el accidente); el control de cambios de SCM es el procedimiento de ingeniería (la esencia) que evalúa y aprueba formalmente las modificaciones únicamente sobre las **Líneas Base congeladas**.

Pregunta 9: ¿Por qué se afirma que la disciplina de SCM funciona como una “actividad paraguas” preventiva?

- **Respuesta breve:** Porque brinda un contenedor y repositorio ordenado, seguro y consistente de los ICs. Sin esta base de control estable, es materialmente imposible que las disciplinas de Testing y QA puedan verificar o validar el software (analogía del dormitorio desordenado).

Pregunta 10: ¿En qué momento del flujo de control de cambios interviene el CCB por segunda vez y con qué fin?

- **Respuesta breve:** Interviene tras la codificación y pruebas del cambio, en el **Paso 8: Conformación de la Nueva Línea Base**, con el fin de auditar que el cambio se haya ejecutado exactamente como fue autorizado y proceder a taguear el nuevo checkpoint estable.

2.4. Plan de Gestión de Configuración (PGC)

2.4.1. Definición formal y propósito del PGC

- **Definición de las notas de clase (NC_Mansilla / NC_Ardiles):** El Plan de Gestión de Configuración (PGC) es **el primer ítem de configuración (IC / SCI) que se debe crear obligatoriamente al iniciar un proyecto de desarrollo**. Consiste en un

documento formal que planifica, estructura y define las reglas organizacionales, herramientas, roles y procedimientos administrativos que regirán el repositorio del proyecto para resguardar la integridad del producto a lo largo de su ciclo de vida.

- **Qué problema resuelve:** Erradica la anarquía y la falta de “accountability” (rendición de cuentas) en el equipo de desarrollo. Evita que cada integrante guarde archivos en rutas arbitrarias (creando una “bolsa de gatos”), nombre los entregables con versiones manuales inconsistentes (ej. ERS_final_v2_este_si.docx), defina criterios de líneas base (baselines) incompatibles o modifique unilateralmente componentes congelados sin el consentimiento técnico del Comité.

Cómo se hace (Paso a paso para su confección):

Para construir un PGC académico y profesional con el estándar UTN-FRC, el equipo debe seguir estas fases secuenciales:

1. **Relevamiento del Ecosistema:** Determinar las características del proyecto (tamaño del equipo, duración, criticidad) para seleccionar la infraestructura tecnológica (Git, GitHub/GitLab).
2. **Definición de Estructuras:** Diseñar el árbol de directorios físico que se subirá al repositorio desde el primer día (trunk/, branches/, documentos/).
3. **Modelado del Inventario:** Listar cada tipo de entregable (IC) del proyecto, asociándole una regla de nombrado unívoca y una sola extensión permitida.
4. **Establecimiento de Roles:** Asignar nombre y apellido real de los integrantes de tu equipo a las funciones de Gestor de SCM y miembros del CCB.
5. **Definición del Flujo del Cambio:** Configurar el procedimiento formal de cambios (pasos de CRF, ECP, ECO) y los criterios de madurez técnica para congelar Líneas Base.
6. **Exportación a PDF:** Firmar el plan por todo el equipo y exportarlo a formato no editable para subirlo a la UV.

2.4.2. Plantilla de estructura formal y secciones del PGC (Estándar TP4)

A continuación te presento la plantilla estructurada con **todas las secciones obligatorias** que debe contener el Plan de Gestión de Configuración de tu grupo para pasar con éxito la auditoría del TP4 práctico:

PLAN DE GESTIÓN DE CONFIGURACIÓN DE SOFTWARE (PGC)

Proyecto: [Nombre del Proyecto de tu Grupo - Ej. “Matrix”]

1. INTRODUCCIÓN Y PROPÓSITO

- **1.1. Propósito:** Definición del alcance de las políticas de SCM para la simulación del proyecto durante la cursada de la materia.
- **1.2. Audiencia y Ámbito:** Lista de interesados (stakeholders) a los que obliga el cumplimiento de este plan.

2. GLOSARIO Y DEFINICIONES

- [Sección obligatoria en el TP4 para explicar abreviaturas técnicas, extensiones y términos propios del negocio o del grupo].

3. ROLES, RESPONSABILIDADES E INTEGRANTES DEL COMITÉ (CCB)

- 3.1. Gestor de SCM (SCM Manager):** [Nombre, Apellido y Legajo del estudiante responsable de la administración física del repositorio Git, tagueo de líneas base y comunicaciones de ECO].
- 3.2. Miembros del CCB (Comité de Control de Cambios):** [Asignación de personas reales a los roles de Analista Funcional, Líder de Arquitectura, Líder de Testing y Representante del Cliente].

4. HERRAMIENTAS Y TECNOLOGÍAS

- [Definición explícita de las herramientas de versionado distribuidas, IDEs y plataformas de alojamiento a utilizar - Ej. Git, GitHub, VS Code].

5. ESTRUCTURA Y UBICACIÓN FÍSICA DEL REPOSITORIO

- 5.1. Esquema de Directorios:** Descripción del árbol unificado de carpetas subido a la rama principal (main/master).
- 5.2. Políticas de Ramificación (Branching):** Criterios para abrir ramas de funcionalidad (`feature_<<nombre>>`), hotfixes de soporte o spikes de experimentación.

6. IDENTIFICACIÓN DE ÍTEMS DE CONFIGURACIÓN (ICs)

- 6.1. Tabla de Inventario de ICs:**

ID_IC	Nombre de Configuración	Regla de Nombrado unívoca (con placeholders << >>)	Ubicación Física (Ruta en Repo)	Extensión	Tipo de IC
IC_01	Plan de Gestión de Configuración	PGC_<<Grupo>>.pdf	/documentos/planes/	.pdf	Proyecto
IC_02	Especificación de Requerimientos	ERS_<<Proyecto>>.docx	/documentos/requerimientos/	.docx	Producto
IC_03	Plan de Iteración	pi_<<Proyecto>>_<<NroIteración>>.docx	/documentos/planes/iteraciones/	.docx	Iteración
IC_04	Código Fuente de Pruebas TDD	test_<<Modulo>>.py	/trunk/src/test/	.py	Producto

7. POLÍTICAS DE VERSIONADO (Incremento de Versiones)

- [Definición de qué constituye un Cambio Mayor, Cambio Menor o Parche técnico, y cómo se incrementan las etiquetas (ej. de 1.0.0 a 2.0.0 o 1.1.0)].

8. CRITERIOS DE CREACIÓN DE LÍNEAS BASE (BASELINES)

- [Definición detallada de los hitos o eventos estables y maduros en los que se aplicará un Tag inalterable de línea base en Git].

9. PROCEDIMIENTO FORMAL DE GESTIÓN DE CAMBIOS

- **9.1. Flujo del Cambio:** Descripción paso a paso de la ruta que sigue una Solicitud de Cambio (CRF) desde el ingreso hasta su tagueo final.
- **9.2. Análisis de Impacto:** Criterios técnicos para evaluar colisiones de arquitectura y desvíos de plazos.

10. PROCESOS DE AUDITORÍA DE CONFIGURACIÓN

- **10.1. Auditoría Física (PCA):** Rúbrica de control para verificar completitud del repositorio.
- **10.2. Auditoría Funcional (FCA):** Validación contra la matriz de trazabilidad y reportes de ejecución de testing con estado “OK”.
- **Ejemplo de clase:** El docente de práctico destaca que, si en el PGC de tu grupo no colocás **nombre y apellido real** de la persona responsable del CCB o del Gestor de SCM, el práctico se coloca en estado de “reprobado”. Poner solo “Líder de Proyecto” o “Responsable de Testing” de forma abstracta anula la responsabilidad (“accountability”) y el realismo de la simulación de ingeniería.
- **Errores comunes y advertencias:**
 - ⚠ *El error de los placeholders ausentes:* Es un error grave de parcial y de entrega omitir los delimitadores < > o << >> en las reglas de nombrado de ítems repetitivos.
 - *Advertencia del docente:* Si un IC se puede repetir (como los Casos de Uso o los Planes de Iteración), la regla de nombrado **debe contener placeholders variables** (ej. BTS_u_caso_uso_<>_<>.docx). Si la regla de nombrado se escribe fija, se asume que solo existirá un Caso de Uso en todo el proyecto, lo cual es inconsistente.
 - ⚠ *Extensiones múltiples:* Colocar en la columna de extensión de un IC una barra con varias opciones (ej..docx /.pdf / .txt) es una mala práctica de SCM. El PGC debe forzar una sola extensión técnica consistente para evitar la desorganización.
- **Relación con otros conceptos:** El PGC es el **esqueleto administrativo** del que se desprenden las actividades físicas del repositorio. Regula el funcionamiento de la disciplina protectora (SCM) sobre la que se asientan el aseguramiento de procesos (PPQA) y el testing técnico de la Unidad 4.

2.5. Gestión de Configuración de Software en ambientes Ágiles

2.5.1. El cambio de paradigma: SCM sin fricción

- **Definición teórica (Agile SCM.pdf / Slides_02):** En enfoques ágiles, la Gestión de Configuración de Software se despoja de su rol restrictivo de control para convertirse en un facilitador continuo del desarrollo lúdico (“frictionless”). Su objetivo prioritario es **servir a los programadores del equipo (practitioners) y no viceversa**, automatizando las políticas preventivas a través de código y herramientas instrumentales, permitiendo responder rápidamente a los cambios imprevistos en lugar de intentar bloquear el repositorio de forma burocrática.

Valores del Manifiesto Ágil aplicados a la Gestión de Configuración (Berczuk / Appleton):

Valor del Manifiesto Ágil	Adaptación e Implementación Técnica en SCM Ágil
Individuos e interacciones sobre procesos y herramientas	Los procesos y herramientas de SCM deben amoldarse a la dinámica humana de trabajo del equipo. El acceso al repositorio y la capacidad de realizar compilaciones del sistema en espacios privados deben ser comunes y descentralizados para todos los integrantes.
Software funcionando sobre documentación detallada	SCM automatiza las políticas organizacionales preventivas mediante conocimiento ejecutable (scripts de integración continua) en lugar de exigir manuales de procedimientos impresos. El estado real de la calidad del software se valida con el resultado inmediato de las ejecuciones del build.
Colaboración con el cliente sobre negociación contractual	SCM facilita la visibilidad técnica y la gestión de expectativas generando compilaciones frecuentes y transparentes de incrementos de software terminados para que el cliente o Product Owner (PO) pueda inspeccionarlos de forma temprana.
Respuesta al cambio sobre seguir un plan	El repositorio se diseña para abrazar y facilitar el cambio dinámico de requerimientos sin bloquear el trabajo. La ramificación semántica automatizada y la integración rápida (frecuente y continua) reducen el tiempo necesario para incorporar una modificación en producción.

2.5.2. Patrones de Compilación en Agile SCM (Kinds of Builds)

El paper de **Steve Berczuk (Agile SCM - Crossroads News, 2003)** establece tres patrones o tipos de compilaciones esenciales para estructurar los flujos de integración y entrega de valor del equipo ágil:

1. Private System Build (Build de Sistema Privado)

- **Definición:** Compilación del sistema lúdico ejecutada de forma local por un desarrollador en su **Playground o Workspace privado (Playground / Workspace)** utilizando su propia computadora antes de subir los cambios al repositorio central.
- **Propósito:** Permite validar de manera aislada que el nuevo código fuente no posee errores de compilación y que los cambios pasan las pruebas locales, evitando subir parches inconsistentes que “rompan” el repositorio principal del equipo.
- **Consumidor:** El programador individual.

2. Integration Build (Build de Integración)

- **Definición:** Compilación automatizada y centralizada que se dispara tras la recepción de un nuevo commit o Pull Request (PR) aprobado en la rama principal (trunk/main).
- **Propósito:** Fusionar e incorporar en un único resultado estable, consistente y verificado el trabajo concurrente de todos los integrantes del equipo. Sirve de base para que los desarrolladores realicen la sincronización de sus workspaces (*Workspace Update*).
- **Consumidor:** El Integrador Técnico o el sistema de Integración Continua (CI).

3. Release Build (Build de Liberación)

- **Definición:** Compilación formal de envergadura global que se ejecuta de forma esporádica (con menor frecuencia que los builds de integración) cuando el equipo técnico decide empaquetar una versión consolidada del sistema para ser enviada a entornos externos de pruebas formales (QA) o de producción final.
- **Propósito:** Generar paquetes de distribución inmutables con notas de release (*Release Notes*) detalladas y controles estrictos de auditoría de calidad.
- **Consumidor:** El equipo de aseguramiento de calidad (QA), el Product Owner y el cliente final.

RESOLUCIÓN DE LAS PREGUNTAS DE DEBATE DE SCM ÁGIL (Meles / Covaro)

[HUECO: Las respuestas concretas a las preguntas de debate de SCM Ágil figuran al final de la SLIDE_02 de SCM pero no poseen ningún desarrollo narrativo oficial en el material de diapositivas cargado en el entorno. Se reconstruyen y resuelven a continuación en base a la integración técnica de Agile SCM.pdf, Sommerville (Capítulo 25) y las explicaciones docentes en clase].

A partir de los audios y la bibliografía consolidada, se resuelven académicamente las cuatro grandes preguntas de debate planteadas por las docentes para los exámenes teóricos:

Debate 1: ¿Qué pasa con el Comité de Control de Cambios (CCB) en ambientes ágiles?

- *Resolución académica:* El CCB tradicional, caracterizado por su rigidez burocrática y reuniones asincrónicas de firmas formales, desaparece. La autoridad central para priorizar, negociar y aceptar cambios en el alcance funcional recae de forma exclusiva en el **Product Owner (PO)**. Las peticiones de cambio funcional se ingresan directamente en el **Product Backlog** (backlog del producto) como Historias de Usuario, Spikes o parches técnicos prioritarios.

- Para cambios de código internos y refactorizaciones arquitectónicas preventivas, la decisión es **100% descentralizada y queda a discreción del equipo de desarrollo** (en XP, cualquier programador puede refactorizar cualquier componente de la base de código para mejorar el diseño sin requerir autorización formal de un comité). Un rol de facilitación (el ScrumMaster) vela por que el proceso fluya y se respete el timebox.

Debate 2: ¿Qué ítems de configuración (ICs) podemos tener en un proyecto ágil?

- *Resolución académica:* Siguiendo la filosofía Lean de eliminación sistemática de desperdicios (*waste*), se descartan de plano todos los documentos de proceso de alta sobrecarga que no agregan valor real de software funcionando. Sin embargo, **el agilismo no es indisciplina ni prohíbe documentar**.
- Los ICs que obligatoriamente se colocan bajo control estricto de configuración son: el código fuente del sistema, la base de datos y sus scripts de migración, las pruebas automatizadas (código de tests de regresión), el Product Backlog virtualizado en herramientas de seguimiento, el Plan de Configuración del sprint (Sprint Backlog) y, si contractualmente se requiere, la ERS y los manuales de usuario exportados.

Debate 3: ¿Qué pasa con las auditorías de configuración en agilidad?

- *Resolución académica:* Se descarta el concepto de auditoría como hito de revisión puramente manual, tardío e individual ejecutado de manera burocrática por inspectores externos antes del lanzamiento.
- **Las auditorías de configuración se automatizan y se ejecutan de manera continua** en cada build a través del servidor de Integración Continua (CI). La Auditoría Funcional (FCA) se delega a las suites de pruebas automatizadas (pruebas de humo, unitarias y de integración) que evalúan en segundos si el software cumple con los criterios de aceptación (Confirmation) pactados con el PO. Si las pruebas corren y dan estado “OK”, la conformidad funcional queda auditada y certificada técnicamente.

Debate 4: ¿Qué pasa con los reportes de estado (Status Accounting) en agilidad?

- *Resolución académica:* La actividad de recolección de metadatos históricos del repositorio de software se delega de forma directa a la **automatización inalterable de las herramientas de control de versiones distribuidas (Git)** y las plataformas de integración (ej. GitHub, Jenkins).
- Las preguntas complejas de negocio se responden consultando en segundos el historial semántico de confirmaciones (*commits*), las firmas digitales de los builds, y los tableros visuales interactivos (transparencia extrema) donde se expone el estado de avance binario (hecho o no hecho) de cada requerimiento de la iteración.

Puntos clave para el examen — 2.5.2

1. **DIFERENCIA ENTRE MVP Y MMP (AUDIO_o5 / AUDIO_o6):** Durante la clase práctica, el docente plantea una advertencia de examen crucial. En agilidad se abusa del término MVP (Producto Mínimo Viable). El **MVP tiene como único propósito validar una hipótesis de negocio** con el menor esfuerzo lúdico posible. Si el producto ya está

vendido, si el cliente sabe exactamente qué quiere y no hay hipótesis de mercado que validar, la primera entrega acotada en funcionalidad que se pone en producción no se llama MVP; se llama **MMP (Minimum Marketable Product / Producto Mínimo Comercializable)** o incremento de producción.

2. **LA VELOCIDAD SE CALCULA, NO SE ESTIMA (AUDIO_05):** Judith Meles enfatiza que la velocidad de un equipo ágil es una métrica puramente **empírica e histórica**. Se calcula de forma matemática al final de cada sprint sumando de manera rígida los Story Points de las **User Stories aceptadas al 100% por el PO**. Si una historia quedó al 90% de avance lúdico, sus puntos no suman velocidad de forma parcial para esa iteración (gestión binaria ágil: 100% o 0%).
3. **EL PGC SE ACTUALIZA (AUDIO_06):** Los docentes prácticos aclaran que el PGC no es un documento estático de piedra. A medida que avanza la materia, el equipo puede re-estimar las reglas de nombrado de nuevos ICs surgidos o retocar el repositorio, siempre y cuando dejen documentadas las versiones de cambio en la tabla de control de cambios del propio documento de PGC.

Preguntas de autoevaluación — 2.5.2

Pregunta 1: ¿Por qué se define académicamente al PGC como el “primer ítem de configuración” de un proyecto?

- **Respuesta breve:** Porque el PGC define de forma anticipada las reglas lógicas del juego de todo el desarrollo. Sin el PGC, el repositorio carece de estructura unificada, reglas de nombrado e integrantes del CCB, imposibilitando la consistencia y la integridad del producto desde el día cero.

Pregunta 2: ¿Qué placeholders sintácticos exige la cátedra para la identificación de ICs repetitivos en el PGC?

- **Respuesta breve:** Exige el uso obligatorio de los placeholders variables delimitados por signos de menor y mayor dobles (ej. <> o), los cuales deben ser reemplazados de forma física por el programador al crear el archivo.

Pregunta 3: ¿Por qué colocar la versión del archivo en su nombre físico dentro de Git es considerado un error grave en el TP4?

- **Respuesta breve:** Porque Git gestiona el historial evolutivo del versionado de forma automática por metadatos internos. Si se renombra físicamente el archivo de forma manual, Git asume que el archivo viejo fue borrado y se creó uno nuevo, destruyendo la trazabilidad histórica del repositorio.

Pregunta 4: ¿Cómo se define conceptualmente el SCM en ambientes ágiles en Agile SCM.pdf?

- **Respuesta breve:** Se define como una disciplina sin fricción (“frictionless”) orientada a coordinar y facilitar el cambio rápido de forma automática en lugar de bloquear el repositorio con burocracia, sirviendo de soporte directo a las prácticas ágiles del desarrollador.

Pregunta 5: Explique brevemente en qué consiste el patrón “Private System Build” de Steve Berczuk.

- **Respuesta breve:** Es la compilación del software ejecutada de forma local y aislada por el desarrollador en su espacio de trabajo privado para verificar sintáctica y técnicamente sus cambios antes de integrarlos al repositorio de la rama principal.

Pregunta 6: ¿Cuál es la diferencia técnica entre un “Integration Build” y un “Release Build”?

- **Respuesta breve:** El Integration Build se ejecuta de forma automatizada y frecuente (CI) tras cada commit para consolidar el trabajo del equipo; el Release Build se ejecuta esporádicamente cuando se decide empaquetar de forma formal una entrega madura para testeo o producción.

Pregunta 7: ¿Cómo se resuelve el rol del Comité de Control de Cambios (CCB) en Scrum?

- **Respuesta breve:** El CCB tradicional desaparece; el poder soberano y la autoridad técnica para decidir la prioridad de los cambios funcionales recae en el Product Owner (PO), quien prioriza las modificaciones en el Product Backlog.

Pregunta 8: ¿Cómo se gestionan las modificaciones de base de código puramente técnicas (refactorizaciones) en un equipo ágil (XP)?

- **Respuesta breve:** Quedan a total y libre discreción de los programadores de forma descentralizada. Cualquier desarrollador puede realizar una mejora en la arquitectura técnica del código para elevar su calidad sin pedir autorización externa.

Pregunta 9: ¿Qué ocurre con la documentación de proceso burocrática tradicional al aplicar Lean SCM?

- **Respuesta breve:** Se descarta en su totalidad por considerársela “desperdicio” (*waste*), priorizando la automatización de políticas mediante scripts ejecutables y tests integrados que verifiquen el sistema de manera transparente.

Pregunta 10: ¿Cómo se automatiza la Auditoría Funcional (FCA) en agilidad?

- **Respuesta breve:** Se automatiza integrando de forma continua suites de testing de aceptación en el servidor de Integración Continua. Si el build corre y el reporte de testing arroja que los casos de prueba y tests de regresión pasaron con estado “OK”, la funcionalidad queda formalmente auditada.

2.6. Casos prácticos de SCM (TP4)

2.6.1. Enunciado general y objetivos del Trabajo Práctico N° 4

El Trabajo Práctico N° 4 es el primer entregable evaluable de carácter grupal del año. Su particularidad es que **no consiste en programar un sistema para un cliente externo**, sino en aplicar de forma introspectiva la disciplina de SCM sobre la propia cursada de la materia.

Enunciado Oficial de la Cátedra:

“Aplicar los conceptos de la gestión de configuración de software (SCM) estudiados utilizando su versionador de confianza favorito (Git/GitHub de forma pública). Como dominio del proyecto, simular el desarrollo de la materia Ingeniería de Software como si fuera un proyecto de software propio en el cual el objetivo a fin de año es lograr la aprobación directa de la asignatura”.

Estructura de Entregables Obligatorios:

Para que la entrega en la UV (Universidad Virtual) sea considerada válida, cada grupo de alumnos debe presentar un único intento por grupo que contenga de forma estricta los siguientes dos entregables:

1. URL de Acceso Público al Repositorio:

- Un enlace directo y activo al repositorio Git (alojado en GitHub, GitLab o similar).
- **Condición de aprobación:** El repositorio **debe ser obligatoriamente público**. Si al momento de la corrección por parte de los docentes de práctico (como los profesores Valerio o Salva) el repositorio está privado, no se puede acceder, o arroja error 404, el grupo recibe un **DOS (2) directo, no aprobado y sin derecho a recuperatorio inmediato**.

2. Plan de Gestión de Configuración (PGC) en PDF:

- Un documento formal exportado en formato **PDF** que declare de antemano el marco administrativo del grupo. No se admite la entrega de links a Google Docs editables de forma asincrónica.
- *Secciones obligatorias del PGC:*
 - **Glosario:** Explicación unificada de siglas y extensiones de archivos.
 - **Roles y Responsabilidades:** Nombres, apellidos y legajos reales de los integrantes, asignando específicamente quién actúa como Gestor de SCM (*SCM Manager*) y quiénes integran el Comité de Control de Cambios (CCB).
 - **Estructura Física del Repositorio:** El árbol detallado de carpetas iniciales (trunk/, branches/, documentos/).
 - **Inventario de Ítems de Configuración (ICs):** Tabla estructurada con los ICs, sus extensiones unívocas y reglas de nombrado.
 - **Criterio de Línea Base:** Definición consensuada de en qué hitos del cuatrimestre se congelará el repositorio mediante etiquetas (*Tags*) de Git.

2.6.2. Casos de discusión en aula con resolución y fundamento teórico

Durante las clases prácticas de SCM (3 Practico SCM.m4a), los docentes plantean tres casos de debate clásicos que sirven para diagnosticar si el equipo asimiló la esencia de la disciplina o si se limitan a usar Git de forma automática.

A continuación se desarrolla cada uno de los tres casos estructurados de forma exhaustiva:

CASO 1: Anotaciones en PPTs (El Origen de la Autoría)

[CASO 1: ANOTACIONES EN PPTs]
¿Qué pasa si edito una diapositiva de clase?
RESOLUCIÓN CORRECTA: Es un IC de PRODUCCIÓN PROPIA

- El estudiante introduce cambios de valor lógico.
- Mutabilidad: el estado del archivo ahora depende de él.
- Su versión e integridad de origen deben ser controladas.
- **Enunciado** **planteado** **por** **el** **docente:**

*“Si yo descargo de la UV una diapositiva (PPT o PDF) de clase provista por los profesores de la cátedra para estudiar, le agrego mis anotaciones, aclaraciones personales del teórico y resúmenes propios, y luego la guardo en mi repositorio: ¿Este archivo representa un Ítem de

Configuración de Cátedra/Clase o de Producción Propia?”

- **Resolución Correcta:** Es un Ítem de Configuración de **Producción Propia**.
- **Fundamento Teórico:** La disciplina SCM se ocupa de **mantener la integridad y el origen del producto** (Product Integrity) [Bersoff]. El origen o autoría de un IC está determinado por **quién tiene la autoridad de modificar su estado lógico y quién responde por su evolución**.
 - La diapositiva original en la UV es “Material de Cátedra” porque su estado lo gobierna la profesora Judith Meles.
 - El momento en que el alumno introduce anotaciones y modificaciones de valor lúdico sobre el archivo, **la propiedad intelectual y la mutabilidad del archivo pasan a ser de su exclusiva responsabilidad**. Si el archivo se corrompe o pierde sus notas, el daño colateral impacta en el grupo de alumnos, no en la cátedra. Por ende, debe ser catalogado como producción propia del equipo y versionado bajo sus propias reglas de nombrado (ej. aplicando un prefijo del grupo como BTS_apunte_SCM_estudiante_go2.pdf).
- **Respuesta Incorrecta Común:** “Es de Cátedra/Clase porque el archivo original lo hicieron las profesoras Meles y Covaro, y nosotros solo le escribimos un par de textos encima”.
 - *Por qué está mal:* Confunde la **plantilla o insumo base** con el **entregable final de estudio**. En SCM, si vos tenés la capacidad física y la necesidad técnica de editar y versionar el estado evolutivo de un componente, ese componente es de tu propia producción.

CASO 2: Variantes de plataforma (El Límite del Proyecto)

[CASO 2: iOS vs. ANDROID (PROYECTOS)]

¿Son variantes de un IC o proyectos distintos?

RESOLUCIÓN CORRECTA: Son DOS PROYECTOS INDEPENDIENTES

- Tienen conjuntos de actividades técnicas disímiles.
- Procesos de build y despliegue (App Store vs. Play Store)

completamente separados y aislados.

- Requieren planes, estimaciones y comisiones individuales.
- **Enunciado planteado por el docente:**

*“Si como equipo de desarrollo tenemos que diseñar y codificar una aplicación para teléfonos celulares, y se define que debe funcionar tanto para el sistema operativo iOS de Apple como para Android de Google: ¿Los códigos y pantallas resultantes representan variantes de un mismo Ítem de Configuración dentro del mismo proyecto, o estamos hablando de proyectos diferentes?”

- **Resolución Correcta:** Son dos proyectos diferentes y separados.

- **Fundamento Teórico:** En el marco teórico formal de la cátedra (y la bibliografía de SWEBOK), un proyecto de software está acotado por su **conjunto de actividades técnicas, de gestión y sus criterios de despliegue**.
 - Aunque conceptualmente e identitariamente (funcionalidad) las aplicaciones de iOS y Android hagan lo mismo, **las actividades lógicas requeridas para construirlas y desplegarlas son asimétricas**.
 - Desplegar una app en la App Store de Apple exige configurar certificados de firma digital de Apple, compilar en Xcode, y someterse a revisiones de código de Apple. Desplegar en la Play Store de Android requiere herramientas, configuraciones gradle y flujos lógicos de validación totalmente diferentes.
 - Debido a que implican actividades, estimaciones, riesgos y ciclos de vida técnicos independientes, **la unidad de gestión debe ser separada**. Cada uno representa un proyecto de software autónomo, con su propio repositorio o rama principal de control.
- **Respuesta Incorrecta Común:** “Son variantes de un mismo proyecto porque la aplicación es exactamente la misma para el usuario final y funcionalmente hacen lo mismo”.
 - *Por qué está mal:* Se comete el error de confundir la **simetría funcional** (el comportamiento que ve el usuario) con la **complejidad de ingeniería del proceso**. SCM no controla “intenciones” abstractas, controla la configuración física ejecutable del producto de software [Sommerville Cap. 25].

CASO 3: Estructura del repositorio (La Planificación Inicial del PGC)

[CASO 3: ESTRUCTURA DEL REPOSITORIO (TP4)]

¿Se pueden crear las carpetas sobre la marcha?

RESOLUCIÓN CORRECTA: NO, debe estar completa

el

DÍA UNO (1)

- Evita la “bolsa de gatos” (desorden descontrolado).
- Permite que cualquier integrante nuevo asimile el orden.
- Se implementa físicamente en Git usando archivos.gitkeep.
- **Enunciado planteado por el docente:**
 - *“Como grupo de TP4, definimos que a lo largo del cuatrimestre vamos a realizar 9 trabajos prácticos. En nuestro documento de PGC detallamos que las carpetas de los prácticos 5, 6, 7, 8 y 9 las vamos a ir creando físicamente en el repositorio Git a medida que transcurra la cursada y se nos vayan asignando las consignas, para no tener el repositorio vacío. ¿Este criterio planteado en el PGC es correcto o está mal?”*
- **Resolución Correcta: Está mal conceptual y prácticamente.** La estructura física del repositorio debe estar **completamente creada y subida a Git desde el primer día** de la entrega del TP4.
- **Fundamento Teórico:** La estructura física del repositorio es la materialización del **Plan de Gestión de Configuración** (PGC). Si se autoriza a los desarrolladores a “ir creando carpetas sobre la marcha”, se destruye la predictibilidad y el control preventivo de SCM.

- Habilitar la creación incremental abre la puerta para que un integrante, por apuro o falta de comunicación, cree la carpeta TP5 con minúsculas, otro cree tp_05 y un tercero suba los archivos sueltos en el directorio raíz, generando la clásica problemática de la **pérdida de un componente** o desorden del repositorio.
- Para evitar esto, la estructura del placard (repositorio) se define y se clava físicamente de entrada. Para lograrlo en Git (que no permite subir directorios vacíos de forma nativa), se aplica la técnica del **marcador de posición** colocando un archivo vacío llamado **.gitkeep** dentro de cada carpeta vacía para obligar al versionador a registrar la estructura de primer día.
- **Respuesta Incorrecta Común:** “Es correcto crearlas incrementalmente porque se optimiza el espacio y se aclara explícitamente en una nota del PGC que se crearán a medida que sucedan las clases”.
 - *Por qué está mal:* Confunden la **carga incremental de archivos** (que sí es correcta y periódica) con la **creación sobre la marcha del mapa de directorios**. El mapa debe estar congelado y validado en la Línea Base de Planificación para asegurar el orden transversal de la cursada.

2.6.3. Tabla Comparativa de Casos Prácticos de SCM

Variable de Análisis	Caso 1: Anotaciones en PPTs	Caso 2: Variantes iOS/Android	Caso 3: Carpetas del Repositorio
Problema que previene	Falta de identificación del origen y pérdida de modificaciones del estudiante.	Caos en la planificación de builds, compilaciones cruzadas y testeos asimétricos.	Desorganización física en el repositorio central (“bolsa de gatos”).
Atributo SCM Clave	Autoría y Mutabilidad (quién responde por el estado del archivo).	Unidad de Gestión y Alcance (separación de actividades técnicas de despliegue).	Control Preventivo e Infraestructura (estructura planificada e inalterable).
Técnica de Implementación	Aplicar regla de nombrado unívoca de Producción Propia con placeholders variables en el PGC.	Crear dos repositorios o planes SCM aislados con sus propias líneas.gitk base operacionales.	Subir todo el árbol de directorios el Día 1 utilizando archivos ocultos eep.
Consecuencia en Cursada	Error de mapeo en el inventario del PGC (confundir base teórica).	Desaprobación teórica en parciales al no saber justificar el “depende” fi ingenieril.	Desaprobación directa del TP4 práctico por inconsistencia sica en GitHub.

Puntos clave para el examen — 2.6.3

1. **EL REPOSITORIO DEBE SER SQUEAKY CLEAN DESDE EL DÍA 1 (AUDIO_03 / AUDIO_06):** Durante la corrección de errores típicos, el docente de práctico recalca que es motivo de baja nota directa que el repositorio de GitHub contenga archivos “basura” sueltos en la raíz (ej: archivos.DS_Store de Mac, carpetas.idea del IDE o archivos temporales de Word ~\$PGC.docx). Es obligatorio configurar de entrada un archivo **.gitignore** bien detallado para filtrar estos accidentes.
2. **CONMUTACIÓN DE ROLES (AUDIO_03):** El docente recalca que las personas reales de tu grupo “se instancian” en los roles de SCM. No es aceptable que una sola persona concentre todas las decisiones. Debe quedar claro en las minutas y el PGC quién actúa como desarrollador y quién audita como Gestor de SCM, garantizando la independencia de criterios para las firmas del CCB.
3. **POLÍTICA DE COMMITS PERIÓDICOS (AUDIO_03):** No sirve de nada que el repositorio esté impecable si tiene solo dos commits: uno el día que crearon el repositorio y otro el día de la entrega final a fin de año. Los docentes prácticos auditarán la historia de Git para verificar que **todos los integrantes del grupo hayan realizado aportes lógicos de forma sistemática y periódica** a lo largo de las iteraciones de la materia.

Preguntas de autoevaluación — 2.6.3

Pregunta 1: ¿Cuál es el dominio real del Trabajo Práctico N° 4 en la materia?

- **Respuesta breve:** El dominio es la simulación de la propia cursada de la asignatura de Ingeniería de Software, tratándola formalmente como si fuera un proyecto de software cuyo entregable final de calidad es la aprobación directa de los estudiantes.

Pregunta 2: ¿Por qué la entrega de un repositorio de GitHub configurado como “Privado” se penaliza con un dos (2) directo sin recuperación?

- **Respuesta breve:** Porque impide físicamente que el equipo docente pueda navegar el repositorio, auditar el historial de commits y certificar el cumplimiento de los criterios de SCM declarados en el PGC.

Pregunta 3: Si un grupo utiliza Git para versionar sus entregables, ¿cómo debe ser la regla de nombrado de sus archivos físicos de código fuente?

- **Respuesta breve:** El nombre físico del archivo de código en el repositorio debe estar limpio y libre de números de versión (ej: calendario.py), dado que Git gestiona y rastrea la evolución cronológica de las versiones por metadatos internos de manera transparente.

Pregunta 4: ¿Por qué un apunte de clase descargado de la UV y modificado por el estudiante para estudiar califica como “Producción Propia”?

- **Respuesta breve:** Porque el estudiante introdujo valor lógico y anotaciones que alteraron su estado. Al ser el responsable de su mantenimiento y de que no se pierdan sus notas, la autoría y la mutabilidad del archivo pasaron a ser de su grupo.

Pregunta 5: ¿Por qué el desarrollo de una app para iOS y otra para Android representa dos proyectos independientes y no variantes del mismo?

- **Respuesta breve:** Porque implican conjuntos de actividades técnicas, de compilación, de herramientas de build y procesos de certificación de despliegue en tiendas de aplicaciones (App Store vs Play Store) completamente asimétricos y separados.

Pregunta 6: ¿Qué es un archivo.gitkeep y qué función técnica cumple en el TP4?

- **Respuesta breve:** Es un archivo marcador de posición vacío que se coloca dentro de los directorios vacíos para obligar a Git a rastrearlos y subirlos al servidor central, permitiendo consolidar la estructura del repositorio desde el primer día.

Pregunta 7: ¿Cuál es el error de gestión más común al redactar la tabla de Ítems de Configuración en el PGC?

- **Respuesta breve:** Definir reglas de nombrado rígidas y estáticas para ítems repetitivos, omitiendo el uso obligatorio de placeholders variables encerrados en signos de menor y mayor dobles (ej: <> o).

Pregunta 8: ¿Qué significa que los trabajos prácticos de la cátedra de SCM “no tienen recuperación”?

- **Respuesta breve:** Significa que la nota que obtiene el grupo en la UV tras la corrección del docente es definitiva; si es menor a 7, el grupo pierde la opción de la aprobación directa y queda en condición de regular para el examen final.

Pregunta 9: ¿Por qué es obligatorio que figuren los nombres y apellidos reales de las personas en el PGC del TP4?

- **Respuesta breve:** Para garantizar la rendición de cuentas (*accountability*) y que los procesos administrativos se ejecuten realmente en la cursada, evitando la abstracción de roles genéricos que nadie asume de forma práctica.

Pregunta 10: ¿Qué criterio auditan los profesores en el práctico de SCM a fin de cuatrimestre además de la completitud de las carpetas?

- **Respuesta breve:** Auditan la periodicidad de las contribuciones, comprobando mediante el historial de commits de Git que todos los integrantes del grupo hayan subido aportes de forma sistemática durante todo el año, y no todo junto al final.

UNIDAD 3 - Requerimientos ágiles: user stories y estimaciones ágiles

3.1. Requerimientos ágiles: Introducción y bases del Manifiesto Ágil

3.1.1. Filosofía ágil y el Manifiesto Ágil (2001)

3.1.1.1. Definición precisa de “Ágil”

- **Definición de la Cátedra (SLIDE_03 / NC_MartinBosi):** Ágil NO es una metodología, ni un proceso rígido, ni una receta de cocina. Ágil es una **ideología o pensamiento** estructurado bajo un conjunto definido de valores y principios que guían el desarrollo de productos de software en entornos de alta volatilidad.
- **Definición de Ian Sommerville (BIBLIO_ - Cap. 3.1):** Los métodos ágiles son enfoques de desarrollo incremental de software enfocados en simplificar el proceso burocrático, evitar tareas con valor dudoso a largo plazo y eliminar documentación innecesaria que probablemente nunca se use.
- **Qué problema resuelve:** Destruye la utopía tradicional del **BRUF (Big Requirements Up Front / Grandes Requerimientos de Entrada por Adelantado)**. Evita que el equipo pase meses o años redactando documentos de requerimientos masivos y rígidos que quedan obsoletos al instante en que se tira la primera línea de código, lo que crónicamente causaba desvíos de costos, retrasos y software inútil que no satisfacía las necesidades reales del usuario.

3.1.1.2. El origen histórico del Manifiesto Ágil

- **La técnica / El hito:** En febrero de **2001**, un grupo de 17 desarrolladores de software líderes en la industria (incluyendo a Ken Schwaber y Jeff Sutherland de Scrum, Kent Beck de Extreme Programming - XP, Mike Cohn, y Alistair Cockburn de Crystal, entre otros) se reunieron en una estación de esquí en Utah, EE. UU.. Cansados de los fracasos crónicos de los proyectos tradicionales, consensuaron un acuerdo político y profesional formalizado en la web **agilemanifesto.org**. Este pacto define **cuatro valores centrales** y **doce principios** que actúan como un compromiso obligatorio de calidad para cualquier framework que se denomine ágil.

Los 4 Valores del Manifiesto Ágil (Explicados uno por uno)

La infografía oficial de la cátedra resalta la regla dorada de lectura: “*Valoramos más lo que está a la izquierda (arriba en la infografía) que lo que está a la derecha (abajo)*”. Valorar los elementos de la izquierda **no significa anular o prohibir los de la derecha**, sino que ante un conflicto de intereses o priorización, la balanza de ingeniería siempre debe inclinarse hacia la izquierda.

|| LOS 4 VALORES ÁGILES ||

1. INDIVIDUOS E INTERACCIONES -> Valorados por sobre -> Procesos y herramientas
2. SOFTWARE FUNCIONANDO -> Valorado por sobre -> Documentación extensiva

- 3. COLABORACIÓN CON CLIENTE -> Valorada por sobre -> Negociación contractual
- 4. RESPUESTA ANTE EL CAMBIO -> Valorada por sobre -> Seguir un plan rígido

Valor 1: Individuos e interacciones por sobre procesos y herramientas

- *Explicación:* El factor más crítico de éxito en un proyecto de software es la calidad humana del equipo y los vínculos que se establecen entre ellos. De nada sirve comprar la herramienta de automatización o el versionador de código más costoso si los desarrolladores están desmotivados o no se comunican de forma efectiva. Se le da poder al equipo para que se autogestione y resuelva sus conflictos de forma directa, sin depender de jefes de proyecto rígidos.

Valor 2: Software en funcionamiento por sobre documentación exhaustiva (o extensiva)

- *Explicación:* Lo único que realmente le genera valor de negocio al cliente y resuelve sus problemas operativos es el **software funcionando** en producción.
- *Advertencia del docente contra la mala interpretación de la industria (AUDIO_01 / NC_Ardiles):* Judith Meles remarca que **Scrum y el agilismo nunca dijeron “no hay que documentar”**. Eso es un facilismo e indisciplina del mercado. El manifiesto dice que preferimos el código ejecutable antes que la documentación detallada e inmutable de especificaciones. Hay que documentar “lo justo y necesario” (documentación lean) para asegurar la trazabilidad y la mantenibilidad futura del producto, no para engrosar manuales burocráticos inertes.

Valor 3: Colaboración con el cliente por sobre negociación contractual

- *Explicación:* Las relaciones contractuales tradicionales se basan en deslindar responsabilidades (ej. “vos me firmaste este contrato y yo te entrego esto aunque no te sirva” o “me equivoqué al pedirte esto pero ya está firmado”). El agilismo propone que el cliente (representado en Scrum por el rol del **Product Owner - PO**) se integre de forma diaria y activa al equipo de desarrollo. La responsabilidad del éxito del software es compartida de forma colaborativa, no una batalla de abogados.

Valor 4: Respuesta ante el cambio por sobre seguir un plan

- *Explicación:* Se acepta como una premisa fundamental que **los requerimientos van a cambiar de forma inevitable** durante el proyecto. El cambio no es un enemigo que deba evitarse o burocratizarse mediante engorrosos flujos de control de cambios del CCB tradicional. Los procesos cambiantes representan una ventaja competitiva si el equipo está preparado técnicamente para actuar sobre ellos con rapidez.

Los 12 Principios del Manifiesto Ágil (Completo)

Durante el dictado de la materia, la cátedra describe los doce principios que complementan a los cuatro valores.

1. **Nuestra mayor prioridad es satisfacer al cliente** a través de la entrega temprana y continua de software con valor.
2. **Aceptamos que los requisitos cambien**, incluso en etapas tardías del desarrollo. Los procesos ágiles aprovechan el cambio para proporcionar ventaja competitiva al cliente.
3. **Entregamos software funcional frecuentemente**, entre dos semanas y dos meses, con preferencia al periodo de tiempo más corto.

4. **Los responsables de negocios y los desarrolladores deben trabajar juntos** de forma cotidiana durante todo el proyecto (Técnicos y no técnicos).
5. **Desarrollamos los proyectos en torno a individuos motivados.** Hay que darles el entorno y el apoyo que necesitan, y confiarles la ejecución del trabajo.
6. **El método más eficiente y efectivo de comunicar información** al equipo de desarrollo y entre sus miembros es la conversación cara a cara.
7. **El software funcionando es la medida principal** de progreso.
8. **Los procesos ágiles promueven el desarrollo sostenible.** Los promotores, desarrolladores y usuarios deben ser capaces de mantener un ritmo constante de forma indefinida.
9. **La atención continua a la excelencia técnica y al buen diseño** mejora la agilidad.
10. **La simplicidad** —o el arte de maximizar la cantidad de trabajo no realizado— es esencial.
11. **Las mejores arquitecturas, requisitos y diseños emergen de equipos autoorganizados.**
12. **A intervalos regulares, el equipo reflexiona sobre cómo ser más efectivo** para a continuación ajustar y perfeccionar su comportamiento en consecuencia.

Los 5 Principios más relevantes para la Gestión de Requerimientos Ágiles

La profesora Judith Meles es sumamente taxativa al remarcar que, para entender por qué los requerimientos ágiles funcionan de manera incremental, **existen cinco principios del manifiesto que configuran su sustento conceptual básico** y que son de evaluación obligatoria:

- **Principio 1 (Satisfacción del cliente mediante releases frecuentes):** Satisface al cliente entregándole porciones de funcionalidad utilizables cada 2 o 4 semanas, en lugar de hacerlo esperar al final del proyecto.
- **Principio 2 (Aceptar requerimientos cambiantes):** El agilismo no penaliza el cambio; lo abraza como una ventaja competitiva del negocio del cliente.
- **Principio 3 (Técnicos y no técnicos trabajando juntos diariamente):** Representa el fin de la barrera de comunicación. En Scrum, el Product Owner (no técnico) y los Product Developers (técnicos) coordinan tareas todos los días, asumiendo el PO la responsabilidad del “qué” y el equipo la del “cómo” construir.
- **Principio 6 (Conversación cara a cara):** Permite que fluya información vocal, subvocal, gestual y con realimentación inmediata. Elimina el desperdicio de interpretar largas especificaciones textuales frías. Se asocia de manera directa al componente “Conversación” de las User Stories.
- **Principio 11 (Emergencia de requerimientos de equipos autoorganizados):** Las decisiones las toma el equipo que está en la cancha y no un superior jerárquico alejado de la realidad del código. Los requerimientos “emergen” y se descubren en la marcha de forma empírica.

3.1.2. Procesos Empíricos vs. Procesos Definidos (La base del empirismo)

3.1.2.1. Definición precisa

- **Procesos Definidos (Predictivos):** Son modelos basados en la teoría del planificador rígida. Asumen que el desarrollo de software es un proceso físico repetible, predecible e inalterable, donde si se siguen exactamente los mismos pasos teóricos descritos en un manual de procesos (ej. el PUD tradicional o RUP), se obtendrán resultados idénticos en plazos y costos estimados.
- **Procesos Empíricos (Adaptables):** Son modelos de trabajo basados en el **empirismo lógico: experimentar, medir y aprender sobre la base de los resultados reales obtenidos**. Asumen que el desarrollo de software es un proceso único, creativo y de alta incertidumbre, donde la experiencia previa de un equipo no se puede extrapolar directamente a otro porque el contexto del cliente, la tecnología y el factor humano son irrepetibles.

3.1.2.2. Los tres pilares del empirismo ágil (Ken Schwaber)

Meles remarca en clase que para que un equipo pueda operar de forma empírica, **debe respetar mandatoriamente los tres pilares de base:**

PILARES DEL EMPIRISMO
[TRANSPARENCIA] [INSPECCIÓN] [ADAPTACIÓN]

Visibilidad extrema para que Puntos de control continuos Capacidad y libertad

todos conozcan el estado real (ej. Demos, retrospectivas) del equipo para ajustar

de la calidad del software. para verificar desviaciones. el proceso de forma JIT.

1. **Transparencia (Transparency):** Todos los aspectos del proceso deben ser visibles y comprensibles para aquellos que son responsables de los resultados (ej. tableros visuales interactivos, código compartido, honestidad en los desvíos).
2. **Inspección (Inspection):** Los artefactos de software y el proceso de trabajo deben inspeccionarse de manera frecuente y diligente para detectar desviaciones o problemas de calidad que atenten contra el objetivo.
3. **Adaptación (Adaptation):** Si el inspector detecta que uno o más aspectos del proceso se desvían de los límites aceptables y el producto final será inconsistente, el proceso o los materiales deben ser ajustados de inmediato de forma autónoma por el equipo.

Tabla de Contraste: Procesos Predictivos (Definidos) vs. Procesos Empíricos (Ágiles)

Variable de Comparación	Procesos Predictivos / Definidos (Ej. PUD / RUP)	Procesos Empíricos (Ej. Scrum, Kanban)
Premisa de la Experiencia	Extrapolable. Asume que la experiencia de proyectos anteriores se puede estandarizar y aplicar linealmente mediante métricas rígidas.	No extrapolable. Cada proyecto, equipo y cliente configuran una realidad única e irrepetible.
Modelo de	Dirigido por el plan (Plan-	Dirigido por el valor (Value-

Variable de Comparación	Procesos Predictivos / Definidos (Ej. PUD / RUP)	Procesos Empíricos (Ej. Scrum, Kanban)
Gestión	driven). Se define el alcance fijo de entrada; tiempo y recursos se estiman de forma lineal.	driven). Tiempo y recursos son fijos (timebox); el alcance es variable.
Rol de la Documentación	Exhaustiva y masiva al inicio del proyecto (BRUF), intentando predecir el 100% de la funcionalidad.	Lean y “Just in Time” (justo a tiempo), especificando al detalle únicamente lo que se va a programar en el sprint.
Estructura de Autoridad	Jerárquica. La figura central es el Jefe de Proyecto o Supervisor que asigna tareas y controla tiempos.	Descentralizada. Equipos autoorganizados, empoderados y horizontales sin jefes intermedios.
Control de Cambios	Restictivo y burocrático. Se intenta evitar la mutabilidad a través de comités CCB tradicionales.	Flexible. El cambio de requerimientos se abraza como una ventaja competitiva del negocio.
Ciclo de Vida del Software	Tradicionalmente en cascada (<i>waterfall</i>) o iteraciones de alcance fijo que postergan el feedback.	Iterativo e Incremental puro, entregando software potencialmente desplegable al final de cada iteración.

Ejemplos concretos de la cátedra para el debate de aula:

- **El ejemplo de la Universidad de California en Irvine (Inspección empírica):** El docente de teoría utiliza este caso para ilustrar la diferencia conceptual. Los planificadores tradicionales (proceso definido) habrían diseñado caminos de hormigón rígidos de antemano sobre planos teóricos, forzando a los estudiantes a caminar por donde el arquitecto imaginó. La universidad aplicó un enfoque empírico: sembraron pasto en todo el predio, esperaron un año a observar por dónde caminaba naturalmente la gente haciendo “caminito” (inspección de la realidad), y recién ahí pavimentaron de forma definitiva sobre esos senderos empíricos (adaptación).
- **El ejemplo del partido de fútbol Argentina vs. España (AUDIO_01):** La profesora explica que el empirismo asume que un proyecto de software es como un partido de fútbol. Es único e irrepetible. De nada sirve tener un “plano rígido de juego” memorizado al inicio si el rival mete un gol a los dos minutos o te expulsan un jugador. El equipo requiere feedback continuo en tiempo real (inspección en el campo de juego) y tomar decisiones de ajuste sobre la marcha, aceptando que la experiencia del partido anterior ayuda, pero no se puede extrapolar de forma matemática lineal al partido actual.

3.2. Requerimientos en el Paradigma Ágil

3.2.1. El Concepto de “Valor” y la diferencia entre Output y Outcome

3.2.1.1. Definición precisa

- **Output (La Salida / El entregable físico):** Es la cantidad de elementos o entregables técnicos que el equipo produce físicamente (ej. cantidad de líneas de código escritas, número de clases creadas, cantidad de pantallas programadas o cantidad de tareas marcadas como terminadas).
- **Outcome (El Resultado de valor / El beneficio de negocio):** Es el beneficio real, tangible y cuantificable que el usuario final y el negocio reciben cuando interactúan con el software para resolver su necesidad operativa (ej. reducción del tiempo de facturación en un 30% o la autogestión de turnos de forma digital).
- **Qué problema resuelve:** Erradica el síndrome de la “**fábrica de características**” (**Feature Factory**). Ataca el mal del informático que programa con la cabeza agachada sin importarle para qué sirve su software en el negocio. Forzar la mirada en el **outcome** evita que el equipo desperdicie recursos construyendo software técnicamente impecable y robusto que luego nadie usa en producción porque no genera valor real.

3.2.1.2. Cómo se hace (La técnica del 80/20 de Pareto y Standish Group)

Para maximizar el valor y reducir el desperdicio lógico (Lean), el Product Owner debe gestionar el Product Backlog basándose en la **ley de Pareto (80/20)** aplicada al software.

- **Estadísticas de Jim Johnson (Standish Group, XP 2002)** presentadas por la cátedra:
 - **45% de las funcionalidades** de un sistema de software de tipo corporativo **NUNCA se usan**.
 - **19% de las funcionalidades** se usan rara vez.
 - **16% de las funcionalidades** se usan ocasionalmente o algunas veces.
 - **13% de las funcionalidades** se usan a menudo.
 - **7% de las funcionalidades** se usan siempre.

|| DESPERDICIO EN SISTEMAS TRADICIONALES ||
[NUNCA SE USA: 45%] -> [RARA VEZ: 19%] -> [ALGUNAS VECES: 16%] -> [OFTEN/ALWAYS: 20%]

64% de DESPERDICIO de código

(Nunca o casi nunca se usa)

- **La regla de ingeniería:** El **64% del software corporativo desarrollado bajo enfoques tradicionales representa desperdicio lógico puro** (nunca o casi nunca se usa). El agilismo se enfoca en priorizar estrictamente el backlog para que los primeros sprints implementen únicamente el **20% de la funcionalidad que concentra el 80% del valor de negocio** de los usuarios.
- **Ejemplo de clase:** El docente pregunta al aula qué porcentaje de las funciones lógicas de **Microsoft Excel** o Word utilizan para estudiar o trabajar. La mayoría de los alumnos no

supera el 3% o 5% de uso real del paquete de oficina. El resto de las características hipercomplejas de Excel representa desperdicio técnico para el 95% de los usuarios comunes.

- **Warnings / Errores comunes:** Caer en la tentación de programar “por las dudas” o por “confort técnico” agregando campos, botones o validaciones que el Product Owner no priorizó formalmente en el backlog, asumiendo erróneamente que “más código equivale a más valor”.

3.2.2. El 50% de los requerimientos emergentes

3.2.2.1. Definición conceptual

- **Definición de la Cátedra (SLIDE_03 / NC_Ardiles):** Es el principio empírico que postula que **el 50% de los requerimientos de un sistema de software no se conocen al inicio del proyecto** porque son de naturaleza emergente. Aparecen, maduran y se descubren de forma inevitable únicamente cuando el usuario interactúa y empieza a operar físicamente el software en un entorno real de producción.

EL MAPA REAL DE LOS REQUERIMIENTOS
[EL PRIMER 50%] [EL SEGUNDO 50%]

Identificable al inicio de forma **REQUERIMIENTOS EMERGENTES**

preventiva por el equipo técnico. Emergen de forma empírica en el proceso de uso real del software.

- **Qué problema resuelve:** Elimina la parálisis por análisis. Destruye la mala práctica tradicional de retrasar el inicio de la programación intentando “adivinar” cómo se comportará el negocio en el futuro.
- **Cómo se hace (Procedimiento JIT - Just in Time):**
 1. Se asume la imperfección y la falta de información al inicio del proyecto.
 2. Se especifica detalladamente únicamente el 50% identificable de forma temprana para los primeros sprints.
 3. Se difiere el análisis detallado de los requerimientos restantes hasta el último momento responsable (Just in Time), que es justo antes de comenzar el desarrollo del sprint respectivo.
 4. Se libera una versión funcional rápida a producción para que el cliente la use, de modo que el 50% emergente comience a materializarse mediante retroalimentación temprana.

Puntos clave para el examen — 3.2.2.1

1. **ÁGIL NO SIGNIFICA RÁPIDO (AUDIO_01):** Meles recalca que el error original o la “manzana que mordió Eva” en la industria es pensar que ágil significa rápido. No necesariamente se desarrolla más rápido; lo que se logra es **entregar valor temprano y reducir drásticamente el riesgo de tirar dinero a la basura** construyendo software innecesario o erróneo.

2. **GESTIÓN BINARIA DE HISTORIAS (AUDIO_05):** En agilidad no existe el “falta poco” ni el avance porcentual formal para medir la velocidad de entrega. Un requerimiento o historia de usuario está **100% terminada y aceptada por el Product Owner, o bien tiene un 0% de avance lúdico** para el cálculo de la velocidad del sprint. Los puntos intermedios no se computan de forma parcial.
3. **EL SCRUMMASTER NO ES UN JEFE (AUDIO_01 / NC_Ardiles):** Las decisiones de diseño, estimación y de proceso en los entornos empíricos son tomadas de manera horizontal y colectiva por el **equipo autoorganizado (product developers)**. El ScrumMaster no asigna tareas a dedo ni actúa como supervisor jerárquico tradicional.

Preguntas de autoevaluación — 3.2.2.1

Pregunta 1: ¿Qué es el Manifiesto Ágil de 2001 y qué elementos básicos lo integran?

- **Respuesta breve:** Es el hito fundacional consensual que unifica la ideología de desarrollo ágil. Está integrado de forma obligatoria por **cuatro valores centrales** y **doce principios** de ingeniería y gestión.

Pregunta 2: Explique la regla de lectura de los 4 valores del manifiesto y dé un ejemplo.

- **Respuesta breve:** Se lee valorando más los elementos de la izquierda (ej. individuos e interacciones) que los de la derecha (ej. procesos y herramientas). No anula los elementos de la derecha, sino que prioriza de manera estratégica a los de la izquierda ante un conflicto de diseño o gestión.

Pregunta 3: ¿Por qué la afirmación “en Scrum no se documenta” es una mala interpretación de la agilidad según Meles?

- **Respuesta breve:** Porque el manifiesto prioriza el software en funcionamiento sobre la documentación detallada, pero en ningún momento prohíbe documentar. Exige una documentación lean, pragmática y de valor que asista de forma real al mantenimiento, sin burocracia inútil.

Pregunta 4: ¿Cuál es la diferencia conceptual entre un “Proceso Definido” y un “Proceso Empírico”?

- **Respuesta breve:** El definido asume predictibilidad, repetibilidad y que la experiencia es extrapolable a otros equipos; el empírico asume que cada proyecto es único, requiere control basado en el ensayo y error en tiempo real, y que la experiencia no se extrapola linealmente.

Pregunta 5: Enumere los tres pilares fundamentales que sustentan al empirismo según Ken Schwaber.

- **Respuesta breve:** Los tres pilares son la **Transparencia**, la **Inspección** y la **Adaptación**.

Pregunta 6: Explique la analogía del “caminito de pasto de Irvine” descrita por el docente en clase.

- **Respuesta breve:** Ilustra el empirismo: en lugar de pavimentar de antemano de forma teórica y rígida (proceso definido), sembraron pasto, esperaron a ver de forma transparente por dónde caminaba la gente en la realidad (inspección), y recién ahí pavimentaron los senderos empíricos (adaptación).

Pregunta 7: ¿Cuál es la diferencia de negocio fundamental entre el “Output” y el “Outcome”?

- **Respuesta breve:** El **Output** es la salida técnica física producida por el programador (ej. líneas de código, tareas hechas); el **Outcome** es el resultado de negocio y beneficio real que percibe el usuario al interactuar con esa salida.

Pregunta 8: Según las estadísticas de Jim Johnson (Standish Group), ¿qué porcentaje de funcionalidad de un software tradicional representa desperdicio lógico directo?

- **Respuesta breve:** El **64% de la funcionalidad** representa desperdicio directo, ya que un 45% de las características nunca se utilizan y un 19% se utilizan muy rara vez en producción.

Pregunta 9: ¿Qué significa conceptualmente que “el 50% de los requerimientos de software son emergentes”?

- **Respuesta breve:** Significa que es imposible conocer y planificar el 100% del alcance al inicio del proyecto, ya que la mitad de los requerimientos verdaderos emergen y se descubren de forma empírica únicamente cuando el usuario empieza a operar el sistema real.

Pregunta 10: ¿Cómo se aplica la técnica “Just in Time” (JIT) a la especificación de requerimientos ágiles?

- **Respuesta breve:** Consiste en diferir el análisis detallado y exhaustivo de cada requerimiento hasta el último momento responsable de gestión, que es justo antes de que se inicie el sprint respectivo de desarrollo, evitando documentar funcionalidades que podrían cambiar o descartarse.

3.3. Estructura de una User Story: Tarjeta, Conversación, Confirmación y Criterios de Aceptación

3.3.1. Las 3 C’s de las User Stories (Kent Beck / Ron Jeffries)

- **Definición de las notas de clase (NC_Ardiles / NC_MartinBosi):** Las tres dimensiones interdependientes de una Historia de Usuario (US) se abrevian como las **3 C’s: Tarjeta (Card), Conversación (Conversation) y Confirmación (Confirmation)**. Ron Jeffries (2001) acuñó esta aliteración para denotar que una US no es simplemente un texto escrito, sino un proceso colaborativo dinámico.

A continuación se detalla y desarrolla cada una de las 3 C’s de forma separada y en profundidad:

1ª C: Tarjeta (Card)

1ª C: TARJETA
(Al Frente)

- Título (Frase verbal en infinitivo)
- Descripción (Sintaxis Connextra con Rol)
- Notas para recordar temas abiertos en la mesa
- **Definición precisa:** Es el componente físico y la manifestación tangible inicial de la Historia de Usuario. Mike Cohn (Capítulo 1) e Ian Sommerville (Capítulo 3) la definen formalmente como una descripción sumamente corta y de alto nivel de una necesidad de negocio, escrita tradicionalmente a mano en tarjetas bibliográficas de cartón para actuar como un recordatorio (*token* o placeholder) de que debe sostenerse una conversación detallada en el futuro.

- **Qué problema resuelve:** Erradica la “falsa ilusión de precisión” de las especificaciones tradicionales. Al estar severamente acotada por el espacio físico del cartón, impide que el analista escriba requerimientos hiperdetallados de manera prematura (lo cual genera un costoso inventario de requerimientos obsoletos).
- **La técnica (Sintaxis estándar de Connextra):** Se escribe utilizando una plantilla estructurada que incorpora de forma obligatoria el rol de negocio del usuario:

Como <>,

quiero <>

de forma tal que <>

- *Placeholders y variables obligatorios de la plantilla:*
 - <>: Representa de forma específica quién realiza la acción o quién recibe el valor en el negocio.
 - <>: Describe la acción concreta que realizará el sistema de soporte.
 - <>: Comunica por qué es necesaria la actividad y qué beneficio concreto le aporta al cliente o a la organización.

Ejemplos completos escritos:

- *Ejemplo de Clase (Buscar Destino por Dirección):*
 - **Título (Frase verbal):** Buscar Destino por Dirección.
 - **Tarjeta:** Como Conductor quiero buscar un destino a partir de una calle y altura para poder llegar al lugar deseado sin perderme.
- *Ejemplo de Bibliografía (Pagar con Tarjeta - Cohn):*
 - **Título (Frase verbal):** Pagar Búsqueda con Tarjeta.
 - **Tarjeta:** Como Compañía quiero pagar por una búsqueda de puestos con una tarjeta de crédito, así resuelvo mi necesidad en forma más eficiente.
- **Clasificaciones de Tarjetas:**
 - *Épicas (Epics):* Historias sumamente grandes, abstractas y complejas que deben ser desagregadas (partidas) en historias más pequeñas para poder ser consumidas en un sprint [110, Image 69].
 - *Historias de Usuario comunes:* Historias con el tamaño ideal para ser estimadas y desarrolladas.
 - *Spikes:* Tipo especial de historia de investigación orientada a reducir riesgos técnicos o funcionales del proyecto.
- **Errores comunes / Advertencias de Judith Meles:**
 - △ *No abuses de la palabra “Y”:* Si la acción de la tarjeta incluye un “y” (ej. “quiero registrarme y pagar”), estás ante una historia compuesta que debe ser rota en dos tarjetas separadas inmediatamente [11, 40, Image 40].

-  *El actor no es el software*: Nunca escribas “Como sistema quiero...” o “Como aplicación...”. La historia se escribe **siempre** desde la perspectiva del ser humano de carne y hueso que opera el negocio lúdico.
- **Relación con otros conceptos**: La tarjeta actúa como un ítem de planificación en el Product Backlog que el Product Owner prioriza dinámicamente según su valor económico.

2ª C: Conversación (Conversation)

2ª C: CONVERSACIÓN

(En la Daily/Refinamiento)

- Cara a cara entre Product Owner y Equipo
- Generación de entendimiento compartido
- Transferencia empírica de conocimiento
- **Definición precisa**: Es el intercambio verbal, colaborativo y dinámico de ideas que se produce de forma cotidiana entre el cliente (Product Owner), los usuarios reales y el equipo de desarrollo para descubrir los detalles del requerimiento.
- **Qué problema resuelve**: Resuelve la brecha de comunicación e interpretación del lenguaje escrito. Las especificaciones textuales tradicionales son frías y propensas a múltiples interpretaciones. La conversación cara a cara permite una realimentación inmediata, aclarando suposiciones en minutos y unificando la visión técnica del equipo con la necesidad del negocio.
- **Cómo se hace**: Se ejecuta a través de reuniones periódicas de refinamiento del backlog (*backlog refinement*) y de forma directa en el área de trabajo durante el sprint. Se prioriza la conversación presencial (cara a cara) y el uso de pizarras analógicas.
- *Transcripción Dudosa aclarada por el docente*:

[TRANSCRIPCIÓN DUDOSA: En el audio “4.Rqs ágiles y usar stories.m4a” figura “La conversación no queda en ningún lugar, o sea, en ningún lugar. No queda en la cabeza”. Se aclara y corrige académicamente: El docente enfatiza que la conversación como tal es efímera y no se redacta de forma textual redundante. “No queda en un papel frío”, sino que queda grabada como conocimiento adquirido en la mente de los integrantes lúdicos que participaron de forma activa, y sus acuerdos concretos se plasman al dorso de la tarjeta en forma de Confirmación].

- **Errores comunes**:
 -  *El silencio del programador*: Asumir que “con leer la tarjeta alcanza para programar” y no consultar al Product Owner sobre los casos de borde del requerimiento.
 -  *Reemplazar la voz por herramientas*: Intentar resolver las dudas complejas mandándose interminables correos electrónicos o comentarios en Jira/Trello en lugar de levantarse y hablar cara a cara.
- **Relación con otros conceptos**: La conversación es la que dota de contenido dinámico a los **Criterios de Aceptación** de la historia.

3ª C: Confirmación (Confirmation)

3ª C: CONFIRMACIÓN (Al Dorso)

- Criterios de Aceptación (Condiciones del PO)
- Pruebas de Aceptación de Usuario (pasa/falla)
- Criterio de Hecho (Definition of Done)
- **Definición precisa:** Es el componente de la User Story que representa los acuerdos y criterios cuantitativos que el Product Owner utilizará para comprobar empíricamente que la funcionalidad se ha construido y funciona exactamente como el negocio lo requiere.
- **Qué problema resuelve:** Desierre la subjetividad del Product Owner al momento de aceptar o rechazar el incremento de software en la Sprint Review. Brinda al desarrollador un criterio de parada inequívoco (*Definition of Done*) para saber cuándo detenerse de agregar código, y provee al área de testing los insumos objetivos para programar las pruebas de regresión.
- **Cómo se hace (La técnica en dos pasos de la UTN-FRC):**
 1. **Identificar los Criterios de Aceptación (AC)** al dorso de la tarjeta física.
 2. **Diseñar las Pruebas de Aceptación de Usuario** de alto nivel utilizando la sintaxis de infinitivo de la cátedra.

Sintaxis Estándar exigida para las Pruebas de Aceptación (UTN-FRC):

Probar <> + <> + (pasa/falla)

- (pasa): Representa un escenario feliz o de éxito donde el sistema debe aceptar la transacción de forma consistente.
- (falla): Representa un escenario de error o caso de borde donde el sistema debe rechazar la transacción y controlar la anomalía de manera segura.

Ejemplo de Confirmación (dorso de la tarjeta) para la historia “Buscar Destino por Dirección”:

- **Criterios de Aceptación (AC):**
 - AC1: La altura de la calle debe ser un número entero positivo.
 - AC2: El sistema debe permitir la ausencia de altura de calle (sin número, ej: “Maestro Marcelo López sin número”).
 - AC3: La búsqueda no puede demorar más de 30 segundos (Requerimiento No Funcional de performance).
- **Pruebas de Aceptación:**
 - Probar buscar un destino con una calle, país y ciudad existentes y con una altura de calle que sea un número entero (pasa).
 - Probar buscar un destino ingresando letras en el campo de altura de calle (falla).
 - Probar buscar un destino ingresando “sin número” o dejando el campo de altura vacío (pasa).

- Probar buscar un destino y verificar que la respuesta del mapa tarde menos de 30 segundos (pasa).
- **Errores comunes / Advertencias:**
 -  *No metas código en las pruebas:* Está mal redactar pruebas diciendo “probar que se ejecute la validación javascript” o “probar que la base de datos tire null”. Las pruebas de aceptación son de **alto nivel** y se expresan 100% en el idioma del negocio del PO.
 -  *Diseñar solo pases:* Omitir los escenarios de falla. Una Confirmación robusta requiere tanto pruebas que pasen como pruebas que fallen para certificar la estabilidad de la validación.
- **Relación con otros conceptos:** La Confirmación es el punto de partida directo para que el equipo de Testing diseñe la suite de pruebas funcionales automatizadas en la Unidad 4.

3.3.2. El Modelo INVEST (Bill Wake)

- **Definición de las Slides de la Cátedra (SLIDE_03 / NC_Ardiles):** Es un modelo de control de calidad o *checklist* de seis atributos de ingeniería de software sugerido por Bill Wake (2003) para evaluar si una Historia de Usuario está bien especificada y “Lista” (*Definition of Ready*) para ser incorporada a la planificación del sprint.

A continuación se detalla cada uno de los seis atributos, comparando un ejemplo que cumple y otro que no cumple con el estándar:

1. Independent (Independiente)

- **Definición:** Las historias de usuario deben poder ser planificadas, estimadas, desarrolladas y probadas en cualquier orden sin depender estrechamente de la finalización de otra historia del backlog.
- *Ejemplo que SÍ cumple:* Como Administrador quiero registrar un nuevo libro ingresando su ISBN para catalogar el inventario.
- *Ejemplo que NO cumple:* Escribir tres historias separadas como “Pagar con Visa”, “Pagar con Mastercard” y “Pagar con American Express”.
 - *Por qué no cumple:* Las tres historias están acopladas en su estimación y desarrollo. El desarrollador necesita 3 días para programar la primera (independientemente de cuál sea por el esfuerzo del motor de pagos de base) y 1 día para las restantes. No se pueden planificar de forma aislada.
 - *Resolución:* Agruparlas en una historia paraguas: “Pagar con tarjetas de crédito (Visa, Mastercard, Amex)”.

2. Negotiable (Negociable)

- **Definición:** La tarjeta no es una especificación rígida ni un contrato cerrado. Debe declarar el “qué” se necesita (el valor lúdico del negocio) pero dejar libre el “cómo” se resolverá a nivel de implementación de software, permitiendo la discusión creativa del equipo técnico.
- *Ejemplo que SÍ cumple:* Como Conductor quiero buscar un destino para poder visualizar el mapa de la ruta.

- *Ejemplo que NO cumple:* Como Conductor quiero que al presionar el botón azul 'Buscar' el sistema llame al Store Procedure SP_BUSCAR_DIRECCION en la base de datos PostgreSQL, abriendo una ventana flotante de 300px con un campo numérico.
 - *Por qué no cumple:* Introduce supuestos de interfaz de usuario de bajo nivel y arquitectura técnica de base de datos prematuros, eliminando la capacidad de los programadores de sugerir soluciones más eficientes durante la conversación.

3. Valuable (Valiosa)

- **Definición:** Cada historia debe aportar un beneficio claro, tangible y comprensible para el cliente, comprador o usuario final del software lúdico. No se admiten historias puramente técnicas.
- *Ejemplo que SÍ cumple:* Como Estudiante quiero comprar un pase de estacionamiento mensual para ingresar con mi vehículo al campus sin demoras.
- *Ejemplo que NO cumple:* Como Programador quiero configurar la base de datos Oracle XE para el entorno de desarrollo.
 - *Por qué no cumple:* Es una tarea técnica de infraestructura necesaria para el equipo, pero no le genera ningún valor de negocio visible al Product Owner, por lo que él no la puede priorizar en la pila. Debe ser tratada como una tarea técnica embebida o un habilitador arquitectónico.

4. Estimatable (Estimable)

- **Definición:** El equipo de desarrollo debe comprender el dominio de negocio y la tecnología lo suficiente como para poder asignarle un tamaño relativo (en puntos de historia) y estimar su esfuerzo.
- *Ejemplo que SÍ cumple:* Como Cliente quiero recuperar mi contraseña ingresando mi correo electrónico registrado.
- *Ejemplo que NO cumple:* Como Investigador Médico quiero procesar secuencias de ADN utilizando algoritmos heurísticos experimentales para curar afecciones genéticas.
 - *Por qué no cumple:* La incertidumbre técnica y de negocio es tan masiva que los desarrolladores no tienen parámetros lógicos para estimar un número de puntos.
 - *Resolución:* Transformar la historia en una **Spike** (historia de investigación de tiempo acotado) para realizar un prototipo rápido y volverla estimable en el siguiente sprint.

5. Small (Pequeña)

- **Definición:** La historia debe tener un tamaño de grano lo suficientemente acotado como para ser iniciada, construida, probada y aceptada al 100% (Hecha) de forma binaria dentro del timebox de una sola iteración.
- *Ejemplo que SÍ cumple:* Como Administrador de la biblioteca quiero modificar la información de un libro existente.
- *Ejemplo que NO cumple (Épica):* Como Comprador quiero poder buscar libros, agregarlos a un carrito, pagar con tarjeta, hacer el seguimiento del envío de la compra en tiempo real y facturar la orden impositiva en la web.

- *Por qué no cumple:* Es un flujo completo de negocio de enorme tamaño que excede el tiempo físico de un sprint de dos semanas, propiciando el arrastre de tareas incompletas (“síndrome del 90% de avance”).

6. Testable (Testeable / Probable)

- **Definición:** La historia debe estar redactada de manera tan clara, objetiva y libre de ambigüedades que sea posible diseñar escenarios de prueba pasa/falla que verifiquen empíricamente su correcto comportamiento.
- *Ejemplo que SÍ cumple:* Como Administrador quiero que las claves de los usuarios contengan de forma obligatoria al menos 8 caracteres.
- *Ejemplo que NO cumple:* Como Jugador quiero que la interfaz gráfica de la aplicación sea sumamente intuitiva, amigable y estéticamente hermosa.
 - *Por qué no cumple:* Los adjetivos “hermosa”, “intuitiva” o “amigable” son juicios de valor completamente subjetivos. Es imposible para un programador o tester diseñar una prueba objetiva binaria para verificar que la interfaz es “hermosa”.

Tabla Comparativa de Criterios de INVEST

Atributo	Foco de Calidad de la US	Responsable directo	Riesgo de No Cumplimiento	Mecanismo de Resolución en SCM
Independent	Aislamiento lógico. Historias sueltas planificables en cualquier momento.	Product Owner y Equipo.	Cuellos de botella en cascada; estimaciones de puntos imposibles de asignar.	Agrupar componentes relacionados o refinar dependencias.
Negotiable	Apertura de diseño. Define el “qué” de negocio y no el “cómo” técnico.	Product Owner.	Sofocamiento creativo del equipo; software intractable; requisitos rígidos de mala calidad.	Remover detalles prematuros de interfaces de usuario o de base de datos.
Valuable	Retorno de inversión. Beneficio visible para el cliente/comprador.	Product Owner.	Desperdicio de horas de desarrollo en módulos inútiles que nadie usa.	Escribir siempre el “para qué” (valor de negocio) en la tarjeta.
Estimatable	Predictibilidad. Disminución de la incertidumbre.	Product Developers.	Imposibilidad de proyectar costos o planificar fechas de release.	Crear una Spike (historia de investigación acotada a 1 iteración).
Small	Factibilidad temporal. Que	Product Developers.	Tareas incompletas al	Romper y desagregar la

Atributo	Foco de Calidad de la US	Responsable directo	Riesgo de No Cumplimiento	Mecanismo de Resolución en SCM
	entre en el timebox del sprint.		final de la iteración; gestión parcial ineficaz.	Épica en historias atómicas independientes [25, 110, Image 69].
Testable	Certificación. Verificación empírica objetiva.	Product Owner y Testers.	Funcionalidades ambiguas imposibles de auditar en la Sprint Review.	Definir Criterios de Aceptación cuantitativos claros al dorso.

3.3.3. Contradicción de Longevidad: ¿Se descartan o se conservan las User Stories?

Existe una tensión académica de fondo muy importante en la materia entre lo que describe el agilismo de los libros de texto tradicionales y lo que exige de forma obligatoria la cátedra de Ingeniería de Software de la UTN-FRC para resguardar la integridad y consistencia de los proyectos.

A. El razonamiento de Mike Cohn (Paradigma Ágil Puro / Agile SCM)

- **Su postura:** Las historias de usuario **son consumibles efímeros de cortísimo plazo y pueden ser descartadas** (literalmente rotas física y administrativamente) tras su correcta implementación, testing y despliegue a producción.
- **Su fundamento:** El valor sustancial de la User Story reside en la **conversación** y en el conocimiento compartido que se transmitió al equipo de forma presencial. Una vez que el código fuente de producción estable está escrito y las suites de pruebas de regresión automatizadas están configuradas y pasando con estado “OK”, **el propio código ejecutable y sus tests automatizados son la única especificación viva, precisa y actualizada** del sistema. Conservar las tarjetas físicas de cartón o actualizar a mano planillas o documentos de texto de requerimientos estáticos es generar **inventario redundante de información desactualizada** (desperdicio o *waste* de Lean).

B. El razonamiento de la Cátedra de la UTN-FRC (Enfoque de Ingeniería de Software)

- **Su postura:** Las historias de usuario son **Ítems de Configuración de Producto (ICs) fundamentales que bajo ningún concepto se descartan**. Se deben identificar, almacenar en el repositorio Git, versionar de forma inalterable y conservar de manera permanente a lo largo de toda la vida útil del software.
- **Su fundamento:** Para que un producto de software posea **integridad**, es un requisito ineludible garantizar la **trazabilidad post-ERS completa**. Las User Stories actúan como el eslabón primario de trazabilidad que conecta de forma inalterable el requerimiento funcional de negocio con el diseño arquitectónico, las porciones físicas de código fuente programadas y los casos de prueba dinámicos. Si las historias se eliminan al final de cada sprint, se rompe de forma catastrófica la cadena de trazabilidad, imposibilitando que el

equipo de soporte pueda auditar en el futuro el origen o justificación de una regla de negocio lúdica durante la etapa de mantenimiento.

¿Qué responder en el examen de esta materia?

- **⚠ En el examen teórico (parciales/final)** de la cátedra de Meles y Covaro, debes responder con absoluta firmeza que **las User Stories se consideran Ítems de Configuración de Producto que NO se descartan**, sino que se versionan, se colocan bajo control en el repositorio Git y se conservan de forma sistemática para asegurar la trazabilidad y la integridad a lo largo de toda la vida útil del producto.
- *La respuesta de excelencia para el 10:* Demuestra una comprensión brillante agregando que: *“A pesar de que el enfoque ágil de Mike Cohn admite descartar las tarjetas físicas de cartón tras el sprint porque la especificación viva pasa a residir en las pruebas de regresión automáticas y el código fuente; la cátedra de la UTN-FRC adopta un enfoque sistémico de ingeniería en el cual se las debe conservar y versionar en el repositorio de SCM para garantizar de manera permanente la trazabilidad trasera y delantera de los entregables del producto”.*

Puntos clave para el examen — 3.3.3

1. **DIFERENCIA ENTRE CRITERIO Y PRUEBA DE ACEPTACIÓN (AUDIO_04):** Es una pregunta de examen clásica y mortal. Los **Criterios de Aceptación** son las condiciones lógicas mínimas que define el PO (ej: “la altura de la calle debe ser un número”). Las **Pruebas de Aceptación** son escenarios dinámicos de pasa/falla que derivan de esos criterios (ej: “Probar ingresar letras en altura de calle y verificar que el sistema falle”). Está mal redactar criterios como si fueran pruebas, o pruebas sin sustento en los criterios.
2. **SINTAXIS RÍGIDA DE PRUEBAS CON “PROBAR” (AUDIO_04 / AUDIO_06):** En los parciales prácticos de requerimientos, la cátedra exige de forma estricta que todas las pruebas de aceptación comiencen con el verbo **“Probar” en infinitivo**, seguido de la acción de la historia, la condición evaluada, y cerrando de manera obligatoria entre paréntesis con la palabra **(pasa)** o **(falla)** (ej. Probar buscar un destino ingresando números negativos en la altura y verificar que el sistema falle (falla)). Escribir “Verificar que...” o “El sistema valida...” es causa de descuento de puntos en la corrección.
3. **EL “COMO QUIERO” SÍ, EL “COMO QUIERO PODER” NO (AUDIO_04):** Durante las correcciones de aula, el docente de teoría remarca que escribir *“Como interesado quiero poder registrarme...”* es un error redundante. El “poder” ya está implícito en la acción lúdica de la tarjeta. La sintaxis limpia y profesional exige escribir directamente: *“Como interesado quiero registrarme...”*.
4. **CORTAR VERTICALMENTE LAS USER STORIES (AUDIO_04):** Las historias de usuario deben representar porciones de funcionalidad verticales que atraviesen todas las capas de la arquitectura (interfaz, lógica de negocio y base de datos). Diseñar una historia de usuario llamada *“Diseñar la base de datos de usuarios”* es un error conceptual grave de agilidad, ya que eso representa una capa horizontal técnica que no le entrega ningún software funcional potencialmente desplegable al Product Owner al final de la iteración.

Preguntas de autoevaluación — 3.3.3

Pregunta 1: ¿Cuáles son las tres partes o dimensiones de una Historia de Usuario (US)?

- **Respuesta breve:** Son **Tarjeta (Card)**, **Conversación (Conversation)** y **Confirmación (Confirmation)**.

Pregunta 2: ¿Por qué la Tarjeta no se considera una especificación detallada de requerimientos?

- **Respuesta breve:** Porque la tarjeta no documenta de manera exhaustiva la funcionalidad; actúa únicamente como una expresión corta de intención y como un recordatorio (*token*) para entablar una conversación detallada en el futuro.

Pregunta 3: ¿Qué error sintáctico se comete en la tarjeta al escribir “Como usuario quiero registrarme”?

- **Respuesta breve:** Se comete el error de utilizar un actor genérico (“Usuario”). Se debe declarar obligatoriamente un **rol específico de negocio** (“Estudiante”, “Conductor”) para comprender el contexto de uso lúdico.

Pregunta 4: ¿Por qué se afirma que la Conversación es el componente más importante de las 3 C’s?

- **Respuesta breve:** Porque es el medio interactivo y presencial en el cual fluye la transferencia de conocimiento cara a cara entre el PO y los desarrolladores, destruyendo la falsa ilusión de precisión que genera el texto escrito.

Pregunta 5: ¿Qué es la Confirmación en una User Story y qué elementos la configuran al dorso de la tarjeta?

- **Respuesta breve:** Es el componente de la historia que permite verificar de forma objetiva y empírica si se ha implementado correctamente el requerimiento. La configuran los **Criterios de Aceptación** y las **Pruebas de Aceptación de Usuario**.

Pregunta 6: Describa el significado de las siglas del modelo “INVEST” de Bill Wake.

- **Respuesta breve:** Representan que una Historia de Usuario de calidad debe ser: Independiente, Negociable, Valiosa (para el negocio), Estimable (en tamaño relativo), Small (pequeña para entrar en el sprint) y Testeable.

Pregunta 7: ¿Por qué una historia que describe de forma rígida componentes de base de datos o interfaces de usuario viola el atributo “Negotiable” (Negociable)?

- **Respuesta breve:** Porque elimina la capacidad del equipo técnico de debatir y proponer soluciones lúdicas o de diseño alternativas más eficientes, transformando a la tarjeta en un contrato rígido de diseño en lugar de una declaración de valor de negocio.

Pregunta 8: Explique la contradicción de longevidad de las historias entre Mike Cohn y la cátedra de la UTN-FRC.

- **Respuesta breve:** Cohn sostiene que las historias pueden ser descartadas o rotas tras el sprint porque la especificación viva reside en el código y los tests; la cátedra sostiene que no se descartan, sino que son ICs de Producto que deben conservarse en el repositorio Git para asegurar la trazabilidad post-ERS completa.

Pregunta 9: ¿Qué sintaxis formal exige la cátedra para la redacción de las Pruebas de Aceptación de Usuario?

- **Respuesta breve:** Exige comenzar con el verbo en infinitivo “**Probar**”, describir el escenario funcional específico evaluado, y cerrar entre paréntesis de forma binaria indicando si la prueba **(pasa)** o **(falla)**.

Pregunta 10: ¿Qué es una “Spike” y qué atributo de INVEST asiste cuando una historia es demasiado compleja?

- **Respuesta breve:** Es una historia especial de investigación técnica o funcional que se ejecuta en un sprint aislado para reducir la incertidumbre. Asiste al atributo de **Estimatable (Estimable)** cuando la historia original posee demasiadas incógnitas técnicas o de negocio.

3.4. Estimación ágil y Story Points (Puntos de Historia)

3.4.1. Definición y Fundamento de la Estimación Relativa

3.4.1.1. Definición de Story Points (Puntos de Historia - SP)

- **Definición de Mike Cohn (BIBLIO_ - Cap. 4):** Los Story Points son una **unidad de medida abstracta y relativa** utilizada por los equipos ágiles para expresar el tamaño general, el esfuerzo y la complejidad de una Historia de Usuario.
- **La denominación de Joshua Kerievsky:** Kerievsky sugirió jocosamente denominar a los Story Points como **NUTs (Nebulous Units of Time / Unidades Nebulosas de Tiempo)**, reflejando de forma satírica que, aunque se asocian de manera cualitativa al esfuerzo temporal, representan una abstracción pura de magnitud que no equivale de forma directa a un número de horas lineales de reloj.
- **Qué problema resuelve:** Destruye la ineficacia humana en la estimación absoluta. Evita el desgaste técnico de forzar a los programadores a dar estimaciones milimétricas en horas de reloj (ej. “esto me va a llevar 7 horas y media”), lo cual es inherentemente propenso al error y varía drásticamente según el nivel de experiencia (seniority) de quien ejecute la tarea.

3.4.1.2. Cómo se hace: La técnica de Estimación Relativa

- **La Premisa Cognitiva (¿Por qué somos relativos?) (SLIDE_04):** Las personas somos biológica y psicológicamente deficientes estimando en términos absolutos, pero **somos sumamente eficaces y rápidos comparando cosas** entre sí.
- **La analogía de clase:** Es sumamente complejo que nos digas a simple vista cuántos metros exactos de altura tiene la Torre Ángela de Córdoba en comparación con el edificio de Tarjeta Naranja (estimación absoluta). Sin embargo, si los parás uno al lado del otro, en un segundo podés certificar con total exactitud cuál es más alto (estimación relativa).
- **El procedimiento en el repositorio:**
 1. Se analiza la historia que se desea estimar.
 2. Se la compara contra una **Historia de Usuario de referencia (Canónica)**.
 3. Se determina cuántas veces “más grande” o “más compleja” es la nueva historia respecto a la base establecida.

3.4.2. Las tres dimensiones de un Story Point

Para que la asignación de Story Points sea consistente y objetiva (dentro de lo probabilístico), la cátedra enseña que el equipo de desarrollo debe desmenuzar y justificar el tamaño de cada Historia de Usuario analizando de manera combinada **tres variables cualitativas**:

DIMENSIONES DE UN STORY POINT

[COMPLEJIDAD] [ESFUERZO] [INCERTIDUMBRE / DUDA]

Dificultad intrínseca Volumen de trabajo físico Falta de información o

tecnológica y de lógica línea que demanda para desconocimiento de dominio

del requerimiento. completar la tarea. tecnológico o de negocio.

1. **Complejidad:** Se refiere a la dificultad intrínseca de la solución técnica o lógica necesaria para resolver el problema. Está ligada a la naturaleza de la tarea o a la arquitectura de software involucrada.
 2. **Esfuerzo (Trabajo):** Representa el volumen cuantitativo de trabajo físico que requiere el desarrollo de la historia de extremo a extremo (programación, testing, base de datos).
 3. **Incertidumbre o Duda:** Mide el nivel de ignorancia o falta de conocimiento que el equipo tiene sobre el requerimiento funcional de negocio o sobre la tecnología que debe utilizar para construirlo. A mayor incertidumbre, mayor es el colchón de puntos que se debe asignar preventivamente para resguardar al equipo.
- **Ejemplo de clase (La matriz de burbujas):** El docente en el pizarrón dibuja tres burbujas de distintos tamaños para tres historias diferentes (Story 1, Story 2, Story 3).
 - La *Story 1* tiene complejidad media, esfuerzo bajo e incertidumbre nula: se le asigna **2 SP**.
 - La *Story 2* tiene complejidad baja, pero un volumen de esfuerzo físico altísimo (ej. cargar a mano 50 pantallas repetitivas de ABM): se le asigna **5 SP**.
 - La *Story 3* tiene complejidad baja y esfuerzo bajo, pero una burbuja gigante de duda e incertidumbre porque el PO no sabe cómo se comunicará el sistema de pago: se le asigna **8 u 13 SP**.

3.4.3. La User Story Canónica (Base Story)

- **Definición de las notas de clase (NC_MartinezBaigorria):** La Historia de Usuario Canónica es el **punto de referencia inicial y patrón de medida estable** que el equipo de desarrollo consensúa para comparar y calibrar el tamaño relativo de todas las demás historias del backlog.
- **Qué problema resuelve:** Evita la descalibración de la escala de estimación. Sin una canónica clara, lo que para el Programador A vale “5 puntos” para el Programador B vale “2”, rompiendo la consistencia de la velocidad y destruyendo la predictibilidad de la planificación.
- **Cómo se hace (Paso a paso de deducción) (AUDIO_06):**
 - Se analiza la pila de Historias de Usuario iniciales.
 - Se busca e identifica de forma consensuada **la historia más simple, pequeña, conocida y libre de incertidumbre** de todo el backlog.

- Se le asigna de manera rígida el valor de **1 Story Point (o 2 de margen)**.
- Cada nueva historia se estima preguntando de forma sistemática: “¿Cuántas historias canónicas de tamaño 1 entran en este nuevo requerimiento?”.
- **Errores comunes / Advertencia de examen:**
 -  *Elegir mal la canónica:* Seleccionar como canónica una historia sumamente grande o compleja (ej. una de 8 puntos). Esto provoca que las historias más pequeñas tengan que ser estimadas con decimales o fracciones (0.5, 0.25, 0.1), lo cual es sumamente incómodo de computar y dificulta el Poker de Estimación.

3.4.4. Escalas de Estimación: Fibonacci Modificada y Taller de Remera

La cátedra enseña que para estimar el tamaño relativo de los ICs se deben descartar las escalas lineales (1, 2, 3, 4, 5, 6, 7...) porque la complejidad del software y el error humano no crecen de forma lineal, sino de forma exponencial conforme aumenta el tamaño del objeto analizado.

escalas de medición del tamaño en SCM:

Escala Fibonacci Modificada (Para el Sprint Backlog)

- **Valores:** 0, 0.5, 1, 2, 3, 5, 8, 13, 20, 40, 100.
- **Fundamento:** Cada número subsecuente es la suma de los anteriores. Refleja matemáticamente que a mayor tamaño, la incertidumbre se expande de forma exponencial. Es más fácil debatir si una tarea es un 5 o un 8 (una diferencia perceptible) que discutir si un archivo vale 13 o 14 puntos en una escala lineal.
- **Cuándo aplica:** Se utiliza de forma obligatoria durante la **Sprint Planning** para estimar las historias refinadas que ingresarán de forma inminente al desarrollo activo.

Escala Taller de Remera (T-Shirt Sizing) (Para el Product Backlog)

- **Valores:** XS, S, M, L, XL, XXL.
- **Fundamento:** Es una escala sumamente blanda y cualitativa. Permite agrupar rápidamente elementos de gran tamaño cuando el nivel de información técnica disponible es sumamente escaso o difuso.
- **Cuándo aplica:** Se utiliza para realizar **estimaciones tempranas de largo plazo (pre-estimaciones)** directamente sobre el Product Backlog o en fases de preventa del producto.

3.5. La Dinámica de Planning Poker (Scrum Poker)

3.5.1. Definición y Características de Base

- **Definición formal (Mike Cohn - Cap. 6):** El Planning Poker es una **técnica de estimación colaborativa, gamificada y basada en el consenso** que combina la opinión de expertos, la estimación por analogía y la desagregación de componentes para asignar de forma rápida y confiable Story Points a las historias de usuario.

- **Qué problema resuelve:** Elimina la influencia del **anclaje cognitivo (anchoring)** y los sesgos de autoridad. En las reuniones tradicionales de estimación, el líder técnico o el programador más extrovertido decían en voz alta: “*Esto lleva 3 días*”, y de inmediato el resto del equipo (por temor o pereza técnica) se acoplaba a su opinión, anulando el pensamiento crítico colectivo y subestimando los riesgos.

3.5.2. El Procedimiento de Planning Poker Paso a Paso

La dinámica se ejecuta respetando de manera rígida los siguientes 6 pasos técnicos (exigidos de forma procedimental en los parciales prácticos):

[FLUJO DE TRABAJO EN EL PLANNING POKER]
 [Paso 1: Presentación] -> [Paso 2: Aclaración] -> [Paso 3: Votación Oculta]
 [Paso 6: Consenso] <- [Paso 5: Debate Extremos] <- [Paso 4: Exposición]

Paso 1: Presentación de la Historia

- **Acción:** El moderador (usualmente el Product Owner) lee la Historia de Usuario, explica el objetivo de negocio y describe detalladamente sus Criterios de Aceptación.

Paso 2: Aclaración de dudas y diseño rápido

- **Acción:** Los desarrolladores realizan preguntas técnicas breves al PO sobre la arquitectura, las validaciones y los casos de borde. Es una discusión técnica de alto nivel, acotada estrictamente en tiempo utilizando un reloj de arena de dos minutos (*timebox*) para evitar la parálisis por análisis.

Paso 3: Votación individual en privado (Secreta)

- **Acción:** Cada estimador del equipo de desarrollo, de forma aislada e independiente, selecciona una carta de su mazo de Fibonacci que represente su estimación por comparación contra la canónica. La carta se coloca **boca abajo** sobre la mesa para que nadie la vea.

Paso 4: Exposición simultánea de cartas

- **Acción:** Cuando todos los estimadores colocaron sus cartas boca abajo, el moderador da la orden y **todos los integrantes muestran sus cartas de forma simultánea**.

Paso 5: Debate y justificación de los valores extremos

- **Acción:**
 - Si hay consenso absoluto (todos votaron el mismo número), se registra ese valor como la estimación oficial y se pasa a la siguiente historia.
 - Si existen discrepancias significativas, **no se realiza un promedio matemático** (eso es un error grave de ingeniería). Se le da la palabra de forma prioritaria a los **dos votos extremos** (el valor más bajo y el valor más alto).
 - *El fin del debate:* El que votó bajo debe explicar qué suposiciones de reutilización o simplicidad técnica vio; el que votó alto debe argumentar qué riesgos ocultos, dependencias o complejidades de base detectó y que el resto del equipo pasó por alto.

Paso 6: Re-votación y Consenso

- **Acción:** Tras la breve conversación técnica de alineación, el equipo vuelve a votar a ciegas (GOTO Paso 3). El proceso se repite (generalmente no más de dos o tres rondas) hasta alcanzar un consenso sólido sobre el tamaño de la historia.

3.5.3. Decodificación de las cartas especiales del mazo

Durante el Planning Poker, los estimadores cuentan con cartas con funciones administrativas específicas definidas por la cátedra para el TP4:

[DECODIFICACIÓN DE CARTAS ESPECIALES]
[CARTA 0] [CARTA ?] [CARTA ☕]

- Esfuerzo nulo. - Incertidumbre total. - Agotamiento mental.
- Se usa para RNF - El requerimiento muta - Se frena el Poker

o restricciones. a un SPIKE, para tomar un café.

- **Carta 0:** Representa que el requerimiento no le demanda ningún esfuerzo físico de codificación ni complejidad agregada al equipo de desarrollo. Se utiliza clásicamente para registrar restricciones o **Requerimientos No Funcionales (RNF)** transversales (ej. “*el sistema debe correr sobre servidores en la nube*”), que deben cumplirse pero no suman Story Points individuales a la velocidad de la iteración.
- **Carta ? (Signo de pregunta):** Denota ignorancia o incertidumbre total sobre la tecnología o las reglas de negocio de la historia. Votar con un “?” frena la estimación y obliga al Product Owner a transformar esa historia en una **Spike** (investigación técnica acotada) para reducir la duda antes de poder ser puntuada.
- **Carta de la Taza de Café ☕:** Indica cansancio y saturación mental del equipo. Cuando un integrante la muestra, se suspende momentáneamente el juego de planificación para realizar un recreo y evitar que se sigan votando números de forma descuidada por apuro u omisión.
- **Carta 40 / 100:** Alerta roja de tamaño.
 - *El 40:* Indica que la historia es **demasiado grande (Épica)** y que es materialmente imposible completarla y testearla al 100% dentro de un sprint de dos semanas. Obliga a aplicar técnicas de partición vertical de historias.
 - *El 100:* Alerta máxima de que **el requerimiento está muy mal definido** o posee una contradicción lógica tan severa que es preferible ni comenzar a programar hasta que el PO lo rediseñe por completo.

3.5.4. Políticas de Reestimación de Historias en SCM (Mike Cohn - Cap. 7)

El manual de **Mike Cohn** y los apuntes prácticos de la cátedra definen reglas sumamente estrictas para normar **cuándo SÍ y cuándo NO** se debe reestimar una Historia de Usuario que ya posee un puntaje en el backlog:

Cuándo SÍ se debe reestimar:

1. **Cambio real en el tamaño relativo:** Únicamente cuando el equipo descubre nueva información técnica o de negocio que altera de forma drástica la complejidad o la

incertidumbre original del requerimiento (ej. se pensaba usar una API interna y a mitad de camino se descubre que hay que desarrollar el motor de comunicación de cero).

2. **Identificación de incoherencias lógicas:** Al incorporar nuevas historias al backlog, se detecta que una historia vieja quedó desalineada en la escala de analogía en comparación con los nuevos activos incorporados.

Cuándo NO se debe reestimar:

1. **Desvíos de esfuerzo temporal:** Nunca se debe cambiar el puntaje de una historia de usuario simplemente porque la tarea demandó más tiempo o esfuerzo del planificado originalmente.
2. *El fundamento de ingeniería:* Si el programador se demoró más horas por cortes de luz, problemas personales, falta de concentración o porque el equipo simplemente estuvo más lento en ese sprint, **el tamaño físico de la funcionalidad (el producto) no varió en absoluto**. Lo único que se vio afectado fue el esfuerzo. Modificar el número de Story Points de forma cosmética para “hacer encajar” las horas destruye la consistencia de la métrica y corrompe la base matemática de la **Velocidad**.

3.6. Relación entre Estimación, Compromiso, Objetivo de Negocio y Velocidad

3.6.1. La Ecuación Ágil Central (La derivación del Calendario)

Una de las grandes revoluciones de la gestión ágil es que **nunca se estima el tiempo de calendario de forma directa**. Estimar fechas directas de entrega en entornos cambiantes es una adivinación costosa.

El agilismo aplica un procedimiento matemático e ingenieril riguroso para derivar el calendario:

$$\text{Duración (Iteraciones)} = \frac{\{\text{Tamaño total del Backlog (Story Points)}\}}{\{\text{Velocidad del equipo (Puntos por Iteración)}\}}$$

El procedimiento técnico para calendarizar un Release:

1. **Estimar el tamaño:** El equipo calcula la sumatoria total de los Story Points de todas las Historias de Usuario prioritarias que conforman el backlog del release (ej. un backlog de **100 SP**).
2. **Medir la velocidad empírica:** Se determina cuántos puntos de historia es capaz de completar con éxito el equipo por cada sprint de duración fija (ej. velocidad histórica demostrada de **20 puntos/sprint**).
3. **Calcular la duración:** Se divide el tamaño sobre la velocidad para obtener el número de sprints requeridos: $(100 / 20 = 5 \text{ sprints})$.
4. **Derivar el calendario físico:** Sabiendo que en Scrum las iteraciones tienen una **duración fija (timebox) e inamovible** (ej. sprints rígidos de 2 semanas de duración), se multiplican los sprints por el timebox: $(5 \text{ sprints} \times 2 \text{ semanas} = 10 \text{ semanas de calendario})$.

3.6.2. Matriz de Desambiguación Conceptual: Estimación, Compromiso y Objetivo

Durante las clases grabadas, la cátedra advierte que confundir estos tres términos en el ámbito de la gestión corporativa es el “pecado original” que arrastra a los proyectos al fracaso sistémico.

Tabla Comparativa de Variables de Gestión

Variable de Control	¿Qué representa conceptualmente?	Naturaleza / Atributo	¿Quién tiene la autoridad de definirlo?	Ejemplo concreto en el proyecto	¿Qué pasa si se confunde con la estimación?
Estimación (Estimation) [Cohn Cap. 1] y	Una predicción analítica, imparcial probabilística ti del tamaño, costo o esfuerzo que demandará un requerimiento.	Probabilística y Objetiva. No ene certeza to absoluta y debe expresarse en rangos.	El Equipo de Desarrollo en su talidad qu (practitioner s).	“Este módulo de E login estimamos e tiene un un tamaño de 5 Story Points ”. i	s el origen del fracaso. Si a estimación se trata como compromiso o nmutable, el equipo se ve forzado a recortar la calidad lúdica de los entregables.
Compromiso (Commitment) [Cohn Cap. 1] al	Una promesa firme de entrega de cance y calidad de asumida de común acuerdo, asimilando riesgos específicos del negocio.	Determinística y Contractual. Se fine sobre ac fechas del calendario.	El Equipo de Desarrollo en uerdo con el en Product Owner.	“Nos S comprometem os a tregar la co funcionalidad básica de cobro el 15 de noviembre ”. t	i el negocio dicta un mpromiso irreal sin basarse en las estimacione s técnicas, se genera un cronograma subestimad o crónico.
Objetivo de Negocio (Target / Goal) [Cohn d Cap. 1] o	Una meta estratégica esecada que la D rganización o el a cliente necesitan alcanzar para que el software genere	Subjetiva y Comercial. e fine la E spiración del P negocio.	El Cliente, la Gerencia de la mpresa o el e roduct Owner. e	“Necesitamos S reducir un 30% l tiempo de o spera de los n usuarios en la fila de	i el equipo asume que el bjetivo de egocio es una estimación de sfuerzo

Variable de Control	¿Qué representa conceptualmente?	Naturaleza / Atributo	¿Quién tiene la autoridad de definirlo?	Ejemplo concreto en el proyecto	¿Qué pasa si se confunde con la estimación?
	retorno de inversión.			cobro". e	técnica, se "finge demencia" y se firma un plan imposible de cumplir.

3.6.3. El Cono de la Incertidumbre (Barry Boehm) y el Rol de la Velocidad

- **Definición teórica (Barry Boehm - 1981):** El Cono de la Incertidumbre es un modelo gráfico empírico que demuestra cómo **la variabilidad y el margen de error de las estimaciones disminuyen de forma sistemática a medida que avanza el ciclo de vida del proyecto** y se reduce la ignorancia sobre el sistema.

Margen de Variabilidad de la Estimación

```

4.0x \ /
1.0x *-> Tiempo de Desarrollo
0.25x/ \
[Inicio del Proyecto] [Entregas de Sprints]

```

Universo de Incertidumbre Certeza con SW Funcionando

(Variabilidad de 400%) (El cono se achica)

- **La Dinámica en el Inicio (Pre-estimaciones):** Al inicio del proyecto (fase de factibilidad), el nivel de desconocimiento del sistema es absoluto. Las estimaciones tempranas poseen un **margen de error de hasta el 400% (variando desde 0.25x hasta 4x del costo real)**.
- **Cómo el agilismo achica el cono:** A diferencia de los procesos tradicionales que intentan forzar la certeza de forma artificial mediante documentos de requerimientos masivos rígidos escritos de entrada (BRUF), **el agilismo reduce la incertidumbre de forma empírica entregando software funcional de forma frecuente**.
- En cada demo y sprint review, el cliente inspecciona el código real, las dudas de negocio desaparecen y la brecha de variabilidad de las estimaciones futuras se reduce drásticamente de forma natural.
- **La Velocidad como el Gran Ecualizador:** La velocidad del equipo de desarrollo (métrica histórica calculada sumando los SP aceptados de forma binaria al final del sprint) actúa como un **corrector automático de las estimaciones**.
- Si el equipo estimó con optimismo desmedido al principio, no importa; tras tres sprints, la velocidad empírica real (el ritmo de quema de puntos demostrado en la cancha) ecualizará el

plan de release, ajustando el cronograma final a la capacidad real de producción de los programadores.

3.6.4. Patologías de la estimación: Sobreestimación y Subestimación

La cátedra advierte sobre las dos patologías de gestión que se gatillan cuando se rompe la transparencia en los procesos de estimación:

1. Subestimación (Underestimation): El cronograma irreal

- *Qué la genera:* Presión comercial del negocio para “hacer encajar” la estimación en una fecha fija deseada por marketing, o el optimismo ingenuo del programador.
- *Consecuencia sistémica en cascada (Explicado en AUDIO_05):*
 1. Se parte de un cronograma subestimado e imposible de cumplir en la realidad lúdica.
 2. Como el equipo está apurado y “el agua le llega al cuello”, se **recortan drásticamente las actividades tempranas de especificación, análisis y diseño de arquitectura**.
 3. Se cae en la trampa del “*lo hagamos rápido y sucio así nomás, total después lo refactorizamos*” (un “después” que el docente aclara que nunca llega porque el negocio nunca habilitará tiempo para mejorar código ya escrito).
 4. El código sucio acumula **deuda técnica (technical debt)**, disparando la tasa de defectos y bugs en testing o, peor aún, en producción ante los ojos del usuario.
 5. Se desata un estado de crisis urgente crónico que obliga a gastar recursos masivos de soporte y horas extras para apagar incendios, resultando en un proyecto deficitario y con el “cartel” de que somos pésimos ingenieros.

2. Sobreestimación (Overestimation): El colchón defensivo

- *Qué la genera:* Una reacción defensiva del equipo técnico ante organizaciones jerárquicas con baja tolerancia al error donde estimar se penaliza como un incumplimiento de contrato.
- *El chiste de clase (AUDIO_05):* El programador sabe que necesita 4 horas para codificar una función. Pero sabe que su jefe de proyecto, al escuchar “4 horas”, le va a aplicar un recorte arbitrario exigiéndole que lo haga en 2. Como mecanismo de supervivencia, el programador infla la estimación y pide 8 horas. El jefe recorta a la mitad, “concediéndole” 4.
- *El problema:* Se destruye la confianza y la transparencia de la organización, transformando la estimación en un juego de negociaciones informales de pasillo en lugar de una actividad científica de diseño de ingeniería.

Puntos clave para el examen — 3.6.4

1. **LA VELOCIDAD NUNCA SE ESTIMAS, SE CALCULA (AUDIO_05 / AUDIO_06):** Es la trampa preferida de la cátedra para bochar alumnos en el parcial práctico. Está conceptualmente mal que en tu TP4 escribas “*estimamos que nuestra velocidad para el Sprint 1 va a ser de 25 puntos*”. La velocidad es una **métrica puramente empírica e histórica**. Se calcula de forma binaria al final de cada sprint sumando únicamente los Story

Points de las **User Stories aceptadas al 100% por el Product Owner**. Si no tenés datos históricos de sprints previos, podés hablar de una “*velocidad esperada o suposicional*” para arrancar el plan de release, pero debés aclarar de inmediato que no es una estimación oficial y que requiere ser ajustada con la métrica real.

2. **REGLA DE GESTIÓN ÁGIL BINARIA DE Calidad (AUDIO_05):** En los exámenes prácticos de estimación, si una Historia de Usuario de 8 Story Points quedó desarrollada en un 90% (ej: el programador codificó la pantalla, la base de datos y andaba perfecto, pero faltó completar el testing de aceptación final de QA), para el cálculo de la velocidad de ese sprint **esa historia aporta exactamente CERO (0) puntos de velocidad**. No se admiten porcentajes de avance parciales (como decir “*sumamos 7.2 puntos porque está casi lista*”) para erradicar de cuajo el síndrome crónico del “90% de avance listo”.
3. **EL DETALLE DE LA JUSTIFICACIÓN EN EL PARCIAL:** El docente práctico es terminante: “*En el examen práctico de estimaciones no nos alcanza con que nos pongan que la historia vale un 3 o un 5. Tienen que justificar obligatoriamente en base a las tres burbujas de dimensiones: por qué para su grupo la complejidad es baja, el esfuerzo es medio y la incertidumbre es nula*”. Un número suelto sin la justificación cualitativa de las dimensiones se califica directamente con cero puntos en el examen.
4. **CAUTELA CON EL INVEST EN HISTORIAS DE 13 PUNTOS (AUDIO_06):** Si en un ejercicio de parcial le asignás **13 o más Story Points** a una Historia de Usuario, debés prender las alertas de inmediato. Un valor de 13 puntos denota que la historia es **demasiado grande o compleja** para entrar con seguridad en un sprint de dos semanas. En el mismo examen debés justificar que, para cumplir con el atributo **Small** del modelo INVEST, la historia de 13 puntos **debe ser particionada verticalmente** en historias de menor denominación (ej. romperla en una de 5, otra de 5 y otra de 3) antes de autorizar su ingreso al Sprint Backlog.

Preguntas de autoevaluación — 3.6.4

Pregunta 1: ¿Qué es un Story Point (Punto de Historia) y qué representa en SCM?

- **Respuesta breve:** Es una unidad de medida abstracta y de tamaño relativo que expresa la combinación de complejidad técnica, volumen de esfuerzo y nivel de incertidumbre de una Historia de Usuario en comparación con otra.

Pregunta 2: ¿Cuáles son las tres dimensiones cualitativas que configuran el tamaño de un Story Point?

- **Respuesta breve:** Son la **Complejidad** (dificultad técnica), el **Esfuerzo** (volumen de trabajo físico) y la **Incertidumbre o Duda** (falta de información).

Pregunta 3: ¿Qué es la Historia de Usuario Canónica y por qué debe deducirse en primer lugar?

- **Respuesta breve:** Es la historia base, de menor tamaño y mayor simplicidad del backlog, que se utiliza como patrón de referencia inicial para estimar todas las demás historias por analogía y comparación relativa.

Pregunta 4: ¿Por qué la complejidad del software exige la adopción de la escala de Fibonacci Modificada en lugar de una lineal?

- **Respuesta breve:** Porque el crecimiento de la complejidad y de la incertidumbre en el software es de naturaleza exponencial; la escala de Fibonacci refleja matemáticamente esta diseconomía de escala a mayor tamaño.

Pregunta 5: ¿En qué consiste el paso de “Votación a Ciegas” en el Planning Poker y qué sesgo evita?

- **Respuesta breve:** Consiste en que cada estimador elija su carta de estimación de forma aislada y la coloque boca abajo en la mesa, mostrándolas todas a la vez. Evita el sesgo de **anclaje cognitivo**, impidiendo que la opinión de los perfiles senior o más extrovertidos condicione la votación del resto.

Pregunta 6: ¿Cómo se resuelve una discrepancia de votos en una ronda de votación de Planning Poker?

- **Respuesta breve:** No se promedian los votos. Se abre un debate técnico priorizando la palabra de los **votos extremos** (el más alto y el más bajo) para alinear el entendimiento y revelar riesgos o supuestos ocultos, y luego se vuelve a votar.

Pregunta 7: ¿Para qué se utiliza formalmente el valor de “0” en las cartas del Planning Poker?

- **Respuesta breve:** Se utiliza para registrar restricciones o Requerimientos No Funcionales (RNF) que no agregan volumen físico de código ni complejidad individual al backlog del sprint, pero que deben cumplirse de manera obligatoria.

Pregunta 8: ¿Cuándo se debe reestimar el puntaje de una Historia de Usuario según la cátedra?

- **Respuesta breve:** Únicamente cuando se descubre nueva información técnica o funcional que altera el **tamaño relativo intrínseco** o la complejidad original de la funcionalidad, nunca por desvíos temporales de horas de trabajo.

Pregunta 9: Defina la métrica de “Velocidad” de un equipo ágil y explique cómo se calcula.

- **Respuesta breve:** Es una métrica empírica de progreso que se calcula al final de cada sprint sumando estrictamente los Story Points de todas las Historias de Usuario que fueron terminadas al 100% y aceptadas formalmente por el Product Owner.

Pregunta 10: ¿Cuál es el peligro de confundir una Estimación técnica con un Compromiso contractual de negocio?

- **Respuesta breve:** Provoca la firma de cronogramas irreales que fuerzan al equipo a recortar las fases tempranas de análisis y diseño, acumulando deuda técnica crónica, disparando la tasa de defectos de software y encareciendo el proyecto en producción.

3.7. El espectro de los Productos Mínimos (MVP, MMP y MLP) y la justificación de la escala

3.7.1. Producto Mínimo Viable (MVP - Minimum Viable Product)

3.7.1.1. Definición precisa y propósito

- **Definición formal (Eric Ries, *The Lean Startup* / Cátedra UTN-FRC):** Un MVP es una versión preliminar de un producto nuevo que permite a un equipo recolectar la máxima cantidad de **aprendizaje validado** (validated learning) sobre los clientes con el menor esfuerzo lúdico y costo posibles.

- **La clave del MVP:** No es un producto final fragmentado ni de mala calidad [Image 45]. Su único propósito ingenieril es **validar una hipótesis de negocio** (comprobar supuestos de “salto de fe”) para mitigar el riesgo del cero absoluto antes de invertir capital o código de producción [43, 72, Image 41, Image 42].
- **Qué problema resuelve:** Erradica el desperdicio masivo de construir “la catedral perfecta” de software que luego nadie usa en producción. Evita la parálisis por análisis y desmitifica la ilusión de que el cliente sabe lo que quiere antes de verlo.

3.7.1.2. Cómo se hace: El ciclo de validación empírica

El desarrollo de un MVP sigue el procedimiento estricto del **empirismo** y la filosofía Lean:

1. **Declaración de la Hipótesis:** Redactar de forma explícita qué comportamiento humano o necesidad del mercado se desea comprobar.
2. **Definición del Alcance:** Seleccionar únicamente las características críticas que atacan directamente la hipótesis planteada. **El alcance se ajusta a la hipótesis, nunca al revés.**
3. **Construcción del Artefacto de Validación:** Diseñar el canal de validación más económico (puede ser una maqueta Figma, un video explicativo, una landing page o un prototipo con características limitadas).
4. **Medición:** Exponer el MVP a usuarios reales y medir la métrica crítica de **retención** (cuántos vuelven de forma recurrente) en lugar de la vanidad de las descargas.
5. **Adaptación / Pivoteo:** Con el feedback rápido, decidir si se preserva la visión original o si se realiza un **pivote** (cambio de estrategia de negocio) hacia un MVP2 [71, 72, Image 47].

3.7.1.3. Ejemplos concretos de la Cátedra (Casos de estudio de examen)

- **El Caso Dropbox (Prueba de Humo sin código)** [Image 44]: Drew Houston enfrentaba un riesgo técnico de infraestructura masivo para sincronizar archivos. En lugar de escribir código complejo de inmediato, grabó un **video banal de 3 minutos** demostrando cómo funcionaría teóricamente la tecnología. La lista de espera saltó de 5.000 a 75.000 personas de la noche a la mañana, validando la demanda del mercado con costo de software cero.
- **El Caso Facebook (Validación de comportamiento humano)** [Image 43]: En 2004, con ingresos casi nulos, validaron la hipótesis de valor demostrando que **más del 50% de los usuarios registrados volvían al sitio todos los días de forma recurrente** (retención), y la hipótesis de crecimiento al captar 3/4 de la población de Harvard en un mes con \$0 de inversión publicitaria [Image 43].

3.7.1.4. Errores comunes y advertencias de Meles

- **⚠ El error de fabricar código de entrada:** Un MVP **no tiene por qué ser software funcionando**. Si es un video, una maqueta o un prototipo “atado con alambre”, es 100% válido mientras capture aprendizaje real de los usuarios.

3.7.2. Producto Mínimo Comercializable (MMP - Minimum Marketable Product)

CRONOLOGÍA DE LIBERACIÓN: LA RUTA DEL VALOR

[Fase 1: MVP (Aprendizaje)] -> Prototipos rápidos, videos, maquetas

[Fase 2: MMP / Release 1] -> SOFTWARE FUNCIONANDO. Primer build en producción con MMFs esenciales.
[Fase 3: MMR 2... MMR N] -> Evolución continua e incrementos.

3.7.2.1. Definición precisa y propósito

- **Definición de las Slides de la Cátedra (SLIDE_05):** El MMP (o MMR₁ - Minimum Marketable Release) es la porción o **versión más pequeña de software funcionando y testeado que entrega valor de negocio real** y que el equipo puede poner físicamente en producción en el servidor del cliente.
- **Qué problema resuelve:** Resuelve el desfase temporal de entrega. Evita postergar el retorno de inversión del cliente, permitiéndole operar de forma básica el sistema y ganar tracción con sus usuarios reales mientras el equipo sigue codificando los siguientes incrementos [100, Image 53].

3.7.2.2. Cómo se hace: Procedimiento de selección de MMFs

El PGC y el Product Backlog se articulan para:

1. Identificar las **Características Mínimas Comercializables (MMF - Minimum Marketable Features)**, que representan tajadas funcionales completas y verticales del sistema [100, 143, Image 49].
 2. Descartar módulos secundarios (ej. la parte de administración avanzada).
 3. Definir las condiciones de aprobación y testing de regresión obligatorios.
 4. Lanzar el release una vez alcanzado el umbral funcional acordado.
- **Ejemplo concreto de la cátedra:** El reemplazo del sistema de autogestión de la UTN-FRC. No requiere hipótesis que validar porque los usuarios ya están cautivos y se conoce la necesidad. Por ende, el primer release en producción con el cobro básico y la inscripción es un **MMP puro**, no un MVP.
 - **Errores comunes / Advertencias:**
 - ⚠ *Llamar MVP a todo:* El 99% de las empresas de la industria dice “MVP” cuando en la realidad técnica están lanzando un **MMP** porque ya están entregando software funcionando directo a producción sin hipótesis comerciales que validar.

3.7.3. Producto Mínimo Adorable (MLP - Minimum Lovable Product)

3.7.3.1. Definición precisa y propósito

- **Definición de las Slides de la Cátedra (SLIDE_05):** Es la evolución del mínimo producto orientado a mercados maduros. Consiste en la porción de software que no solo cumple con la viabilidad funcional (MVP) y comercial (MMP), sino que **incorpora desde el primer día características de diseño, usabilidad y experiencia de usuario (UX)** de tal nivel que logran deleitar al cliente y evitan su deserción.
- **Qué problema resuelve:** “Cruza la grieta de la experiencia” [Image 54]. Resuelve la deserción silenciosa de usuarios. Hoy en día, si un usuario se descarga una app y la interfaz es fea, lenta o frustrante, **la desinstala de inmediato y no le da una segunda oportunidad**. El MLP evita tratar a la UX como “algo que vemos al final” del proyecto.

3.7.3.2. Los 10 bloques de construcción de la adorabilidad (De la pirámide de la cátedra)

[Image 57]

Para que un producto trascienda de “usable” a “adorable”, la cátedra detalla una jerarquía estructural de diez bloques [Image 57]:

▲

/

/HAL

/

```
/\  
/ MOT   DIV \  
/\  
/CON_M   CON_S   ESC   SOS\  
/\  
/ ESP   SAT   CUI \  
/\  

```

1. **Halo** (Cúspide de la pirámide): La magia o identidad intangible que hace única a la marca o al producto [Image 57].
2. **Motivación**: Generar una razón de peso lúdico y emocional para que el usuario use la app de forma diaria [Image 57].
3. **Diversión**: Incorporar gamificación y elementos visuales interactivos amenos [Image 57].
4. **Confianza en uno mismo**: Que la usabilidad de la interfaz empodere al usuario y lo haga sentir inteligente [Image 57].
5. **Confianza en el sistema**: Seguridad de datos y ausencia total de fallas técnicas o cuelgues [Image 57].
6. **Escala**: Estabilidad de performance del sistema ante el crecimiento de usuarios concurrentes [Image 57].
7. **Sostenibilidad**: Que el ritmo de valor lúdico y de soporte técnico sea constante en el tiempo [Image 57].
8. **Esperanza**: Transmitir al usuario que el sistema resolverá de forma real sus problemas de vida [Image 57].
9. **Satisfacción**: Cumplimiento del núcleo funcional demandado con holgura [Image 57].
10. **Cuidado** (Base de la pirámide): Atención extrema a los pequeños detalles de accesibilidad y atención al cliente [Image 57].

3.7.4. Justificación valor por valor de la Escala de Fibonacci Modificada (Mazu de Cartas del TP4)

Para el examen práctico de la UTN-FRC, se exige de forma rigurosa que el grupo declare en su PGC la **escala de Fibonacci Modificada** y justifique de forma milimétrica el significado cualitativo de cada valor, indicando cuándo se usa y proveyendo un ejemplo funcional específico.

Tabla General de Decodificación y Justificación de la Escala de SCM

Valor de Fibonacci	Significado Cualitativo SCM	Cuándo se usa operativamente	Ejemplo de Funcionalidad Concreta en el Repositorio
0	Esfuerzo nulo. No aporta peso a la velocidad.	Para restricciones del sistema o Requerimientos No Funcionales (RNF) transversales.	<i>“El sistema debe ejecutarse en servidores LinuxUbuntu en la nube”</i> (No requiere codificar un flujo de negocio).
0.5 (1/2)	Fracción mínima de tamaño. Menor que la canónica.	Para funcionalidades ultra simples o cosméticas surgidas tras elegir la canónica.	Cambiar el color de un botón de confirmación o corregir una errata ortográfica en el título del login.
1	El “Metro P Patrón” base. d Unidad base de comparación.	Para la **Historia * e Usuario q Canónica** inicial elegida por el equipo.	<i>“Como Administrador quiero visualizar el listado de libros registrados”*</i> (Complejidad nula, esfuerzo mínimo, duda cero).
2	Funcionalidad pequeña. Doble que la canónica.	Para requerimientos sumamente sencillos, conocidos y de bajo impacto lógico.	<i>“Como Cliente quiero eliminar un ítem seleccionado de mi carrito de compras”.</i>
3	Funcionalidad pequeña a mediana. “El dulce spot” de c la planificación.	Para historias de usuario estándar con ógica de negocio p ontrolada y sin (dependencias.	<i>“Como Cliente quiero modificar mis datos de perfil de usuario nombre y teléfono”.</i>
5	Funcionalidad mediana. Límite del confort técnico.	Para historias que combinan esfuerzo físico de codificación y lógica de base de datos.	<i>“Como Cliente quiero registrarme en la plataforma completando mi e-mail y clave con validación”.</i>
8	Funcionalidad grande. Alerta de límite de sprint.	Para historias complejas de programar y probar de forma individual.	<i>“Como Cliente quiero pagar mi compra integrando el SDK externo de Mercado Pago”</i> (Complejidad alta, duda media).
13	Alerta roja de partición. Dificultad para entrar en el sprint.	No debería permitirse su ingreso directo al desarrollo; se debe partir de forma obligatoria.	<i>“Como Estudiante quiero inscribirme a materias, pagar la cuota online y descargar el comprobante”</i> (Debe dividirse).

Valor de Fibonacci	Significado Cualitativo SCM	Cuándo se usa operativamente	Ejemplo de Funcionalidad Concreta en el Repositorio
20	Épica o incertidumbre masiva.	Para funcionalidades de gran envergadura técnica que no cumplen con INVEST.	“Como Gerente quiero generar reportes predictivos de facturación con gráficos dinámicos” (Muta a Spike de investigación).
40	Épica pura de Product Backlog.	Para agrupar requerimientos abstractos de largo plazo en el backlog inicial.	“Módulo completo de Facturación Electrónica e Integración con AFIP”.
100	Incoherencia lógica o desastre técnico.	Denota que la historia está tan mal redactada o posee contradicciones que mejor ni arrancar .	“Como Sistema quiero validar las transacciones offline sin conexión y garantizar sincronización instantánea 100% de datos”.
?	Incertidumbre absoluta.	Cuando los desarrolladores no comprenden las reglas de negocio de la US.	Un requerimiento que involucra normativas impositivas de exportación que el PO no ha definido en absoluto.
🕒	Saturación y cansancio mental.	Se muestra de forma unánime para solicitar un recreo temporal en el poker de estimación.	El equipo lleva 4 horas consecutivas debatiendo puntos de historia y la calidad de la estimación empieza a decaer de forma drástica.

3.7.5. La Contradicción de Estimación: Story Points frente a Días Ideales (Ideal Days)

Una de las discusiones teóricas y de gestión más intensas que plantea la cátedra se basa en la confrontación entre los dos métodos de medición del tamaño sugeridos por **Mike Cohn (Capítulo 8 de Agile Estimating and Planning)**.

|| CONTRADICCIÓN DE LA ESTIMACIÓN ||
 [STORY POINTS (SP)] [DÍAS IDEALES (IDEAL DAYS)]

- Medida abstracta de tamaño. - Tiempo de desarrollo puro y aislado
- Justificada por las 3 burbujas. sin interrupciones cotidianas.
- Estimable 100% por analogía. - Es más fácil de explicar al negocio.

[EXIGIDO POR LA CÁTEDRA] [RECHAZADO POR LA CÁTEDRA]

Asegura transparencia y velocidad Muta a horas absolutas, rompe el empírica constante en Git. consenso y tienta al micromanagement.

A. El punto de vista de Mike Cohn (La viabilidad del debate)

En su bibliografía, Cohn admite que **ambas son opciones viables** y describe las ventajas de cada una:

- **Las ventajas de los Días Ideales:** Son sumamente fáciles de comprender y explicar a los gerentes de la empresa y personas externas al equipo técnico. Además, es más sencillo y natural para un equipo novato comenzar a estimar pensando en “días de trabajo sin interrupciones” que asimilar la abstracción mental de los Story Points.
- **Sin embargo, Cohn prefiere los Story Points** porque son una medida pura de tamaño que no se degrada con el aumento de la excelencia técnica del equipo, promueve la colaboración cross-funcional y evita disputas de tiempos.

B. El punto de vista y exigencia de la Cátedra de la UTN-FRC (Rechazo absoluto de Días Ideales)

La cátedra de Meles y Covaro **prohíbe de forma tajante estimar en Días Ideales**, exigiendo de manera excluyente el uso de **Story Points** basados en las tres dimensiones de justificación cualitativa.

El argumento destructivo del docente para rechazar los Días Ideales en los exámenes:

El profesor fundamenta que el concepto de “Días Ideales” es una **trampa de gestión** debido al contexto cultural de las organizaciones tradicionales:

1. **La comparación errónea con el Calendario:** Al utilizar la palabra “Días”, la gerencia o los clientes inevitablemente caerán en la tentación de **comparar linealmente los días ideales con los días reales de calendario**. Los desarrolladores se verán sometidos a la presión de tener que justificar administrativamente de forma constante por qué “solo entregaron 7 días ideales de trabajo en una iteración real de 10 días”.
2. **La degradación de la métrica:** Para sobrevivir a esta presión gerencial de control, el equipo técnico terminará redefiniendo tramposamente el “día ideal” como “un día de 6 horas reales”, **destruyendo la abstracción del tamaño** y volviendo de manera encubierta a la ineficiente estimación absoluta de horas lineales.
3. **“Mis días ideales no son tus días ideales” (La asimetría del Seniority):** Si un desarrollador Senior estima una historia en 2 días ideales, pero por cuestiones de asignación del sprint la termina programando un Junior, la tarea demandará 5 días reales de calendario. El cronograma se rompe de inmediato porque el “día ideal” es una variable que depende estrechamente de la habilidad individual de quien tira código, a diferencia de los Story Points que expresan el tamaño físico del producto de forma unificada e independiente del ejecutor.

Puntos clave para el examen — 3.7.5

1. **EL MVP PUEDE SER UN VIDEO O UNA MAQUETA (AUDIO_05):** Durante la clase teórica de SCM y Requerimientos, la profesora Judith Meles es sumamente enfática: en los

exámenes parciales de requerimientos ágiles, si te piden diseñar un MVP para validar una hipótesis de negocio lúdica, **no cometes el error de proponer codificar el software entero** de entrada. El MVP busca el menor costo posible de aprendizaje; validar mediante una maqueta interactiva de Figma o un video explicativo (como hizo Dropbox) es la respuesta de excelencia ingenieril.

2. **LA CONTRADICCIÓN ENTRE MVP Y MMP EN LA INDUSTRIA (AUDIO_05):** El docente de práctico destaca que en las empresas de desarrollo se suele usar el término MVP de forma sistemática y desvirtuada para referirse a la “Versión 1” o primer release funcional del software. Como ingenieros, debemos marcar la diferencia en el examen: **el MVP se enfoca en el aprendizaje y validación de hipótesis; el MMP exige obligatoriamente software funcionando de calidad para ser lanzado a producción.**
3. **EL PESO DE LA JUSTIFICACIÓN EN EL PARCIAL:** El profesor advierte de forma explícita que colocar un número suelto de la escala de Fibonacci sin justificar de manera textual el balance cualitativo de las tres burbujas de dimensiones (Complejidad, Esfuerzo, Incertidumbre) representa un **cero (o) directo en el puntaje de esa pregunta en el parcial práctico.**

Preguntas de autoevaluación — 3.7.5

Pregunta 1: ¿Cómo se define el Producto Mínimo Viable (MVP) según la cátedra y qué busca validar?

- **Respuesta breve:** Es una versión acotada y de bajo costo de un producto diseñada exclusivamente para recolectar el máximo aprendizaje validado sobre los usuarios, permitiendo verificar de forma empírica supuestos e hipótesis de negocio.

Pregunta 2: ¿Por qué un MVP puede no ser software en funcionamiento? Dé un ejemplo.

- **Respuesta breve:** Porque su único fin es validar la demanda y el interés humano. Un ejemplo es Dropbox, que validó su mercado y capturó miles de usuarios usando un simple video narrativo de 3 minutos sin escribir código.

Pregunta 3: ¿Cuál es la diferencia de negocio fundamental entre un MVP y un Producto Mínimo Comercializable (MMP)?

- **Respuesta breve:** El **MVP** se enfoca en el aprendizaje y validación de hipótesis sin necesidad de código; el **MMP** es software funcionando, testado y de calidad que se pone físicamente en producción para generar valor inmediato al negocio.

Pregunta 4: ¿Qué es el Producto Mínimo Adorable (MLP) y por qué surge en el mercado actual?

- **Respuesta breve:** Es la porción de software que incorpora desde el día uno diseño y usabilidad de excelencia para deleitar al usuario. Surge ante la enorme oferta de software actual, donde una mala experiencia de usuario (UX) provoca la desinstalación inmediata de la app.

Pregunta 5: ¿Qué representa el valor de “o” en la escala de Fibonacci Modificada de la cátedra?

- **Respuesta breve:** Representa restricciones o Requerimientos No Funcionales (RNF) transversales que el equipo de desarrollo debe cumplir obligatoriamente pero que no suman Story Points ni peso físico a la velocidad de la iteración.

Pregunta 6: ¿Para qué se reserva el valor de “13” en la escala de estimación y qué exige hacer el modelo INVEST?

- **Respuesta breve:** Denota una funcionalidad demasiado grande o compleja para un sprint. Exige de forma obligatoria particionar verticalmente la historia en porciones de menor denominación para cumplir con el atributo **Small**.

Pregunta 7: ¿Qué significan y para qué sirven las cartas especiales “?” y de la taza de café ☕?

- **Respuesta breve:** El ? indica incertidumbre lógica total, obligando a transformar la US en un Spike de investigación; la taza de café ☕ denota saturación mental del equipo, deteniendo el poker de estimación para realizar un recreo.

Pregunta 8: ¿Por qué la cátedra de Ingeniería de Software de la UTN-FRC rechaza la estimación en Días Ideales?

- **Respuesta breve:** Porque la gerencia tradicional inevitablemente los compara con días reales de calendario, desvirtuando el tamaño por el seniority del desarrollador asignado y tentando a los programadores a colocar “colchones” de seguridad que rompen la transparencia.

Pregunta 9: Explique la analogía de Cohn de “los dos corredores en el sendero” aplicada a la estimación.

- **Respuesta breve:** Dos corredores (uno rápido y uno lento) acuerdan que el sendero mide 1 km (medida de tamaño / Story Point), pero discreparán sobre cuánto tardarán en correrlo (tiempo de calendario / Días Ideales), demostrando que el tiempo absoluto no sirve de consenso de equipo.

Pregunta 10: ¿Cómo define el manifiesto ágil la simplicidad en el desarrollo de productos mínimos?

- **Respuesta breve:** El principio número 10 lo define como “*el arte de maximizar la cantidad de trabajo no realizado*”, enfocando al equipo en eliminar de cuajo funcionalidades de desperdicio que no aportan valor real.

3.8. Poker de Estimación (Poker Estimation)

3.8.1. Definición y Fundamento Metodológico

- **Definición de las Notas de Clase (NC_MartinBosi / NC_MartinezBaigorria):** El Poker de Estimación es la dinámica sistemática de carácter colaborativo, horizontal y gamificado que utiliza el equipo de desarrollo para estimar el tamaño relativo (en puntos de historia) de las historias de usuario del backlog.
- **Definición de la Bibliografía (Mike Cohn, Agile Estimating and Planning, Cap. 6)**:** El Poker de Estimación (Planning Poker) es un enfoque ágil de estimación que combina la opinión de expertos, la analogía y la desagregación de componentes en una actividad grupal amena, rápida y confiable.
- **Qué problema resuelve:** Destruye de raíz el sesgo de **anclaje cognitivo (anchoring)** y la asimetría de autoridad en el equipo. En las reuniones tradicionales, los perfiles más extrovertidos o senior imponían de forma prematura sus estimaciones absolutas en horas, silenciando e influenciando al resto. Al forzar una votación ciega y simultánea, se incentiva la

total transparencia, la participación equitativa y la emergencia de supuestos de diseño técnicos que de otro modo pasarían desapercibidos.

3.8.2. Contradicción Terminológica: “Poker Estimation” vs. “Planning Poker”

En la cátedra de Ingeniería de Software de la UTN-FRC, las profesoras Judith Meles y Laura Covaro exigen de forma excluyente y punitiva utilizar el término **“Poker Estimation” (Póker de Estimación)** y rechazan tajantemente las denominaciones comerciales y populares de **“Planning Poker”** o **“Poker Planning”**.

El argumento de desaprobado directo del docente (AUDIO_05):

El docente explica en clase que esta distinción no es un capricho semántico, sino un límite conceptual crítico en la formación de un ingeniero [AUDIO_05]:

1. **La diferencia entre estimar y planificar:** Estimar es predecir una situación que ocurrirá en el futuro en base a observaciones sistemáticas (en este caso, predecir el **tamaño relativo** o la magnitud física del software a construir). Planificar es un proceso inmensamente más complejo que toma esa estimación de tamaño como entrada y, cruzándola con variables de negocio, compromisos, plazos de calendario y la capacidad del equipo, diseña un plan y asume un compromiso de entrega.
2. **La trampa del “Planning Poker”:** Utilizar la palabra “Planning” (Planificación) para describir lo que en realidad es una técnica de estimación de tamaño relativo fomenta la peligrosa confusión de **asociar la estimación con un compromiso contractual directo**. Si los programadores creen que están “planificando” al votar una carta, se verán tentados a inflar los números con “colchones de seguridad” (sobreestimación defensiva) para resguardarse de la gerencia, corrompiendo la transparencia de la métrica.
3. **La regla de oro de la cátedra:** El Póker de Estimación se usa **únicamente para estimar tamaño relativo** (story points). La planificación del sprint (Sprint Planning) es un evento posterior y separado de Scrum donde el equipo, basándose en la velocidad calculada y el objetivo del sprint, se compromete con un alcance funcional específico.

3.8.3. La Dinámica Paso a Paso y el Rol de cada Participante

La dinámica de Poker Estimation exige la participación de roles específicos que operan bajo reglas de juego rígidas para garantizar el éxito y la agilidad de la reunión.

```
[ EVENTOS Y PASOS EN EL POKER ESTIMATION ]  
[Paso 1: Presentación (PO)] -> [Paso 2: Aclaración (PO+Devs)] -> [Paso 3: Selección Oculta (Devs)]  
[Paso 6: Consenso y Registro] <- [Paso 5: Debate de Extremos] <- [Paso 4: Revelación (Simultánea)]
```

Los Roles en la Reunión:

- **El Product Owner (PO):** Es el dueño del backlog de requerimientos. Su función es funcional y de negocio; no técnica. **No vota ni estima**, ya que de acuerdo a las pautas de Cohn y de la cátedra, **“las personas más competentes en resolver una tarea deben ser quienes la estimen”** (los que van a meter las manos en el código y el testing).

- **El ScrumMaster / Facilitador:** Es el guardián del proceso y las reglas de SCM. Modera la reunión, gestiona los bloqueos y vigila que no se violen las reglas lúdicas de la estimación.
- **Los Product Developers / Estimadores (Programadores, Testers, DBAs, UX designers):** Son el equipo multidisciplinario que construirá el software. Tienen la responsabilidad y el derecho exclusivo de votar y definir el tamaño final de las historias de usuario.

Procedimiento de Trabajo Detallado de la Dinámica:

Paso 1: Determinación de la US Canónica y Presentación del Requerimiento

- **Qué se hace:** Se debe establecer primero la historia de referencia (canónica) de tamaño 1 o 2 SP. Una vez fijado el “metro patrón”, se toma la primera US prioritaria de la pila. El moderador lee la descripción textual de la tarjeta (frente) y explica detalladamente el valor que aporta al negocio.
- **Acción del PO:** Lee la historia en voz alta y expone las intenciones y necesidades de negocio de los usuarios.
- **Acción del ScrumMaster:** Asegura que todo el equipo tenga a mano sus mazos de Fibonacci Modificada (ya sea físicos o mediante una aplicación móvil remota).
- **Acción de los Developers:** Escuchan con atención el alcance de la funcionalidad.

Paso 2: Aclaración de dudas y diseño técnico rápido (Timebox)

- **Qué se hace:** Los estimadores realizan preguntas de diseño, casos de prueba y reglas de negocio para despejar ambigüedades. Es un debate técnico y funcional de alto nivel.
- **Acción del PO:** Responde de manera concreta y precisa las consultas de los programadores, definiendo qué se considera un escenario feliz o de error.
- **Acción del ScrumMaster:** Activa un **reloj de arena o timer de 2 minutos** (timebox) en el centro de la mesa para evitar que el debate se extienda infinitamente en discusiones de diseño de bajo nivel.
- **Acción de los Developers:** Preguntan sobre dependencias, validaciones de interfaz, estructura de datos y estrategias de testing. Proponen ideas rápidas de implementación.

Paso 3: Selección de carta en privado (Votación Oculta)

- **Qué se hace:** Cada estimador decide de forma autónoma qué número de la escala refleja mejor el tamaño comparativo de la historia.
- **Acción del PO:** Se mantiene al margen de la mesa en silencio para no inducir presión psicológica.
- **Acción del ScrumMaster:** Controla que se respete la regla de oro: **nadie puede decir números en voz alta ni hacer gestos** que revelen su voto antes de tiempo.
- **Acción de los Developers:** Comparan la US bajo análisis contra la canónica y sus tres burbujas (Complejidad, Esfuerzo, Incertidumbre). Seleccionan una carta de su mazo y la colocan **boca abajo** sobre la mesa.

Paso 4: Revelación simultánea (El momento del “Poker”)

- **Qué se hace:** Se exponen todos los votos de forma colectiva y transparente.
- **Acción del PO:** Observa la distribución y variabilidad de los números de tamaño.
- **Acción del ScrumMaster:** Da la señal de partida (a la cuenta de tres) para dar vuelta las cartas al mismo tiempo.
- **Acción de los Developers:** Voltean sus cartas simultáneamente para que todos los integrantes visualicen los puntajes asignados.

Paso 5: Debate y justificación de los valores extremos (Divergencia)

- **Qué se hace:** Si existe consenso unánime, se registra ese valor y se avanza a la siguiente US. Si hay diferencias notables en los votos (ej. un estimador votó 2 y otro 13), se abre una instancia formal de aclaración orientada **únicamente a los dos extremos**.
- **Acción del PO:** Escucha la discusión técnica para tomar nota de restricciones funcionales o supuestos que deba documentar al dorso de la tarjeta (criterios de confirmación).
- **Acción del ScrumMaster:** Evita de forma estricta que el equipo cometa el error grave de “promediar matemáticamente” las discrepancias (ej: promediar un 3 y un 13 y clavar un 8). Concede la palabra de manera prioritaria al voto más bajo y al voto más alto.
- **Acción de los Developers:**
- **El voto más bajo (Optimista):** Explica por qué considera que la tarea es sumamente sencilla, detallando si existe una biblioteca preconstruida que resuelva la lógica o si el diseño arquitectónico es más simple de lo que parece.
- **El voto más alto (Pésimo/Cauteloso):** Argumenta qué complejidades de integración, volumen físico de datos, riesgos imprevistos de testing o dependencias funcionales detectó y que el resto del equipo pasó por alto.

Paso 6: Re-votación y Consenso final

- **Qué se hace:** Tras escuchar los dos puntos de vista extremos, el equipo adquiere un entendimiento compartido superior de la funcionalidad. Se vuelve a barajar y se ejecuta una nueva ronda de votación a ciegas.
- **Acción del PO:** Registra el valor final de Story Points en el Backlog una vez alcanzado el consenso.
- **Acción del ScrumMaster:** Modera la re-votación (GOTO Paso 3). Habitualmente no se requieren más de dos o tres rondas de votación por historia para que los números converjan de forma natural.
- **Acción de los Developers:** Vuelven a votar de forma silenciosa e independiente con su carta reajustada en base al conocimiento técnico compartido durante la ronda de discusión.
- **Ejemplo concreto de la cátedra:**

En el simulacro de la cursada del TP4 práctico, el Product Owner del grupo lee la US de “Inscripción a Parciales Prácticos”. En la primera ronda de votación, el tester vota con un **8** por el esfuerzo que implicará diseñar los casos de prueba de borde con notas negativas, el programador front-end vota un **2** asumiendo que es solo un formulario simple con dos botones, y el DBA vota un **5** por la

conurrencia de base de datos a la hora de asignación de turnos. Al abrir el debate de extremos (tester y programador front-end), el equipo comprende que hay que validar de forma segura que el sistema no asigne bancos excedentes en el laboratorio, lo que demanda lógica de backend intermedia. En la segunda ronda, todos convergen de forma unánime y consensúan la estimación de la US en **5 Story Points**.

3.9. Políticas de Reestimación (Re-Estimating)

- **Definición del Concepto (Mike Cohn - Cap. 7):** La reestimación es la actividad periódica de mantenimiento del backlog mediante la cual el equipo de desarrollo ajusta el tamaño relativo (Story Points) de las historias de usuario que aún no se han completado.
- **Qué problema resuelve:** Mantiene la consistencia del “metro patrón” de estimación relative a lo largo del tiempo, absorbiendo los cambios en la tecnología, el dominio del negocio o la madurez del propio equipo para asegurar la predictibilidad de los planes de release.

ÁRBOL DE DECISIÓN DE SCM EN REESTIMACIÓN
¿Cambié el TAMAÑO RELATIVO
de la historia (US)?
(SÍ) (NO)
[REESTIMAR LA US] [NO REESTIMAR LA US]
(Aprender del error técnico (La Velocidad absorbe el desvío;
y reajustar el Backlog) el tamaño físico es idéntico)

3.9.1. Cuándo SÍ se debe reestimar una US:

1. **Descubrimiento de nueva información técnica o funcional:** Únicamente cuando el equipo, al avanzar en el desarrollo, adquiere información disruptiva de arquitectura o de negocio que demuestra que el tamaño relativo asignado originalmente era erróneo (ej. descubrimos que la API de integración no provee el set de datos esperado y hay que desarrollar un web scrapper de cero).
2. **Identificación de incoherencias en la escala comparativa:** Al incorporar nuevas historias al Product Backlog, el equipo detecta de forma evidente que una historia vieja de 5 SP tiene exactamente la misma magnitud de complejidad y esfuerzo que una nueva historia calificada con unánime consenso en 2 SP. En este caso, se reestima la historia desalineada para preservar la coherencia y balance de la escala de analogía.
3. **Partición y redefinición de historias parcialmente completadas:** Cuando una historia de usuario queda incompleta al final de un sprint y se decide particionarla para el siguiente. La porción restante (un subconjunto del requerimiento) se redacta como una nueva historia de usuario y se estima de cero en base al estado del conocimiento actual del equipo.

3.9.2. Cuándo NO se debe reestimar una US:

1. **Desvíos en el esfuerzo temporal o retrasos de calendario:** Nunca se debe alterar el puntaje de una historia de usuario simplemente porque demandó más horas reales de trabajo de las previstas originalmente. Si el desarrollador se demoró por cortes de energía, problemas personales, falta de concentración, reuniones de pasillo o porque el equipo estuvo

más lento durante el sprint, **el tamaño físico de la funcionalidad (el producto) no varió en absoluto**; lo único que se vio afectado fue el esfuerzo temporal. La estimación se mantiene fija y se deja que la métrica de **Velocidad** disminuya de forma natural, revelando de forma transparente la caída de productividad a la gerencia.

2. **Historias de usuario completadas con éxito:** Una vez que una historia se ha completado al 100%, testeado y aceptado por el PO, **su valor de Story Points se clava de forma inalterable** para el cálculo de la velocidad. Aunque se reconozca que un ítem estimado en 3 SP costó el esfuerzo real de una de 8 SP, se le otorgan al equipo los 3 SP originales. Modificar cosméticamente el puntaje de las historias terminadas para “hacer encajar” las horas de trabajo desvirtúa de forma catastrófica la base matemática de la velocidad, destruyendo su capacidad de ecualizar futuros planes de release.
3. **Variación por el Seniority del programador asignado:** El tamaño relativo de una US es una propiedad física intrínseca del software, independiente de quién la codifique. Que a un programador junior le cueste 5 días reales de calendario resolver una historia de 2 SP que el senior liquida en 3 horas de reloj no justifica en absoluto modificar la estimación de la US. La asimetría técnica se verá reflejada y absorbida de forma natural por la velocidad promedio histórica del equipo.

3.10. La Ecuación Central Ágil y la Métrica de Velocidad

3.10.1. La Ecuación Central de SCM y Derivación del Calendario

- **Definición de las Slides de la Cátedra (SLIDE_04 / NC_Ardiles):** Es la regla de oro que rige la planificación ágil de proyectos. Postula la prohibición absoluta de estimar plazos y fechas de calendario de forma directa e intuitiva al inicio de un desarrollo. En su lugar, el agilismo exige **estimar el tamaño del software en Story Points** (magnitud física), medir empíricamente la **velocidad real de quema de puntos del equipo** en la cancha y, a partir de la relación matemática de ambas variables, **calcular y derivar la duración del calendario**.

Fórmula Central Ágil:

$$\begin{aligned} & Duración(Iteraciones) \\ & = \frac{Tamaño\ total\ del\ Backlog(Story\ Points)}{Velocidad\ del\ equipo(Puntos\ por\ Iteración)} \end{aligned}$$

- **Qué problema resuelve:** Erradica el fracaso crónico que provocan los **cronogramas irreales dictados de forma unilateral por el negocio o marketing**. Al basar la proyección temporal en una ecuación matemática alimentada con datos empíricos de producción real del equipo, evita la subestimación defensiva y la acumulación sistemática de deuda técnica en el repositorio.

Ejemplos Numéricos Resueltos Paso a Paso

EJEMPLO 1: Planificación de un Incremento (MMP) con Velocidad Histórica Estable

- **Contexto:** El Product Owner del proyecto “Matrix” de la UTN-FRC requiere planificar el primer release comercial (MMP) del sistema. El equipo de desarrollo trabaja con sprints fijos e inamovibles (**timebox**) de **2 semanas de duración (10 días hábiles de trabajo)**.
- **Datos de entrada:**
- **Suma de Story Points de todas las US priorizadas para el MMP: 120 SP.**
- **Velocidad real de los últimos 4 sprints completados por el equipo:**
- Sprint 1: 22 SP completados y aprobados por el PO.
- Sprint 2: 18 SP completados y aprobados por el PO.
- Sprint 3: 25 SP completados y aprobados por el PO.
- Sprint 4: 15 SP completados y aprobados por el PO.
- **Resolución Paso a Paso:**
- 1. **Calcular la Velocidad Promedio Histórica (*promediodeV*):**
$$promediodeV = \text{Sumatoria de Velocidades} / \text{Cantidad de Sprints} = (22 + 18 + 25 + 15) / (4) = (80) / (4) = 20 \text{ Story Points por Sprint}$$
- 2. **Aplicar la Ecuación Central para derivar la Duración en Iteraciones (*D*):**
$$D = \text{Tamaño del Backlog (SP)} / \text{Velocidad Promedio} = 120 \text{ SP} / 20 \text{ SP/Sprint} = 6 \text{ Sprints}$$
- 3. **Calcular la Duración en Calendario Físico Real (*C*):**
$$C = D \times \text{Timebox del Sprint} = 6 \text{ Sprints} \times 2 \text{ semanas/Sprint} = 12 \text{ semanas físicas}$$
- **Resultado del Plan de SCM:** Con un nivel de confianza extremadamente sólido respaldado por la velocidad real medida del equipo, el release comercial (MMP) requerirá un calendario fijo de **6 sprints completos, equivalentes a 12 semanas reales de calendario (aproximadamente 3 meses)**.

EJEMPLO 2: Proyección de Proyecto en Cono de Incertidumbre Alto con Rango de Desempeño

- **Contexto:** El grupo de desarrollo inicia un módulo para el sistema de soporte meteorológico “Clima”. Al ser un desarrollo con tecnología no familiar, no poseen historial de velocidad previa y el nivel de ignorancia al inicio es muy alto. El equipo ha definido un **timebox** rígido de sprint de **3 semanas (15 días hábiles)**.
- **Datos de entrada:**
- **Tamaño total estimado del Product Backlog: 160 Story Points (SP).**
- **Velocidad Suposicional / Esperada para el Sprint 1:** El equipo desglosa tareas funcionales para la primera iteración y estima de forma suposicional que su capacidad técnica inicial será de **16 SP por Sprint**.
- **Margen del Cono de Incertidumbre de Barry Boehm / Mike Cohn (Fase Inicial):** Dado que no hay historial empírico, la cátedra y la bibliografía exigen proyectar un rango de velocidad del **60% para el peor escenario (velocidad baja)** y del **160% para el mejor escenario (velocidad alta)** para reflejar la variabilidad intrínseca del inicio del proyecto.

- **Resolución Paso a Paso:**
- 1. **Calcular el Rango de Velocidades Probables:**
 - **Velocidad Pesimista** (V_{pes}): $VelocidadSuposicional \times 0.60 = 16SP \times 0.60 = 9.6SP$ por Sprint Redondeado a 10 SP/Sprint
 - **Velocidad Optimista** (V_{opt}): $VelocidadSuposicional \times 1.60 = 16SP \times 1.60 = 25.6SP$ por Sprint Redondeado a 26 SP/Sprint
- 2. **Calcular la Duración del Proyecto en Iteraciones para ambos extremos:**
 - **Escenario Pesimista** (Velocidad Baja = 10 SP):

$$D_{pésima} = 160SP / 10SP/Sprint = 16Sprints$$
 - **Escenario Optimista** (Velocidad Alta = 26 SP):

$$D_{óptima} = 160SP / 26SP/Sprint = 6.15$$
 Redondeado al entero superior: 7 Sprints
- 3. **Calcular el Rango de Calendario Físico:**
 - **Fecha de Entrega Pesimista:** $16Sprints \times 3semanas/Sprint = 48semanas$ de calendario
 - **Fecha de Entrega Optimista:** $7Sprints \times 3semanas/Sprint = 21semanas$ de calendario
 - **Resultado del Plan de SCM:** Al no tener velocidad empírica medida en el campo de juego, el equipo reporta al Product Owner que la duración del proyecto se expresa como un rango probabilístico de **entre 7 y 16 sprints (de 21 a 48 semanas físicas de calendario)**. El cono de incertidumbre se irá achicando y la brecha se reducirá de forma drástica de forma natural a partir del sprint 3, cuando ya contemos con datos reales medidos en el repositorio Git.

3.10.2. El Criterio Binario de la Velocidad (La Regla de Oro)

- **La Regla de Oro del Empirismo (AUDIO_05 / NC_Ardiles):** La velocidad de un equipo ágil es una métrica binaria y libre de subjetividad. **Se rige de forma estricta por el criterio del “todo o nada” (all-or-nothing).**
- Si al finalizar el sprint la historia de usuario está **100% completada, testeada en producción y aceptada formalmente por el Product Owner**, el equipo suma el 100% de sus Story Points a la velocidad de la iteración.
- Si a la historia de usuario le falta el más mínimo detalle funcional, de base de datos o de testing de QA (incluso si está al 99% de avance y el programador dice “falta subir un commit”), **la historia aporta exactamente CERO (0) puntos a la velocidad de ese sprint.**

Ejemplo Práctico de Velocidad de la Cátedra (TP4):

Supongamos que para la iteración actual el equipo se comprometió a desarrollar 3 Historias de Usuario:

- **US_01** (Modificar Datos de Perfil): **3 Story Points.**
- **US_02** (Pago con Tarjeta de Crédito): **5 Story Points.**
- **US_03** (Recuperar Contraseña por E-mail): **2 Story Points.**

Al finalizar el Sprint, la situación física en el repositorio Git es la siguiente:

- **US_01:** Completada al 100%, testeada y el PO firmó la aceptación funcional.
- **US_02:** El código backend está listo y el programador integró la API, pero el equipo de testing no llegó a ejecutar los casos de prueba de regresión automatizados (avance físico real estimado: 90%).
- **US_03:** El desarrollador front-end diseñó la pantalla y los estilos CSS, pero falta programar el script de envío de e-mail (avance físico real estimado: 50%).

El Cálculo de la Velocidad del Sprint:

$$Velocidad = SPUS_{01}(Terminada) + SPUS_{02}(Incompleta) + SPUS_{03}(Incompleta)$$

$$Velocidad = 3SP + 0SP + 0SP = 3StoryPoints$$

- **¿Qué pasa con los 7 puntos de las historias incompletas?:** Vuelven de inmediato al Product Backlog. El Product Owner analizará de forma soberana si siguen siendo una prioridad del negocio o si prefiere descartarlas por cambios de mercado. Si decide mantenerlas en el backlog para el siguiente sprint, **conservan su puntaje original intacto** (la **US_02** sigue valiendo 5 SP y la **US_03** sigue valiendo 2 SP, bajo ningún punto de vista se les cambia el tamaño de forma parcial a 2.5 SP o 1 SP para “cobrar el trabajo hecho”). Cuando se completen al 100% en la iteración futura, sumarán todo su peso de golpe a la velocidad de ese nuevo sprint.

Puntos clave para el examen — 3.10.2

1. **LAS ESTIMACIONES NUNCA SE NEGOCIAN, SE NEGOCIAN LOS COMPROMISOS (AUDIO_05 / NC_MartinBosi):** Judith Meles repite hasta el cansancio en clase que estimar es una actividad de diseño científico de ingeniería, libre de intereses comerciales. Si el equipo de desarrollo estimó que un requerimiento complejo vale **8 Story Points**, y el jefe de proyecto o el cliente presiona diciendo que “necesita que valga 3 para que entre en el plan”, **el estimador debe defender su mano con firmeza profesional**. Lo que sí se puede y debe negociar de forma flexible es el compromiso: **“Dado que este requerimiento de 8 puntos excede nuestra capacidad del sprint, negociemos recortar el alcance funcional de la US o postergar otras historias secundarias para la próxima entrega”**.
2. **LA VELOCIDAD NUNCA SE ESTIMAS, SE CALCULA (AUDIO_05 / SLIDE_04):** En los exámenes prácticos de estimaciones ágiles, es un error fatal de desaprobado directo que en tu documento de TP4 o respuesta de parcial escribas **“estimamos que la velocidad de nuestro equipo va a ser de 20 puntos para el Sprint 1”**. La velocidad es una **métrica puramente empírica, histórica y observada en producción**. Solo se calcula sumando y multiplicando al finalizar una iteración real. Al principio del proyecto, ante la ausencia total de datos de campo, se puede hablar exclusivamente de una **“velocidad esperada o suposicional”** de carácter temporal para iniciar el plan de release, aclarando explícitamente que carece de rigurosidad matemática y que debe ser descartada y ajustada de inmediato con la velocidad medida del Sprint 1.
3. **LA VELOCIDAD TIENE UN EFECTO ECUALIZADOR (AUDIO_05 / Cohn Cap. 7):** El docente de teoría resalta que la velocidad del equipo actúa como un **corrector automático de desvíos de estimación**. Si un equipo peca de optimismo ingenuo y estima

que todas las historias del backlog son extremadamente pequeñas (ej: asigna 2 SP a lo que en realidad merecía 5 SP), no importa. Su velocidad medida en la cancha será proporcionalmente baja (ej: completará solo 10 puntos por sprint en vez de los 25 previstos). Al aplicar la Ecuación Central, la baja velocidad ecualizará el plan de release, estirando de forma matemática el número de sprints reales necesarios para entregar el backlog completo, ajustando la fecha final de entrega a la capacidad productiva fáctica de los programadores.

4. **JUSTIFICAR CADA DIMENSIÓN EN EL PARCIAL (AUDIO_o6):** En la evaluación práctica del examen de la materia, los profesores no calificarán positivamente a aquellos alumnos que se limiten a volcar un número de Fibonacci de forma arbitraria sobre una historia de usuario. **Es obligatorio documentar de forma textual y cualitativa la justificación de las tres aristas del Story Point:** por qué para el grupo la complejidad es baja o alta, qué esfuerzo físico demanda la tarea y qué grado de incertidumbre o vacíos funcionales o tecnológicos persisten en el requerimiento. Un puntaje sin justificación de las tres burbujas se califica con **cero (0) puntos directos**.

Preguntas de autoevaluación — 3.10.2

Pregunta 1: ¿Cómo define Mike Cohn la técnica de “Planning Poker”?

- **Respuesta breve:** Es una técnica de estimación ágil colaborativa que combina de manera gamificada la opinión de expertos, el razonamiento por analogía y la desagregación de componentes para asignar puntos de historia relativos.

Pregunta 2: ¿Por qué las docentes de la UTN-FRC exigen denominar a la dinámica de cartas como “Poker Estimation” y rechazan el nombre “Planning Poker”?

- **Respuesta breve:** Porque “Planning” hace referencia a la planificación y el compromiso contractual, mientras que la dinámica es una técnica pura de **estimación de tamaño relativo**. Confundirlas atenta contra la transparencia de la medición.

Pregunta 3: ¿Cuál es el rol del Product Owner (PO) en la dinámica de Poker Estimation y por qué no tiene derecho a voto?

- **Respuesta breve:** El PO actúa como moderador e informador funcional, leyendo la historia y aclarando dudas de negocio. No tiene derecho a voto porque no forma parte del equipo técnico y las estimaciones deben realizarlas de forma exclusiva quienes ejecutarán el desarrollo.

Pregunta 4: ¿Qué función cumple la carta de Fibonacci Modificada “40” o “100” en el mazo de un estimador?

- **Respuesta breve:** El **40** indica que la US es una Épica de enorme magnitud imposible de completar en un sprint, obligando a su partición; el **100** delata una contradicción lógica grave o desastre en la definición, exigiendo rediseñarla antes de estimar.

Pregunta 5: ¿Cuándo corresponde reestimar de forma correcta una historia de usuario según la cátedra?

- **Respuesta breve:** Únicamente cuando se descubre nueva información técnica o de negocio que altera el **tamaño relativo intrínseco** o la complejidad real del requerimiento respecto al supuesto original del backlog.

Pregunta 6: ¿Por qué es un error grave de SCM modificar el puntaje de una historia de usuario que demandó más horas de desarrollo de las previstas inicialmente?

- **Respuesta breve:** Porque el tamaño físico de la funcionalidad no varió; solo se vio afectado el esfuerzo temporal debido a desvíos o ineficiencias del equipo. Reestimar destruye la métrica de velocidad, que debe bajar de forma transparente.

Pregunta 7: Escriba la fórmula matemática de la Ecuación Central de la planificación ágil de proyectos.

- **Respuesta breve:** La ecuación es: $Duración(Iteraciones) = \frac{Tamaño\ total\ del\ Backlog(SP)}{Velocidad\ real(puntos/iteración)}$.

Pregunta 8: ¿Por qué la duración de las iteraciones (timebox) en metodologías como Scrum es fija e inamovible?

- **Respuesta breve:** Para asegurar el ritmo sostenible de trabajo, forzar hitos constantes de control de calidad mediante software funcionando y asegurar que la variable de ajuste ante desvíos sea el alcance y no los plazos del calendario.

Pregunta 9: Explique el “criterio binario” del cálculo de la velocidad en la agilidad.

- **Respuesta breve:** Es el principio del “todo o nada”: solo suman puntos a la velocidad de la iteración las historias de usuario completadas al 100% y aprobadas por el PO. Las historias inconclusas (incluso al 99% de avance) aportan exactamente **cero (0) puntos**.

Pregunta 10: ¿Qué sucede administrativamente con una historia de usuario que quedó incompleta (porcentaje parcial) al finalizar el sprint?

- **Respuesta breve:** Vuelve de forma íntegra al Product Backlog manteniendo intacto su puntaje de Story Points original para que el PO evalúe si la prioriza para el siguiente sprint. No se le computan puntos parciales a la velocidad.

3.11. El espectro de los Productos Mínimos (MVP, MMP, MLP), validación de hipótesis y el corte vertical

La gestión ágil de requerimientos y la planificación de entregas no se limitan a redactar tarjetas o asignarles puntos mediante dinámicas lúdicas. El verdadero desafío de la Ingeniería de Software reside en diseñar estrategias de liberación incremental que permitan validar hipótesis de negocio, mitigar riesgos técnicos extremos y asegurar el retorno de inversión del cliente lo antes posible.

En este bloque, se desarrolla en profundidad la arquitectura del valor de los productos mínimos y el arte de la partición de requerimientos.

3.11.1. Producto Mínimo Viable (MVP - Minimum Viable Product)

Definición precisa del concepto

- **Definición formal (Eric Ries, The Lean Startup - Capítulo 6 - BIBLIO_):** El Producto Mínimo Viable (MVP) es una versión de un producto nuevo que permite a un equipo recolectar la máxima cantidad de **aprendizaje validado (validated learning)** sobre el comportamiento y las necesidades de los clientes con el menor esfuerzo y costo de desarrollo posibles.
- **Qué problema resuelve:** Elimina de raíz la **parálisis por análisis** y el desperdicio financiero catastrófico de construir características complejas que los usuarios no necesitan o no están dispuestos a usar. Mitiga el **riesgo del cero absoluto** (la audacia de lanzar un

producto asumiendo supuestos de mercado que no han sido contrastados con la realidad empírica).

Cómo se hace: El procedimiento del ciclo Lean

El desarrollo de un MVP no consiste en programar un software fragmentado o incompleto (un “producto cortado de forma horizontal” donde solo se entrega la base técnica, como muestra la mala práctica de la pirámide de construcción).

El MVP es un **experimento científico** cohesivo diseñado para confrontar la realidad lo antes posible. Se implementa mediante el siguiente flujo secuencial:

```
[ EL PROCESO CIENTÍFICO DE VALIDACIÓN DEL MVP ]  
[ DECLARAR HIPÓTESIS ] [ AJUSTAR EL ALCANCE ] [ CONSTRUIR EXPERIMENTO ]  
Definir el supuesto lúdico Seleccionar solo las MVFs Video, landing page, maqueta  
de "salto de fe" a probar. indispensables ("table stakes") o prototipo sin backend.
```

1. **Declarar la Hipótesis:** Redactar una suposición empírica medible sobre la interacción del usuario (ej. **“los visitantes del parque están interesados en autogestionar su comida a través de una app”**).
2. **Ajustar el Alcance:** El alcance técnico se subordina de forma estricta a la hipótesis de valor. Solo se incluyen las **Características Mínimas Viables (MVF - Minimum Viable Features)** que actúan como “nodos translúcidos” para explorar e investigar la nueva área de valor.
3. **Construir el Experimento:** Desarrollar el canal de validación más barato y rápido posible (entrevistas, maquetas interactivas en Figma, o videos demostrativos).
4. **Medir y Pivotar:** Recolectar datos duros de comportamiento (métricas de retención y recurrencia, no de descargas vanidosas). Si los datos refutan la hipótesis, se realiza un **pivote (MVP 2)** para cambiar la dirección estratégica del producto.

Anatomía de una Validación Temprana: El Caso Facebook (2004)

La cátedra destaca a **Facebook** como el ejemplo paradigmático de que un MVP exitoso no requiere validar factibilidad técnica (la tecnología web para armar un perfil ya existía), sino **comportamiento humano**. Mark Zuckerberg y sus socios validaron dos hipótesis críticas de manera secuencial:

Track 1: Hipótesis de Valor

- **La Pregunta:** ¿El producto realmente entrega valor al usuario cuando lo opera de forma cotidiana?
- **Métrica Utilizada:** Tasa de Retención de Usuarios (User Retention Rate).
- **Resultado Empírico (2004):** Descubrieron que **más del 50% de los estudiantes registrados volvían a iniciar sesión en el sitio todos los días de forma recurrente**, demostrando un enganche emocional y de utilidad altísimo.

Track 2: Hipótesis de Crecimiento

- **La Pregunta:** ¿Cómo descubren el producto los nuevos usuarios? ¿Se requiere marketing agresivo o se propaga por recomendación orgánica?

- **Métrica Utilizada: Net Promoter Score (NPS)** o tasa de referencias directas (**viral coefficient**).
- **Resultado Empírico (2004):** Sin invertir un solo dólar en publicidad, **casi el 75% de la población estudiantil de Harvard adoptó y usó el sistema en menos de un mes**, probando que el crecimiento era orgánico, viral y exponencial.

La Lección del Caso: Con escasos 150.000 usuarios y facturación nula, Facebook levantó \$500.000 de capital semilla de inversores ángeles en 2004 gracias a que tenían estas dos hipótesis validadas con datos reales de retención humana en el campus de Harvard, destruyendo “la trampa del cero” (la fantasía de proyectar números grandes sobre la nada).

Caso de Estudio Técnico: Dropbox y la “Prueba de Humo”

- **El Reto Técnico (Drew Houston):** Crear una plataforma que sincronizara archivos en tiempo real entre múltiples sistemas operativos (Windows, Mac, Linux) y bases de datos locales distribuidas de forma consistente. El riesgo técnico de arquitectura de software y el costo financiero de codificar la solución eran extremos.
- **El MVP (Prueba de Humo sin código):** Houston no escribió una sola línea de la lógica de sincronización profunda para validar el mercado. Grabó un **video banal de 3 minutos narrado por él mismo**, demostrando con maquetas animadas cómo funcionaría de forma fluida el sistema lúdico desde la perspectiva del usuario de escritorio.
- **Hipótesis Validada:** La lista de espera beta saltó de **5.000 a 75.000 usuarios registrados en una sola noche**, demostrando que existía una demanda real y masiva en el mercado. Dropbox consiguió la financiación necesaria habiendo construido solo un video demostrativo de bajo costo.

Errores comunes y advertencias de examen

- **⚠ El error del producto fragmentado:** Intentar hacer un MVP entregando solo la “capa funcional” sin usabilidad ni fiabilidad (ej. un software que calcula el IVA pero se cuelga a cada rato y tiene interfaz de consola de comandos incomprensible). El MVP debe ser **cohesivo aunque limitado**, cortando una rebanada fina que toque de forma equilibrada la funcionalidad, la fiabilidad, la usabilidad y el diseño del sistema.

3.11.2. Producto Mínimo Comercializable (MMP - Minimum Marketable Product)

Definición precisa del concepto

- **Definición de las Slides de la Cátedra (SLIDE_05 / NC_Ardiles):** El Producto Mínimo Comercializable (MMP) —también conocido en el roadmap técnico de liberación como **MMR 1 (Minimum Marketable Release 1)**— es la porción o **versión más pequeña de software funcionando y testeado que entrega valor de negocio directo** y que el equipo puede desplegar de forma segura en producción para que empiece a ser utilizado de forma masiva.
- **Qué problema resuelve:** Ataca el **costo del retraso (cost of delay)**. Evita que el cliente tenga que esperar a que el 100% de la funcionalidad de largo plazo esté construida para empezar a capturar retorno de inversión (ROI), facilitando el posicionamiento temprano de la marca en producción real.

Cómo se hace: La gestión del Roadmap de Liberación

El MMP no se basa en conjeturas de “salto de fe”. Se planifica cuando ya se conoce el dominio de negocio y se tiene un conjunto de características estables. El procedimiento exige:

```
[ Pila de User Stories (Product Backlog) ] -> [ Matriz de Priorización ] -> [ Selección de MMFs ]  
[ Liberación del MMP a early adopters ] <- [ Pruebas de Regresión (QA) ] <- [ Integración (CI) ]
```

1. **Priorización de Características:** Clasificar los requerimientos del Product Backlog mediante una matriz de priorización cruzando Urgencia e Importancia.
 2. **Identificar las Características Mínimas Comercializables (MMF - Minimum Marketable Features):** Seleccionar las piezas sólidas de alto valor comercial comprobado que reducen el tiempo de llegada al mercado (**time to market**).
 3. **Filtrar el Desperdicio (CRUD boundaries):** Separar los módulos operativos críticos de las tareas administrativas diferibles. Por ejemplo, en una ticketera web, la MMF crítica es “Comprar Entrada”. Las funciones de “Alta, Baja y Modificación” (CRUD) de eventos para los administradores del sistema se delegan para los releases futuros (MMR 2 o MMR 3) o se gestionan de forma temporal mediante accesos directos a la base de datos.
 4. **Establecer Criterios de Calidad:** Pasar de forma obligatoria las suites de pruebas de regresión técnica de QA antes de autorizar el empaquetado del build de liberación [Release Build].
- **Ejemplo Concreto de la Cátedra:** El reemplazo del sistema de autogestión de la UTN-FRC. Aquí no hay hipótesis que validar porque los usuarios son estudiantes reales y la necesidad funcional está totalmente clara. La versión uno que permite inscribirse a exámenes y ver la regularidad de las materias representa un **MMP puro**, no un MVP.

3.11.3. Producto Mínimo Adorable (MLP - Minimum Lovable Product)

Definición precisa del concepto

- **Definición formal (SLIDE_05 - BIBLIO_):** Es la porción más acotada de software que no solo cumple con las características de viabilidad funcional (MVP) y comercial (MMP), sino que incorpora desde su primer release atributos de **usabilidad, diseño visual de vanguardia y experiencia de usuario (UX)** de tal nivel que logran evocar emociones positivas y deleitar profundamente al cliente, asegurando su lealtad inmediata.
- **Qué problema resuelve:** Resuelve la **deserción silenciosa** y cruza con éxito la “**Grieta de la Experiencia**”. En el mercado altamente maduro de hoy, los competidores están bien financiados y la funcionalidad básica es un estándar de mercado (**commodity**). Si un usuario descarga un software y la interfaz es frustrante, la desinstala en un segundo y no le da una segunda oportunidad al producto, arruinando la reputación de la marca.

Cómo se hace: Implementando los 10 Bloques de la Adorabilidad

Para construir un MLP, el equipo de SCM y los diseñadores de UX deben asegurar que el primer release del incremento vertical toque de forma simultánea los diez bloques de la pirámide de la cátedra:

1. **Cuidado (Base):** Atención extrema a los detalles finos de interfaz, accesibilidad y soporte.

2. **Satisfacción:** Cumplimiento del núcleo de la necesidad de forma limpia y transparente.
 3. **Esperanza:** Transmitir al usuario que el sistema es su aliado para mejorar su vida.
 4. **Sostenibilidad:** Mantener un ritmo constante de calidad técnica y soporte sin degradación.
 5. **Escala:** Estabilidad de respuesta y performance de la UI ante altos volúmenes de usuarios concurrentes.
 6. **Confianza en el sistema:** Ausencia total de fallas, cuelgues o inconsistencias transaccionales.
 7. **Confianza en uno mismo:** Interfaces sumamente intuitivas que hagan sentir al usuario inteligente y autónomo.
 8. **Diversión:** Incorporación de gamificación y microinteracciones amenas.
 9. **Motivación:** Brindar una razón poderosa para que el usuario retorne de forma periódica al software.
 10. **Halo (Cúspide):** El factor “magia” intangible que genera pertenencia emocional y fanatismo de marca.
- **Ejemplos Reales:**
 - **Spotify:** El deleite del usuario no reside solo en poder escuchar música (MMP), sino en la sección “Discover Weekly”, que utiliza algoritmos de recomendación curados que hacen sentir al usuario profundamente comprendido por el sistema.
 - **Instagram (2010):** No era la única app para compartir fotos de la época, pero triunfó como MLP gracias a su interfaz increíblemente intuitiva y la incorporación de filtros creativos instantáneos que transformaban fotos ordinarias de celulares de baja calidad en imágenes estéticamente hermosas en segundos.
 - **WhatsApp:** Su adorabilidad reside en su fiabilidad extrema, ausencia total de fricción en el registro (usa tu número de celular directo) y simplicidad de diseño radical.

3.11.4. Matriz Comparativa: El Espectro de los Productos Mínimos

El ingeniero de calidad y el Product Owner deben diagnosticar con precisión matemática en qué fase del ciclo de vida se encuentra el producto para aplicar los conceptos y las métricas de control adecuadas:

Criterio de Comparación	MVP (Producto Mínimo Viable)	MMP (Producto Mínimo Comercializable)	MLP (Producto Mínimo Adorable)
Objetivo Principal	Validar conceptos e hipótesis de negocio de forma rápida y económica.	Atender a una base amplia de usuarios con funcionalidad clave para producción.	Crear una conexión emocional profunda y deleite desde el primer día.
Enfoque del Diseño	Investigar, empatizar y capturar aprendizaje técnico o funcional.	Construir, refinar e integrar código fuente y testing de regresión de calidad.	Diseñar para el deleite visual (UI avanzada) y reducción de la fricción de uso.

Criterio de Comparación	MVP (Producto Mínimo Viable)	MMP (Producto Mínimo Comercializable)	MLP (Producto Mínimo Adorable)
Foco de Calidad Clave	Investigación exploratoria, prototipos, maquetas y pruebas de usuario rápidas.	Estabilidad de arquitectura, velocidad del release y salida formal al mercado.	Experiencia de usuario (UX) unificada, microinteracciones y empatía humana.
Beneficio de Negocio	Informar con datos empíricos el alcance real del producto final.	Asegurar la calidad, consistencia y capturar tracción comercial en producción.	Evitar la deserción de usuarios (churn rate) y construir lealtad y retención .
Ejemplo Típico del Mercado	Video demostrativo de Dropbox o landing page de reserva anticipada.	Versión Beta pública con el núcleo funcional crítico operando estable.	Aplicaciones sofisticadas como Spotify o Instagram con alto valor de deleite.

3.11.5. El Arte del Corte de Requerimientos: Corte Vertical vs. Construcción Horizontal

Para que los conceptos de MVP, MMP o los incrementos de Scrum funcionen técnicamente sin corromper la arquitectura de software, el equipo de desarrollo debe dominar la técnica de **partición de requerimientos**.

1. Construcción Horizontal (La Mala Práctica): “El Enfoque de Componentes Técnicos”

- **En qué consiste:** Es el error sistemático de subdividir el desarrollo de software de forma horizontal siguiendo las capas de la arquitectura técnica del sistema: primero codificar toda la capa de Base de Datos (Persistencia), en el siguiente sprint programar toda la Capa de Lógica de Negocio (Servicios), y al final del proyecto diseñar la Interfaz de Usuario (UI).
- **Por qué es un error grave de Ingeniería (Pérdida de Valor):** Ninguna de las capas técnicas horizontales por sí solas entrega software funcional potencialmente desplegable al final del sprint. Si se entrega solo la Base de Datos, el Product Owner no puede interactuar con ella ni probarla de forma empírica. Esto posterga la integración técnica hasta las fases finales del proyecto, gatillando el riesgo de encontrar colisiones catastróficas de arquitectura que obliguen a reescribir código masivo a último momento.

2. Corte Vertical (La Buena Práctica): “Slicing the Cake” (Bill Wake, 2003)

- **En qué consiste:** Es la técnica de cortar las Historias de Usuario de forma vertical, atravesando de manera simultánea todas las capas técnicas y arquitectónicas del software (Interfaz, Lógica y Persistencia) para entregar una porción funcional atómica completa.
- **Qué problema resuelve:** Reduce drásticamente el riesgo tecnológico integrando de forma continua todas las capas del sistema desde el sprint uno. Habilita la retroalimentación temprana del cliente e interesados al interactuar con código operable real, incluso si el alcance general del incremento es sumamente limitado.

Representación Gráfica y Descriptiva de la Partición de Requerimientos

[MALA PRÁCTICA: CONSTRUCCIÓN HORIZONTAL] [BUENA PRÁCTICA: CORTE VERTICAL]

Usuario (UI)	U	1. Interfaz de
Negocio (Services/Logic)	L	2. Lógica de
Datos / Persistencia (DB)	B	3. Base de
* No hay entrega de valor parcial. * Rebanada de torta (Slice)		
* Alto riesgo de integración tardía. * Atraviesa todas las capas.		
* Genera desperdicio (BRUF). * Potencialmente desplegable.		

El Ejemplo de la Torta de Tres Capas (The Cake Slice Analog):

- Imaginá que un sistema de software es una torta de bodas de tres capas: la superior es el glaseado vistoso (**Interfaz de Usuario**), la capa intermedia es el dulce de leche con nueces (**Lógica de Negocio**), y la base es el bizcochuelo esponjoso (**Persistencia/Base de Datos**).
- **Si aplicás el Enfoque Horizontal:** Intentar servirle a tus invitados solo el glaseado en el primer plato, en el segundo plato solo el dulce de leche, y al final solo el bizcochuelo seco. Tus invitados estarán asqueados y frustrados porque las porciones de forma aislada carecen del sabor combinado unificado de la torta.
- **Si aplicás el Corte Vertical (Slicing):** Utilizar un cuchillo afilado de ingeniería y cortar una **rebanada fina de torta** que contenga de forma proporcional glaseado, dulce de leche y bizcochuelo. La rebanada es pequeña, pero representa la experiencia completa y el sabor real de la torta.
- En el software, una rebanada vertical es programar una funcionalidad atómica como: **“Comprar un boleto de cine con tarjeta de débito”**. No tiene todas las opciones de pago ni el mapa interactivo de asientos, pero el flujo completa un objetivo de negocio real, atraviesa las tres capas del sistema y puede ser probado y consumido por el Product Owner en su demo.

Puntos clave para el examen — 3.11.5

1. **DIFERENCIA ENTRE MVP Y PROTOTIPO (AUDIO_05):** Durante el debate en clase, la docente aclara que un prototipo es una herramienta puramente estática o interactiva simulada (como las pantallas hilvanadas en Figma) para validar diseño visual o flujos funcionales rápidos. El MVP, si bien puede ser un prototipo, tiene un propósito mucho más amplio: **validar empíricamente hipótesis de negocio y comportamiento humano de retención en el mercado**.
2. **LA ADORABILIDAD NO ES “DECORAR” AL FINAL (AUDIO_05):** Judith Meles advierte de forma explícita que es un error conceptual grave de diseño considerar que el Producto Mínimo Adorable (MLP) consiste en “hacer el software y después ver de ponerle colores bonitos” en la última iteración. El MLP exige que la experiencia de usuario (UX) y el deleite emocional se diseñen de forma nativa e integrada desde la **primera rebanada vertical** del producto.

3. **EL RIESGO DEL ALCANCE INTERMINABLE:** El docente de práctico destaca en las correcciones que para el parcial práctico, al definir un MVP, debes obligatoriamente declarar de forma simétrica **qué incluye y qué no incluye (No incluye / Exclusiones)** el alcance del experimento, justificándolo siempre en base a la hipótesis declarada. Un alcance difuso o sin exclusiones explícitas es causa directa de baja nota.

Preguntas de autoevaluación — 3.11.5

Pregunta 1: ¿Cómo define Eric Ries al Producto Mínimo Viable (MVP)?

- **Respuesta breve:** Es la versión de un producto nuevo que permite recolectar la mayor cantidad de aprendizaje validado sobre el mercado y los clientes con el menor esfuerzo y costo de desarrollo posibles.

Pregunta 2: ¿Cuál es el riesgo de negocio crítico que mitiga la implementación de un MVP?

- **Respuesta breve:** Mitiga el riesgo del cero absoluto, es decir, el peligro de gastar recursos masivos de ingeniería construyendo un software que luego nadie quiere o está dispuesto a usar en la realidad.

Pregunta 3: Explique las dos hipótesis que validó Facebook de forma empírica en 2004.

- **Respuesta breve:** Validó la **Hipótesis de Valor** (verificando que más del 50% de los usuarios volvían al sitio todos los días) y la **Hipótesis de Crecimiento** (comprobando que el 75% del campus lo adoptó orgánicamente en un mes sin marketing).

Pregunta 4: ¿Por qué el video demostrativo de Dropbox de 3 minutos es calificado técnicamente como un MVP?

- **Respuesta breve:** Porque sin escribir código de sincronización real, permitió recolectar aprendizaje validado sobre el mercado, haciendo saltar la lista de espera de 5.000 a 75.000 usuarios en una sola noche con costo cero de software.

Pregunta 5: ¿Qué es un Producto Mínimo Comercializable (MMP) y en qué difiere conceptualmente de un MVP?

- **Respuesta breve:** El MMP exige software funcionando de calidad para ser puesto de forma segura en producción para el usuario final; el MVP se enfoca únicamente en capturar aprendizaje estratégico y puede consistir en maquetas, videos o experimentos sin código.

Pregunta 6: ¿Qué es una Característica Mínima Comercializable (MMF) según la cátedra?

- **Respuesta breve:** Es la porción más pequeña de funcionalidad del sistema que posee alta certeza comercial y que, de forma aislada, es capaz de entregar valor de negocio inmediato y tracción al ser liberada en producción.

Pregunta 7: ¿Cómo se define el Producto Mínimo Adorable (MLP) y qué problema de comportamiento humano resuelve?

- **Respuesta breve:** Es la versión acotada de software que incluye usabilidad, diseño y deleite desde el primer día. Resuelve el problema de la deserción silenciosa de usuarios, quienes en mercados maduros no dan una segunda oportunidad a interfaces frustrantes.

Pregunta 8: Enumere al menos tres de los diez bloques de construcción de la pirámide de adorabilidad del MLP.

- **Respuesta breve:** Los bloques fundamentales de construcción son el **Cuidado** (atención extrema a los detalles), la **Satisfacción** de la necesidad, y el factor **Halo** (magia e identidad de marca).

Pregunta 9: ¿Qué significa conceptualmente que las User Stories deben ser “porciones verticales” en el desarrollo?

- **Respuesta breve:** Significa que cada historia debe atravesar de manera simultánea todas las capas técnicas del software (interfaz, servicios y persistencia) para entregar un incremento operable que aporte valor real al final del sprint.

Pregunta 10: Explique la analogía de la “rebanada de torta” (slicing the cake) sugerida por Bill Wake.

- **Respuesta breve:** Indica que el desarrollo horizontal (capa por capa de base de datos o lógica) es como servir la torta por ingredientes aislados; la rebanada vertical corta todas las capas simultáneamente, ofreciendo el sabor combinado completo y una experiencia funcional real de menor tamaño.

3.12. Caso de Estudio Resuelto: “El Parque del Sol” (Gestión de Requerimientos y Estimación)

3.12.1. Introducción al Caso de Estudio

El presente caso de estudio asimila de forma introspectiva el ejercicio de taller e integración ágil discutido en las clases prácticas de la cátedra. El dominio consiste en el diseño, especificación de requerimientos y estimación de tamaño relativo de una aplicación móvil autogestionada orientada a los visitantes de un parque zoológico (“Parque del Sol”).

- **Qué problema resuelve:** Demuestra cómo un equipo de desarrollo asimila las pautas pragmáticas de SCM y Requerimientos de la UTN-FRC para:
 1. Traducir un enunciado narrativo informal en un inventario estructurado de Historias de Usuario (US).
 2. Establecer un Producto Mínimo Viable (MVP) basado exclusivamente en la validación de una hipótesis de negocio clara, aplicando criterios rigurosos de exclusión funcional.
 3. Ejecutar el proceso de Poker de Estimación (Poker Estimation) sobre requerimientos complejos y resolver anomalías mediante la división vertical de historias.

3.12.2. Listado de Historias de Usuario (US) Identificadas

A partir del análisis del dominio del parque, se identifican las siguientes 14 Historias de Usuario (US). Cada tarjeta está redactada utilizando la sintaxis estándar de Connextra exigida por la cátedra, incorporando de forma explícita el rol de negocio específico (evitando el uso del actor genérico “Usuario”):

1. **US_01: Comprar Entradas**
 - Como Visitante quiero comprar entradas digitales para el parque a través de la aplicación, de forma tal que pueda asegurar mi acceso al parque de manera ágil sin hacer colas físicas.
2. **US_02: Registrar Visitante**

- Como Visitante quiero registrar un usuario con mi e-mail y clave, de forma tal que el sistema pueda persistir mis datos de perfil, historial de compras y preferencias de visita.
3. **US_03: Iniciar Sesión**
 - Como Visitante Registrado quiero iniciar sesión en la aplicación, de forma tal que pueda acceder a mi perfil personalizado y mis entradas previamente adquiridas.
 4. **US_04: Cerrar Sesión**
 - Como Visitante Registrado quiero cerrar mi sesión activa, de forma tal que se proteja la privacidad de mis datos de perfil y pago en caso de prestar el dispositivo.
 5. **US_05: Consultar Horarios de Alimentación**
 - Como Visitante quiero visualizar el listado de horarios de alimentación de las especies del parque, de forma tal que pueda planificar mi recorrido para presenciar estos eventos lúdicos.
 6. **US_06: Consultar Exhibiciones**
 - Como Visitante quiero consultar el cronograma detallado de exhibiciones especiales, de forma tal que conozca qué espectáculos y demostraciones técnicas se realizarán durante mi estadía.
 7. **US_07: Consultar Actividades**
 - Como Visitante quiero visualizar el listado de actividades disponibles (palestra, safari, tirolesa) con sus respectivos cupos, de forma tal que conozca las opciones recreativas adicionales del parque.
 8. **US_08: Inscribirse a Actividades**
 - Como Visitante Registrado quiero inscribirme a una actividad específica en un horario determinado, de forma tal que pueda reservar un cupo de participación y asegurar mi lugar.
 9. **US_09: Visualizar Mapa Interactivo**
 - Como Visitante quiero visualizar el mapa digital del parque con sus puntos de interés destacados por colores, de forma tal que pueda orientarme y explorar las instalaciones de manera eficiente.
 10. **US_10: Visualizar Ubicación en Tiempo Real**
 - Como Visitante quiero visualizar mi ubicación física en tiempo real reflejada en el mapa interactivo mediante el GPS de mi dispositivo, de forma tal que conozca mi posición exacta y qué atracciones tengo cerca.
 11. **US_11: Recibir Notificación de Alimentación**
 - Como Visitante Registrado quiero recibir un aviso de notificación en mi dispositivo una hora antes de que comience una alimentación que marqué como favorita, de forma tal que pueda apurarme a llegar al sector.
 12. **US_12: Configurar Parámetros de Notificaciones**
 - Como Visitante Registrado quiero configurar manualmente con cuánta antelación deseo recibir los avisos de alimentación o exhibición, de forma tal que pueda personalizar mi experiencia de alerta lúdica.
 13. **US_13: Alimentar Animal Virtual**
 - Como Visitante quiero alimentar de forma virtual e interactiva a un animal representativo desde la pantalla mediante un minijuego lúdico, de forma tal que me entretenga mientras hago filas de espera físicas.
 14. **US_14: Filtrar Horarios de Alimentación**
 - Como Visitante quiero filtrar el listado de horarios de alimentación por sector o especie, de forma tal que pueda visualizar rápidamente únicamente los eventos de mi interés inmediato.

3.12.3. Definición del Producto Mínimo Viable (MVP)

A. Hipótesis de Negocio del MVP

La cátedra destaca que todo MVP requiere un sustento empírico basado en una hipótesis de valor o crecimiento que se desea validar en el mercado:

Hipótesis del MVP: “Creemos que al proveerle al visitante una aplicación móvil autogestionada que le permita comprar entradas de manera digital, consultar el mapa interactivo estático y visualizar los horarios de alimentación de hoy, lograremos que el visitante autogestione su recorrido disminuyendo en un 30% las colas en la boletería física y en los centros de información, validando así la adopción tecnológica de la plataforma en producción”.

B. Qué Entra y Qué Queda Afuera en el MVP

Para lograr este objetivo con el **menor costo y esfuerzo técnico posible (Mínimo)**, se definen de manera rigurosa las fronteras del alcance:

ID_US	Ítem de Configuración Funcional (US)	¿Entra en el MVP?	Justificación y Criterio de Exclusión / Inclusión
US_01	Comprar Entradas (con Mercado Pago)	SÍ	Entra. Es el núcleo transaccional que valida la hipótesis de descongestionamiento de la boletería física. Incluye la integración básica de pasarela de pago lúdica.
US_02	Registrar Visitante	SÍ	Entra. El caso de uso específico del negocio obliga a que el usuario se registre para poder procesar la compra digital de la entrada y validar su identidad.
US_03	Iniciar Sesión / Cerrar Sesión	SÍ	Entra. Es el soporte básico indispensable para la persistencia del perfil y la seguridad de las transacciones de compra de entradas.
US_05	Consultar Horarios de Alimentación	SÍ	Entra. Permite descongestionar las consultas recurrentes de información física de los folletos tradicionales. Se presenta como un listado estático básico de la agenda diaria.
US_09	Visualizar Mapa Interactivo (Estático)	SÍ	Entra. Provee la asistencia de ubicación espacial de primer nivel mediante un mapa vectorial estático estructurado por sectores de colores, sin geolocalización dinámica.
US_06	Consultar Exhibiciones / Actividades	NO	Excluido. Se posterga para futuras iteraciones comerciales. En el MVP, estas actividades lúdicas se consideran complementarias y no afectan la validación de la autogestión de recorrido básica.
US_08	Inscribirse a	NO	Excluido. Requiere el desarrollo del motor de cupos y

ID_US	Ítem de Configuración Funcional (US)	¿Entra en el MVP?	Justificación y Criterio de Exclusión / Inclusión
	Actividades		reservas en tiempo real en la base de datos, lo cual eleva la complejidad técnica y demora el lanzamiento.
US_10	Visualizar Ubicación en Tiempo Real (GPS)	NO	Excluido. El algoritmo de geolocalización bajo techo y la integración con APIs de mapas dinámicos en tiempo real es de altísima complejidad. Para validar la hipótesis, al visitante le alcanza con ver el mapa estático y orientarse por cartelería.
US_11	Recibir Notificación de Alimentación	NO	Excluido. Demanda configurar servidores de notificaciones push y gestionar servicios en segundo plano en los sistemas operativos. El visitante puede consultar manualmente la pantalla de horarios (US_05).
US_12	Configurar Parámetros de Notificaciones	NO	Excluido. Al no incluirse las notificaciones en el MVP, su módulo de configuración personalizado carece de sentido lógico y representa desperdicio técnico puro.
US_13	Alimentar Animal Virtual (Minijuego)	NO	Excluido. Es una característica accesorio cosmética (“un chiche lúdico”) que no asiste a la hipótesis de descongestionamiento ni autogestión de recorrido. Excluido por criterio de simplicidad.
US_14	Filtrar Horarios de Alimentación	NO	Excluido. En el MVP la cantidad de animales y horarios cargados en la base de datos es muy reducida. Un listado simple ordenado cronológicamente es suficiente para no abrumar al usuario, haciendo innecesario el filtro técnico complejo.

3.12.4. Ejercicio Práctico de Poker de Estimación (Poker Estimation)

Durante la sesión de Poker de Estimación para la estimación del tamaño relativo de la pila del backlog, el equipo se enfrenta a la Story: **US_05: Consultar Horarios de Alimentación.**

A. Criterios de Aceptación de la US (Al Dorso de la Tarjeta):

- **AC1:** La pantalla debe mostrar de manera obligatoria el horario de inicio del evento, la especie animal, el nombre del animal individual, la edad del mismo, el sector del parque (color y nombre) y el nombre y apellido del cuidador a cargo.
- **AC2:** El listado de horarios debe mostrarse ordenado cronológicamente, figurando primero los eventos más próximos a suceder.
- **AC3:** La pantalla debe mostrar la agenda de eventos planificados hasta una semana en el futuro.

- **AC4:** El sistema debe mostrar un mensaje destacado de alerta de proximidad de tipo **“Apurate que ya comienza”** si falta menos de una hora para que inicie la alimentación en un sector.

B. El Debate de la Votación: Por qué da 13 Puntos de Fibonacci

Cuando el equipo realiza la revelación simultánea de las cartas de Fibonacci Modificada, el resultado arroja una enorme divergencia técnica:

- **Desarrollador Junior:** Vota **5 puntos** porque asume que es una consulta de base de datos estándar (“un SELECT simple de SQL con un ORDER BY”).
- **Desarrollador Senior (Experto):** Vota **13 puntos** porque detecta complejidades ocultas en los criterios de aceptación.

El argumento del Senior en el debate de extremos:

1. **La lógica de fechas extendidas (AC3):** Consultar y ordenar de forma eficiente eventos hasta una semana en el futuro exige lidiar con zonas horarias, formatos de fecha asimétricos y el consumo de APIs con paginación para no colgar la memoria del celular.
2. **El cálculo dinámico de la alerta (AC4):** El requisito del mensaje de proximidad **“Apurate que ya comienza”** no es un dato estático de base de datos. Requiere programar un algoritmo dinámico que compare de manera constante en segundo plano la hora local del dispositivo del visitante con la hora de inicio del evento de alimentación, disparando la renderización reactiva de la interfaz.
3. **Acoplamiento de Entidades (AC1):** Para renderizar una sola fila de horario, el sistema debe consumir e integrar datos físicos de 5 tablas de base de datos distintas (Horarios, Animales, Especies, Sectores y Cuidadores).

Debido a la sumatoria de estas tres burbujas de complejidad, esfuerzo e incertidumbre, el equipo alcanza el consenso de asignarle **13 Story Points** a la historia.

C. Qué Atributo del Modelo INVEST se Incumple y por qué

La asignación de 13 puntos en el Poker de Estimación enciende de inmediato una alarma de calidad en SCM.

- **Atributo Incumplido: Small (S - Pequeña).**
- **Fundamento Técnico de SCM:** Una historia que vale 13 puntos es **demasiado grande y compleja** para ingresar de forma segura a un sprint de dos semanas. Existe un riesgo inaceptable de que el desarrollador no logre completar el código, el testing dinámico y el manual de usuario antes del timebox (síndrome del “casi terminado”), lo que aportará cero puntos a la velocidad de la iteración, arrastrando deuda técnica. El modelo INVEST exige de forma obligatoria realizar un **particionado vertical (slicing)** del requerimiento.

D. Resolución: División Vertical (Slicing) de la US

Para subsanar el incumplimiento de calidad y volver estimables las historias de forma atómica, se realiza un particionado vertical de la US_05 original de 13 puntos, dando como resultado tres historias independientes y negociables que cumplen de forma estricta con el modelo INVEST:


```
|| PARTICIONADO VERTICAL DE US_05 ||  
[ US_05: Consultar Horarios de Alimentación ] (13 SP)  
[ US_05A: Listado de Hoy ] [ US_05B: Listado Semanal ] [ US_05C: Alerta Proximidad ]  
(3 SP) (3 SP) (3 SP)
```

1. US_05A: Consultar Horarios de Alimentación del Día (Hoy)

- **Tarjeta:** Como Visitante quiero visualizar el listado de horarios de alimentación programados para el día de hoy, de forma tal que conozca qué eventos sucederán de manera inminente.
- **Estimación: 3 Story Points** (Complejidad baja, esfuerzo físico medio, incertidumbre nula por ser un SELECT simple de fecha de hoy).
- **Prueba de Aceptación:**
- Probar consultar horarios de alimentación de hoy con registros cargados para el día de la fecha y verificar que el sistema los muestre ordenados cronológicamente (pasa).

2. US_05B: Consultar Horarios de Alimentación Semanales (Extendidos)

- **Tarjeta:** Como Visitante quiero consultar los horarios de alimentación planificados hasta una semana en el futuro, de forma tal que pueda planificar con anticipación mi regreso al parque en los próximos días.
- **Estimación: 3 Story Points** (Complejidad media por control de fechas extendido, esfuerzo medio, incertidumbre nula).
- **Prueba de Aceptación:**
- Probar consultar horarios ingresando un rango de búsqueda de 8 días en el futuro y verificar que el sistema limite la vista estrictamente a 7 días según AC3 (falla).

3. US_05C: Visualizar Alerta de Proximidad de Evento de Alimentación

- **Tarjeta:** Como Visitante quiero visualizar un mensaje destacado de advertencia en la pantalla cuando falte menos de una hora para que inicie la alimentación en un sector, de forma tal que pueda acelerar mi caminata.
- **Estimación: 3 Story Points** (Complejidad alta por el algoritmo comparador de hora local reactiva, esfuerzo bajo, incertidumbre baja).
- **Prueba de Aceptación:**
- Probar visualizar la pantalla de horarios faltando exactamente 45 minutos para el inicio de la alimentación del León y verificar que se renderice el mensaje destacado de advertencia (pasa).
- **Resultado de SCM:** Al dividir verticalmente el requerimiento original, el equipo pasa de tener una historia inmanejable de 13 puntos a contar con **tres historias atómicas de 3 puntos cada una**, totalmente independientes, estimables y testeables que ingresarán de forma fluida a la planificación del sprint.

3.12.5. La Contradicción Terminológica de Examen: “Poker Estimation” vs “Planning Poker”

En los exámenes de la cátedra, los profesores penalizan severamente el uso de los términos de mercado tradicionales “Planning Poker” o “Poker Planning”, exigiendo de forma unívoca denominar a la técnica como **Poker de Estimación (Poker Estimation)**.

¿En qué consiste esta contradicción y por qué desaprueban al alumno?

Término de Mercado (No admitido)	Término Exigido por la Cátedra	Justificación Conceptual de Ingeniería de Software
Planning Poker (o Poker Planning)	Poker de Estimación (Poker Estimation)	Evita confundir la estimación del tamaño con la planificación del tiempo de calendario .

El razonamiento de la cátedra para rechazar “Planning Poker”:

1. **La Estimación es Imparcial y de Tamaño:** El objetivo único y exclusivo del juego de cartas es estimar de forma empírica y por analogía el **tamaño relativo (magnitud intrínseca)** de las Historias de Usuario (medido en Story Points abstractos). Es una predicción de ingeniería probabilística y técnica libre de compromisos comerciales.
2. **La Planificación es una Actividad Gerencial de Fechas:** Planificar (Planning) consiste en contrastar ese tamaño estimado contra la velocidad real del equipo y la disponibilidad de recursos de calendario para establecer un compromiso contractual de entrega con el Product Owner.
3. **Confundir los conceptos degrada el proceso:** Llamar “Planning Poker” al juego de cartas confunde al equipo técnico, haciéndoles creer que al votar las cartas están “planificando” cuándo entregarán el software. Esto provoca que los programadores alteren de forma defensiva el tamaño relativo de las historias pensando en “cuántos días reales de calendario les va a llevar”, corrompiendo la consistencia métrica de los Story Points y destruyendo la velocidad histórica del equipo. Por lo tanto, **en el examen teórico y en el práctico se debe responder strictly “Poker de Estimación” (Poker Estimation)**.

Puntos clave para el examen — 3.12.5

1. **LA HISTORIA CANÓNICA DEBE SER LA MÁS PEQUEÑA (AUDIO_o6):** Para que la escala de analogía funcione de forma consistente en el práctico de SCM, la historia canónica de referencia debe ser la de menor tamaño y complejidad posible de todo el backlog (asignándole el valor de 1 o 2). Si elegís una historia mediana de 5 puntos como canónica, las historias más chicas requerirán valores con fracciones (0.5, 0.25), lo cual complica de manera innecesaria el Poker de Estimación y destruye el consenso cuantitativo.
2. **LA JUSTIFICACIÓN COMPLETA DE LAS DIMENSIONES EN EL PARCIAL:** El docente de práctico reitera que un número suelto de Fibonacci en las preguntas prácticas del examen se califica con **cero (0) directo** si no viene acompañado de la justificación textual

explícita detallando por qué para tu grupo se equilibran las tres dimensiones cualitativas del Story Point: Complejidad, Esfuerzo e Incertidumbre.

3. **EL MVP NO SE AJUSTA AL ALCANCE, EL ALCANCE AL MVP (AUDIO_05):** Durante el debate del práctico, se recalca que es un error común que el grupo dibuje una hipótesis gigante de negocio que intente acoplarse a toda la funcionalidad que tienen ganas de programar. El procedimiento correcto exige: primero definir la hipótesis de aprendizaje mínima, y luego recortar de forma drástica todo requerimiento que no aporte datos inmediatos para validar esa hipótesis funcional.

Preguntas de autoevaluación — 3.12.5

Pregunta 1: ¿Cuál es el dominio lúdico que se utiliza de ejemplo en el caso de estudio resuelto del práctico?

- **Respuesta breve:** Consiste en la gestión de requerimientos y estimación de una aplicación móvil de autogestión de recorrido orientada a los visitantes de un parque zoológico (“Parque del Sol”).

Pregunta 2: Escriba la sintaxis estándar Connextra utilizada para redactar la US de compra de entradas en el caso.

- **Respuesta breve:** Como Visitante quiero comprar entradas digitales para el parque a través de la aplicación, de forma tal que pueda asegurar mi acceso al parque de manera ágil sin hacer colas físicas.

Pregunta 3: ¿Cuál es la hipótesis de negocio que justifica el lanzamiento del MVP del parque?

- **Respuesta breve:** Validar que proveer compra digital de entradas, mapa interactivo estático y consulta de horarios de hoy disminuye en un 30% las colas físicas en boletería, demostrando la adopción tecnológica de los usuarios en producción.

Pregunta 4: ¿Por qué la historia de “Visualizar Ubicación en Tiempo Real (GPS)” queda excluida del MVP?

- **Respuesta breve:** Por su alta complejidad técnica de geolocalización. El visitante puede cumplir el objetivo de autogestionarse orientándose de forma básica con el mapa vectorial estático de sectores y la cartelería física del parque.

Pregunta 5: ¿Qué criterios de aceptación del caso hacían que la US de “Consultar Horarios de Alimentación” fuese votada con 13 puntos?

- **Respuesta breve:** Mostrar datos cruzados de 5 tablas independientes (horario, especie, animal, sector, cuidador), proyectar la agenda hasta una semana en el futuro y calcular dinámicamente un mensaje de alerta de proximidad.

Pregunta 6: ¿Qué atributo del modelo INVEST se incumple cuando una Historia de Usuario se estima en 13 puntos?

- **Respuesta breve:** Se incumple el atributo **Small (S - Pequeña)**, ya que un tamaño de 13 puntos excede la capacidad segura de desarrollo y testeado de un solo sprint de dos semanas.

Pregunta 7: ¿Cómo se subsana mediante ingeniería de requerimientos la asignación de 13 Story Points a una historia?

- **Respuesta breve:** Se realiza un particionado vertical (**slicing**) de la funcionalidad para romperla en tres historias independientes de menor envergadura (3 puntos cada una en el caso de estudio).

Pregunta 8: ¿Por qué la cátedra exige denominar a la actividad de votación de cartas “Poker de Estimación” y no “Planning Poker”?

- **Respuesta breve:** Porque el objetivo exclusivo del juego es estimar de forma imparcial el tamaño relativo (Story Points) y no “planificar” fechas o tareas en el calendario, previniendo que los desarrolladores pongan “colchones de horas” por presiones comerciales.

Pregunta 9: ¿Qué significa la carta especial “?” en el Poker de Estimación del parque y qué acción obliga a tomar?

- **Respuesta breve:** Indica incertidumbre absoluta sobre las reglas de negocio o la tecnología, obligando al Product Owner a retirar la historia y transformarla en una Spike de investigación antes de poder ser votada.

Pregunta 10: ¿Por qué se excluye el módulo de administración en su totalidad del Producto Mínimo Viable (MVP) de la aplicación?

- **Respuesta breve:** Porque no agrega valor directo para validar el comportamiento y la adopción tecnológica de los visitantes finales (hipótesis); los datos de base de datos se pueden mquear o inyectar de manera directa por atrás.

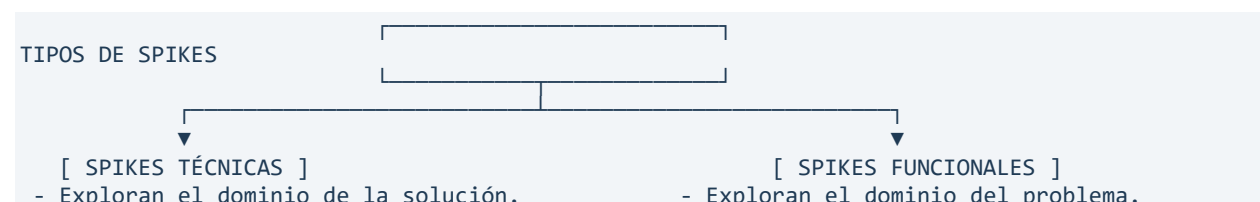
3.13. Spikes: investigación, gestión de incertidumbre y reducción de riesgos

3.13.1. Definición conceptual y formal de “Spike”

- **Definición de las notas de clase (NC_Ardiles / NC_MartinBosi):** una **Spike** es un tipo especial de Historia de Usuario (US) de carácter no funcional o de investigación, utilizada de manera preventiva por el equipo de desarrollo para eliminar riesgos y reducir la incertidumbre técnica o de negocio asociada a un requerimiento antes de proceder a su estimación definitiva.
- **Definición formal de Mike Cohn (Caps. 6 y 12):** un Spike es un experimento o tarea técnica acotada temporalmente (**timebox**) incluida dentro de la planificación de una iteración, que se acomete específicamente con el propósito de adquirir conocimiento, responder a una pregunta técnica o funcional, o viabilizar la estimación de un requerimiento complejo.
- **Qué problema resuelve:** destruye la parálisis técnica y los cronogramas ficticios. Evita que el equipo asuma compromisos sobre funcionalidades que desconoce por completo y previene la mala práctica de asignarle estimaciones desmedidas o erróneas (colchones de puntos de historia) a requerimientos ambiguos.

3.13.2. Clasificación completa de Spikes

La cátedra clasifica las Spikes en dos vertientes según el dominio de la duda que intentan despejar.



- | | |
|---|------------------------------------|
| - Performance, arquitectura, "make or buy". | - Interacción, diseño, flujos UX. |
| - Validación de viabilidad con prototipos. | - Prototipos rápidos con usuarios. |

Spikes técnicos (Technical Spikes)

- *Foco de acción:* exploran el dominio de la **solución técnica**.
- *Cuándo se usan:*
 - Para evaluar la viabilidad de implementar una tecnología o biblioteca nueva.
 - Para analizar la performance o rendimiento potencial de un algoritmo bajo carga.
 - Para tomar decisiones estratégicas de arquitectura, como el dilema “hacer o comprar” (*make or buy*).
 - Cualquier situación donde el equipo requiera una comprensión fiable antes de comprometerse con una funcionalidad en un tiempo fijo.

Spikes funcionales (Functional Spikes)

- *Foco de acción:* exploran el dominio del **problema de negocio y usabilidad**.
- *Cuándo se usan:*
 - Cuando existe una alta incertidumbre sobre cómo el usuario final interactuará con el sistema para obtener el valor esperado.
 - Para ensayar diferentes flujos de pantallas o de interfaz de usuario.
 - Se validan mediante la construcción de prototipos rápidos de baja o alta fidelidad (por ejemplo, maquetas en Figma) para obtener retroalimentación (*feedback*) inmediata de los involucrados (*stakeholders*).

3.13.3. Lineamientos y políticas de gestión de Spikes

Para evitar que las Spikes se conviertan en agujeros negros de horas que resten productividad al sprint, se aplican cinco reglas organizacionales:

1. **Estimables, demostrables y aceptables:** aunque no entreguen software funcional de producción, las Spikes deben poder estimarse (por tiempo fijo), su resultado técnico debe ser demostrable (un reporte de rendimiento, un diagrama o un prototipo que compile) y el Product Owner debe aceptarla formalmente al final de la iteración.
2. **La excepción, no la regla:** el agilismo es empírico y toda historia de usuario posee cierta incertidumbre y riesgo. El equipo debe aprender a tolerar y resolver dudas menores sobre la marcha. Las Spikes se reservan exclusivamente como última opción ante incógnitas críticas y masivas.
3. **El timebox obligatorio:** nunca se abre un Spike de tiempo indefinido. Se le asigna un límite temporal estricto medido en horas o días (por ejemplo, “máximo 12 horas de investigación”). Si al agotarse el tiempo no se despejó la duda, el Spike se cierra igual con la información obtenida para re-evaluar la estrategia.
4. **Separación temporal de iteraciones:** por política de consistencia de SCM, se debe implementar la Spike en una iteración separada y previa a la iteración donde se desarrollarán las historias de usuario resultantes.
5. **La excepción a la regla anterior:** solo si la Spike es pequeña, sencilla y es altamente probable encontrar una solución técnica rápida, se autoriza incluir la Spike y la US resultante en el mismo sprint.

3.13.4. Ejemplos concretos de la cátedra y la bibliografía

Caso A: el histograma de consumo eléctrico (Cohn, Cap. 12)

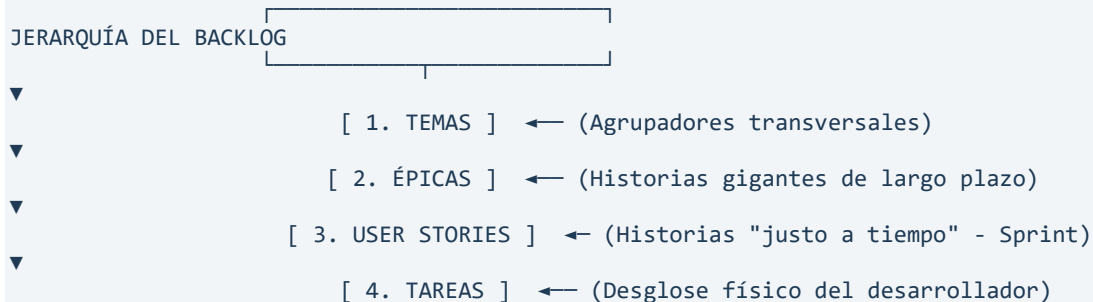
- *Historia de usuario:* Como Cliente quiero ver mi uso diario de energía en un histograma para comprender rápidamente mi consumo pasado, presente y proyectado.
- *La problemática:* el equipo desconoce si la base de datos tolerará el tráfico de actualización constante y el cliente no sabe qué diseño visual prefiere.
- *Resolución (doble Spike):* el equipo abre un **Spike técnico** (para investigar el ancho de banda y si los datos se actualizan por formato *push* o *pull*) y un **Spike funcional** (para crear una maqueta del histograma y testear con los usuarios el estilo y los atributos gráficos antes de codificar).

Caso B: la interfaz CORBA de Java (Cohn, Cap. 2)

- *La problemática:* el cliente exige una interfaz de comunicación CORBA, pero ningún programador del equipo utilizó jamás esa tecnología.
- *Resolución:* se define una Spike con un timebox de 3 días para que un desarrollador configure un entorno mínimo y valide la comunicación, permitiendo estimar con seguridad la historia real en el siguiente sprint.

3.14. Estructura y jerarquía del backlog (líneas de abstracción)

Para evitar la desorganización de los requerimientos, la cátedra enseña que el Product Backlog debe estructurarse de manera jerárquica en diferentes niveles de grano o abstracción.



3.14.1. Tema (Theme)

- **Definición:** es un agrupador lógico o transversal que reúne un conjunto de Historias de Usuario relacionadas funcionalmente.
- **Cuándo se usa:** para organizar el backlog en módulos o áreas temáticas, facilitando la priorización del Product Owner.
- **Ejemplo de la cátedra (NC_Ardiles):** el tema “**Cientes**” o “**Facturación**”, que engloba todas las user stories de alta, baja, modificación y reportes impositivos de los clientes.

3.14.2. Épica (Epic)

- **Definición:** una Historia de Usuario sumamente grande, abstracta, compleja o de grano grueso.

- **Cuándo se usa:** al inicio del proyecto o para funcionalidades lejanas en el tiempo, ubicadas en el fondo del Product Backlog. Permite capturar intenciones del negocio a largo plazo sin gastar tiempo en detallar requerimientos de forma prematura (**Just in Time**).
- **Ejemplo (Cohn, Cap. 6):** Como Usuario quiero planificar mis vacaciones completas en la aplicación. (Contiene múltiples reservas, vuelos, hoteles, etcétera).

3.14.3. Historia de usuario (User Story)

- **Definición:** unidad de especificación de requerimientos de grano fino, pequeña, negociable e independiente que cumple con INVEST.
- **Cuándo se usa:** es la unidad de asignación y trabajo técnico del equipo durante un sprint de desarrollo. Debe poder completarse al 100% dentro del timebox de la iteración.
- **Ejemplo:** Como Visitante quiero comprar una entrada digital para asegurar mi acceso al parque.

3.14.4. Tarea (Task)

- **Definición:** desglose técnico, físico e instrumental de las actividades requeridas para implementar una Historia de Usuario.
- **Cuándo se usa:** se identifican y estiman en **horas ideales de trabajo** durante la **Sprint Planning**, por los desarrolladores que ejecutarán la tarea. No se colocan en el Product Backlog general.
- **Ejemplo:** “Crear la tabla ENTRADAS en PostgreSQL”, “Escribir el caso de prueba unitario en pytest”.

3.15. Técnicas de particionado de épicas (splitting / slicing vertical)

3.15.1. Qué problema resuelve el particionado

Cuando una épica o una historia de usuario obtiene una estimación de **13, 20 o más Story Points**, incumple el atributo **Small** del modelo INVEST y no entra en el sprint. El particionado vertical descompone esa funcionalidad gigante en historias de grano fino más pequeñas, fáciles de estimar y de testear, manteniendo el valor de negocio en cada una.

3.15.2. Los cuatro métodos técnicos de particionado (Cohn, Cap. 12)

Método 1: particionado por límites de datos (splitting across data boundaries)

- **Procedimiento:** identificar los diferentes tipos de datos, formatos o variaciones de entrada que soporta el requerimiento y dividir la historia en función de la madurez de estos datos.
- *Ejemplo de la bibliografía:* la épica “Como usuario quiero ingresar la información de mi balance contable” se divide verticalmente en:
 - Como usuario quiero ingresar información de mis activos fijos (efectivo, vehículos). (Fase 1)
 - Como usuario quiero ingresar información de mis pasivos y deudas bancarias. (Fase 2)

- *Ejemplo técnico:* dividir el soporte de teléfonos en dos historias, una para números locales y otra para números internacionales con sus códigos de país.

Método 2: particionado por operaciones CRUD (Create, Read, Update, Delete)

- **Procedimiento:** romper la historia gigante de “administración” o “gestión” de un maestro de datos en tres o cuatro user stories independientes asociadas a cada una de las operaciones básicas.
- *Ejemplo de la cátedra (NC_Ardiles):* la épica “Como entrenador quiero gestionar los nadadores de mi equipo” se particiona verticalmente en:
 - Como entrenador quiero agregar un nuevo nadador con su legajo al equipo. (Create)
 - Como entrenador quiero editar la información de contacto de un nadador existente. (Update)
 - Como entrenador quiero dar de baja a un nadador que abandonó el equipo. (Delete)

Método 3: aislamiento de aspectos transversales (removing cross-cutting concerns)

- **Procedimiento:** desacoplar el núcleo de la regla de negocio de los requerimientos de soporte transversales de la arquitectura (seguridad, manejo de excepciones complejo, logging o auditoría).
- *Ejemplo:* en lugar de programar el registro con validación LDAP y encriptación de alta seguridad de entrada, se crean dos historias:
 - Como Visitante quiero registrar mi usuario ingresando e-mail y clave básica. (El núcleo funcional).
 - Como Administrador de Seguridad quiero que el registro de visitantes cumpla con los estándares de encriptación SHA-256. (Aspecto transversal aislado).

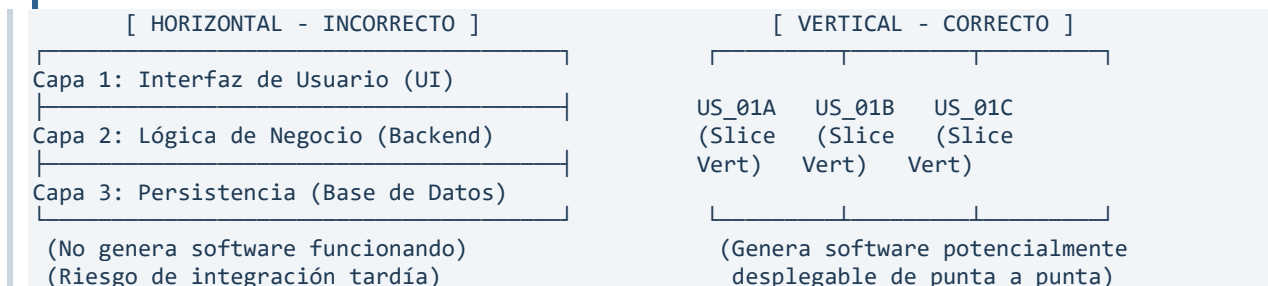
Método 4: postergación de requerimientos de rendimiento (ignore performance targets)

- **Procedimiento:** hacer funcional una historia rápidamente ignorando momentáneamente los objetivos duros de performance, tiempo de respuesta o escalabilidad, que se tratan en un sprint posterior.
- *Ejemplo:*
 - Como Visitante quiero visualizar el mapa del parque de forma básica. (Satisface la funcionalidad en el sprint actual).
 - Como Visitante quiero que el renderizado gráfico del mapa demore menos de 1.5 segundos bajo conexiones 3G. (Requerimiento de performance tratado como historia separada para la siguiente iteración).

3.15.3. El concepto de “tracer bullet” (bala de trazado / slicing vertical)

La cátedra y la bibliografía (Cohn, Cap. 12) son terminantes con una regla de oro:

Regla: nunca partas una épica de forma horizontal, por capas técnicas. El particionado de requerimientos debe ser siempre vertical.



- **Particionado horizontal (mala práctica):** crear historias como “Diseñar la interfaz de cobro”, “Programar la lógica de cálculo del IVA” o “Crear la base de datos de transacciones”. Ninguna de estas capas por separado aporta valor de negocio ni genera software funcionando que el PO pueda inspeccionar en la Sprint Review.
- **Particionado vertical (tracer bullet):** se dispara una rebanada vertical que atraviesa de punta a punta todas las capas del sistema (diseño del botón en UI, lógica de negocio básica en backend y guardado en base de datos) para una sola transacción atómica pequeña. Esto valida de inmediato la arquitectura de integración, reduce el riesgo de acoplamiento tardío y entrega software potencialmente desplegable al final del sprint.

3.16. Errores comunes y advertencias en Spikes y estructura del backlog

- ⚠ **Tratar la Spike como un comodín para procrastinar el análisis:** creer que ante cualquier requerimiento medianamente complejo se debe abrir una Spike. Esto dilata el desarrollo. La Spike es la excepción, no la regla.
- ⚠ **Olvidar el timebox de la Spike:** permitir que un desarrollador pase dos semanas investigando una API sin entregar un reporte técnico intermedio concreto. El timebox es inamovible.
- ⚠ **El desequilibrio de la pirámide del backlog:** tener un backlog saturado de épicas gigantes en la parte superior listas para programar, o cientos de mini-historias atómicas de 1 SP en el fondo que queden obsoletas por el cambio de requerimientos. El backlog se gestiona de forma dinámica y adaptativa, con grano fino arriba y grano grueso abajo (Just in Time).

3.17. Relación de Spikes y backlog con otros conceptos del apunte

- **Relación con la Unidad 1 (proceso y producto):** las Spikes y las historias son artefactos de especificación de requerimientos que alimentan la dimensión **Producto** y protegen la dimensión **Proyecto** al blindar de incertidumbre al equipo de desarrollo.
- **Relación con el modelo INVEST (Unidad 3):** el particionado vertical de épicas es el mecanismo de salvaguarda obligatorio para forzar el cumplimiento del atributo **Small** e impedir el ingreso de requerimientos sobredimensionados de 13 o 20 SP al Sprint Backlog.
- **Relación con SCM (Unidad 2):** al ser considerados ítems de configuración de producto, tanto las épicas como las historias de usuario resultantes y los reportes de salida de las Spikes se versionan en Git, asegurando la trazabilidad desde el requerimiento original hasta la línea de código que lo satisface.

APÉNDICE A — Glosario bilingüe (ES/EN)

Este glosario consolida los términos técnicos, siglas y conceptos fundamentales desarrollados a lo largo de las Unidades 1, 2 y 3 del apunte de estudio. Está ordenado alfabéticamente por su término en español, incluyendo su denominación equivalente en inglés, una definición concisa y la unidad temática a la que pertenece.

A

- Aceptación, Pruebas de (Acceptance Tests)
Unidad 3
Escenarios de verificación dinámicos de alto nivel (pasa/falla), redactados desde la perspectiva del negocio, que permiten comprobar empíricamente si una Historia de Usuario cumple con los criterios de aceptación del cliente.
- Adaptación (Adaptation)
Unidad 3
Uno de los tres pilares del empirismo de Scrum. Consiste en la capacidad autónoma del equipo para ajustar los procesos o materiales inmediatamente al detectar desvíos fuera de los límites aceptables de calidad.
- Auditoría Física de Configuración (Physical Configuration Audit - PCA)
Unidad 2
Actividad estática de SCM que verifica de forma administrativa la completitud e integridad del repositorio, controlando que estén presentes todos los archivos físicos que el plan decía que debían estar.
- Auditoría Funcional de Configuración (Functional Configuration Audit - FCA)
Unidad 2
Actividad dinámica de SCM que evalúa que la funcionalidad y performance reales del software construido coincidan de forma exacta con las especificaciones de requerimientos (ERS) aprobadas.

B

- Bespoke Software / Custom Software (Software a Medida)
Unidad 1
Sistemas de software desarrollados de manera específica bajo contrato para un cliente particular, quien tiene el control exclusivo del alcance y las especificaciones de requerimientos.
- Branching (Ramificación)
Unidad 2
Estrategia de SCM en herramientas de versionado (como Git) que permite aislar líneas de desarrollo paralelas (ramas) para codificar funcionalidades o correcciones de forma independiente sin alterar el código estable.

- Build (Compilación / Construcción)
Unidad 2 / 3
Proceso técnico de traducir el código fuente en artefactos ejecutables terminados, integrando y validando las porciones lógicas concurrentes del equipo.

C

- Card (Tarjeta)
Unidad 3
La primera de las “3 C’s” de las Historias de Usuario. Es el soporte físico (tradicionalmente de cartón) que actúa como un recordatorio o *token* de una necesidad de negocio que requiere discusión futura.
- Commit (Confirmación)
Unidad 2
Operación en sistemas de control de versiones (como Git) que registra de forma física, inalterable y con metadatos asociados (autor, timestamp) un conjunto de cambios lógicos en el repositorio.
- Comité de Control de Cambios (Change Control Board - CCB)
Unidad 2
Órgano colegiado de un proyecto de software responsable de evaluar sistemáticamente el impacto de las solicitudes de cambio para decidir de forma unánime su aprobación, rechazo o postergación.
- Compromiso (Commitment)
Unidad 3
Una promesa contractual firme sobre fechas y alcance asumida de común acuerdo por el equipo de desarrollo, asimilando riesgos específicos del negocio (no debe confundirse con la estimación).
- Confirmación (Confirmation)
Unidad 3
La tercera de las “3 C’s” de las Historias de Usuario. Representa el dorso de la tarjeta física, donde se plasman los criterios y pruebas de aceptación cuantitativas para validar la entrega.
- Cono de la Incertidumbre (Cone of Uncertainty)
Unidad 3
Modelo gráfico empírico de Barry Boehm que demuestra cómo el margen de variabilidad y error de las estimaciones disminuye de forma sistemática a medida que avanza el proyecto y disminuye la ignorancia sobre el sistema.
- Conversación (Conversation)
Unidad 3
La segunda de las “3 C’s” de las Historias de Usuario. Es el intercambio de ideas cara a cara entre el Product Owner y el equipo técnico para descubrir de forma colaborativa los detalles funcionales del requerimiento.
- Criterios de Aceptación (Acceptance Criteria)
Unidad 3

Condiciones lógicas y de negocio mínimas acordadas que el Product Owner define para que una Historia de Usuario sea considerada funcionalmente correcta.

D

-
- | | | | |
|---------------|---------|------------|-------|
| Deuda | Técnica | (Technical | Debt) |
| <i>Unidad</i> | | | 3 |

Metáfora de Ward Cunningham que describe el costo a largo plazo que asume un proyecto al priorizar la velocidad de entrega prematura por sobre la excelencia del diseño y la calidad del código (“rápido y sucio”).
 - | | | | |
|---------------|---------|--------|-------|
| Días | Ideales | (Ideal | Days) |
| <i>Unidad</i> | | | 3 |

Unidad de medida absoluta basada en estimar el tiempo puro de codificación sin interrupciones del día a día (prohibido por la cátedra por atentar contra la transparencia y asimilarse al micromanagement).

E

-
- | | | |
|---------------|--|---------|
| Epic | | (Épica) |
| <i>Unidad</i> | | 3 |

Historia de usuario de gran envergadura lógica, esfuerzo e incertidumbre que no puede ser completada de forma segura en un único sprint, requiriendo ser particionada de manera vertical.
 - | | | | | | |
|---------------|-----------|------------|-----------|-----------|--------|
| Escala | Fibonacci | Modificada | (Modified | Fibonacci | Scale) |
| <i>Unidad</i> | | | | | 3 |

Escala numérica no lineal (0, 0.5, 1, 2, 3, 5, 8, 13, 20, 40, 100) adoptada para estimar Story Points, reflejando matemáticamente que a mayor tamaño de requerimiento, la incertidumbre crece exponencialmente.
 - | | | | |
|--|---|---|---|
| Especificación de Requerimientos de Software (Software Requirements Specification - SRS / ERS) | | | |
| <i>Unidad</i> | 2 | / | 3 |

Documento técnico formal que detalla exhaustivamente todos los requerimientos funcionales, no funcionales y de interfaz que el sistema a construir debe cumplir de forma obligatoria.
 - | | | | |
|---------------|----------|-----------|-------------|
| Estimación | Relativa | (Relative | Estimating) |
| <i>Unidad</i> | | | 3 |

Técnica cognitiva que evalúa el tamaño de una funcionalidad comparándola por analogía contra una historia base conocida (canónica), en lugar de intentar calcular horas de reloj absolutas.

G

-
- Gitignore
Unidad 2
Archivo de configuración en el directorio raíz de Git que declara de antemano qué carpetas, archivos temporales o compilados locales deben ser ignorados por el versionador para mantener limpio el repositorio.
 - Gitkeep
Unidad 2
Archivo oculto vacío que se utiliza de forma artificial dentro de carpetas vacías para forzar a Git a subir y conservar la estructura física unificada del repositorio desde el primer día.

I

-
- Inspección (Inspection)
Unidad 3
Pilar del empirismo lúdico. Consiste en la evaluación recurrente de los artefactos de software y de los procesos de trabajo para detectar anomalías o desviaciones de calidad de forma temprana.
 - INVEST
Unidad 3
Checklist de calidad de Bill Wake (2003) para evaluar Historias de Usuario. Exige que sean: Independientes, Negociables, Valiosas, Estimables, Small (pequeñas) y Testeables.
 - Ítem de Configuración de Software (Software Configuration Item - SCI / IC)
Unidad 2
Cualquier elemento de información o artefacto físico o digital (documento, código, scripts, manual) producido a lo largo del desarrollo de software que se coloca bajo control de versiones formal en el repositorio.

L

-
- Línea Base (Baseline)
Unidad 2
Especificación o producto de software que ha sido formalmente revisado y aprobado por el CCB, que actúa como base para desarrollos posteriores y que solo puede modificarse mediante el flujo de control de cambios.

M

-
- Manifiesto Ágil (Agile Manifesto)
Unidad 3
Acuerdo consensuado en 2001 por 17 líderes del desarrollo de software que formaliza la

filosofía ágil a través de 4 valores centrales y 12 principios de gestión de requerimientos y equipos autoorganizados.

O

-
- Orden de Cambio de Ingeniería (Engineering Change Order - ECO)
Unidad 2
Documento emanado del CCB que autoriza formal y administrativamente a un desarrollador a modificar una línea base para implementar un cambio aprobado de forma controlada.
 - Output (Salida / Entregable)
Unidad 3
La cantidad física de elementos de software producidos por el equipo técnico (ej. líneas de código, número de pantallas programadas).
 - Outcome (Resultado / Valor de Negocio)
Unidad 3
El beneficio de negocio cuantificable y la mejora operativa real que el usuario final recibe cuando interactúa con el software.

P

-
- Plan de Gestión de Configuración (Configuration Management Plan - PGC)
Unidad 2
Documento administrativo y de ingeniería que planifica y detalla las reglas de nombrado de ICs, la estructura de carpetas, los roles, los criterios de baselines y los procedimientos de cambio para el repositorio.
 - Poker de Estimación (Poker Estimation)
Unidad 3
Técnica de estimación colaborativa, gamificada y basada en el consenso grupal a ciegas para puntuar Story Points comparativos, evitando los sesgos de anclaje de perfiles senior. (Término exigido estrictamente por la cátedra).
 - Proceso (Process)
Unidad 1
Definición conceptual y abstracta de lo que se debería hacer para construir software, especificando actividades, flujos de trabajo, pre/postcondiciones y roles asociados (ej. PUD).
 - Producto (Product)
Unidad 1
El software profesional final entregado, concebido como conocimiento representado en distintos niveles de abstracción junto con toda su documentación asociada y datos de configuración.
 - Producto Mínimo Adorable (Minimum Lovable Product - MLP)
Unidad 3
La versión de software mínima que, además de ser funcional, incorpora desde el inicio diseño y usabilidad excepcionales para deleitar a los usuarios en mercados competitivos.

- Producto Mínimo Comercializable (Minimum Marketable Product - MMP)
Unidad 3
La versión de software utilizable y testeada más pequeña que entrega valor de negocio real e inmediato, y que el equipo puede poner físicamente en producción.
- Producto Mínimo Viable (Minimum Viable Product - MVP)
Unidad 3
Versión preliminar y de bajísimo costo de un producto concebida con el único propósito de capturar el máximo aprendizaje validado sobre las necesidades y comportamiento de los usuarios (validación de hipótesis).
- Propuesta de Cambio de Ingeniería (Engineering Change Proposal - ECP)
Unidad 2
Documento que recopila el análisis de impacto técnico, estimación de esfuerzo en horas y cambios colaterales que causaría aplicar una solicitud de modificación en el repositorio.
- Proyecto (Project)
Unidad 1
Unidad de gestión temporal, acotada por presupuesto, cronograma y recursos humanos asignados para obtener un producto de software único.

R

-
- Requerimientos Emergentes (Emergent Requirements)
Unidad 3
Principio empírico que postula que el 50% de los requerimientos verdaderos no se conocen al inicio del proyecto y emergen de manera inevitable recién cuando el usuario opera el software real en producción.

S

-
- SCM (Software Configuration Management)
Unidad 2
Gestión de Configuración de Software. Disciplina de soporte de la ingeniería de software encargada de identificar, controlar, auditar e informar el estado evolutivo del repositorio para proteger la integridad del producto.
 - Spike
Unidad 3
Historia de usuario experimental orientada exclusivamente a la investigación técnica o de dominio funcional, acotada en tiempo, diseñada para reducir la incertidumbre y habilitar estimaciones futuras.

T

-
- T-Shirt Sizing (Taller de Remeras)
Unidad 3

Escala cualitativa blanda de estimación (XS, S, M, L, XL) utilizada en el Product Backlog para dimensionar requerimientos a largo plazo con bajo nivel de información.

- Transparencia (Transparency)
Unidad 3
Uno de los tres pilares del empirismo. Exige que todos los aspectos del desarrollo y el estado real de la calidad del software sean visibles y comprensibles para todos los interesados.
- Trazabilidad (Traceability)
Unidad 2
Atributo de ingeniería que permite vincular unívocamente un requerimiento funcional de negocio con sus documentos de diseño, módulos de código fuente físicos y casos de prueba que lo certifican.

U

-
- User Story (Historia de Usuario - US)
Unidad 3
Descripción corta y de alto nivel de una necesidad de negocio contada desde el rol de usuario, estructurada en tres dimensiones (3 C's) que facilitan la especificación ágil Just in Time.

V

-
- Velocidad (Velocity)
Unidad 3
Métrica empírica que mide la capacidad de entrega de un equipo por sprint, calculada al sumar de forma binaria (100% de éxito o cero) los Story Points de las historias de usuario aceptadas por el Product Owner.
 - Verificación y Validación (V&V)
Unidad 2 / 4
Procesos paralelos de aseguramiento de calidad. La Verificación controla de manera estática que el producto cumpla con las especificaciones técnicas de su baseline. La Validación evalúa de forma dinámica que el software resuelva el problema real del usuario en el negocio.

APÉNDICE B — Mapa de relaciones entre unidades

Este documento establece la red de conexiones conceptuales de las primeras tres unidades de la materia. Analiza cómo cada concepto de ingeniería de software se apoya en, contradice o extiende a otro, clasificado por su unidad de origen.

1. CONEXIONES CONCEPTUALES DESDE LA UNIDAD 1 (Introducción e Integridad)

```
[ MAPA DE IMPACTO DE LA UNIDAD 1 ]  
[ EL PENTÁGONO DEL ECOSISTEMA ] [ PROCESOS EMPÍRICOS (SCRUM) ] [ INHERENCIA DE LA COMPLEJIDAD ]  
-> Extiende: ICs de Producto (U2) -> Extiende: Reqs Emergentes (U3) -> Contradice: BRUF tradicional (U3)  
-> Contradice: Dormitorio Desord. (U2)-> Se conecta: Auditoría Continua (U2)-> Se apoya en: Spikes y MVP (U3)
```

1.1. El Pentágono del Ecosistema del Software (Proceso, Proyecto, Producto, Personas, Herramientas)

- **Se apoya en: Personas / Personal** (Dimensión 4), dado que la capacidad humana, la motivación y la autoorganización del equipo son las que ejecutan materialmente el proceso teórico.
- **Extiende a: Identificación de Ítems de Configuración** (Unidad 2). Los entregables lógicos que componen el **Producto** (Dimensión 3, ej. código, especificaciones, scripts de base de datos) se convierten en Ítems de Configuración de Software (ICs / SCIs) individuales que deben protegerse administrativamente.
- **Contradice a:** La patología de SCM del “**Dormitorio Desordenado**” (Unidad 2). Cuando la dimensión “Herramientas” (Git/GitHub) y el “Proceso” de SCM no se coordinan de forma unificada, el “Producto” se degrada de forma sistemática y el “Proyecto” pierde el control del alcance.

1.2. Procesos Empíricos (Scrum / Kanban)

- **Se apoya en:** Los tres pilares fundamentales de Ken Schwaber: **Transparencia, Inspección y Adaptación**.
- **Extiende a: Requerimientos en el Paradigma Ágil** (Unidad 3), asumiendo empíricamente que el 50% de los requerimientos verdaderos son de naturaleza emergente y solo se descubrirán mediante la inspección en producción.
- **Se conecta con: Auditorías de Configuración Continuas** (Unidad 2). El enfoque empírico de testing de regresión en el servidor de Integración Continua (CI) reemplaza de forma ágil a las auditorías de configuración funcionales (FCA) tradicionales y burocráticas de fin de fase.

1.3. Inherencia de la Complejidad del Software (Fred Brooks)

- **Contradice a:** La estimación absoluta en horas lineales de reloj y el enfoque del **BRUF (Big Requirements Up Front)** de la Unidad 3. Intentar predecir el comportamiento de un sistema inherentemente complejo en un plano de requerimientos estático inicial es una falacia de ingeniería.
- **Se apoya en:** La técnica de las **User Stories** (Unidad 3) y el desarrollo incremental de **Productos Mínimos Viables (MVP)** para descubrir las dificultades de integración técnica y de negocio de forma temprana.

2. CONEXIONES CONCEPTUALES DESDE LA UNIDAD 2 (SCM - Control de Configuración)

```
[ MAPA DE IMPACTO DE LA UNIDAD 2 ]  
[ IDENTIFICACIÓN DE ICs ] [ CONTROL DE CAMBIOS ] [ AUDITORÍA DE CONFIGURACIÓN ]  
-> Extiende: Líneas Base (Baselines) -> Se apoya: Análisis de Impacto -> Se apoya:  
Verificación y Val.  
-> Se conecta: Trazabilidad Post-ERS -> Contradice: SCM "Frictionless" -> Extiende:  
Confirmación de USs
```

2.1. Identificación de Ítems de Configuración (ICs)

- **Se apoya en:** El inventario de entregables del **Producto** definidos en el PGC académico.
- **Extiende a: Líneas Base (Baselines).** Es el cimiento lógico indispensable: no se puede conformar ni congelar un punto de referencia estable (Baseline) en Git sin haber definido de forma previa las reglas de nombrado unívocas y las extensiones rígidas de cada componente involucrado.
- **Se conecta con:** La **Trazabilidad Post-ERS** (Unidad 3). Permite acoplar unívocamente las tarjetas de Historias de Usuario con sus ramas físicas de desarrollo en Git y sus clases de código fuente asociadas.

2.2. Control de Cambios (Procedimiento formal: CCB, CRF, ECP, ECO)

- **Se apoya en:** El **Análisis de Impacto (ECP)** para estimar colisiones en la arquitectura del producto y desvíos de cronograma del proyecto.
- **Contradice a:** El principio del **SCM sin fricción (Frictionless)** de los ambientes ágiles (Unidad 3). En Scrum, el flujo rígido y secuencial del CCB tradicional se descentraliza por completo: el Product Owner prioriza de forma directa los cambios funcionales en el Product Backlog, mientras que el equipo técnico decide de manera autónoma las refactorizaciones de código (XP).

2.3. Auditorías de Configuración (Física - PCA y Funcional - FCA)

- **Se apoya en:** Las disciplinas de **Verificación y Validación (V&V)** del software.
- **Extiende a:** La **Confirmación (3ª C de las User Stories)** de la Unidad 3. Las pruebas de aceptación con sintaxis de infinitivo (ej: **Probar...**) actúan como la rúbrica de auditoría de

configuración funcional (FCA) para certificar de forma binaria que el software construido cumple con los criterios de aceptación del PO.

2.4. Informes de Estado / Status Accounting

- **Se apoya en:** El registro sistemático de metadatos históricos de Git (commits, autores, timestamps, diffs de código).
- **Extiende a:** El cálculo de la **Velocidad del equipo** (Unidad 3) al reportar de manera inalterable qué historias de usuario cumplieron con la Definition of Done y fueron aceptadas de forma binaria (100% de avance o 0%) al final de cada iteración.

3. CONEXIONES CONCEPTUALES DESDE LA UNIDAD 3 (Requerimientos y Estimación)

```
[ MAPA DE IMPACTO DE LA UNIDAD 3 ]  
[ USER STORIES (3 C's) ] [ STORY POINTS / FIBONACCI ] [ ESPECTRO DE MÍNIMOS ]  
-> Se apoya: Principios 6 y 11 -> Se apoya: Complejidad/Esfuerzo/Duda -> Se apoya: Hipótesis Lean  
-> Contradice: BRUF (Waterfall) -> Contradice: Días Ideales (Cohn) -> Contradice: Const. Horizontal  
-> Extiende: Pruebas de Aceptación -> Extiende: Velocidad / Cronograma -> Extiende: MMP (Release 1)
```

3.1. User Stories (Dimensión de las 3 C's e INVEST)

- **Se apoya en:** El **Principio 6** (conversación cara a cara para eliminar especificaciones frías) y el **Principio 11** (emergencia de requerimientos en equipos autoorganizados) del Manifiesto Ágil.
- **Contradice a:** Las especificaciones textuales de requerimientos del **modelo tradicional (Cascada / Waterfall)**.
- **Extiende a:** Las **Pruebas de Aceptación**, que actúan como el insumo directo para que el equipo de testing diseñe la suite de pruebas funcionales automatizadas y manuales de la Unidad 4.

3.2. Story Points y Escala de Fibonacci Modificada

- **Se apoya en:** Las tres dimensiones cualitativas del tamaño: **Complejidad, Esfuerzo intrínseco e Incertidumbre/Duda**.
- **Contradice a:** La estimación de **Días Ideales (Ideal Days)** de Mike Cohn y las horas de reloj lineales absolutas.
- **Extiende a:** El cálculo de la **Velocidad de sprint**, que a su vez se inyecta en la **Ecuación central de proyección** de SCM para calendarizar de manera matemática las fechas estimadas del plan de release.

3.3. El Espectro de Mínimos (MVP, MMP y MLP)

- **Se apoya en:** Las hipótesis de negocio y los ciclos de experimentación empíricos de la filosofía Lean.
- **Contradice a:** La **construcción de software por capas horizontales** (ej: programar primero toda la base de datos, luego la API y al final la pantalla). Exige de forma excluyente la técnica de **corte vertical (slicing)** del pastel (las 3 C acopladas de inicio a fin).
- **Extiende a:** El **MMP (Minimum Marketable Product)** que se consolida formalmente en el primer release en producción del repositorio (Línea Base del MMR1).

4. LAS TRES CONTRADICCIONES CLAVE DE EXAMEN (Bibliografía vs. Cátedra)

A continuación, se consolidan y contrastan las tres grandes tensiones teóricas y terminológicas identificadas en la materia, indicando el razonamiento de cada postura y la **respuesta obligatoria que debés dar para aprobar el examen teórico y práctico**:

CONTRADICCIÓN 1: Longevidad de las User Stories (¿Se descartan o se conservan?)

[BIBLIOGRAFÍA (COHN)] [CÁTEDRA (UTN-FRC)]
Las historias son efímeras. Las historias son ICs de Producto.
Se descartan tras el sprint. Se conservan y versionan en Git.
La especificación viva es el código. Aseguran la trazabilidad Post-ERS.

El Razonamiento de la Bibliografía (Mike Cohn - User Stories Applied)

- La User Story es un placeholder de cortísimo plazo que sirve para gatillar la conversación.
- Una vez que el desarrollador codificó la funcionalidad, el tester la verificó y el Product Owner la aceptó en la Sprint Review, **la historia física de papel o su registro administrativo se pueden descartar o romper**.
- La especificación viva y actualizada del sistema pasa a ser **el código fuente y sus tests de regresión automatizados**. Conservar documentos de requerimientos estáticos paralelos genera **desperdicio lógico (waste)** y desincronización de información.

El Razonamiento de la Cátedra (Judith Meles / Laura Covaro)

- Las User Stories son **Ítems de Configuración de Producto (ICs)** fundamentales de la Gestión de Configuración.
- Para que un desarrollo de software posea integridad y calidad formal, es mandatorio garantizar de forma inalterable la **trazabilidad de punta a punta (post-ERS)** a lo largo de todo el ciclo de vida.
- Si las historias se descartan tras el sprint, se rompe la trazabilidad trasera, impidiendo que el equipo de mantenimiento audite en el futuro por qué se programó determinada lógica de

negocio o bajo qué acuerdo comercial se aprobó. Las historias se identifican en el PGC, se suben al repositorio Git, se versionan y se conservan de forma permanente.

⚠ QUÉ RESPONDER EN EL EXAMEN:

En la UTN-FRC, se debe responder categóricamente que **las User Stories se consideran Ítems de Configuración de Producto que NO se descartan**, sino que se catalogan en el inventario del PGC, se suben al repositorio Git y se conservan y versionan de forma inalterable para garantizar la consistencia, la integridad y la trazabilidad de largo plazo del producto de software.

CONTRADICCIÓN 2: Story Points frente a Días Ideales (Ideal Days)

[BIBLIOGRAFÍA (COHN)] [CÁTEDRA (UTN-FRC)]
Días Ideales: opción viable. Días Ideales: PROHIBIDOS.
Fáciles de explicar a gerentes. Tientan al micromanagement burocrático.
Suponen trabajo sin interrupción. Asimétricos por seniority del programador.

El Razonamiento de la Bibliografía (Mike Cohn - Agile Estimating and Planning)

- Cohn describe que **los Días Ideales representan una escala viable de estimación**. Consiste en estimar cuánto tiempo demandaría una funcionalidad si se programara de forma aislada y continua, sin ninguna interrupción cotidiana (reuniones, soporte, correos, pausas).
- Su ventaja radica en que es una métrica sumamente intuitiva de entender para los perfiles de gerencia y comerciales ajenos al desarrollo técnico de software. No obstante, Cohn prefiere los Story Points por ser una medida pura de tamaño que facilita el consenso de equipo.

El Razonamiento de la Cátedra (Judith Meles / Laura Covaro)

- La cátedra **prohíbe de forma tajante e inflexible estimar en Días Ideales**, exigiendo únicamente el uso de **Story Points** respaldados por las tres dimensiones de la estimación.
- **El argumento del docente:** El concepto de “Días Ideales” es una **trampa de gestión tradicional**. Al contener la palabra “Días”, los clientes y jefes de proyecto inevitablemente la compararán de forma lineal con el calendario real. Esto forzaría al equipo a desvirtuar la métrica y degradarla a horas reales para justificar desvíos.
- Además, el “día ideal” es **asimétrico y dependiente de la habilidad de cada persona**: 3 días ideales de un programador Senior representan 10 días reales de un Junior, lo que destruye el consenso colectivo de tamaño en el repositorio.

⚠ QUÉ RESPONDER EN EL EXAMEN:

En los parciales teóricos y prácticos de la materia, se debe responder que **se estima exclusivamente utilizando Story Points**, justificando su puntaje en base a la Complejidad, el Esfuerzo y la Incertidumbre. Se debe fundamentar técnicamente el rechazo de los Días Ideales explicando que destruyen la transparencia del equipo al asociar el tamaño del producto con la velocidad del calendario, tentando al micromanagement y variando de forma inaceptable según el seniority de quien codifique la solución.

CONTRADICCIÓN 3: Planning Poker frente a Poker de Estimación (Poker Estimation)

[MERCADO / BIBLIOGRAFÍA] [CÁTEDRA (UTN-FRC)]
Término común: "Planning Poker". Término exigido: "Poker de Estimación".
Propuesto comercialmente por Cohn. Evita confundir el "Tamaño" técnico
Se asocia directamente a Scrum. con el "Compromiso" del calendario.

El Razonamiento del Mercado y Bibliografía

- El término comercial patentado y más popularizado en los frameworks ágiles de la industria de software es **“Planning Poker”** (o “Poker Planning”), acuñado por James Grenning y popularizado de forma masiva por Mike Cohn. Se utiliza para describir la reunión gamificada donde se votan los puntos de historia.

El Razonamiento de la Cátedra (Judith Meles / Laura Covaro)

- El docente en el audio (5. Estimaciones ágiles.m4a) es terminante: **“En esta materia no se dice ‘Planning Poker’ ni ‘Poker de Planificación’. Decir eso en el examen es un error conceptual grave. Se debe denominar obligatoriamente* ‘Poker de Estimación’ (Poker Estimation)”**.
- El fundamento ingenieril:** El término “Planning Poker” induce al error de creer que el equipo está “planificando compromisos o plazos temporales” mientras vota las cartas. El poker tiene como único fin **estimar el tamaño relativo** del producto de forma probabilística. La planificación del sprint (**Sprint Planning**) es una ceremonia posterior y diferente, donde el PO toma el tamaño estimado, evalúa la capacidad del equipo y se pactan los compromisos. Mezclar los términos demuestra una falta de distinción básica entre tamaño y calendario.

⚠ QUÉ RESPONDER EN EL EXAMEN:

En cualquier instancia de evaluación de esta cátedra, se debe utilizar de forma excluyente el concepto de **“Poker de Estimación” (Poker Estimation)**. Si se te pregunta el motivo de esta exigencia, se debe justificar explicando que la estimación es la predicción del tamaño del producto (actividad técnica y probabilística en Story Points), mientras que la planificación es el compromiso del alcance en el tiempo de calendario (actividad de gestión de plazos). Denominarlo “Planning Poker” fomenta la confusión conceptual de estas dos disciplinas de control diferenciadas.